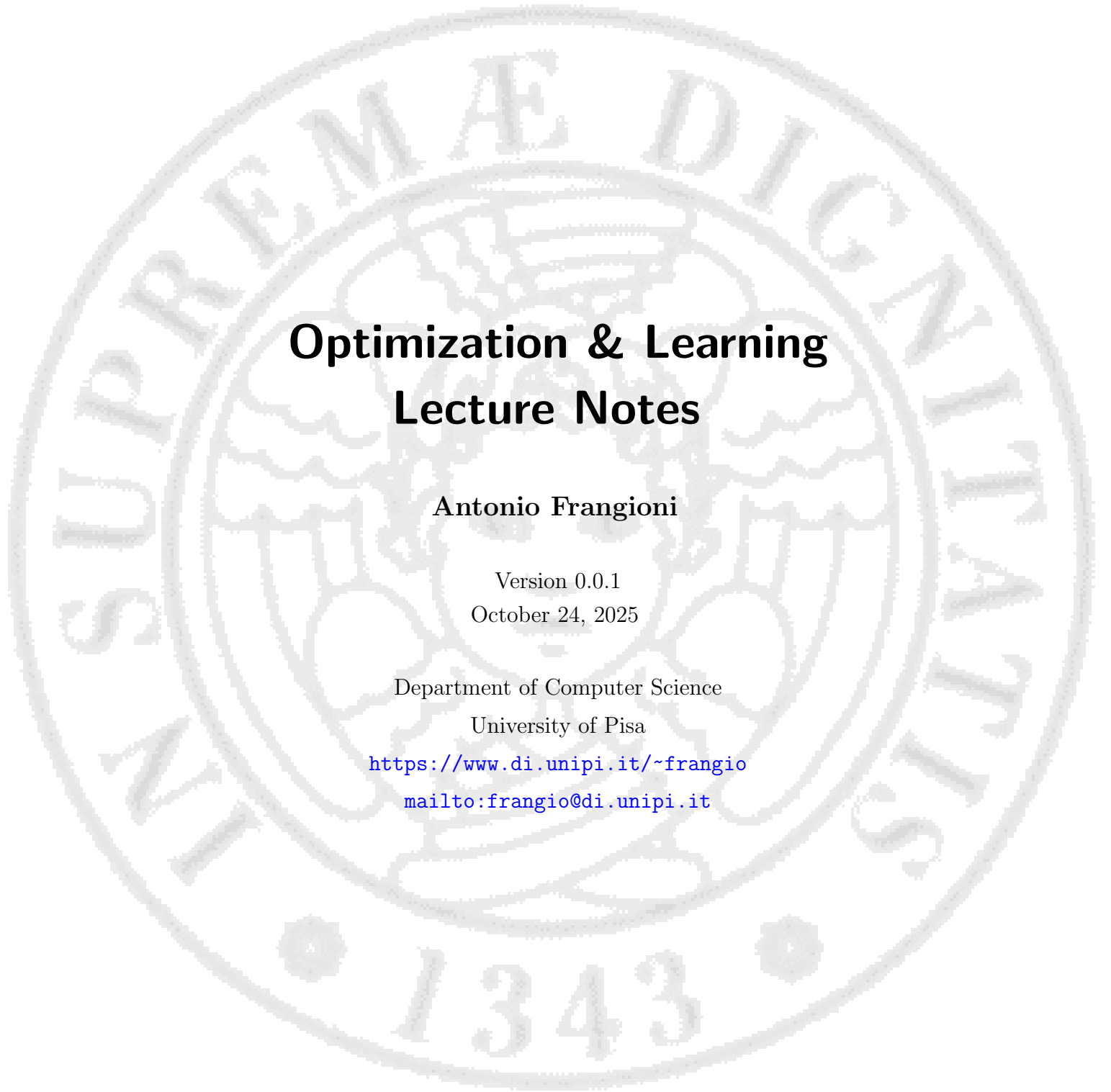


The background of the slide features a large, faint watermark of the University of Pisa seal. The seal is circular and contains the Latin motto "SUPREMAE DIGINITATIS" around the top and "1343" at the bottom. In the center of the seal is a detailed crest featuring a lion and other heraldic elements.

Optimization & Learning Lecture Notes

Antonio Frangioni

Version 0.0.1
October 24, 2025

Department of Computer Science
University of Pisa

<https://www.di.unipi.it/~frangio>
<mailto:frangio@di.unipi.it>

Preface

These lecture notes are intended as a support for some courses at the University of Pisa dealing with the intersection between Data Science / Artificial Intelligence / Machine Learning and Optimization, such as (the optimization part of) Computational Mathematics for Learning and Data Analysis of the Master Program in Computer Science, Optimization for Data Science in the Master Program in Data Science and Business Informatics, and Optimization Methods and Game Theory in the Master Program in Artificial Intelligence and Data Engineering.

The aim of these courses is to provide mathematical optimization concepts that are useful in the design and analysis of methods for learning out of (large sets of) data. It has to be intended that the fundamental DS/AI/ML concepts are *not* covered in these lectures; some of the main models (regression, Neural Networks, Support Vector Machines, Clustering) are described, but only as relevant examples of applications of the optimization techniques. Issues like the learning capabilities of these models and how to improve them are completely outside of the focus of these lectures, since they are dealt with in specific courses. What is covered here is purely some of the mathematical optimization methods that are relevant among several others, for DS/AI/ML applications.

Several students of the courses have contributed to early versions of these lectures, among which Gemma Martini, Ivan Grujic, Federica Di Pasquale, Erica Neri, and Andrea Cardia. The responsibility of any error, inconsistency or incompleteness in the lecture notes fully remains with the extensor.

These lecture notes are distributed under the Creative Commons license

Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0)

<https://creativecommons.org/licenses/by-nc-nd/4.0>



This corresponds to the fact that these lectures can be freely used in courses at any other institution provided that proper attribution is provided and no economic gain is obtained by providing the material to the students; the best approach should be that interested students be just directed to the web page

<https://commalab.di.unipi.it/courses>

where the most recent version will be hosted. It is appreciated if any use of this material in courses is made known to the extensors, and even more that feedback about errors or possible improvements is provided. Experts willing to directly contribute to the improvement of the material, subject to the current terms of the license, are welcome to contact the extensors.

Contents

I	A Gentle Introduction	7
1	Simple Optimization Problems	9
1.1	Optimization Problems	9
1.1.1	First definitions	9
1.1.2	Approximate optimality	12
1.1.3	(Simple) reformulations	13
1.1.4	Representing the feasible set	13
1.1.5	Approximate feasibility	15
1.1.6	Domains and extended-valued functions	16
1.1.7	An aside: (why not) multivariate optimization	17
1.2	(Outrageously) Simple (Univariate) Optimization	19
1.2.1	Linear univariate functions	19
1.2.2	Homogeneous quadratic univariate functions	20
1.2.3	Quadratic non-homogeneous univariate functions	21
1.3	(Not always) Simple Multivariate Optimization	22
1.3.1	Linear multivariate functions	22
1.3.2	Very simple quadratic functions: separable (non-homogeneous)	24
1.3.3	The multivariate homogeneous quadratic function	25
1.3.4	Optimizing non-homogeneous quadratic functions	29
1.4	Multivariate Quadratic optimization: Gradient Method	31
1.4.1	Some generalities on iterative methods	31
1.4.2	The Gradient Method for Quadratic Functions	32
1.4.3	Convergence of the gradient method for $Q \succ 0$	34
1.4.4	Some notes on the stopping criterion	36
1.4.5	Convergence of the gradient method for $Q \succeq 0$	37
1.5	The Conjugate Gradient Method	39
1.5.1	Q -conjugate directions	39
1.5.2	The conjugate gradient method for quadratic functions	40
1.5.3	Preconditioning the Conjugate Gradient method	42
1.6	Multivariate Quadratic optimization: a Direct Method	43
1.6.1	Backsolve	44
1.6.2	Cholesky factorization	44
1.6.3	Using the Cholesky factorization	46
1.6.4	Efficiency of the Cholesky factorization	47
1.7	Ex-post motivation: Polynomial Interpolation	48
1.8	Wrap up	50
1.9	Solutions	51
II	Unconstrained Optimization	55
2	Univariate Optimization	57
2.1	General Univariate Optimization Problems	57
2.2	Lipschitz (Global) Optimization	58
2.3	Local optimization	59
2.4	First local optimization algorithms	60
2.5	Towards faster local optimization algorithms	64

2.6	Dichotomic Search	68
2.6.1	Improving the dichotomic search by interpolation	69
2.7	Newton's method	72
2.8	A Fleeting Glimpse to Global Optimization	74
2.8.1	A very quick glimpse to convexity	75
2.8.2	Exploiting convexity	75
2.9	Wrap up	78
2.10	Solutions	79
3	Unconstrained Multivariate Optimality and Convexity	81
3.1	Unconstrained Multivariate Optimization	81
3.1.1	Multivariate (box-constrained) global optimization	81
3.1.2	Multivariate (unconstrained) local optimization	82
3.2	Gradients, Jacobians, and Hessians	82
3.2.1	Directional/partial derivatives, gradient	83
3.2.2	Differentiability in $\mathbb{R}^n$	84
3.2.3	Geometric interpretation of the gradient	85
3.2.4	Derivatives of vector-valued functions	86
3.2.5	Second order (partial) derivatives and the Hessian	87
3.3	Optimality conditions	88
3.4	A Quick Look to Convex Functions	90
3.5	Ex-post Motivation: (Artificial, Deep) Neural Networks	93
3.6	Wrap up	97
3.7	Solutions	97
4	Smooth Unconstrained Optimization	101
4.1	Gradient method for general functions	101
4.1.1	Gradient method basics	101
4.1.2	Inexact Line Search: Armijo-Wolfe	104
4.1.3	Gradient method with fixed stepsize	107
4.2	Newton-type methods	110
4.2.1	"Twisting" the gradient	111
4.2.2	Newton's method basics	111
4.2.3	Newton's method: the nonconvex case	113
4.2.4	Quasi-Newton methods	115
4.2.5	Limited-memory Quasi-Newton methods	118
4.3	Deflected gradient methods	118
4.3.1	Nonlinear Conjugate gradient methods	119
4.3.2	Heavy Ball gradient methods	120
4.3.3	Accelerated Gradient	123
4.4	Wrap up	125
4.5	Solutions	125
5	Nonsmooth Convex Unconstrained Multivariate Optimization	129
5.1	Motivations	129
5.1.1	Incremental (a.k.a. Stochastic) Gradient in ML	129
5.1.2	Nondifferentiable Regularisation	131
5.1.3	Support Vector Machine / Regression	132
5.2	Convex Nondifferentiable functions	134
5.3	Nondifferentiable optimization is hard(er)	134
5.4	Subgradient methods	134
5.5	Smoothed gradient methods	134
5.6	Bundle methods	134
5.7	Wrap up	134
5.8	Solutions	134
III	Constrained Optimization	139
6	Constrained Optimality and Duality	141

<i>CONTENTS</i>	5
7 Constrained Optimization	143
IV Combinatorial Optimization	145
8 A Fleeting Glimpse to Combinatorial Optimization	147
V Supplementary Material	149
References	151
A Miscellaneous Mathematical Background	155
A.1 Infima, suprema and $\mathbb{R}$	155
A.2 Vector space, scalar product	156
A.3 Matrices, transpose, symmetry, products	157
A.4 Eigenvalues and the determinant, in practice	158
A.5 Limits and optimization	159
A.6 Continuity	161
A.7 (Univariate) Derivatives	162
A.8 Topology and limit in $\mathbb{R}^n$	163
A.9 Eigenvalues & matrix norms	164
A.10 Topology and feasibility	164
A.11 Stronger forms of regularity	166
A.12 A super-brief recall on integrals	167

Part I

A Gentle Introduction

Chapter 1

Simple Optimization Problems

1.1 Optimization Problems

1.1.1 First definitions

An *optimization problem* corresponds, roughly speaking, to the following question: “among all my choices, which one is the best?” In order to poise the question in mathematical terms, there are two things that need be defined mathematically: what are the choices, and how to define “best”.

For the first part we start assuming that we are given a *set* X that contains all the choices. We don’t formally discuss the intuitive notion of set [13, §1.3], and we will postpone to much later the discussion of how the set is actually represented/implemented. Anyway, most sets considered in these lecture notes are “easy”. The set of choices is called *feasible region*, and any element $x \in X$ is called *feasible solution* (the adjective “feasible” would seem to be redundant, but we’ll see later on that the concept of unfeasible solution also makes sense in some contexts). The set may be finite (but, often, “very large”) or infinite, provided that it can be appropriately handled algorithmically.

Choosing among the elements in X requires a way to define “best”; this is done via a *function*, i.e., a mapping $f : X \rightarrow Y$ between the x *input space* X and the *output space* Y . Again we don’t insist on mathematical details of the concept (cf. e.g. [12, §2.2.1]); anyway all our functions will be (relatively) “easy”, i.e., either algebraic or with a “reasonably cheap” algorithm available to evaluate them pointwise (that is, produce $y = f(x)$ given x). While we will be quite free (within limits) with our input space X , using some f as the *objective* (function) in an optimization problem severely limits the choices for Y . In fact, the objective is there to answer the following question: “given two different feasible solutions $x' \in X$ and $x'' \in X$, which one is the best?” The idea is that f maps this concept into

$$x' \text{ is better than } x'' \iff f(x') = y' < y'' = f(x'') .$$

Intuitively, $y = f(x)$ is the “cost” of choosing x , and we want to select the least costly choice. However, for this to make sense the notion “ $y' < y''$ ” must be well defined; mathematically, Y must be a (total) *order* [12, Def. 2.2.8], [13, §1.1]. For our purpose this will mean that $Y = \mathbb{R}$, i.e., our objective is a *real* function (produces real numbers). This is, in some sense, a strong simplifying assumption: there are many situations in life where decisions need to be taken considering multiple aspects of the choices. Indeed, *multi-objective optimization* exists; yet, many concepts then become much less convenient. Furthermore, as briefly discussed in §1.1.7, there are a few “dirty tricks” that are regularly used to turn a multi-objective optimization problem into an ordinary (single-objective) one. These are more than enough (and, in fact, frequently used) in the applications of interest here, and therefore in these notes we will only work with real functions as objectives.

This gives us the basic form of *optimization* (minimization) *problem*

$$(P) \quad f_* = \min\{f(x) : x \in X\} . \tag{1.1}$$

Problems are often given “names”, in this case “ P ”, to be able to speak about them (especially if one is speaking about multiple ones at the same time, as it often happens). A distinction need be made between “optimization problem” and “instance of an optimization problem”, though. The latter is one very specific question (= instance) regarding a completely defined f and X , and therefore (possibly, as discussed below) has a very specific answer. The former is rather a class, or (often, infinite) set, of questions, each one characterised by the values of some *parameters* that can be set in (typically, infinitely) different ways. This is relevant because *algorithms are devised for optimization problems*: one does not want to write an algorithm just to solve one specific questions, but (infinitely) many of them. The following example illustrates the concept.

Example 1.1. The equipartition problem

The equipartition problem corresponds to the following question: given a set of n natural numbers, $N = \{a_1, a_2, \dots, a_n\}$, what is the subset S of N such that the difference in magnitude between the sum of the numbers in S and that of the numbers in $N \setminus S$ is the smallest possible? A mathematical formulation of the problem is

$$(EQ) \quad \min \{ f(x) = \left| \sum_{a \in x} a - \sum_{a \in N \setminus x} a \right| : S \subseteq N \} . \quad (1.2)$$

In this case, X is the set of all subsets of N ; in fact, the only condition that an answer must satisfy is to be a subset of the given n numbers. For this problem, the *parameters* are the number “ n ” and the n numbers “ $a_1, a_2, \dots, a_n$ ”; choosing for example $n = 4$ and $N = \{7, 3, 4, 6\}$ we obtain a particular instance of the problem, in which all the parameters are specified. The specific instance can be given a definite answer (in this case it is easy to see that $x = \{3, 7\}$ with $f(x) = 0$ is one); yet, when designing algorithms one would want them to work for any possible instance of the problem, i.e., irrespectively on how the parameters are fixed.

Because algorithms generally work on sets of problems, a fundamental issue that immediately rises is that of *efficiency*, i.e., how fast the solution time grows as the size of the problem does; this will be a central concern in these lecture notes. The problem in Example 1.1 is *combinatorial*: the number of possible different decisions to the problem, i.e., the size of X , grows as 2^n . In fact, despite its apparent simplicity the problem is “difficult” ($\mathcal{NP}$ -hard): there are no known algorithms capable of solving it in general in polynomial time in n , which can be thought at first to be due to the fact that “there are too many possible decisions to look at”. Most problems considered in these lectures are “even worse”, in that they are *continuous*: X is composed of (uncountably) *infinitely many* solutions. Finding the optimal choice in such a set may well be impossible, and relaxed notions of “solving the problem” will have to be devised, as we shall see. With appropriate assumptions there are, of course, continuous problems that admit “efficient” solution algorithms; in fact, these will be the main focus of these lecture notes.

The value f_* as defined in (1.1) is called the *optimal value of problem P* , and it is denoted as $\nu(P)$ when it is useful to make explicit its dependency on P . Since the minimum of a set of real numbers is unique (when it is well defined, as discussed below) one can then easily use it to define the *optimal solutions* of P , i.e., any $x^* \in X$ such that $f(x^*) = f_*$; assuming that any exists. In fact, what one is primarily interested in is x^* rather than f_* , hence the problem should actually be regarded as

$$(P) \quad x^* \in \operatorname{argmin} \{ f(x) : x \in X \} \quad (1.3)$$

with “argmin” denoting the set of $x \in X$ that attain the minimal value of f ; in other words, the set X^* of all optimal solutions of P . While f_* is unique (when it is well-defined), x^* is not, i.e., it often happens that X^* is not a singleton. Indeed, X^* may easily contain (uncountably) infinitely many solutions, which might make the task impossible if one were required to produce them *all*. However, as (1.3) underlines, the idea is that one is tasked with finding *any one* of the optimal solutions, regardless to which one. This makes sense, in that “in the eyes of the objective all optimal solutions are equal”, and “there are no other eyes except the objective’s”: there is no way to distinguish two different solutions $x' \in X^*$ and $x'' \in X^*$, and therefore to choose among them. If, from the viewpoint of the practical application driving the definition of (P) , there were any reason to actually distinguish x' from x'' , and favour one over the other, then f would not be a “complete” objective function, in that it would not properly represent all the preferences of the decision-maker on behalf of whom the problem is being solved. Although devising a “complete” objective may entail difficulties (cf. again §1.1.7), in these lecture notes we will always assume that this has in fact been done: as a consequence, solving an optimization problem requires finding just any one of its optimal solutions.

As it often happens, these intuitive notions require specification and clarification once they need to be expressed into exact mathematical notations. Indeed, when P is defined there are no less than *four* different situations that may arise:

1. The “normal” one in which $f_* \in \mathbb{R}$ is well-defined and at least one optimal solution x^* exists: this is what will happen in almost all the cases of interest in these lecture notes. As we will see, this by no means implies that finding f_* / x^* is efficiently doable, or in fact at all possible. One specific issue (in some sense, the crucial one) is whether or not, given some x , it is possible to efficiently detect whether or not $x \in X^*$. This might seem simple in the sense that computing $y = f(x)$ is efficiently doable (by our basic assumption on f), so it would seem that one is only required to check that $y = f_*$: this is true, except that f_* is *unknown* as well. Indeed, in some sense *finding f_** is “the really difficult part” of solving (P) , at least in a number of cases. We will return to this concept later on.
2. The case in which f_* is not a real value: the obvious example is the optimization problem

$$\min \{ x : x \in \mathbb{R} \} .$$

There clearly is no optimal solution: however fixed $M \in \mathbb{R}$ there exist some $x_M \in X$ (in our case, just $x = M$) such that $f(x_M) \leq M$. Formally speaking, the *image of X through f*

$$\text{im}_X(f) = \{y : \exists x \in X \text{ s.t. } y = f(x)\} \subset \mathbb{R}$$

is *unbounded below*: there is no finite value $\underline{f} \in \mathbb{R}$ such that $y \geq \underline{f}$ for all $y \in \text{im}_X(f)$. In plain words, one can obtain solutions that are “as good as one wants them”. In this case one customarily says that $f_* = -\infty$. That is, f_* is still well-defined provided one is happy to let the value live in the set of *extended reals* $\bar{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\}$ [13, p. 7]. The two special symbols “ $-\infty$ ” and “ $+\infty$ ” have their intuitive meaning and can, most of the times, be treated like ordinary real numbers with a bit of tweaking of the standard algebraic rules [13, p. 7]. In fact it will be sometimes expedient to allow f to be an *extended real* function, i.e., $f(x) = \pm\infty$ to happen for some x . Even if this is not the case, however, $f_* = -\infty$ (or $+\infty$ for maximization problems, as we will see) still “naturally happens”. This is not, in principle, an issue: in fact, in practice one would likely be very happy (were it not that such an occurrence almost invariably means that the model that one is solving does not correctly represent the reality). Yet, formally speaking this means that there cannot be any optimal solution: $X^* = \emptyset$. Also, *proving* that $f_* = -\infty$ is possibly even trickier than proving it has a finite value. Indeed, one should basically provide a(n easy to compute) function that, given any $M \in \mathbb{R}$, provides the $x_M \in X$ such that $f(x_M) \leq M$. This is indeed possible in a few cases, but not in many others. Fortunately, in the applications of interest here f is usually bounded below (on X) and the issue is hardly ever significant. Yet, in general this is a case that optimization algorithms have to properly take into account.

3. The case in which there cannot be an optimal solution because there is no solution at all: $X = \emptyset$. This may seem weird on a first look, but deciding about the emptiness of a set may not be trivial (in fact, it may be extremely difficult) depending on how the set is specified. This will not be an issue for most of the problems and algorithms studied in these notes because X will be “very simple” and often clearly nonempty; however, in Chapter 7 some algorithms will be discussed where emptiness of the feasible set is a possibility that have to be dealt with. Proving that a set is empty may be nontrivial, but once it is achieved it is a valid and complete answer to (P) : there are no solutions to choose from, and therefore no optimal solution. For good mathematical reasons, when $X = \emptyset$ it is customary to set $f_* = +\infty$ ($-\infty$ for a maximization problem), which confirms that f_* “naturally lives in $\bar{\mathbb{R}}$ ” even if f is real-valued.

The three cases above are succinctly described by the fact that $f_* \in \mathbb{R}$, $f_* = -\infty$ or $f_* = +\infty$. However, a fourth case exists (in theory) where f_* is ill-defined. This is simply the case when $\text{im}_X(f)$ is bounded below (therefore, f_* is not $-\infty$) but it does not have a minimum: the obvious example is the optimization problem

$$\min\{1/x : x \in \mathbb{R}_+\} \tag{1.4}$$

(with $\mathbb{R}_+ = \{x \in \mathbb{R} : x \geq 0\}$, and $\mathbb{R}_-$ defined accordingly). Clearly, $\text{im}_X(f)$ contains all strictly positive numbers, but not 0: that is, any value $v > 0$ cannot be f_* , but on the other hand there is no $x \in X$ such that $f(x) = 0$. This is because any bounded set of real numbers have an *infimum* (cf. §A.1, [13, Theorem 1.1.8]), but not necessarily a *minimum*: thus, f_* would be well-defined, and $= 0$ in the example, if one would write “inf” instead of “min” in the definition. Yet, the problem would not have any optimal solution.

There are reasonably simple (even if not always practical) conditions under which the fourth case can be shown not to happen (e.g., [12, Theorem 16.2.2]), but this is not really an issue in practice. The point is that finding the *exact* solution of an optimization problem is, in general, impossible. This will be argued in general terms in Chapter 2, but a case will be done immediately on the grounds that, in practice, writing “ $x \in \mathbb{R}$ ” is an abstraction that does not really represent what happens in actual solution methods on actual computers. There, “real numbers” are actually “fixed-size floating-point numbers”, typically 64-bits `double`. Although these numbers have a quite good accuracy, corresponding to about 16 digital digits, they do have a finite one: in other words, two “close enough” real numbers are indistinguishable once represented as `double`. In our example, anything that is closer to 0 than the machine precision (roughly 10^{-16} , or `1e-16` in the computer notation that we will favour) is basically indistinguishable from 0, and this even before considering the inevitable numerical errors that have typically been incurred while computing it and that further decrease the accuracy. While there are even more accurate floating-point numbers (e.g., 128-bit quadruple-precision floating-point), and even algebraic systems capable of doing computations in theoretically infinite precision, these are usually way too slow for the kind of applications of interest here. In fact, floating-point numbers with even *less* accuracy are common, such as 32-bit single-precision (`float`), 16-bit half-precision (`Float16`). Recently, the trend in Machine Learning applications has been towards *even less* accuracy, such as 8-bit or even (almost incredibly) 4-bit mini-floats [77], which have been found useful to reduce the tremendous memory requirements of very large models. Without going in any detail about the issue of the errors that this entails and their impact on the algorithms, we use this discussion just to motivate the fact that “solutions with an objective value close enough to f_* are good enough answers”.

1.1.2 Approximate optimality

The previous discussion leads to the following definition: the (absolute) *gap* of some $x \in X$, intended as a solution to (P) , is

$$A(x) = f(x) - f_* (\geq 0). \quad (1.5)$$

Any solution $x \in X$ with $A(x) \leq \varepsilon$ is an ε -*optimal solution*, and “solving (P) ” will in general always be intended as: “*given* ε , find a ε -optimal solution.” Of course ε should not be chosen too small, i.e., smaller than the machine precision, for the task to be at all possible. Even more importantly, *most often the solution cost of the algorithms will depend on ε* , and in particular grow (possibly rather fast) as ε is reduced. This is very intuitive: more accurate solutions are more costly to obtain than less accurate ones. In fact, in many cases choosing ε even remotely close to the machine precision will not be feasible because of the very high cost that this would have. This justifies the fact that we will not be too concerned about the impact of the accuracy of the floating-point numbers used to “approximate” $\mathbb{R}$: the accuracy attainable in practice by an optimization process is usually way lower (the error way larger) than the inherent accuracy of the floating point numbers because of the computational cost.

To make this discussion more practical we introduce a slightly different concept: the *relative gap*

$$R(x) = (f(x) - f_*) / |f_*| = A(x) / |f_*| (\geq 0) \quad (1.6)$$

of some solution $x \in X$. There are good reasons for wanting to define ε -*optimal solution in a relative sense* any $x \in X$ with $R(x) \leq \varepsilon$. Some will be discussed shortly, but one is already implicit in the working of floating-point numbers. As the name implies, the accuracy of these numbers is *relative to their size*. That is, saying that `double` have (roughly) `1e-16` accuracy means that they have (about) 16 significant digits in their decimal scientific representation; any digit (way) beyond the 16th is not really conveying useful information. This is illustrated by the following snippet of `Matlab` code:

```
>> fprintf( '%1.16e\n' , pi )
3.1415926535897931e+00
>> pi3 = pi^(pi*pi);
>> fprintf( '%1.16e\n' , pi3 )
1.3401641830063398e+18
>> fprintf( '%1.16e\n' , pi3 - ( pi3 + pi ) )
0.0000000000000000e+00
```

That is, the number π , which is order of `1e+0`, is represented with an error that is $\approx 1e-16$: all the figures that are printed are correct. Its thrice exponentiation is of the order of `1e+18`: it means that its digit corresponding to `1e+0` is not significant. In fact, adding π to it gives a number that is identical to the original one. That is, the inherent error in a floating-point number is related to its magnitude.

This implies that knowing that, say, $A(x) \approx 1e-3$ gives no meaningful information unless one also knows f_* : for $f_* \approx 1e+10$ this would mean that x is a very accurate solution, with its function value $f(x)$ agreeing with f_* at least on the first 13 decimal digits, while for $f_* \approx 1e+0$ it would rather be a rather poor one (only the first 3 decimal digits right). Relative errors immediately solve the issue: $R(x) \approx 1e-D$ means that $f(x)$ agrees with f_* at least on the first D decimal digits. Hence, $R(x) \approx 1e-16$ is arguably the best possible result while using `double`, and often too much to ask: $R(x) \approx 1e-12$ is generally considered a very good accuracy already. Any $R(x)$ between `1e-8` and `1e-6` is usually regarded as “decent but not exceptional”, `1e-3` to `1e-4` is “low accuracy but adequate in some cases”, and anything larger than that is usually regarded as “poor”. There can be exceptions, in that for very difficult problems even accuracies of the order of `1e-2` (a few percentage points) can be acceptable. However, these lecture notes are about “easy” problems (possibly “cheating about the difficulty”, as we shall see), and hence relative accuracies at least of the order of `1e-4`, or better, are usually expected.

In practice, the relative error is considered a much better measure of the accuracy of a solution. Besides for the reason previously outlined, this is useful to make the condition *scale invariant*, as discussed in §1.1.3. Yet, when analysing the behaviour of algorithms in theory, using the relative gap may complicate the arguments; among other things, for the reason outlined in Exercise 1.1 below. Hence, it is more common to run the analysis for the absolute error, knowing that in principle the two are only different by fixed factor $1/|f_*|$. In these notes “gap” and “ ε -optimal solution” will therefore have to be intended, unless otherwise specified, as referring to the absolute notion. However, in practical implementations the relative versions should be used.

Exercise 1.1. $R(x)$ ill-defined if $f_* = 0$: propose solutions and justify why your proposal makes sense. [solution]

A final important remark is the following: while the (absolute or relative) gap is a very natural measure of

(sub)optimality, *actually computing* (1.5) or (1.6) requires *knowing* f_* , which is typically *unknown*. In fact, one could argue that computing (an estimate of) f_* “is the issue” in optimization: in a number of cases, any efficient approach for doing this would translate in an efficient approach for solving the problem (or at least make existing approaches much more efficient). This is in fact more relevant for other classes of problems and algorithms than the ones that these lecture notes are concerned with, hence we will not focus too much on this particular aspect. It is however useful to point out immediately that knowledge of f_* , and therefore the ability to gauge how much any given x is a “good” solution, is absolutely *not* given. This is relevant in the current discussion because, while it is true that $A(x)$ and $R(x)$ only differ by the fixed factor $1/|f_*|$, it is also true that this factor is unknown during the actual execution of the algorithms, which is one of the reasons why the analysis of $A(x)$ is usually easier.

1.1.3 (Simple) reformulations

Exercise 1.1 has given a first hint of an important concept in optimization, that of *reformulation*. The idea is that of rewriting a given optimization problem in a different but equivalent form that is more convenient for some reason. This is a deep concept, and very sophisticated (and intricate) reformulation schemes have been devised that can have a substantial impact on the ability to efficiently solve some optimization problem. A much less complex, but still useful, use of the concept is that of establishing *normal forms* of optimization problems, so that one can always assume (P) to have a specific form while in fact allowing flexibility in actual implementations.

The first obvious example is that of *maximization* problems

$$(P_{\max}) \quad f_* = \max / x^* \in \operatorname{argmax} \{ f(x) : x \in X \}$$

where one is seeking for the choices with larger profit rather than the one with smaller cost. This clearly is not significantly different from a minimization problem, in the sense that

$$(P_{\min}^-) \quad \min \{ -f(x) : x \in X \}$$

has the property that $\nu(P_{\max}) = -\nu(P_{\min}^-)$: in plain words, maximizing f is equivalent to minimizing $-f$, and then changing the sign to the optimal value. It is immediate to see that every optimal solution to $(P_{\max})$ (if any) is an optimal solution to $(P_{\min}^-)$ and vice-versa, which is a very reasonable definition (although not the most general one) of “ $(P_{\max})$ and $(P_{\min}^-)$ are equivalent problems”; in fact, the equivalence immediately extends to the unbounded and unfeasible cases $(-\infty = +\infty)$. It is important to remark that maximizing and minimizing the *same* function f (without changing its sign) is instead *not* at all equivalent; in fact they can be “very different problems”, as we shall see. However, by just changing a sign of the computed value $y = f(x)$ (an operation that can only be trivial), one can assume without loss of generality that any optimization problem is a minimization one.

A similar property holds between (P) and

$$\begin{aligned} (P_c) \quad f_* &= \max / x^* \in \operatorname{argmax} \{ f(x) + c : x \in X \} \\ (P_\alpha) \quad f_* &= \max / x^* \in \operatorname{argmax} \{ \alpha f(x) : x \in X \} \end{aligned}$$

provided that $\alpha > 0$, i.e., optimal solutions are conserved while the optimal value is modified in the obvious way (namely, $\nu(P_c) = \nu(P) + c$ and $\nu(P_\alpha) = \alpha\nu(P)$). In plain words, optimization is not affected by any *constant* or *scale factor* applied to the objective. This will, for instance, allow to disregard constant terms in the functions that are analysed in §1.2 and §1.3. Note that this provides another rationale for the use of the relative gap, in that any solution x has the same value of $R(x)$ in (P) and in (P_α) , irrespectively to any scale factor α applied to the objective that would, instead, impact on $A(x)$.

1.1.4 Representing the feasible set

So far, membership in X has been implicitly considered “easy”, in the sense that checking whether “ $x \in X$ ” is true or not has never been discussed. In reality checking membership in a set can be hard and may have some of the same issues as checking optimality, starting with the accuracy ones. It all, of course, hinges on how the set X is specified. In our setting, this will always be done by specifying an “easy” *ground set* $F \supseteq X$, and then (possibly) some *constraints* that impose the conditions that the elements $x \in X$ must satisfy. This means that elements $x \in X$ (if any) are designated *feasible solutions* to (P) , while those $x \in F \setminus X$ (if any) are designated *unfeasible solutions*. In these lecture notes, the choice of the ground set will be fixed: F will always be the *finite-dimensional vector space* $\mathbb{R}^n$, typically for any value of n . That is, (feasible or unfeasible) solutions will be n -vectors of reals $x \in \mathbb{R}^n$, i.e., $x = [x_1, x_2, \dots, x_n] = [x_i]_{i=1}^n$; hence, the objective will be a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ of n real (in practice, **double** or **suchlike**) variables

$$f(x) = f(x_1, x_2, \dots, x_n).$$

We refer to §A.2 for the reminders about finite-dimensional vector spaces and their fundamental operations.

One standard way to represent a subset $X \subseteq \mathbb{R}^n$ is via (more than) one function. In particular, some *constraint function* $g: \mathbb{R}^n \rightarrow \mathbb{R}$ and some *right-hand side* $\delta \in \mathbb{R}$ can be used to define

- an *equality constraint* $g(x) = \delta$;
- an *inequality constraint* $g(x) \leq \delta$.

Note that this correspond to the definition of *level set* of the function g at value δ

$$L(f, v) = \{x \in \mathbb{R}^n : f(x) = v\} \subset \mathbb{R}^n$$

and *sublevel set* of the function g at value δ

$$S(f, v) = \{x \in \mathbb{R}^n : f(x) \leq v\} \subset \mathbb{R}^n$$

In these lecture notes, the feasible region X will always be specified by a *finite number* of equality and inequality constraints (semi-infinite optimization where the constraints are infinitely many is also possible under some conditions, but not explored here). Note that the assumption is always that constraints operate in logical conjunction: “ $g_1(x) \leq 0$, $g_2(x) \leq 0$ ” must always be taken as “the set of x that satisfy *both* the first condition *and* the second condition”. In other words, one is always looking at the *intersection* of (sub)level sets of a finite number of functions.

This potentially paves the way for many different forms of problems, but, taking a leaf from §1.1.3, we can consider simple *reformulation rules* that allow to define standard forms for optimization problems:

1. The right-hand side δ of a constraint can be “hidden in the constraint function g ”; that is, one can define $\bar{g}(x) = g(x) - \delta$, and the constraint as $\bar{g}(x) = 0$ (or ≤ 0). It is often expedient to do that, thus unless otherwise specified the right-hand side of the constraint(s) is always assumed to be 0.
2. It may be natural to rather define constraints via a $\geq$ relation. For instance, in (1.4) $X = \mathbb{R}_+$ is naturally defined as $X = \{x \in \mathbb{R} : x \geq 0\}$. Yet, it is always possible to assume all inequality constraints to have the $\leq$ form since $g(x) \geq 0 \implies -g(x) \leq 0$, i.e., again, by re-defining the constraint function $\bar{g}(x) = -g(x)$.
3. It would be possible to assume that there are *no equality constraint*, since $g(x) = 0$ is equivalent to

$$g(x) \leq 0 \quad , \quad g(x) \geq 0 \quad [\equiv -g(x) \leq 0] ;$$

in plain words, one equality constraint is equivalent to two inequality ones. This will sometimes be expedient, but in some sense (as we shall see) “equality constraints work very differently from inequality ones” and in a number of cases it will be appropriate to keep the distinction.

4. Somehow in the opposite direction, it would be possible to assume that *all constraints are equality ones save for very special inequalities, i.e., sign constraints*. This is due to the following equivalence

$$g(x) \leq 0 \quad \equiv \quad g(x) + s = 0 \quad , \quad s \geq 0$$

where s is a “new variable” (called *slack variable*) that “appears nowhere else in the optimization problem save in these two constraints”; in other words, none of the constraint functions (nor the objective) save these two must depend on s . This of course comes at the cost of introducing new variables (as the previous trick came at the cost of introducing new constraints), which may impact the efficiency of the solution algorithms; but this will be discussed in due time. On the other hand, sign constraints are “very easy” inequality constraints, that in a number of cases can be treated in special ways, making the solution approaches (and/or their theoretical analysis) simpler and/or more efficient.

5. Finally, in a problem with only inequality constraints (which can be assumed w.l.o.g., cf. 3.) one could as well assume that *there is only one inequality constraint*, owing to the fact that

$$g_i(x) \leq 0 \quad \forall i = 1, \dots, m \quad \equiv \quad \bar{g}(x) = \max\{g_i(x) : i = 1, \dots, m\} \leq 0.$$

While having only one constraint (as opposed to, say, one million) may appear convenient, in practice this last trick is seldom used because the max operator it involves is not “nice” (as we shall see, $\bar{g}$ is not differentiable even if all the g_i are, which is a significant downside). However, it is worth reporting it since it underlines the power of reformulation techniques to rewrite optimization problems in different forms that can be more or less well-suited for specific algorithmic approaches.

These lecture notes are mainly concerned with optimization problems where X is “simple”. Indeed, most of the work is on problems where $X = \mathbb{R}^n$ outright, i.e., there are no constraints at all: not surprisingly these are

called *unconstrained optimization problems*. In a number of other cases constraints will be present, but they will always be “easy”, more or less in the same vein as the sign constraints. In particular, common and useful classes of (inequality) constraints are *upper / lower bound constraints*, i.e., $x \geq \underline{x}$ / $x \leq \bar{x}$, and their intersection $\underline{x} \leq x \leq \bar{x}$ known as *box constraints*. In the above formulae, $\underline{x}$ and $\bar{x}$ have to be intended as constant vectors. However, it is customary to allow that

$$\underline{x} \in [\mathbb{R} \cup \{-\infty\}]^n \quad \text{and} \quad \bar{x} \in [\mathbb{R} \cup \{+\infty\}]^n .$$

That is, each variable has a lower and/or upper bound, $x_i \geq \underline{x}_i$ and/or $x_i \leq \bar{x}_i$; however, it is allowed that $\underline{x}_i = -\infty$, i.e., that there actually is no lower bound, and/or $\bar{x}_i = +\infty$, i.e., that there actually is no upper bound. Note that obviously it has to be $\underline{x}_i < \bar{x}_i$ (which is why $\underline{x}_i = +\infty$ or $\bar{x}_i = -\infty$ cannot be allowed): in fact, $\underline{x}_i > \bar{x}_i$ immediately makes X empty (and it is very easy to detect), while $\underline{x}_i = \bar{x}_i$ means that the variable x_i is fixed to their common value and is basically eliminated from the problem (fixing a variable almost invariably makes the problem easier). Despite their simplicity box constraints somehow “capture a good part of the complexity of (inequality) constraints” (cf. point 4. above) and therefore are a good compromise between generality and simplicity, which will be repeatedly exploited.

1.1.5 Approximate feasibility

When X is specified by a finite number of constraints, checking membership becomes “easy”. This is of course true if pointwise evaluation of all constraint functions g_i is efficient and the number of constraints is “not too large”. Both conditions are of course satisfied in the case of box constraints, that are at most twice the number of variables (which can be “many”, but this means that the problem is inherently large, and not so because of the presence of constraints). However, this is still subject to the same accuracy issues as the computation of the objective (cf. §1.1.2). That is, once the constraint function $y = g(x)$ has been computed, one is still left with the issue of deciding whether or not $y = 0$, $y > 0$, or $y < 0$ (as the case may be). Numerically speaking this still requires finding, say, an appropriate threshold $\varepsilon > 0$ and implementing the checks as $|y| \leq \varepsilon$, $y \geq \varepsilon$, and $y \leq -\varepsilon$, respectively. ε should clearly not be smaller than the precision of the floating numbers, ideally considering the magnitude of all intermediate results that concurred in the computation of y . This underlies the fact that even membership in a “simple” X characterised by a “small” number of “easy” functions is in principle not necessarily trivial.

On the other hand, it also reveals that some situations that would be issues in a mathematical sense are not such in practice. This is related to the fact that (inequality) constraints have been defined by means of the $\leq$ relationship rather than the $<$ one. Mathematically speaking this is necessary, in that an optimization problem with strict inequality constraints may have analogous undesirable behaviour as (1.4), as illustrated, e.g., by

$$\min\{x : x > 0\} . \tag{1.7}$$

Such a problem has ill-defined optimal value (save accepting the inf version) and no optimal solution. However, it does have ε -optimal solutions for any $\varepsilon > 0$. Numerically speaking, there is something like the smaller ε that is distinguishable from 0 (considering the magnitude of numbers ...), and “ $x > 0$ ” means “ $x \geq \varepsilon$ ”.

Things are of course potentially easier for “simple” forms of the constraints like the box ones. In principle, membership there could just be checked by the natural form $\underline{x} \leq x \leq \bar{x}$, assuming that any numerical error made in translating the “true” value of $\underline{x}$ and $\bar{x}$ into their floating-point form is acceptable (and it’s better be, since it’s unavoidable). However, usually the preferred implementation is rather akin to one among

$$\begin{aligned} \underline{x}(1 - \text{sign}(\underline{x})\varepsilon) &\leq x \leq \bar{x}(1 + \text{sign}(\bar{x})\varepsilon) \\ \underline{x}(1 + \text{sign}(\underline{x})\varepsilon) &\leq x \leq \bar{x}(1 - \text{sign}(\bar{x})\varepsilon) \end{aligned} ,$$

with $\text{sign}(x) = -1$ if $x < 0$, $\text{sign}(0) = 0$, $\text{sign}(x) = 1$ if $x > 0$, and ε “close to machine precision but not too much”, say `1e-12` for `double`. The first form slightly enlarges the interval ensuring that x can get the value of the extremes, the second form slightly restricts it ensuring that x is not beyond the value of the extremes. This does not really work in case of $\underline{x}$ or $\bar{x}$ being 0 (which is not uncommon, these being then sign constraints), but this can be managed the same way as in Exercise 1.1. This becomes more important for general (possibly, two-sided) constraints

$$\underline{\delta} \leq g(x) \leq \bar{\delta}$$

where the error in computing the constraint function may be significant.

These aspects are important for the practical implementation of the algorithms, especially if one aims at developing production-quality code that can be used in demanding environments where correctness of the produced solution may have significant (possibly, even life-and-death) consequences. It is less so in prototypical codes like the ones that students may be expected to develop in a course using these lecture notes, especially because most of the effort will be directed to problems that are either unconstrained or have very simple

constraints like box ones. Furthermore, the mechanisms for dealing with these issues are not terribly complex and quite well-established, although they would significantly complicate the notation if one would insist in detailing them all the time. For this reason in the rest of these lecture notes these issues will be purposely ignored and the “natural” form of the constraints (if any) will always be used.

1.1.6 Domains and extended-valued functions

A relevant detail has been (purposely) overlooked in all the previous discussion: a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ may *not* be well-defined for *all* possible input values $x \in \mathbb{R}^n$. This is well-known already for univariate functions, such as:

- $f(x) = 1/x$ is not well-defined for $x = 0$;
- $f(x) = \log(x)$ is not well-defined for $x \leq 0$;
- $f(x) = \sqrt{x}$ is not well-defined for $x < 0$;

and many others. Since multivariate functions are often obtained by the algebraic combination of univariate ones, they may not be well-defined in a “complicated” subset of $\mathbb{R}^n$.

Indeed, for a proper mathematical treatment one should identify the *domain* $D \subseteq \mathbb{R}^n$ of f in which $f(x)$ is well-defined, and always take care that mathematical statements about $f(x)$ are only made for $x \in D$. When multiple functions (e.g., constraint functions) are involved, the *intersection* of their domains should be used to ensure that all function values are available. This usually makes the notation, and the arguments, significantly more complex without, most of the time, adding really significant information for the practical applications of interest in these lecture notes. Here we will therefore take the stance that, unless explicitly stated, all involved functions are well-defined over all $\mathbb{R}^n$: every domain (and hence their intersection) is $D = \mathbb{R}^n$. This is a potentially strong assumption, but can be justified in two (somewhat opposite) ways.

The first is to assume that the functions can be (typically, one-sided) *extended-valued* ones, i.e., $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$. If f is the objective function of a minimisation problem, $f(x) = +\infty$ clearly means “for no reason whatsoever x can be considered as a possible optimal solution”, and therefore is in fact equivalent to “ $x \notin X$ ”. For a constraint function $g(x) \leq 0$, $g(x) = +\infty$ clearly implies that x is unfeasible. Of course $+\infty$ should be exchanged with $-\infty$ for a maximisation problem or $\geq$ constraint, but §1.1.3 and §1.1.4 have already illustrated how to deal with this. In fact, by this token one could even *completely disregard constraints and assume that an optimization problem is only characterised by its objective*. That is, given some $X \subset \mathbb{R}^n$ one can define its *characteristic function*

$$1_X(x) = \begin{cases} 0 & \text{if } x \in X \\ +\infty & \text{if } x \notin X \end{cases}$$

and define the *essential objective* of (P) as $f_X(x) = f(x) + 1_X(x)$. Then, clearly (P) is mathematically equivalent to the unconstrained problem of minimising f_X over all $\mathbb{R}^n$. Note that if $X = \emptyset$ then $f_X(x) = +\infty$ for all $x \in \mathbb{R}^n$ and “the optimal value is $+\infty$ ”, as expected. Such an approach is not, in practice, as outlandish as it may seem since floating-point numbers like `double` do have an explicit representation for infinite values [64] that obey the rules of extended-value real algebra [13, p. 8] and that are readily accessible by programming languages (e.g., `Inf` in `Matlab` and `std::numeric_limits<double>::infinity()` in `C++`). One could then envision actual implementation of the *oracle* that computes the function values $f_X(x)$ producing $\pm\infty$ values, and this can even work well in some cases.

However, this is not a convenient approach in general. The main reasons, that will be discussed in details later on, are:

1. the function value $f(x)$ is *not* the only information one needs to be computed by the oracle when devising algorithms for solving optimisation problems: other information (derivatives) is as important (or even more), and it would be “seriously messed up” in functions like 1_X (that are “ferociously nondifferentiable”);
2. efficiently solving optimization problems requires to exploit as much as possible their *structure*, i.e., all available knowledge upon their objective and constraints: “hiding the latter into the former” would conceal a great deal of the structure, possibly destroying many opportunities for building more efficient algorithmic approaches.

For these reasons, in these lecture notes we will keep pretending that all functions have the whole $\mathbb{R}^n$ as domain with the following implicit justification: if f is not defined on some x , then some constraint has been imposed (in the problem-building phase, which is not discussed) so that $f(x)$ will never be evaluated. This is in some sense the opposite approach, in that one is possibly concealing some information about the objective in some

properly constructed constraints. Such an approach is only partly satisfactory, as not all algorithmic approaches guarantee that $f(x)$ (and even less the constraint functions) will only be evaluated at feasible points. However, in all the applications of interest here, either all functions are actually well-defined in all $\mathbb{R}^n$, or there are effective ways to “keep out of the danger zone” where they are not. Thus, even if not completely satisfactory mathematically speaking, the simplifying assumption that all functions are defined everywhere is appropriate for these lecture notes, and it will make for a leaner and cleaner treatment. Anyway, this choice is far from being uncommon: many books/articles about optimization problems/algorithms have at the very beginning the apparently inconspicuous statement that “ $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ”, which in fact means “ f is not an extended-value function, and therefore defined everywhere”, i.e., “we do not want to bother with domains”. These lecture notes just follow this well-established tradition.

1.1.7 An aside: (why not) multivariate optimization

Before moving on with the solution of actual optimization problem, it is appropriate to briefly comment—once and for all—on the underlying assumption that the objective is a real-valued function. In practice, this means that all the “value” of any choice $x \in X$ can be expressed by using a single (real) number. However, it is common experience that, when evaluating a set of choices (e.g., buying a new car/house ...) there are *multiple criteria* that must be used to evaluate different aspects of the decision. In real life some of these criteria are even hard to be given numerical values; but assuming this is done somehow, the result is that to each $x \in X$ one has naturally associated a *vector* of (real) values $y = [y_1, y_2, \dots, y_k] \in \mathbb{R}^k$ that represent the evaluation of the k different criteria on x . Thus, $f : \mathbb{R}^n \rightarrow \mathbb{R}^k$ is a *vector-valued function*, which is basically nothing else than “a vector of real-valued functions”:

$$f(x) = [f_1(x), f_2(x), \dots, f_k(x)].$$

One is therefore faced with the *multicriteria*, or *vector-valued*, optimization problem

$$(M) \quad \text{“min”} \{ [f_1(x), f_2(x), \dots, f_k(x)] : x \in X \},$$

with the scare quotes underlying the fact that the meaning of the minimisation is now not obvious. To simplify the discussion we will assume $k = 2$ (bi-criteria optimization problem) since all arguments readily generalise to any $k > 2$.

The issue with vector-valued optimization is clearly that $\mathbb{R}^k$ is not totally ordered: given $y' \neq y''$, it is not always the case that either $y' < y''$ or $y' > y''$ must occur. Two vectors can be *incomparable*, leading to the situation where the objective is not capable to decide between two choices; and if the objective cannot, who can? We use the following textbook example to illustrate the issue.

Example 1.2. The portfolio selection problem

In the financial sector, portfolio managers are given some amount of money and have to decide how to invest it, over some investment period (anything between a lifetime and a few milliseconds), choosing among a large array of financial instruments: government bonds (from different states/regions, with different maturities, ratings, ...), corporate bonds (from many different emitters in different geographical regions and different industrial sectors, ...), equities (in many different publicly traded companies in different markets and different industrial sectors, ...), commodities, ETFs, derivatives, cryptocurrencies, and many others. It is therefore possible to construct (infinitely) many different *portfolios* in which different fractions of the total wealth are invested in different instruments. Let X be the set of all these portfolios, and let us assume for illustration purposes that it is finite. Each $x \in X$ must be evaluated according to *two* different criteria:

- $f_1(x)$ is the (*expected*) *return* of the portfolio, i.e., how much the instruments will (presumably) be worth at the end of the investment period;
- $f_2(x)$ is the (*estimated*) *risk* of the portfolio, i.e., a measure of how likely it is that the return will not be the expected one.

Crucially, the two criteria may have different (and incomparable) units of measure: while the first is “money” (say, €) the second may be a probability. This depends upon which of the *risk measures* (variance, VaR, CVaR, Sharpe ratio, ...) are used, an issue that is clearly way beyond our interest here. As an added twist, f_1 should be maximised while f_2 should be minimised, so one change in sign is also in order. A possible example is depicted in Figure 1.1(left), where each $\star$ marks the position $[f_1(x), f_2(x)]$ corresponding to one $x \in X$ in the return / risk plane.

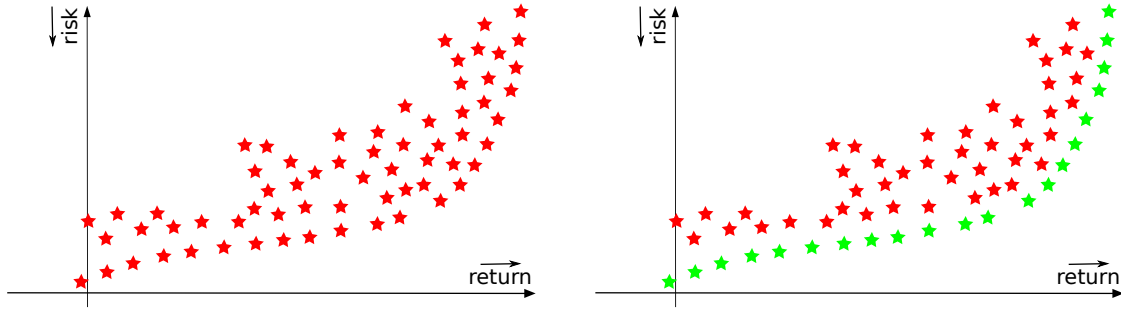


Figure 1.1: A portfolio example (left) and its Pareto frontier (right)

As it can be expected, there is no single portfolio that is better than all others on both criteria: typically, as the return of a portfolio increases (good), so also does its risk (bad). Such *contrasting* objectives—when one improves, the other deteriorates—are very common. What can be identified are portfolios x' that are *dominated* by others, i.e., for which there exist some x'' such that $f_1(x') > f_1(x'')$ and $f_2(x') < f_2(x'')$ (it has better return and lower risk). These can reasonably be excluded from the choices one should make. However, some portfolios x' are *non dominated*: if there exists some x'' such that $f_1(x') > f_1(x'')$ (it has better return), then $f_2(x') > f_2(x'')$ (it has worse risk). These are represented as the green $\star$ in Figure 1.1(right). Basically, these are the portfolios that offer the minimum risk for their given level of return, or, conversely, the maximum level of return for their given level of risk. Without entering in the mathematical details (as usual, the devil can be found in there), these identify the *Pareto-optimal frontier* of the problem. Each non-dominated solution (portfolio) is a rational choice, but which one should be chosen among these depends on the *risk propensity* of the investor: how much (s)he is willing to tolerate risk in view of possible higher returns. This is *not* encoded in any way in the problem, and therefore there is no way in which an algorithm can select one specific solution and declare it “optimal”. The idea in this case is that algorithms will produce *all* the solutions in the Pareto-optimal frontier (which can be infinitely many), and somehow present them to another decision-making level (typically, a human being) where the final choice will be made according to some criteria that is unknown, and possibly difficult to formalise.

While the above “interactive” approach can be sensible in some settings, it is typically not in the ones these lecture notes are focused on. Thus, multicriteria (a.k.a. *multiobjective*) optimization proper has to, and will, be avoided. Yet, the contrasting needs of, on the one hand, considering multiple criteria, and on the other hand automatically producing one final answer, can be satisfied by means of some “dirty tricks”. The two classical forms are *scalarization* and *budgeting*

$$\begin{aligned} (M_\alpha) \quad & \min \{ f_1(x) + \alpha f_2(x) : x \in X \} \\ (M_\beta) \quad & \min \{ f_1(x) : f_2(x) \leq \beta, x \in X \} \end{aligned} \quad .$$

In plain words, in the first one concocts a “magic” constant α that “translates the units of measure of f_1 into those of f_2 ”, allowing to sum them and treat them as a single objective. In portfolio parlance this may be termed a *risk-adjusted return*, i.e., providing a monetary value to the incurred level of risk, as depicted in Figure 1.2(left). In the second approach one is rather setting a bound on the highest possible value of one criteria that can be tolerated (e.g., the maximum risk) and is optimising on the other criteria subject to that constraint (e.g., find the portfolio with maximum return among those with acceptable risk), as depicted in Figure 1.2(right). Note that in the budgeting approach one can freely choose which one of the criteria has to be turned into constraint and which one will be left as the objective; that is, there would as well be the choice of setting a minimum level of return and seeking the least-risk portfolio attaining it.

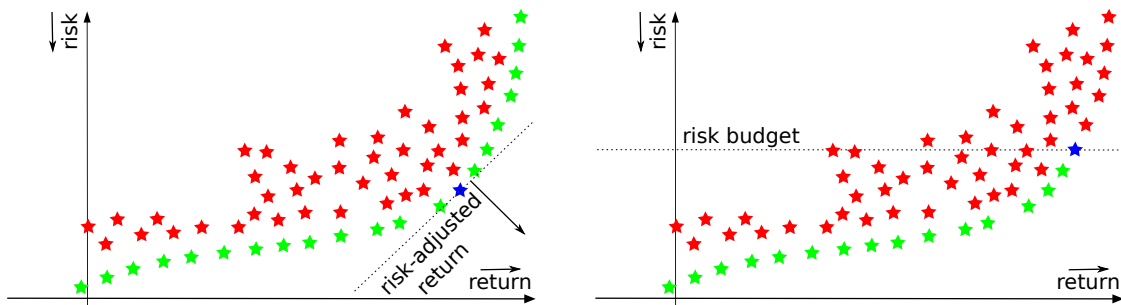


Figure 1.2: Scalarization (left) and budgeting (right)

Of course, the issue in the two approaches is to identify the constants α or β ; this typically requires knowledge of the actual decision process, which in the financial sector in particular is heavily regulated (you should not sell high-risk derivatives to an unknowledgeable pensioner ...). Whenever this is possible, multicriteria optimization problems can be recast as ordinary (single-criteria) ones and solved with the tools discussed in these lecture notes.

The two approaches outlined in the Example, and in particular scalarization, are actually used in the applications of interest for these lecture notes. In particular, in (supervised) Machine Learning applications *models* are constructed that should reconstruct an unknown function $h : I \rightarrow O$ between some input and some output space (the latter not restricted to be $\mathbb{R}$) out of a set of examples. The chosen models are parametrised over a (large) set of variables (weights) w , and an optimization problem is constructed and solved to find the “best” weights. For this, a *loss function* $\mathcal{L}(w)$ is devised that, roughly speaking, measures how well the model, with

the given weights w , reconstructs the function h on the known examples. However, doing so “too well” (that is, optimising $\mathcal{L}(w)$ alone) is usually detrimental to the learning outcomes; cf. §5.1.1 for (a bit) more details. The issue, very broadly speaking, is that a “complex” model is needed to fully reconstruct h on the given input. However, such a model may “overfit”: in striving to perfectly representing h on the known examples it does poorly in its real task, which is to predict it for unknown data. This is called the *bias / variance dilemma*. It turns out that “simpler” models predicting a little (but not too much) less accurately h on the known examples may predict it much more accurately the unknown ones. For this reason a *regularisation term* $\Omega(w)$ is devised that “measures the complexity of the model”, and one is in principle confronted with a bi-criteria problem of optimising $[\mathcal{L}(w), \Omega(w)]$; cf. §5.1.2. The standard way in which this is approached is precisely scalarization: an arbitrary real (hyper)parameter λ is chosen, and $\mathcal{L}(w) + \lambda\Omega(w)$ is optimised. The “right” value of λ is typically unknown, but standard approaches (k -fold cross-validation) are available to “measure how good a given value of λ is” for the “true objective” of having weights w that provide a good approximation of h on all (known and unknown) data. Basically, this is repeatedly done with λ chosen in a finite (small-ish) set of values (grid search), and the best value is retained as the final choice. None of the crucial details of this process are of interest for these lecture notes, since they will be dealt with in Machine Learning courses.

What is relevant here is that the objective function $f(x)$ that is optimised, or some of the constraints, may well be the result of a scalarization or budgeting approach applied to a multi-criteria problem. Whether or not this is the case, and how the crucial (hyper)parameters α and β have been chosen, is completely outside the scope of these lecture notes and supposedly decided during the *modelling phase* in which the optimization problem has been constructed. While the proper construction of (optimization) models is an interesting subject in itself, it will not be pursued in details here. The rationale is that for the applications of interest here a relatively small set of models have been identified over the years that “work well”, and the issue is rather how to solve them efficiently. This is mostly what these lecture notes are about.

1.2 (Outrageously) Simple (Univariate) Optimization

We will now proceed with the exercise of picking up two extremely simple classes of *univariate* functions (i.e., x is a single real number: $n = 1$), linear and quadratic ones, and discuss solving optimization problems with them. Although the arguments here are so simple as to be almost offensive, the ability of solving “simple” problems is typically crucial when solving “complex” ones. In particular, we will see that solving problems with general functions typically entails the use of *models*, i.e., simpler functions that approximate the given one close to some given points. The useful models are linear and quadratic functions, so it makes sense to just quickly review the properties of these, if only to set the nomenclature right, and be done with it. Skipping this section on first reading, and coming back to it only if needed, is a sensible option. However, when dealing with quadratic function the section introduces the first example of a simple but interesting mechanism that will be repeatedly useful in the following.

1.2.1 Linear univariate functions

Linear univariate functions are perhaps the simplest possible imaginable ones: they have the form $f(x) = bx$ for some *fixed* value $b \in \mathbb{R}$. There are, therefore, as many different linear functions as real numbers. These are in fact *homogeneous* linear functions; a different definition that is often also used is that of $f(x) = bx + c$, i.e., allowing some constant term. These are called *affine* functions. Due to the discussion in §1.1.3, constant terms are irrelevant when f is the objective of an optimization problem and we therefore dispense with c . b is called the *linear coefficient* or *slope* of the function. Linear functions are very easy to visualise by plotting their *graph*, i.e., the (two-dimensional) curve

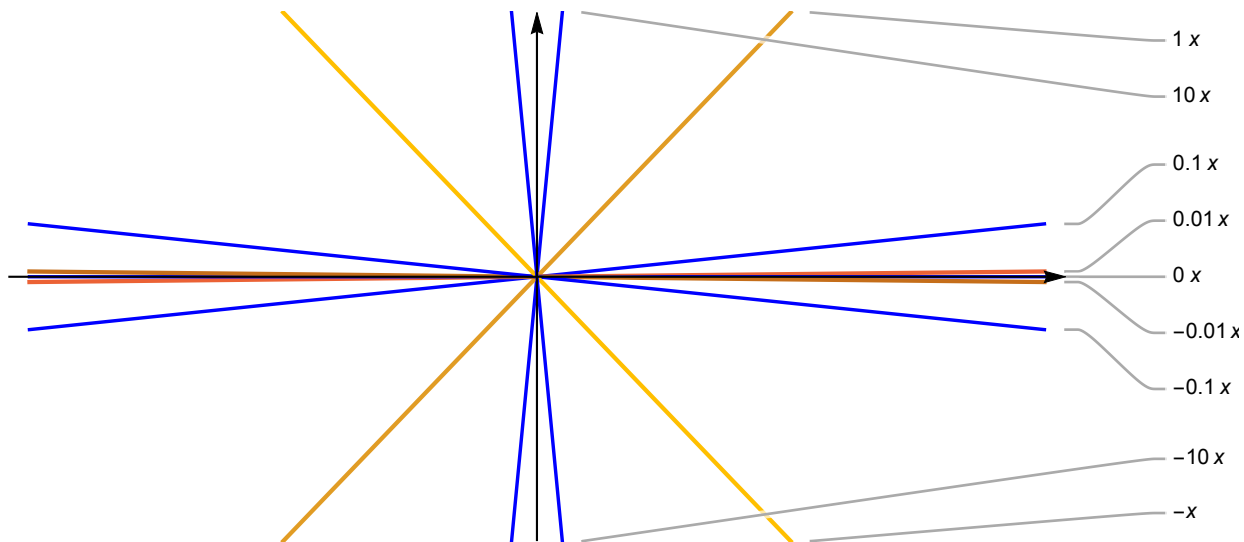
$$\text{gr}(f) = ((f(x), x) : x \in \mathbb{R}) \subset \mathbb{R}^{n+1}.$$

This is done in Figure 1.3 for some different values of b .

The picture makes it plain how the function behaves, starting of course from justifying its name. In terms of optimization, a fundamental property one is interested in is whether the function is (*strictly*) *increasing* / *non decreasing* / *constant* / *non increasing* / (*strictly*) *decreasing* on some (not necessarily bounded) interval $[\underline{x}, \bar{x}] \subseteq \mathbb{R}$, which trivially means that

$$f(x) > / \geq / = / \leq / < f(z) \quad \forall \underline{x} \leq x < z \leq \bar{x}.$$

This is obviously related with b : f is increasing / constant / decreasing if b is > 0 / $= 0$ / < 0 . Albeit trivial, one may want to step aside for a second to prove the fact, is only to start getting used to the fact that every mathematical statement made in these lecture notes can eventually be proven.

Figure 1.3: The graph of linear functions for some values of b

Exercise 1.2. Formally prove the stated properties. [solution]

Optimising a linear function without any constraint is obviously pointless: the minimization problem is unbounded below ($\nu(P) = -\infty$) and the maximization one unbounded above ($\nu(P) = +\infty$) unless of course $b = 0$, in which case $\nu(P) = 0$ and every $x \in \mathbb{R}$ is an optimal solution. This can be made ever so slightly more amusing by considering the box-constrained optimization

$$(L) \quad \min\{bx : \underline{x} \leq x \leq \bar{x}\}$$

on a not necessarily bounded interval, i.e., allowing $\underline{x} = -\infty$ and/or $\bar{x} = +\infty$ to happen. The result is fairly obvious: thanks to the rules of extended-real arithmetic, if $b > 0$ then $\nu(L) = f(\underline{x})$, if $b < 0$ then $\nu(L) = f(\bar{x})$, and if $b = 0$ then $\nu(L) = 0 = f(\underline{x}) = f(\bar{x})$. Intuitively, if the function is increasing the optimal solution is on the left endpoint of the interval (the function “grows when going from left to right”), if the function is decreasing the optimal solution is on the right endpoint of the interval (the function “decreases when going from left to right”). One could even say that $b > 0$ then $x^* = \underline{x}$, if $b < 0$ then $x^* = \bar{x}$, and if $b = 0$ then x^* is any point in the interval, accepting the slight abuse of notation that $\pm\infty$ is the optimal solution in case of an unbounded problem.

Exercise 1.3. Formally prove the above result. [solution]

Ultimately, optimizing a linear function over a possibly unbounded interval can only be defined as trivial: an $O(1)$ closed formula achieves it. We avoid commenting again on the issue that “ $b > 0$ ”, “ $b = 0$ ” and “ $b < 0$ ” need be treated with some care over floating-point numbers. Thus, as expected optimization of (exceedingly) simple functions is (exceedingly) easy. We will see this happening for a while, and breaking surprisingly soon.

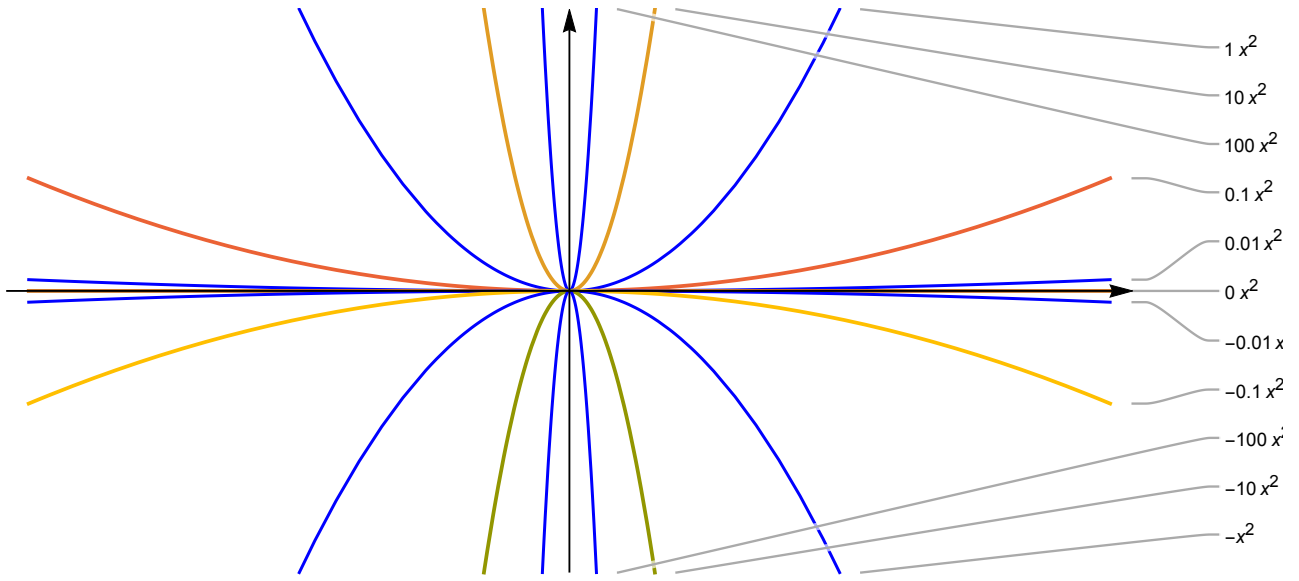
1.2.2 Homogeneous quadratic univariate functions

The next simplest functions besides linear ones are the *homogeneous quadratic* ones: $f(x) = ax^2$ for some *fixed* value $a \in \mathbb{R}$. Again, there are as many different homogeneous quadratic functions as real numbers, ignoring the possibility of adding constant terms. a is called the *quadratic coefficient* or *curvature* of the function. The graphs plotted in Figure 1.4 basically tell all the story.

There are two different cases, excluding the special one in which $a = 0$ and the function is constant (and therefore linear): $a > 0$, in which the graph is a U-shaped parabola, and $a < 0$, in which the graph is a $\cap$ -shaped parabola. The first is called *convex* and the other is called *concave*, as special cases of extremely important definitions with which it is useful to get familiar as early as possible. Similarly to the linear case, the *sign* of the curvature is crucial to determine the overall shape of the function (and, therefore, drive its minimization or maximization), where the *magnitude* influences “how quickly the function increases / decreases”, i.e., “how steep the parabola is”. The fundamental property for our purposes is: if $a > 0$ then f is decreasing for $x \leq 0$ and increasing for $x \geq 0$, while if $a < 0$ then (vice-versa) f is increasing for $x \leq 0$ and decreasing for $x \geq 0$.

Exercise 1.4. Formally prove the stated properties. [solution]

The optimization of a homogeneous quadratic function

Figure 1.4: The graph of homogeneous quadratic functions for some values of a

$$(Q) \quad \min\{ax^2 : \underline{x} \leq x \leq \bar{x}\}$$

is therefore only slightly less trivial than that of a linear one in the unconstrained case, i.e., where $\underline{x} = -\infty$ and $\bar{x} = +\infty$:

- if $a > 0$ then $\nu(Q) = 0$ and the unique optimal solution is $x^* = 0$;
- if $a < 0$ then $\nu(Q) = -\infty$.

Obviously, the rule reverses for maximization: $x^* = 0$ is the unique optimal solution if $a < 0$, and (Q) is unbounded above if $a > 0$. The “true” box-constrained case, where $\underline{x}$ and $\bar{x}$ can have finite values, is ever so slightly more nuanced. For minimization, and $a > 0$, there are already three different cases, depending on “where the extremes stand w.r.t. 0”:

- if $\bar{x} < 0$ ($\implies \underline{x} < 0$) then $\nu(Q) = f(\bar{x})$;
- if $\underline{x} > 0$ ($\implies \bar{x} > 0$) then $\nu(Q) = f(\underline{x})$;
- if $\underline{x} \leq 0 \leq \bar{x}$ then $\nu(Q) = 0$.

In plain words, the optimal solution is 0 if it is feasible, and otherwise the one of the two extremes that is closer to it. Not surprisingly, the situation is *not* entirely symmetric if (Q) is a maximization problem

- if $\bar{x} < 0$ ($\implies \underline{x} < 0$) then $\nu(Q) = f(\underline{x})$;
- if $\underline{x} > 0$ ($\implies \bar{x} > 0$) then $\nu(Q) = f(\bar{x})$;
- if $\underline{x} \leq 0 \leq \bar{x}$ then $\nu(Q) = \max\{f(\underline{x}), f(\bar{x})\}$.

In plain words, if 0 is not feasible then the optimal solution is the extreme that is *farthest* from 0, otherwise one have to look to the value of both extremes and pick the largest (this basically again boils down to find the one that is farthest). As for the linear case, the formulæ work even if any of the extremes is $\pm\infty$, and one can turn them in formulæ for x^* if accepting $\pm\infty$ answers in the unbounded cases.

Exercise 1.5. Formally prove the above result, state and prove the case $a < 0$ [solution]

Again, optimizing a homogeneous quadratic function over a possibly unbounded interval requires trivial $O(1)$ closed formulæ. They are, however, not symmetric for minimization and maximization, with the maximization of a concave function ever so slightly more complex than its minimisation. As we shall see, this is only the presage of much more significant differences.

1.2.3 Quadratic non-homogeneous univariate functions

The natural next step is the *non-homogeneous* quadratic function $f(x) = ax^2 + bx$. Needless to say, this generalise the previous cases where only one among the slope and the curvature were nonzero. Basically, a non-homogeneous quadratic function is just the sum of a homogeneous quadratic and a linear one: however, for the

avoidance of doubt let us state upfront that minimizing it is *not* equivalent to minimizing the two components separately and then “putting the results together” somehow. In general—and save a few relevant cases that we will discuss—an optimization problem is more complex than the sum of its individual parts.

Optimizing a non-homogeneous quadratic function (with both $a \neq 0$ and $b \neq 0$) cannot obviously be overly too complicated than an homogeneous one, but it is still somewhat different. While there are many ways to address this, we exploit the chance to introduce in a very simple setting a general concept that will repeatedly prove useful: *if $f(x)$ is “too complicated”, make it “simpler”*. That is, *reformulate* (cf. §1.1.3 and §1.1.4) the problem in a form that makes it simpler to solve. A powerful way of doing this, in general, is that of *changing the space of variables*. In this simple case the idea is just to change (translate) the input space so that f becomes homogeneous.

The idea is to pick some fixed $\bar{x}$ and to translate the variable x so that $\bar{x}$ is the origin. That is, we define a new variable z by $z = x - \bar{x}$, which just means $x = z + \bar{x}$. With simple algebra we can then rewrite f as a function of this new variable:

$$f(x) = a(z + \bar{x})^2 + b(z + \bar{x}) = az^2 + 2az\bar{x} + a\bar{x}^2 + bz + b\bar{x} = az^2 + (2a\bar{x} + b)z + f(\bar{x}) = g(z) .$$

It is then plain to see that by choosing $\bar{x} = -b/2a$ (which is possible since $a \neq 0$) one obtains an homogeneous quadratic function in the z -space (save the constant term $f(\bar{x})$ that we know we can ignore). This immediately provides the complete solution to the optimization problem by just applying the formulæ of §1.2.2 to g in the z -space and then translating back the solution in x -space. Graphically, a non-homogeneous quadratic function is still a parabola (be it convex or concave), except its *apex* is not in the origin, but rather in $\bar{x} = -b/2a$. The point $x = 0$ is still a *root* of f (i.e., such that $f(x) = 0$), but while in the homogeneous case the apex is the only root, in the non-homogeneous one there are two separate ones. The z -space is chosen so that its origin coincides with the apex, and the constant $f(\bar{x})$ ensures it is the only root. All in all, by a “clever trick” the optimization of a non-homogeneous quadratic function can still be performed in $O(1)$.

Exercise 1.6. Discuss the position of $\bar{x}$ and of the roots of f depending on a and b [solution]

1.3 (Not always) Simple Multivariate Optimization

We now move to “simple” multivariate optimization: $f : \mathbb{R}^n \rightarrow \mathbb{R}$ with f a “simple” (linear or quadratic) function. Even when the problem remains a “simple” one—and, as we shall see, this is often but not always the case—the complexity of solving it cannot be dismissed as irrelevant right away because it cannot but depend on n . While examples will be given in small (2, 3, 100) dimension, practical applications may require largish sizes (10^4), large sizes (10^6) or *heinously large* ones (10^9 , 10^{11}). Thus, by dint of their sheer size alone, multivariate optimization problems can be costly even for “simple” functions. In some cases, however, the cost grows linearly with n , which is the best one can hope for.

1.3.1 Linear multivariate functions

A linear function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is defined as $f(x) = \langle b, x \rangle = \sum_{i=1}^n b_i x_i$, i.e., as the scalar product between the vector of variables $x \in \mathbb{R}^n$ and a fixed vector $b \in \mathbb{R}^n$. In other words, $f(x)$ is the sum of n univariate linear functions, one in each of the variables. It is expedient to define the set of indices of variables $I = [1, 2, \dots, n]$. A linear function has the following properties:

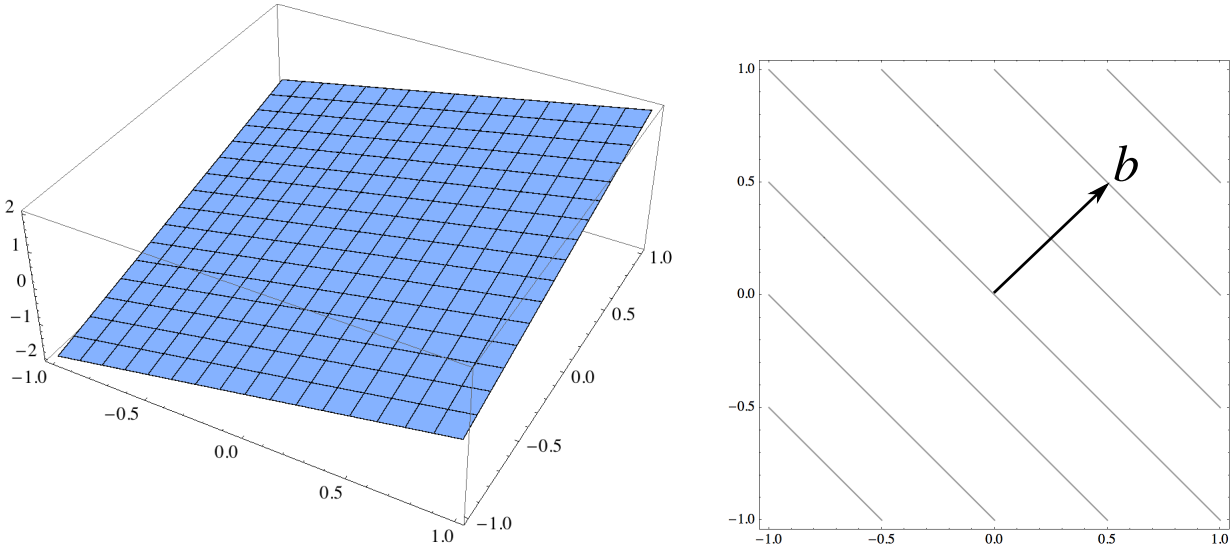
- i) $f(\gamma x) = \gamma f(x)$ however chosen $\gamma \in \mathbb{R}$ and $x \in \mathbb{R}^n$;
- ii) $f(x + z) = f(x) + f(z)$ however chosen $x \in \mathbb{R}^n$ and $z \in \mathbb{R}^n$.

This is easy to prove by the fundamental properties of the scalar product (cf. §A.2); in fact, the two properties are an alternative definition of linear function.

Exercise 1.7. Prove that if $f(x)$ has the properties i. and ii. then it necessarily has the form $f(x) = \langle b, x \rangle$ for some fixed $b \in \mathbb{R}^n$. Discuss if this extends to *affine* functions, i.e., of the form $f(x) = \langle b, x \rangle + c$. [solution]

Linear functions are “very simple” ones, as one can easily verify by picturing two-dimensional cases. The *graph* $\text{gr}(f)$ for a bi-variate linear function ($f : \mathbb{R}^2 \rightarrow \mathbb{R}$) is a *plane* in $\mathbb{R}^3$, as shown in Figure 1.5(left). It becomes an *hyperplane* in $\mathbb{R}^{n+1}$ in the general case $f : \mathbb{R}^n \rightarrow \mathbb{R}$. The *level sets* $L(f, v)$ in $\mathbb{R}^2$ are parallel lines orthogonal to the vector b defining f , as shown in Figure 1.5(right); these become parallel hyperplanes in $\mathbb{R}^n$ in the general case $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

Picturing functions with more than two variables is challenging; level sets of function of three variables can be represented in 3D pictures, but that’s about the maximum (moving pictures allow to reach four variables, but

Figure 1.5: Graph and level sets of the linear function $f(x) = x_1 + x_2$

they are not easy to manage). A simple concept that is useful to visualise—and, actually, optimise—functions with any number of variables is the *tomography* of the function. This is obtained by choosing some fixed $x \in \mathbb{R}^n$ (the origin) and $d \in \mathbb{R}^n$ (the direction), and building the univariate function

$$\varphi_{x,d}(\alpha) = f(x + \alpha d) : \mathbb{R} \rightarrow \mathbb{R} .$$

Here, the value α is the *step* that identifies a point obtained by starting from x and moving “ α units” along d . Often, but not always, the tomography is only considered “in the direction of d ”, i.e., for $\alpha \geq 0$. Indeed, clearly, d has to be interpreted as a direction, which means that $\|d\|$ does not really change it except for a “scale factor”: formally speaking, $\varphi_{x,\beta d}(\alpha) = \varphi_{x,d}(\beta\alpha)$. As a consequence, often (but not always) it is convenient to use a normalised direction ($\|d\| = 1$). It is also often useful to consider the (vertically) *translated* tomography defined by $\varphi_{x,d}(\alpha) = f(x + \alpha d) - f(x)$, so that $\varphi_{x,d}(0) = 0$; which one of the two definitions is used will be made clear from the context. Similarly, in many cases x and / or d will be clear from the context and simplified notations like $\varphi(\alpha)$ will be used. One advantage of the tomography is that $\text{gr}(\varphi_{x,d})$ can always be pictured, allowing to get some insight on the features of the function. Of course, the issue is that this depends on the choice of x and d . Often, however, there will be “natural” choices for the origin and the direction that will simplify the analysis.

For instance, in the case of linear functions the translated tomography is

$$\varphi_{x,d}(\alpha) = f(x + \alpha d) - f(x) = \langle b, x + \alpha d \rangle - \langle b, x \rangle = \alpha \langle b, d \rangle .$$

Hence, the choice of the centre x does not change the tomography, confirming that linear functions “look similar everywhere”. Then, the tomography of a multivariate linear function is a univariate linear function whose slope is $\langle b, d \rangle$. This immediately allows, for instance, to formally verify the property suggested by Figure 1.5(right) that the level sets of f are lines (in $\mathbb{R}^2$, planes in $\mathbb{R}^3$, hyperplanes in general) orthogonal to b . In fact, consider some x and the corresponding level set $L(f, f(x))$. For any direction d orthogonal to b , $\varphi_{x,d}(\alpha) = \alpha \langle b, d \rangle = 0$ (since $\langle b, d \rangle = 0$, cf. §A.2), i.e., a constant function. Thus, all points $x + \alpha d$ have the same function value and therefore belong to the level set. Conversely, for any z belonging to the level set one has $f(x) = f(z)$, i.e., $\langle b, x \rangle = \langle b, z \rangle$, i.e., b is orthogonal to $z - x$.

Regarding the minimisation of a linear function, the unconstrained case is trivial (the problem can only be unbounded) so let us immediately consider the box-constrained case. That is bounded if the box is *compact* (all bounds are finite), but also trivial due to an important property that we encounter for the first—but by means not last—time and that is worth commenting: the problem is *decomposable*. Formally,

$$\min \{ \langle b, x \rangle : \underline{x} \leq x \leq \bar{x} \} = \sum_{i \in I} \min \{ b_i x_i : \underline{x}_i \leq x_i \leq \bar{x}_i \} .$$

In plain words, the variables “do not talk to each other”: the optimal choice for the variable x_i and that for any other variable x_j with $j \neq i$ are completely independent on one another. This means that one is not facing a “monolithic” problem in n variables, but rather n independent problems, each with just one variable. This is clearly a very desirable property, since of course the solution of many smaller problems cannot be more costly than that of the large one—if only because the former can be performed in parallel, yielding a prime example of “embarrassingly parallel” computation. Furthermore, in this particular case the solution of the individual

problems is trivial by means of the formulæ devised in §1.2.1. This problem is therefore as simple as one can hope an n -variables optimization problem to be; indeed its solution would hardly merit being discussed, were it not for the opportunity to introduce some concepts that will be useful later on.

1.3.2 Very simple quadratic functions: separable (non-homogeneous)

We immediately put the last concept into action by briefly considering a “very simple” class of multivariate quadratic functions: the *separable* (non-homogeneous) quadratic ones

$$f(x) = \sum_{i \in I} [f_i(x_i) = a_i x_i^2 + b_i x_i],$$

i.e., the sum of n (non-homogeneous) univariate quadratic functions, one over each of the variables. This is then easy to optimise it, possibly over a (bounded or not) box $\underline{x} \leq x \leq \bar{x}$, by applying the formulæ of §1.2.3 to each x_i individually. However, the fact that there are multiple variables opens the way to a somewhat variegated panorama:

- the minimisation problem is guaranteed to have an optimal solution only if both bounds are finite ($-\infty < \underline{x}_i \leq \bar{x}_i < +\infty$) whenever $a_i < 0$, for otherwise the minimisation of x_i alone is unbounded below; the symmetric condition holds for the maximisation problem, which means that there are cases where neither is bounded;
- hence, in the unconstrained cases the problem is bounded only if minimising with $a_i > 0$ for all $i \in I$, or maximising with $a_i < 0$ for all $i \in I$; in fact the problem can be bounded even if $a_i = 0$ for some i , but only if also $b_i = 0$ for those same i (the objective value does not really depend on those variables, that can therefore take any value).

It is somewhat curious, and instructive, that such a simple problem can already give rise to a number of different cases. Although the problem is “too simple to be interesting”, its study helps understanding the significantly more complex “true quadratic problems” discussed in the subsequent sections. For this reason alone we keep illustrating some of its features.

The first observation is that whenever $a_i \neq 0$ for all $i \in I$, then it is clearly possible to restrict the analysis to the homogeneous case. Indeed, by applying the trick of §1.2.3 one can rewrite the function as $f(z) = \sum_{i=1}^n a_i z_i^2$ (ignoring constant terms) where $z = x - \bar{x}$ with $\bar{x} = [-b_1/2a_1, -b_2/2a_2, \dots, -b_n/2a_n]$. Again, basically one is translating the space so that the origin is moved to $\bar{x}$, and the function is homogeneous in that space. The case where some $a_i = 0$ deserves some discussion, but one could argue that it is “irrelevant”, as soon as optimization is concerned. Indeed, either $b_i \neq 0$, and therefore the problem (irrespectively if minimising or maximising) is unbounded, or $b_i = 0$ and therefore the variable x_i is irrelevant and can be fixed to any value (say, 0). This allows us to concentrate our discussion on the homogeneous case.

Clearly the sign pattern of the a_i is crucial for the “shape” of the function. This is illustrated in Figure 1.6 for bi-variate examples:

- On the top left the graph of $f(x) = 3x_1^2 + x_2^2$, i.e., both $a_1 = 3 > 0$ and $a_2 = 1 > 0$, is shown. The graph is a three-dimensional (convex) “paraboloid”. To better understand its shape one can consider the special form of tomography corresponding to the restriction along i -th coordinate: that is, $\varphi_{0,[1,0]}(\alpha) = f(\alpha, 0) = 3\alpha^2$ and $\varphi_{0,[0,1]}(\alpha) = f(0, \alpha) = \alpha^2$. This immediately extends to any other center than 0, save for a constant term. Thus, for the direction along the one axis x_i the tomography is just the parabola with the corresponding a_i , as the figure clearly shows. The case with $a_1 < 0$ and $a_2 < 0$ would be analogous but “upside-down”: the parabolas would be concave and 0 would be the maximum rather than the minimum.
- On the top right the graph of $f(x) = 3x_1^2 - x_2^2$, i.e., with $a_1 = 3 > 0$ but $a_2 = -1 < 0$, is shown. The graph is an “hyperboloid”, or “saddle-shaped”, and 0 is neither a maximum nor a minimum (it is called a *saddle point*, and the picture clearly justifies the name). This is because on one variable the tomography (restriction along i -th coordinate) is a convex parabola, but on the other is a concave one.
- On the bottom left the graph of $f(x) = 3x_1^2$, i.e., with $a_1 = 3 > 0$ but $a_2 = 0$, is shown. This is a “degenerate paraboloid”: on one variable the tomography is a convex parabola, but on the other it is a constant function. In this case $[0, 0]$ is still a minimum, but so are all points $[0, x_2]$ however chosen x_2 . This is because $b_2 = 0$, which can *not* be assumed without loss of generality since $a_2 = 0$.
- Indeed, on the bottom right the graph of $f(x) = 3x_1^2 + x_2$, i.e., with the same a_i pattern but non homogeneous on x_2 , is shown. This is still a “degenerate paraboloid”, but “not flat in the x_2 variable”. In fact, $\varphi_{0,[0,1]}(\alpha) = f(0, \alpha) = b_2\alpha$, i.e., the tomography (restriction along 2nd coordinate) is linear (and *not* constant), and the function value is bounded neither below nor above.

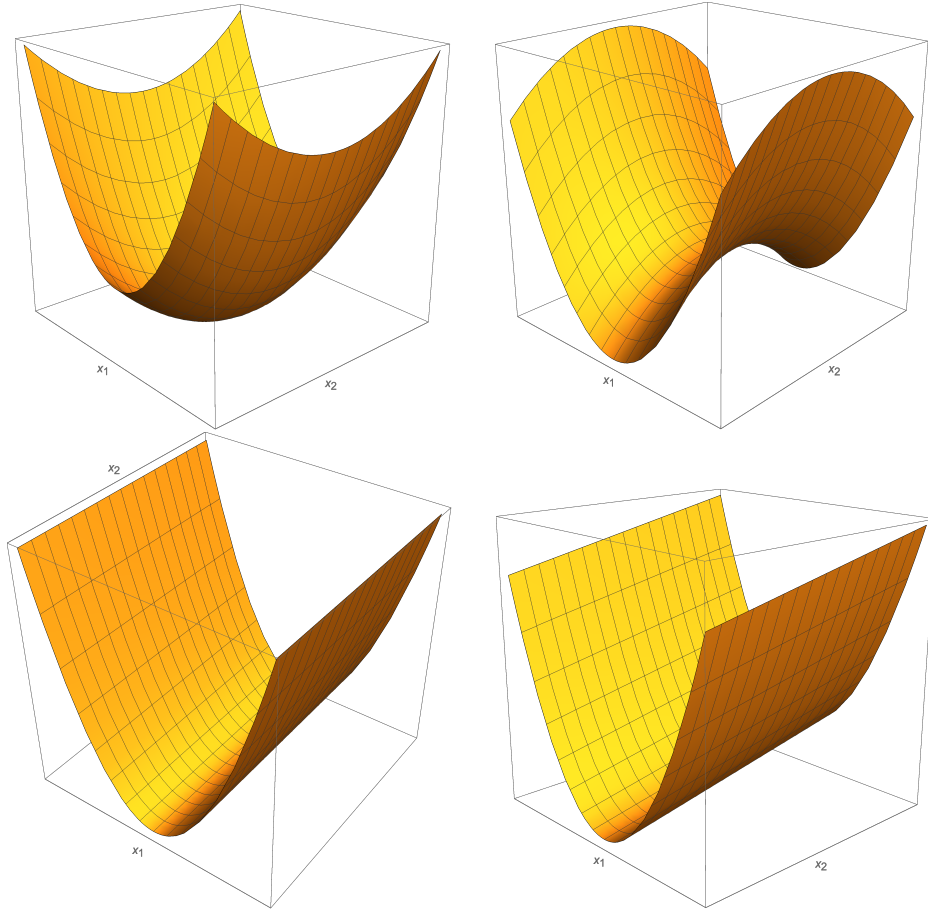


Figure 1.6: Graphs of four examples of separable quadratic functions

A final aspect that is useful to discuss is the impact of the a_i on the “shape” of the function. We focus on the case where both a_i are non-negative: then, the level sets of the function are ellipses (ellipsoids for general n) whose axes are aligned with the Cartesian ones. This is shown in Figure 1.7 and it is kept as an observation, without proof, since it will come as an immediate consequence in the more general analysis in the next section. The Figure shows $L(f, 1)$, i.e., the level sets at $v = 1$, of the parametric function $f(x) = a_1 x_1^2 + x_2^2$ for some different values of a_1 (while keeping $a_2 = 1$). The picture shows a clear trend: the larger a_1 the shorter the x_1 axis of the ellipse, and vice-versa. In fact, for $a_1 = 0$ the ellipse is “degenerate along x_1 ”, with an “infinitely long axis” and therefore turns into two parallel lines. The rationale for this is made clear by considering, e.g., the point where the x_1 axis meets the level set: this is for $f(x_1, 0) = a_1 x_1^2 = 1$, i.e., $x_1 = \pm\sqrt{1/a_1}$. Thus, the smaller a_1 the farther from 0 this point is, and vice-versa, up until for $a_1 = 0$ the two only meet “infinitely far away”. This chimes well with Figure 1.6(bottom left), showing that if $a_i = 0$ then the graph is “infinitely elongated” along the axis of the corresponding variable. While this analysis may seem rather pointless for a function that is so easy to optimize, it is a good introduction to the “real” quadratic functions to be discussed next.

1.3.3 The multivariate homogeneous quadratic function

The “real” quadratic function whose optimization is worth studying is the *nonseparable* one, whereby the impact of a variable x_i on the objective value *does* depend on the other variables x_j for $j \neq i$. This in practice means that there will be *bilinear* terms depending on the product $x_i x_j$. These functions are arguably “significantly more complex” than all those we have seen before since they “depend on a lot more data”: that is, they are characterised by a $n \times n$ matrix Q , i.e., n^2 real numbers as opposed at most $2n$ in any previous case. Formally

$$f(x) = x^T Q x = \left[\sum_{i \in I} Q_{ii} x_i^2 + \sum_{i \in I} \sum_{j=1, j \neq i}^n Q_{ij} x_i x_j \right]$$

(we assume minimal familiarity with matrix/vector operations, cf. §A.3). However, w.l.o.g. one can assume Q *symmetric*, i.e., $Q_{ij} = Q_{ji}$ for all different pairs (i, j) : in fact,

$$x^T Q x = [(x^T Q x) + (x^T Q x)^T] / 2 = x^T [(Q + Q^T) / 2] x$$

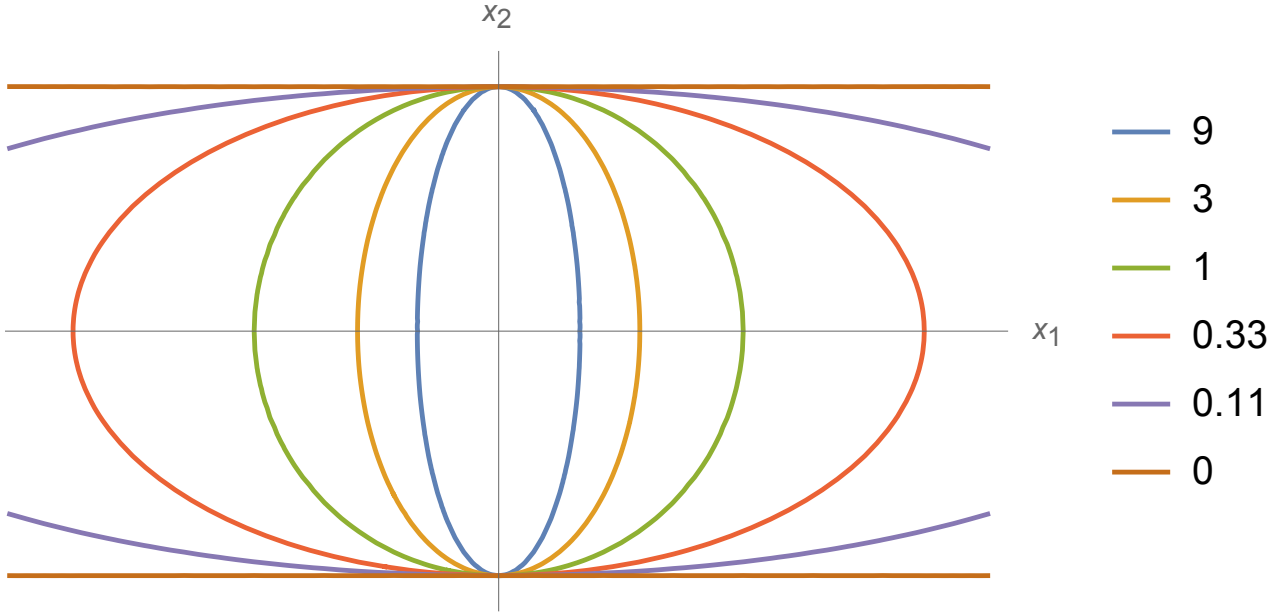


Figure 1.7: $L(f, 1)$ of $f(x) = a_1 x_1^2 + x_2^2$ for some values of a_1

and $(Q + Q^T)/2$ is symmetric; in plain words, if Q is not symmetric to start with one can just replace Q_{ij} with $(Q_{ij} + Q_{ji})/2$ and forget about Q_{ji} . This is advantageous if only because it requires roughly half of the data of a non-symmetric matrix to characterise the function. Not surprisingly, such a function is *not* linear: $f(x+z) = (x+z)^T Q(x+z) = f(x) + f(z) + 2z^T Qx \neq f(x) + f(z)$. It would be natural to consider the non-homogeneous version with an extra linear term, but this extension will be examined later: not doing it ensures that the function itself is symmetric, i.e., $f(x) = f(-x)$ for all $x \in \mathbb{R}^n$. This immediately implies that the level sets of the function are “centred in $x = 0$ ”, and therefore suggest to consider the tomography with 0 centre

$$\varphi_d(\alpha) = f(\alpha d) = \alpha^2 (d^T Q d).$$

which is then a homogeneous quadratic univariate function whose curvature is $d^T Q d$. Thus, in order to understand the “shape” of this function it is necessary to characterise how the number $d^T Q d$ varies when d changes. This is well known to be related to the *spectral decomposition* of Q

$$Q = \lambda_1 H_1 H_1^T + \dots + \lambda_n H_n H_n^T \quad (1.8)$$

where $H_i \in \mathbb{R}^n$ are its *eigenvectors* and λ_i are the corresponding *eigenvalues*, i.e., $QH_i = \lambda_i H_i$, for all $i \in I$. Since Q is symmetric $\lambda_i \in \mathbb{R}$ (otherwise they could be complex numbers, a case we will always carefully avoid). It is obvious that the norm of the eigenvectors is irrelevant, since $QH_i = \lambda_i H_i$ implies $Q(\alpha H_i) = \lambda_i (\alpha H_i)$; thus eigenvectors can always be considered normalised ($\|H_i\| = 1$). They can also always be taken as orthogonal to each other, i.e., $\langle H_i, H_j \rangle = 0$ for all $i \neq j$; in other words, $H^T H = I$ (H is *orthonormal*). Since they are also necessarily linearly independent, they therefore form an orthonormal basis of $\mathbb{R}^n$ (cf. §A.2).

Exercise 1.8. Prove the linear independence of the eigenvectors [solution]

Thus, with $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$, (1.8) can be rewritten as $Q = H \Lambda H^T$.

Exercise 1.9. Prove the above result. [solution]

It is customary to consider (w.l.o.g.) eigenvalues ordered in non increasing sense, i.e., $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$: λ_1 is the maximum eigenvalue, λ_n the minimum. Again, this crucial property would break if Q were non-symmetric and its eigenvalues were *complex* numbers, which—by dint of being *pairs* of real values (cf. §1.1.7)—are not totally ordered. Anyway, the spectral decomposition immediately allows to characterise products $d^T Q d$ (for $d \neq 0$). It is useful to consider the direction $v = d / \|d\|$ corresponding to d : hence, $d = \|d\|v$. Since the eigenvectors are a basis of $\mathbb{R}^n$, one has

$$v = \alpha_1 H_1 + \dots + \alpha_n H_n$$

for a vector $\alpha = [\alpha_i]_{i=1}^n \in \mathbb{R}^n$ of multipliers, where

$$1 = \|v\|^2 = \|\alpha_1 H_1 + \dots + \alpha_n H_n\|^2 = \alpha_1^2 + \dots + \alpha_n^2 = \|\alpha\|^2$$

because $\langle H_i, H_j \rangle = 0$ for all $i \neq j$ and $\|H_i\| = 1$ for all i . Then, by the same token

$$d^T Q d = \|d\|^2 [v^T Q v] = \|d\|^2 [\alpha_1^2 \lambda_1 + \dots + \alpha_n^2 \lambda_n] \quad (1.9)$$

which immediately gives

$$\lambda_1 \geq d^T Q d / d^T d \geq \lambda_n \quad (1.10)$$

a relationship that will be repeatedly useful. Indeed this gives the *variational characterization of eigenvalues*

$$\begin{aligned} \lambda_1 &= \max\{d^T Q d / d^T d : d \neq 0\} = \max\{d^T Q d : \|d\| = 1\} \\ \lambda_n &= \min\{d^T Q d / d^T d : d \neq 0\} = \min\{d^T Q d : \|d\| = 1\} \end{aligned} \quad (1.11)$$

by showing that $d = H_1$ and $d = H_n$ attain respectively the maximum and minimum value in (1.10). The maximum and minimum eigenvalues of a matrix Q are especially important, and they will be denoted by $\lambda_1(Q)$ and $\lambda_n(Q)$, respectively, or simply λ_1 and λ_n when Q is clear from the context. Indeed, the sign of $d^T Q d$ is characterised by that of the eigenvalues of Q , and in particular by that of λ_1 and λ_n , giving rise to five different cases:

- $Q \succ 0$, i.e., is *positive definite*, if all eigenvalues are positive ($\lambda_n > 0$), which implies that $d^T Q d > 0$ however chosen $d (\neq 0)$;
- $Q \succeq 0$, i.e., is *positive semidefinite*, if all eigenvalues are non-negative ($\lambda_n \geq 0$), which implies that $d^T Q d \geq 0$ however chosen d ;
- $Q \succcurlyeq 0$, i.e., is *indefinite*, if some eigenvalues are positive and some are negative ($\lambda_1 > 0$ and $\lambda_n < 0$), which implies that $d^T Q d$ can be both positive and negative according to how d is chosen;
- $Q \preceq 0$, i.e., is *negative semidefinite*, if all eigenvalues are non-positive ($\lambda_1 \leq 0$), which implies that $d^T Q d \leq 0$ however chosen d ;
- $Q \prec 0$, i.e., is *negative definite*, if all eigenvalues are negative ($\lambda_1 < 0$), which implies that $d^T Q d < 0$ however chosen $d (\neq 0)$.

For future reference, let us immediately comment that $\succeq$ defines a *partial order* in the space of $n \times n$ symmetric matrices, that is a relationship “ $A \succeq B$ ” that is reflexive, antisymmetric and transitive (plus it has other nice properties that are not relevant here): this is simply defined by $A \succeq B \iff A - B \succeq 0$. Then, $\succ$ defines a strict ordering, and $\preceq$, $\prec$ obviously do the same. Technically, the set of symmetric positive semidefinite matrices is pointed convex cone with nonempty interior, but we will not need to discuss this in details. The definition immediately implies a relationship between the eigenvalues of A and those of B , via

$$A - B \succeq 0 \implies d^T A d \geq d^T B d \quad \forall d \in \mathbb{R}^n.$$

Passing to the maximum and minimum in (1.11) immediately gives that $\lambda_1(A) \geq \lambda_1(B)$ and $\lambda_n(A) \geq \lambda_n(B)$. This will give, if nothing else, compact forms for giving bounds on eigenvalues: for instance, writing “ $LI \succeq Q \succeq lI$ ” for two constants $L \geq l$ implies that $\lambda_1(Q) \leq L$ and $\lambda_n(Q) \geq l$ (since αI has n identical eigenvalues of value α , as it is immediate to verify), i.e., that the eigenvalues of Q all live in the finite interval $[l, L]$. This is significant, for instance, in view of the relationship between the Euclidean norm of Q and its maximum eigenvalue (cf. §A.9).

This analysis provides a clear way to characterize the solution of the unconstrained optimization (minimization) problem

$$(QU) \quad \min(f(x) = x^T Q x : x \in \mathbb{R}^n).$$

It is instructive to consider the tomography along the eigenvectors, i.e.,

$$\varphi_{H_i}(\alpha) = (\alpha H_i)^T Q (\alpha H_i) = \alpha^2 [H_i^T (\lambda_i H_i)] = \lambda_i \alpha^2$$

(note the use of $\|H_i\| = 1$), which is a homogeneous quadratic function whose curvature is the corresponding eigenvalue. This allows to “recycle” the analysis of the simple case of §1.3.2 in the following sense: the eigenvectors H_i take the place of the axes u^i , and the eigenvalues λ_i take the place of the coefficients a_i . This is indeed correct: the separable case corresponds to the diagonal matrix $Q = \text{diag}(a_1, \dots, a_n)$, and therefore $\lambda_i = a_i$ and $H_i = u^i$. Thus, defining the *span* of any finite set of vectors $V = \{V_1, V_2, \dots, V_k\} \subset \mathbb{R}^n$ as

$$\text{span}(V) = \{x = \sum_{h=1}^k \beta_h V_h : \beta \in \mathbb{R}^k\},$$

and $I^0 = \{i \in I : \lambda_i = 0\}$ the—possibly, empty—set of (indices of) null eigenvalues, one has for (QU)

- if $Q \succ 0$ then $x = 0$ is the unique optimal solution;
- if $Q \succeq 0$ then $x \in \text{span}(\{H_i : i \in I^0\})$ is the set of all optimal solutions;
- if either $Q \succcurlyeq 0$, $Q \preceq 0$ or $Q \prec 0$ (i.e., there exists at least a negative eigenvalue), then the problem is unbounded.

The result for the maximization version of (QU) is obtained by exchanging $\succ/\succeq$ with $\prec/\preceq$. This is illustrated in Figure 1.8, which shows the level sets of quadratic functions with Q all having the same eigenvectors

$$H = [H_1, H_2] = \frac{\sqrt{2}}{2} \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix}$$

but different eigenvalues. In particular, on the top left $\lambda = [\lambda_1, \lambda_2] = [8, 4]$, on the top right $\lambda = [8, 2]$, on the bottom left $\lambda = [8, 0]$, and on the bottom right $\lambda = [8, -2]$. As expected, the axis along $H_2 = [1, 1]$ is longer than that along $H_1 = [-1, 1]$ since $\lambda_2 < \lambda_1$. In particular, its length grows as λ_2 shrinks (top, left to right), up until it becomes “infinitely long” (bottom left). However, when $\lambda_2 < 0$ (bottom right) the graph is no longer a paraboloid but an hyperboloid: the tomography along H_1 is convex but that along H_2 is concave (cf. Figure 1.6 bottom right).

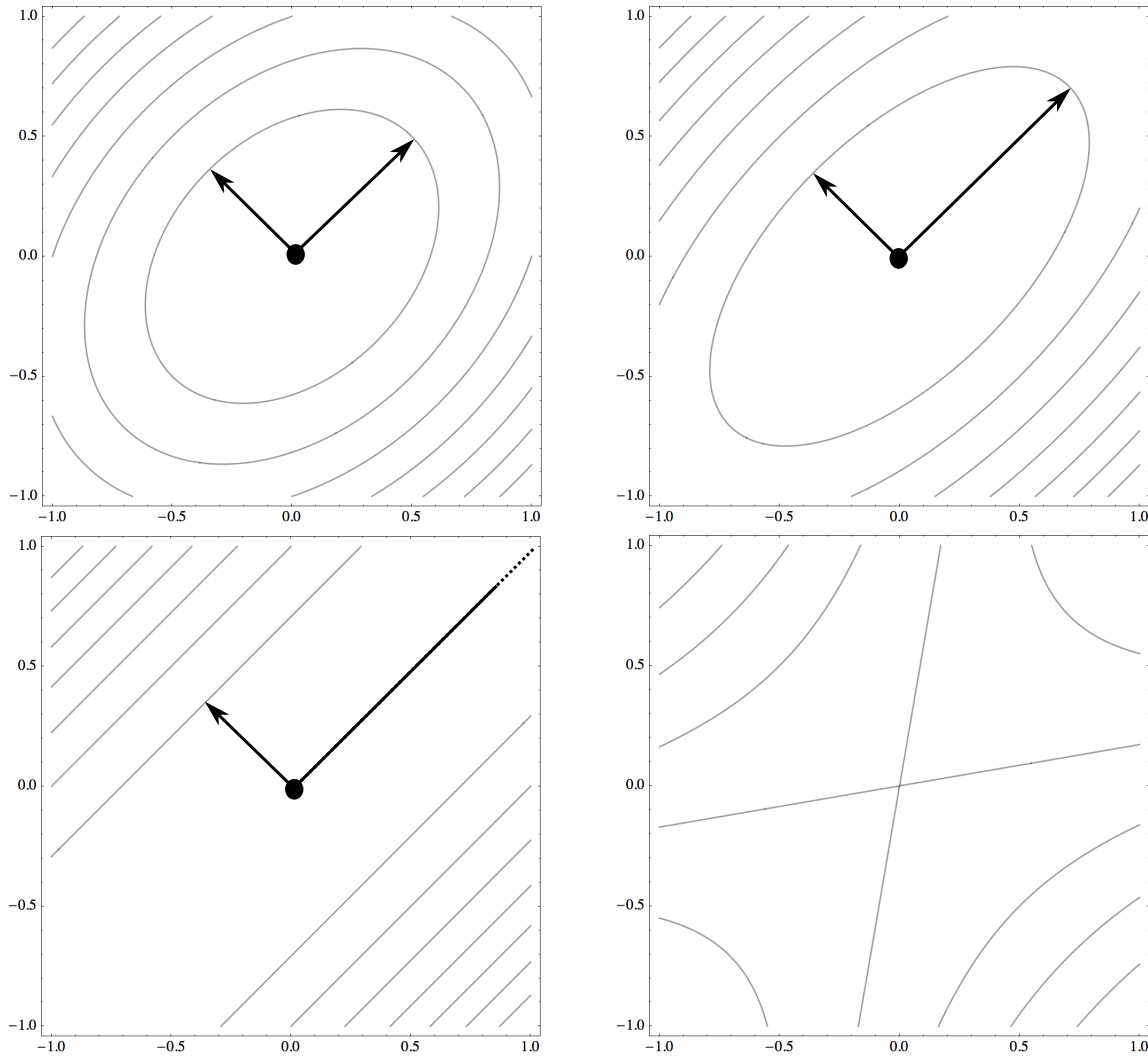


Figure 1.8: Level sets of some homogeneous quadratic functions

Exercise 1.10. Compute the Q matrices corresponding to the four examples. [solution]

These results precisely characterise when (QU) has an optimal solution, and what it is. This does not mean that the solution process is necessarily straightforward, since computing the eigenvalues is nontrivial. Furthermore, the “really interesting” form of the quadratic (unconstrained) optimization problem is that with non-homogeneous functions, where the results—discussed in the next section—are somewhat more complex (although following the same general lines). In fact, in the following some nontrivial solution procedures will have to be developed. However, before doing so it is worth pointing out that the problems are *significantly more challenging in the constrained case*, even for “simple” constraints like the box ones. This may seem surprising, since

$$(QB) \quad \min\{f(x) = x^T Q x : \underline{x} \leq x \leq \bar{x}\}$$

is at least guaranteed to have an optimal solution whenever all bounds are finite. However:

- if $Q \succeq 0$, the problem can be solved “efficiently” but with significantly more complex algorithms, as discussed in Chapter 7;
- if either $Q \succ 0$, $Q \preceq 0$ or $Q \prec 0$ (i.e., there exists at least a negative eigenvalue) the problem is *extremely difficult* ($\mathcal{NP}$ -hard) [18]: there are no known algorithms to solve it with worst-case complexity that grows less than exponentially fast in n , and it is generally conjectured that there cannot be any.

Again, the results hold and the maximization version is obtained by exchanging $\succ/\succeq$ with $\prec/\preceq$. This is a striking example showing that minimizing one function f or maximizing it (as opposed to maximizing $-f$) are by no means equivalent problems. Since these lecture notes are about “easy” problems, a detailed discussion about the very significant difference of complexity between the two problems is not appropriate here; anyway, there is still no formal proof that the “difficult problems” are really such ($\mathcal{P} \neq \mathcal{NP}$ is still a conjecture, although generally believed to be true). Yet, a very brief illustration about the nature of the complexity may be useful.

Example 1.3. Concave minimization

Useful assumptions to simplify somewhat the discussion are that $Q \prec 0$, i.e., all eigenvalues are negative, and that $\underline{x}_i = -1$ and $\bar{x}_i = 1$ for all $i \in I$, i.e., X is the *unitary hypercube*. It is then easy to show that, in the optimal solution x^* to (QB) , either $x_i^* = -1$ or $x_i^* = 1$ for all $i \in I$ (the solution is on a *vertex* of the unitary hypercube). Indeed, assume that $-1 < x_i^* < 1$ for some i , and consider the tomography $\varphi_{x^*, u^i}(\alpha)$: in plain words, all variables x_j save the i -th are fixed to the value x_j^* , while x_i is free to change provided it is feasible, that is, $-1 \leq x_i^* + \alpha \leq 1$ must hold, i.e., $-1 - x_i^* = \underline{\alpha} \leq \alpha \leq \bar{\alpha} = 1 - x_i^*$, where by the hypotheses $\underline{\alpha} < 0$ and $\bar{\alpha} > 0$. Since

$$\varphi_{x^*, u^i}(\alpha) = (x^*)^T Q x^* + 2(x^*)^T Q u^i \alpha + (u^i)^T Q u^i \alpha^2$$

where $(u^i)^T Q u^i < 0$, φ is a concave quadratic function that is minimised on the interval $[\underline{\alpha}, \bar{\alpha}]$, and by the analysis of §1.2.3 its minimum can only lie in one of the extremes of the interval, which clearly correspond to the values of α such that either $x_i = -1$ or $x_i = 1$. No point inside the interval—such as $\alpha = 0$, corresponding to $x = x_i^*$ —can be optimal, contradicting the fact that x^* was optimal.

The continuous problem (QB) is then equivalent to the combinatorial one

$$(QC) \quad \min \{ f(x) = x^T Q x : \{-1, 1\}^n \}$$

whose feasible region contains a finite, but *exponential* (2^n) number of solutions. It is therefore possible to solve it by enumerating all these solutions and keeping the best one, but such a process would have at least $O(2^n)$ cost. Unfortunately, all the known algorithms for solving such a problem have an analogous exponential worst-case complexity: in plain words, no algorithm has ever been found that is “substantially more clever” than blind enumeration in the worst case. In practice clever and sophisticated algorithms exist that may be able to solve some instances of such problems efficiently, but none of them with provable non-exponential accuracy in all cases. Indeed, even the best algorithms may actually require extremely long time, compatible with a real exponential complexity, to solve many instances of the problem. This is true for a huge class of problems that all have “the same complexity” [7], leading to the belief that no polynomial-time algorithm exists for any of them. Unless a very surprising result proves otherwise, the only safe assumption is therefore that (QB) cannot be solved efficiently.

The example shows how “brittle” optimization problems are, at least with respect to complexity: an apparently minor change—in this case, just one negative eigenvalue in Q —brings an enormous difference in the ability to solve large-scale instances. This is why these lecture notes will tread very carefully to remain on the “easy” side of optimization, possibly “cheating” whenever appropriate and necessary.

1.3.4 Optimizing non-homogeneous quadratic functions

The “truly interesting” case is that of the non-homogeneous quadratic function

$$f(x) = \frac{1}{2} x^T Q x + \langle q, x \rangle$$

with $q \neq 0$, i.e., a homogeneous quadratic function plus a linear one. The constant factor of $1/2$ on the quadratic part may look weird at first, but it is useful to streamline the results somewhat. Taking a leaf from §1.2.2, the result is actually at hand under one assumption: Q *nonsingular*, i.e., $\lambda_i \neq 0$ for all $i = 1, \dots, n$ (regardless of the sign). Indeed, in this case one can again translate the variables space as $z = x - \bar{x}$ for $\bar{x} = -Q^{-1}q$ to obtain

$$f(x) = g(z) = \frac{1}{2} z^T Q z + f(\bar{x}),$$

i.e., an homogeneous quadratic function in the z -space (disregarding the constant).

Exercise 1.11. Prove the result, but it should look familiar [solution]

This immediately allows to apply the results of the previous section:

- if $Q \succ 0$ then $x = \bar{x}$ is the unique optimal solution;
- if $Q \succeq 0$ then $x \in \bar{x} + \text{span}(\{H_i : i \in I^0\})$ is the set of all optimal solutions;
- if either $Q \succ 0$, $Q \preceq 0$ or $Q \prec 0$ (i.e., there exists at least a negative eigenvalue), then the problem is unbounded.

Again, exchanging $\succ/\succeq$ with $\prec/\preceq$ give the result for the maximization version, and the box-constrained cases remains hard, or at least nontrivial.

However, this does not conclude the analysis like in the univariate case. Indeed, there if $a = 0$ then the function was linear. In this case, “different pieces of Q can be 0 without all Q being”, which means that a more complicated analysis is needed; however, this is so only in the minimization case where $Q \succeq 0$ and in the maximization one where $Q \preceq 0$, since in the other ones the problem is surely unbounded (below/above). We will then focus on minimization with $Q \succeq 0$, since the other case is symmetric. The set I^0 of (indices of) null eigenvalues is nonempty, i.e., $k = |I^0| > 0$; for the set of (indices of) *positive* eigenvalues $I^+ = I \setminus I^0$, with $h = |I^+| = n - k$, we can also assume $h > 0$ (since the only matrix with all 0 eigenvalues is the null one, and in this case f is “truly linear”). Clearly, it is crucial to distinguish the directions corresponding to the null eigenvalues (where the tomography is linear) from those corresponding to the non-null ones (where the tomography is convex quadratic). The first is $\ker(Q) = \text{span}(\{H_i : i \in I^0\})$: it is easy to see that $Qv = 0$ for all $v \in \ker(Q)$.

Exercise 1.12. Prove the result. [solution]

Thus, this is the set of all vectors “along which the quadratic part is linear”: note that obviously $0 \in \ker(Q)$, but the assumption is that the subspace contains other vectors than 0. The other interesting subspace, of course, is $\text{im}(Q) = \text{span}(\{H_i : i \in I^+\})$. Its fundamental property is that $\forall w \in \text{im}(Q)$ exists some $x \in \mathbb{R}^n$ s.t. $Qx = w$; in plain words, $\text{im}(Q)$ is the set of all right-hand-sides of solvable linear systems with Q .

Exercise 1.13. Prove the result (recall $Q = \lambda_1 H_1 H_1^T + \dots + \lambda_n H_n H_n^T$, use [84]). [solution]

Because H is a (orthonormal) base of $\mathbb{R}^n$ and the index of each H_i either belongs to I^0 or I^+ , $\mathbb{R}^n = \text{im}(Q) + \ker(Q)$ and $\text{im}(Q) \perp \ker(Q)$. In other words, every vector $v \in \mathbb{R}^n$ can be written as $v = v^0 + v^+$, with $v^0 \in \ker(Q)$, $v^+ \in \text{im}(Q)$, and $\langle v^0, v^+ \rangle = 0$. In particular, hence, $q = q^0 + q^+$ with $q^0 \in \ker(Q)$ and $q^+ \in \text{im}(Q)$. This means that the system

$$Q\bar{x} = -q^+$$

has at least one solution (obviously, $v \in \text{im}(Q) \iff -v \in \text{im}(Q)$). Picking *any* such $\bar{x}$ and defining as usual $z = x - \bar{x}$, it is then easy to see that

$$f(x) = g(z) = \frac{1}{2} z^T Q z + q^0 z + f(\bar{x}).$$

Exercise 1.14. Prove the result, but it should look very very familiar. [solution]

In plain words, f is “truly quadratic” on $\text{im}(Q)$ but actually linear on $\ker(Q)$, with the linear coefficient being q^0 . This was also true in the homogeneous case, but since there $q = 0 \implies q^0 = 0$, the linear function was constant, and any point in $\ker(Q)$ was a minimum. In this case, this may or may not happen. That is:

- if $q^0 \neq 0$, then the problem is unbounded below: indeed, the tomography—in the z -space—along with centre 0 and $q^0 \in \ker(Q)$ is

$$\varphi_{0,d}(\alpha) = \alpha \langle q^0, q^0 \rangle + f(\bar{x}),$$

i.e., a linear (affine) function with slope $\|q^0\|^2 > 0$;

- if $q^0 = 0$, then every $x \in \bar{x} + \ker(Q)$ is optimal for the problem.

Exercise 1.15. Prove the result. [solution]

Considering that $q^0 = 0 \iff q^+ = q \iff$ there exists some $\bar{x} \in \mathbb{R}^n$ such that $Q\bar{x} = -q$, all the above discussion leads to the following result:

Theorem 1.1. If $Q \succeq 0$, the optimal solutions of the (unconstrained) problem

$$(QU) \quad \min\{f(x) = \frac{1}{2}x^T Q x + \langle q, x \rangle : x \in \mathbb{R}^n\}$$

are all and only the solutions to the linear system

$$Qx = -q$$

if any (that is, the problem is unbounded below if the system has no solution).

Thus, if $Q \succ 0$ then $\bar{x} = -Q^{-1}q$ is the unique optimal solution to (QU) . If, rather, Q is only positive semidefinite, the problem may or may not have optimal solutions. In plain words this has to do with the fact that “along an eigenvector corresponding to a zero eigenvalue the quadratic part of the function is constant”, which means that “the linear part now decides the outcome”: the problem is bounded below (if and) only if any direction along which the quadratic part of f is constant is also orthogonal to q , which means that also the

linear part of f is constant.

Despite being somehow simple, the above result merits some comments:

- it is the first example of (nontrivial) *optimality condition* in these lecture notes, i.e., a condition that allows to precisely identify the optimal solutions to a problem; optimality conditions are crucial for the development of solution algorithms, in that they provide the *stopping criterion*, i.e., a way to be able to terminate the search when the desired solution is found, but even more—as we shall see—because they typically guide the design of the algorithms themselves;
- it underlines the fundamental role of (non)linear algebra in the development of optimization algorithms: the task of solving an optimization problem, or proving it has no solution, is traced back to the task of proving that a certain set of linear (algebraic) conditions are satisfied.

Of course this is just the simplest possible result: more complex problems will require more complex optimality conditions. However, the previous analysis has already revealed—only using the simplest possible mathematical tools—many of the fundamental ideas of all the subsequent developments.

1.4 Multivariate Quadratic optimization: Gradient Method

Having established the optimality conditions of the optimization problem (Q) paves the way to developing approaches to *actually construct* a solution satisfying them (or prove that none exists), which is the central issue in these lecture notes. For the “very simple” current case, in which it is sufficient to solve the linear system $Qx = -q$ (with Q real, symmetric and ideally positive semidefinite), *direct* methods are available that can construct the solution $\bar{x}$. One of these will even be briefly recalled later on. However, this is a very special case that only occurs for quadratic functions and does not readily generalise. For most problems, computing the exact solution with a finite approach is not possible. As previously discussed, this is not really an issue, though. Even directly solving $Qx = -q$ will necessarily incur in numerical errors, which may be significant, so the exact solution is not available anyway (although some exceptions may exist, and numerical errors will typically be “small” w.r.t. those inherent in other approaches). By the same token, any approach that eventually constructs a solution x that “satisfies the system $Qx = -q$ well enough” will be appropriate. This leads to the concept of *iterative* methods, that is central in these lecture notes.

1.4.1 Some generalities on iterative methods

By and large, all iterative methods work in the same way: they start from an *initial guess* x^0 of the solution, upon which little assumptions are usually made (that is, x^0 is not required to be a “good” solution in any sense of the term, $x^0 = 0$ being one usual choice even if 0 may be arbitrarily far from the desired final answer), and then employ *some process* that take the *current iterate* x^i and transforms it in the *next iterate* x^{i+1} . This process is iterated, in principle for an arbitrary number of times, giving rise to a (n infinite) sequence $\{x^i\}$ of iterates. The idea is that the sequence should “eventually approach the desired solution x^* ”, the optimal solution to the optimization problem at hand, in a sense to be formally specified in the following. In practice, the process is finitely interrupted when some iterate x^i is “close enough to x^* ”, again in a sense to be better specified.

The fundamental aspects of iterative methods that will be primarily investigated in these lecture notes are:

- *correctness*, or *convergence*: proving that the sequence will eventually reach “close enough to x^* ”, with “enough” dictated by one (or more) parameter(s) that can be freely set by the “user” of the approach (possibly subject to limits due to the numerical accuracy);
- *efficiency*, or *complexity*: characterising how rapidly the algorithm will achieve its intended result, depending on the chosen accuracy parameter(s), typically measured in terms of *number of iterations* required to reach the *stopping criterion* proving that the current iterate x^i is “close enough” to x^* , and possibly taking into account the cost (complexity) of each single iteration so as to be able to estimate the total computational cost of the whole process.

The above concepts are purposely stated in a vague way, since, as we will see, there will need to be different choices for the “close enough to x^* ” definition according to the details of the problem and algorithm at hand. The most obvious version is given by the fact that the sequence of iterates $\{x^i\}$ naturally defines the *sequence of (objective) function values* (“ f -values” for short) $\{f^i = f(x^i)\}$; one then wants that “ $\{f^i\}$ go towards $f_* = f(x^*)$ ”. The natural mathematical interpretation is that

$$\lim_{i \rightarrow \infty} f^i = f_* \quad \text{usually written as} \quad \{f^i\} \rightarrow f_*.$$

A sequence of iterates $\{x^i\}$ such that the corresponding sequence $\{f^i\}$ of f -values have the above property is called a *minimizing sequence* (for a minimization problem). Since not all sequences have limits, this may not always be possible; cf. §A.5 for alternative notions of convergence that can be used. However, a very natural assumption on the sequence of f -values produced by an algorithm is that it is *decreasing*: $f^{i+1} = f(x^{i+1}) < f^i = f(x^i)$, corresponding to the fact that “ x^{i+1} is strictly better than x^i ”. This is called a *descent*, or *monotone*, algorithm. Although not all algorithms have this feature many do, among which all those of interest in this section; hence for a start one can focus on descent algorithms. Since monotone sequences always have a limit, the asymptotic value $f^\infty = \lim_{i \rightarrow \infty} f^i$ is always well-defined. It is immediate to realise that $f^\infty \geq f_*$, which means that $f^\infty = -\infty$ “proves” that the problem is unbounded; but this is largely a theoretical concept, as in practice no algorithms run forever and “one cannot really wait for the sequence to go all the way down to $-\infty$ ”. Indeed, *proving unboundedness* is usually nontrivial (the current case being one of the few positive exceptions to this rule); fortunately, most applications of interest for these lecture notes are naturally bounded, which means that necessarily $f^\infty > -\infty$. The first step when designing a (descent) algorithm is therefore to ensure that $f^\infty = f_*$, i.e., *convergence*. The second step is to try to estimate “how fast” f^i get “close” to f_* , i.e., *efficiency*. A first, elementary algorithm will now be introduced for (Q) providing a first example of convergence and efficiency analysis. Although the algorithm is simple, its analysis is not trivial. Furthermore, the results obtained in this case will be shown to be mirrored in many more complex contexts, providing a useful baseline for the analysis of (unconstrained) optimization algorithms.

Before moving along with an actual algorithm, it is appropriate to remark that $f^\infty = f_*$ is not necessarily a completely satisfactory notion of “convergence”. Indeed, the reason for solving problems such as (Q) is typically that of finding an approximately optimal *solution*. While the intuition is that a minimizing sequence will correspond to a sequence of iterates x^i that “get close” to some optimal solution, this may only be partly true. Indeed, consider the minimization of

$$f(x_1, x_2) = x_1^2,$$

with $f_* = 0$ corresponding to the set of optimal solutions $X^* = \{(0, x_2) : x_2 \in \mathbb{R}\}$. The sequence of iterates given by

$$x^i = [1/i, (-1)^i i]$$

clearly gives $\{f^i = 1/i^2\} \rightarrow 0 = f_*$; that is, $\{x^i\}$ is a minimizing sequence. However, the iterates themselves can hardly be seen to “converge” somewhere: $\{\|x^i\|\} \rightarrow +\infty$, and every two subsequent iterates are “very far apart”. In some sense, of course, the sequence is correctly identifying the optimal set: $\{x_1^i\} \rightarrow 0$, while the x_2^i values “span all of $\mathbb{R}$ ”. However, at the very least an algorithm producing such a minimising sequence should be considered as “unstable”, in the sense that it could easily produce very different (approximate) solutions. This is not to say that practical algorithms would naturally produce such iterates; rather, the intent here is to mention how *convergence of f -values* and *convergence of iterates* (to be defined in details later) are in principle different. Of course, the issue here is that X^* is not a singleton, which may in principle lead an algorithm to be “attracted to different optimal solutions”: if $X^* = \{x^*\}$ is a singleton, convergence of f -values is equivalent to convergence of iterates (again, to be discussed in details later). In the quadratic case, this—and the analysis of §1.3.4—suggests that having $Q \succ 0$ may be “more convenient” than having $Q \succeq 0$. This will indeed be shown to be true, justifying why initially $Q \succ 0$ —implying that (Q) always has the unique solution $x^* = -Q^{-1}q$ —will be assumed.

1.4.2 The Gradient Method for Quadratic Functions

The basic idea of the approach is quite simple: given one iterate x^i , it is reasonably necessary to compute $g^i = Qx^i + q$, called (for reasons that is preferable not to discuss right now) the *gradient of f at x^i* . Clearly, if $g^i = 0$ then one optimal solution x^* has been obtained and the algorithm can stop. As previously discussed, a check such as “ $g^i = 0$ ” is not doable in floating point arithmetic, and it is typically substituted with “ g^i is close enough to be the all-0 vector”, i.e., “ $\|g^i\| \leq \varepsilon$ for some appropriate $\varepsilon > 0$ ”. The choice of ε is itself not trivial and it will be discussed in some detail later on; for now it is just a user-defined value upon which the algorithm has no control (it is one, and in fact the only, *algorithmic parameter* of the approach). The idea is that whenever $\|g^i\| > \varepsilon$ (“bad news”, x^i is not “close to being optimal”) then it is possible to *use g^i to produce a x^{i+1} “better” than x^i* , in the sense already discussed below. Indeed, consider the (scaled) tomography

$$\varphi_{x^i, -g^i}(\alpha) = f(x^i - \alpha g^i) - f(x^i)$$

of center x^i and direction $-g^i$ (called the *antigradient*). Simple algebra shows that

$$\varphi_{x^i, -g^i}(\alpha) = \frac{1}{2}\alpha^2(g^i)^T Q g^i - \alpha[(g^i)^T Q x^i + q g^i] = \frac{1}{2}\alpha^2(g^i)^T Q g^i - \alpha\|g^i\|^2.$$

That is, the tomography is (not surprisingly) a non-homogeneous quadratic function with curvature $a = \frac{1}{2}(g^i)^T Q g^i > 0$ and slope $b = -\|g^i\|^2 [< -\varepsilon^2] < 0$. The analysis of §1.2.3 now immediately proves that some $\bar{\alpha} > 0$ exists such that $\varphi_{x^i, -g^i}(\bar{\alpha}) < 0$, i.e., $f(x^i - \bar{\alpha}g^i) < f(x^i)$: this immediately provides a clue about how to construct a next iterate x^{i+1} of a descent algorithm.

Exercise 1.16. Check all the above. [solution]

In plain words, the same information (g^i) indicating that the algorithm *cannot* stop at the same time provides the means to construct a “better” next iterate x^{i+1} . To fully turn this intuition into an algorithm it is only necessary to specify how exactly the appropriate step $\bar{\alpha}$ is selected. Although there may be different choices, the most natural one is to pick *the best possible iterate x^{i+1} that can be obtained*, i.e., the value

$$\alpha^i = \operatorname{argmin}\{ \varphi_{x^i, -g^i}(\alpha) : \alpha \in \mathbb{R} \}.$$

Again, the results in §1.2.3 ensure that such a value exists, is unique, and

$$\alpha^i = \|g^i\|^2 / ((g^i)^T Q g^i). \quad (1.12)$$

Using (1.11) it is easy to provide finite bounds on the *stepsize*: $1/\lambda_1 \leq \alpha^i \leq 1/\lambda_n$.

Exercise 1.17. Prove the last statement. [solution]

All this finally gives the *Gradient Method* for the unconstrained optimization of quadratic functions:

Algorithm 1.1. Gradient Method for Quadratic Function

```

procedure  $x = SDQ(Q, q, x, \varepsilon)$ 
  for ( ; )
     $g \leftarrow Qx + q$ ;
    if ( $\|g\| \leq \varepsilon$ ) then break;
     $\alpha \leftarrow \|g\|^2 / g^T Q g$ ;  $x \leftarrow x - \alpha g$ ;

```

Although having been devised for the case where $Q \succ 0$, the algorithm can be easily adapted to the case where $Q \succeq 0$, or even $Q \succcurlyeq 0$ (basically, no hypotheses on Q). Starting from the former case, the issue clearly is that $g^T Q g = 0$ can happen; numerically speaking, this means that it may be a “very small” number creating issues (e.g., overflows) while computing α . This is, however, easy to solve: $g^T Q g = 0$ means that $g \in \ker(Q)$, i.e., is a direction along which $\varphi_{x, -g}$ is (almost) linear. However, $b = -\|g^i\|^2$ is not 0, and “sizeably so”: thus, $\varphi_{x, -g}$ is *not* constant, and therefore unbounded below. Thus, $f_* = -\infty$ for (Q) . Adapting the algorithm then just requires adding the instruction

if ($g^T Q g \leq \delta$) **then break**;

for a “very small” value δ . In this case, however, it is also necessary to signal the different kind of outcome: that is, the algorithm should return information allowing the user to distinguish whether the returned x is indeed an (approximately) optimal solution, or the problem has been found to be unbounded below. This is usually done by returning a “status code” (often, a simple integer value) encoding this information. It would also be appropriate to return $x = -g$, as this is the *certificate of unboundedness* of the problem: a direction along which the tomography (hence, f) is unbounded below; depending on the status code the user would know in which of the two ways the returned vector should be interpreted. Interestingly, this is all that is needed to extend the algorithm to the case of $Q \succcurlyeq 0$. Indeed, in that case the condition above can be verified when $g^T Q g < 0$ ($g^T Q g \leq -\delta < 0$, i.e., “negative and not very close to zero”), which still means that $\varphi_{x, -g}$ is unbounded below; except, in this case it’s not a linear function, but a quadratic one with negative curvature. With possibly a different returned status code, the algorithm should stop all the same. It should be remarked, however, that in none of the cases the algorithm is guaranteed to be able to determine the certificate; rather, a sequence of iterates with decreasing f -values may be produced where f^i may “numerically approach $-\infty$ ” (possibly eventually yielding overflow errors). Again, *proving* unboundedness is nontrivial. However, all this means that adapting the code for maximization is trivial.

Exercise 1.18. Discuss how to change the code to solve $\max\{f(x)\}$ instead. [solution]

In terms of complexity of the each iteration, all operations are linear in n , i.e., $O(n)$, except the matrix-vector products in $g \leftarrow Qx + q$ and the computation of $a = g^T Q g$. Thus, the complexity of each iteration is $O(n^2)$. Although the algorithm does not, strictly speaking, need to compute the f -value $f^i = f(x^i)$, this is also $O(n^2)$ due to the matrix-vector product $x^T Q x$. Thus, the costly part of the algorithm is the computation of two (or three) matrix-vector products involving Q . With some creativity these can be reduced to *only one*; this is called a *streamlining* of the algorithm.

Exercise 1.19. Discuss how to streamline the algorithm. [solution]

1.4.3 Convergence of the gradient method for $Q \succ 0$

In order to prove convergence one needs to show that $f^\infty = \lim_{i \rightarrow \infty} f^i = f_*$. This is usually rewritten in terms of the absolute gap $a^i = f^i - f_* \geq 0$ as $\{a^i\} \rightarrow 0$. Since the algorithm is of descent, a^i is monotone decreasing and therefore $\{a^i\} \rightarrow a^\infty \geq 0$; it is however not trivial to prove that $a^\infty = 0$. Usually, an issue is that f_* is not known, and hard to characterise a-priori, which makes it difficult to estimate a^i . In this simple case, however, the *homogeneous form of the error*

$$A(x) = f(x) - f_* = \frac{1}{2}(x - x^*)^T Q (x - x^*) \quad (1.13)$$

is valid for all $x \in \mathbb{R}^n$ and allows to considerably simplify the arguments.

Exercise 1.20. Prove the formula above. [solution]

The above relationship, the definition of $x^{i+1} = x^+ - \alpha^i g^i$ with the quantities defined in the algorithm, and $Q \succ 0$ allow to precisely characterise a^{i+1} in terms of a^i :

$$A(x^{i+1}) = \left(1 - \frac{\|g^i\|^4}{((g^i)^T Q g^i)((g^i)^T Q^{-1} g^i)} \right) A(x^i). \quad (1.14)$$

Proving (1.14) requires some simple, albeit somewhat tedious, algebra [9, Lm. 8.6.1]. One of the steps is worth pointing out for later reference: the gradients at two subsequent iterations are orthogonal, i.e., $\langle g^{i+1}, g^i \rangle = 0$.

Exercise 1.21. Prove the last statement. [solution]

Exercise 1.22. Prove (1.14). [solution]

Although exact, (1.14) is not completely informative because it depends on g^i , which in turn depends on x^i . It would therefore be useful to bound from below the crucial term of the form

$$s(x) = \|x\|^4 / [(x^T Q x)(x^T Q^{-1} x)]$$

for all possible $x \in \mathbb{R}^n$ (except 0, since we know $g^i \neq 0$). Since $Q \succ 0$, this is indeed possible using the *condition number of Q* $\kappa = \lambda_1 / \lambda_n \geq 1$, i.e., the ratio between its largest and smallest eigenvalue. Simple arguments using the variational characterisation (1.11) and the fact that eigenvalues of Q^{-1} are the inverse of those of Q (cf. §A.4) yield

$$s(x) \geq s = \lambda_n / \lambda_1 = 1 / \kappa \quad \forall x \in \mathbb{R}^n, \quad (1.15)$$

which is independent of the iterations and only depends on inherent properties of Q .

Exercise 1.23. Prove (1.15). [solution]

Plugging (1.15) in (1.14) gives

$$A(x^{i+1}) \leq r A(x^i) \quad \text{with} \quad r = (1 - 1/\kappa) = (\kappa - 1)/\kappa = (\lambda_1 - \lambda_n)/\lambda_1 < 1. \quad (1.16)$$

This immediately confirms that the error strictly decreases, since $a^{i+1} = r a^i < a^i$. In fact, the error decreases “exponentially fast”: given the initial error $a^0 = A(x^0)$ depending on the initial iterate x^0 —upon which there is little control, but that it is at least finite—one has

$$A(x^i) \leq r^i A(x^0), \quad (1.17)$$

where $r^i \rightarrow 0$ as $i \rightarrow \infty$.

Exercise 1.24. Prove (1.17). [solution]

As a consequence, the algorithm converges: $a^i \rightarrow 0$ as $i \rightarrow \infty$, i.e., $\lim_{i \rightarrow \infty} f^i = f_*$. Due to the specific formula above, it is also possible to estimate its complexity, i.e., the (maximum) number of iterations that are required to achieve a *prescribed accuracy* δ . That is, one wants to compute a function $c(\cdot)$ such that the algorithm is guaranteed (if ran in exact arithmetic) to produce within $c(\delta)$ iterations an iterate x^i such that $A(x^i) \leq \delta$. One just has to plug the requirement in (1.17) to get

$$i \geq \lceil 1 / \log(1/r) \rceil \log(a^0 / \delta). \quad (1.18)$$

Exercise 1.25. Prove (1.18). [solution]

Thus, the complexity of the algorithm is $O(\log(1/\delta))$, which is on the outset a quite good result. In plain words, assume that k iterations are needed to get an accuracy of, say, $\delta = 1\text{e-}4$; this means that

$$k \geq R \log(1\text{e} + 4a^0) = R[\log(1\text{e} + 4) + \log(a^0)] = R[4 + \log(a^0)]$$

for some fixed $R > 0$. Assume now one wants a solution with $\delta = 1\text{e-}8$; if $f_* \approx 1$, this is basically asking 8 decimal digits of accuracy rather than 4, i.e., a significantly better accuracy. By the same token as above one now have

$$k \geq R[8 + \log(a^0)] ;$$

even if the term $\log(a^0)$ is insignificant, this means that “doubling the number of accurate decimal digits requires at most doubling the number of iterations”. Since one can have at most 16 decimal digits (and usually way less), this means that the algorithm should terminate “quickly” even if highly accurate solutions are required. It is particularly relevant that k in the above formulæ *does not (explicitly) depend on n* . This *dimension independent* complexity estimate can be taken to mean that highly accurate solutions may be obtained efficiently even for very-large-scale problems, since the number of iterations may not grow as n does.

The above analysis is, however, disregarding the role of the two constants a^0 and $R = 1 / \log(1/r)$. For the former, little can be said since in general there is hardly any control on how accurate the initial estimate x^0 is. The general gist is that *starting with a more accurate initial estimate will lead to requiring less iterations*, which, albeit a nice property, is hardly surprising. After all, if x^0 were approximately optimal already, in the sense that $\|g^0 = Qx^0 + q\| \leq \varepsilon$, then the algorithm would stop immediately (it would be right to wonder how this is related to the accuracy δ , but discussing this is left for later). The impact of R is, however, much more significant. The issue here is that, as the problem becomes *ill-conditioned*, i.e. $\lambda_1 \gg \lambda_n$ which implies that $r = (1 - 1/\kappa) \approx 1$, R grows large. In fact, it is easy to see that

$$r \approx 1 \implies k \approx [r / (1 - r)] \log(a^0 / \delta) \approx \kappa \log(a^0 / \delta) .$$

Exercise 1.26. Verify the formula above. [solution]

In plain words, all other things— δ and a^0 —being equal, the number of iterations can be expected to grow proportionally with the condition number of Q . This is in some sense the “bad news” version of a dimension independent efficiency result: the number of iterations required to solve the problem can grow rather large even when n remains the same. In fact, things may be even worse in case there is a *hidden dependency*: λ_1 and λ_n may depend on n , which means that κ may grow as n does.

Unfortunately, this analysis is far from only pointing to a remote possibility: the actual computational behaviour of the approach does trail the result quite closely in practice. This is shown in Figure 1.9, which illustrates the behaviour of the gradient method on small-scale ($n = 2$) instances. In particular, the top row corresponds to the matrix

$$Q = \begin{bmatrix} 11 & 9 \\ 9 & 11 \end{bmatrix} \quad \text{with} \quad \lambda_1 = 20, \lambda_2 = 2 \quad \text{and therefore} \quad \kappa = 10,$$

while the bottom row corresponds to the matrix

$$Q = \begin{bmatrix} 101 & 99 \\ 99 & 101 \end{bmatrix} \quad \text{with} \quad \lambda_1 = 200, \lambda_2 = 2 \quad \text{and therefore} \quad \kappa = 100.$$

On the left the iterates performed by the algorithm are plotted upon some level sets of the objective, while on the right the *convergence plot* shows the evolution of the relative error $a^i / |f_*|$ along the iterations on a log-linear scale.

In the first instance the algorithm performs 34 iterations to reach $\|g^i\| \leq 1\text{e-}6$, which corresponds to $\delta \approx 1\text{e-}13$, starting from $a^0 \approx 2.4\text{e-}1$. In the second, starting from $a^0 \approx 1.4\text{e-}1$, it rather requires 254 iterations to get a solution of comparable quality. Although the number of iterations do not scale exactly as the condition number, it does significantly increase as κ grows. Geometrically, a larger condition number corresponds to the fact that the level sets are “more elongated” (Figure 1.9(left), top to bottom) and the iterates of the algorithm “zig-zag a lot”, leading to slow convergence. This is related to the fact that, as previously mentioned, “the trajectory always turns right angles”: the gradients at two subsequent iterations are orthogonal, and therefore so are the lines along which the tomography is made. This is confirmed by Figure 1.9(right), which clearly illustrates why (1.17) is known as *linear convergence*: while in both cases the gap decreases linearly in a log-linear plot, which means exponentially, the slope of the line is much steeper (convergence is much faster) when κ is smaller, i.e., r is farther away from 1.

The fact that the scaling in the previous example is not exact should not come as a surprise: (1.15) is a worst-case estimate, and the decrease may actually be faster. In fact, the estimate is not even the tightest possible one: a more refined analysis using the Kantorovich inequality [9, 8.6.(34)] gives better estimate

$$\frac{\|x\|^4}{(x^T Q x)(x^T Q^{-1} x)} \geq \frac{4\lambda_1 \lambda_n}{(\lambda_1 + \lambda_n)^2} \implies r \leq \left(\frac{\lambda_1 - \lambda_n}{\lambda_1 + \lambda_n} \right)^2 = \left(\frac{\kappa - 1}{\kappa + 1} \right)^2 \quad (1.19)$$

This is close but significantly different from the previous one. For instance, $\kappa = 10$ gives $r = 0.9$ with the rougher estimate and $r = (9/11)^2 \approx 0.67$ with (1.19), while $\kappa = 100$ gives $r = 0.99$ against $r = (99/101)^2 \approx 0.96$.

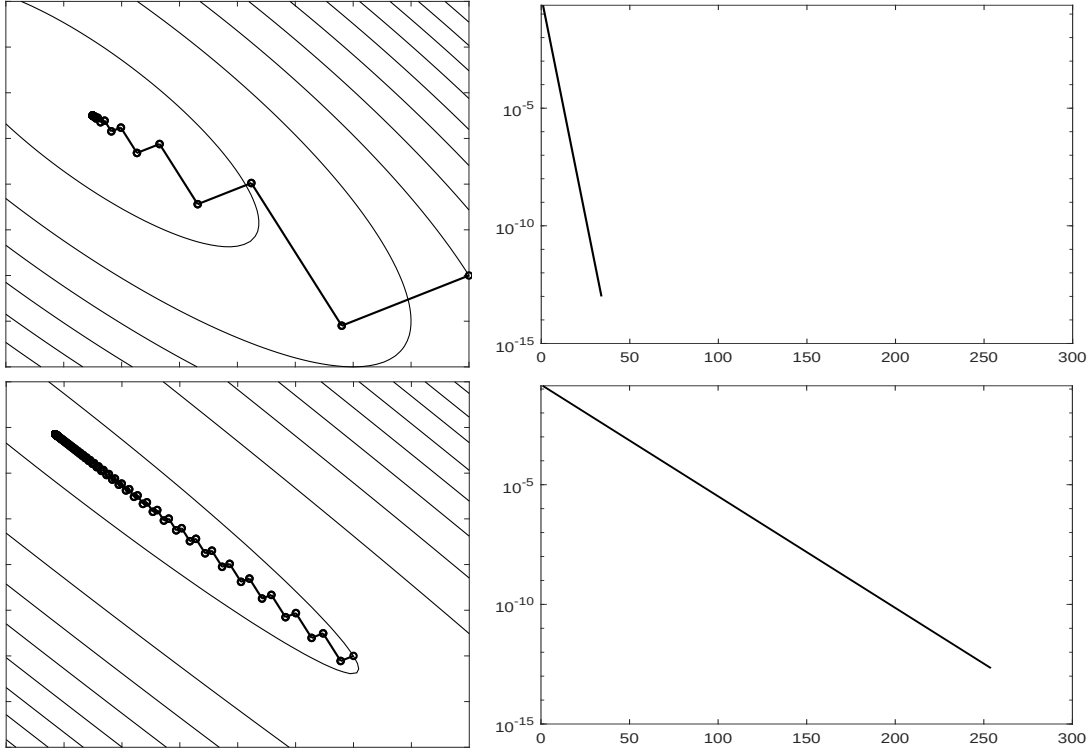


Figure 1.9: Convergence of the gradient method in two examples

Although the difference may seem minor, exponentials are very sensitive to changes of the basis. For instance, $0.99^{100} \approx 0.37$, whereas $0.96^{100} \approx 0.017$. In plain words, according to the rougher estimate 100 iterations of the method would only reduce the (absolute) gap to 37% of its initial value. However, the better estimate reveals that in fact the remaining gap will only be at most 1.7% of its initial value.

There are of course other factors at play in the actual behaviour of the algorithm, mainly the choice of the starting point x^0 and its relationship with q (which has not been mentioned before, but it clearly plays a role). For instance, the results in Figure 1.9 have been obtained with $x^0 = 0$ and $q = [2, 2]$. The apparently minor change $q = [1, 1]$ leads to the somewhat dramatic impact whereby the algorithm terminates at *just the second iteration* with $\|g^1\| \approx 3e-16$ and δ numerically 0. This is due to the fact that, by sheer chance, the optimal solution $x^* = [-0.005, -0.005]$ happen to be exactly on the line $x^0 - \alpha g^0 = -\alpha q$: thus, the minimization along this line immediately achieves $x^1 = x^*$ and the algorithm promptly terminates, even if κ is “large”. Thus, in this lucky case the algorithm “does not zig-zag” and it converges (very) efficiently.

In fact, for future reference it is worth mentioning a very special case, suggested by both (1.16) and (1.19), in which the algorithm will always terminate in one iteration: that of $\kappa = 1$, i.e., $\lambda_1 = \lambda_n$, i.e., all eigenvalues being equal. This is typically the case where Q is a scalar multiple of the identity, so let us examine $Q = I$ for simplicity. Clearly, $x^* = -Q^{-1}q = -q$. Also, $g = Qx + q = x + q$. Hence, whatever the initial guess x^0 , one has that for $i = 1$ the algorithm is minimising the tomography over the line

$$x^0 - \alpha g^0 = x^0 - \alpha(x^0 + q) .$$

Thus, $\alpha^0 = 1$ immediately provides $x^1 = x^*$ and the algorithm terminates, irrespectively of what x^0 was. This is of course not a surprising feat: due to the special properties of Q , x^* could be computed directly with $O(n)$ effort. In fact, the corresponding quadratic function is separable, and the solution could have been obtained working on each x_i individually as done in §1.3.2. Yet, it is positive that the algorithm “automatically recognises an easy case and is efficient there”. Furthermore, this simple observation will turn out to be surprisingly useful later on.

1.4.4 Some notes on the stopping criterion

Before moving on, it can be useful to remark about the stopping criterion of the algorithm. The examples above have already illustrated one significant issue, which is that the “natural” stopping criterion would be $A(x^i) \leq \varepsilon$, or—even better— $R(x^i) \leq \varepsilon$. However, such a condition cannot typically be implemented in practice for the simple reason that f_* is typically unknown.

Indeed, as already mentioned, it could be argued that “the real difficulty in optimization comes from estimating

the optimal value”. In this case this would boil down to computing *lower bounds* $\underline{f}^i \leq f_*$, so as to be able to estimate the (absolute) gap by $a^i = f^i - f_* \leq f^i - \underline{f}^i$. For this to be useful, the lower bounds should be “tight” *at least towards termination*; that is, one would want that $\{\underline{f}^i\} \rightarrow f_*$ (or some weaker convergence notion, cf. §A.5) as well. In plain words, one would have *two sequences*, of upper and lower bounds $\underline{f}^i \leq f_* \leq f^i$, which should be made to simultaneously converge to the optimal value. This is often “hard” but in fact possible in some cases, some of which will be discussed in these lecture notes, but *not* (easily) in the current one.

For the current algorithm (and, as we shall see, unconstrained optimization in general), the stopping criterion is rather that $\|g^i\|$ is “small”. This is a “proxy” of the required one, in the sense that $\|g^i\| \leq \varepsilon$ for some “small ε ” hopefully implies that $A(x^i) \leq \delta$ for some “small δ ”: however, the exact relationship between ε and δ is nontrivial. This is significant in that it is ε that has to be chosen by the user of the algorithm, and the cost of the algorithm grows as ε gets smaller: hence, it would be reasonable to use the largest ε that attains the desired gap $A(x^i)$.

This is nontrivial to do even in the current particularly simple case, where (1.13) gives

$$a^i = \frac{1}{2}(x^i - x^*)^T Q(x^i - x^*) = \frac{1}{2}\langle x^i - x^*, g^i \rangle \leq \frac{1}{2}\|g^i\| \|x^i - x^*\| \quad (1.20)$$

(using $Q(x^i - x^*) = Qx^i + q = g^i$). Thus, if one knew a bound $\Delta \geq \|x^i - x^*\|$, then $\|g^i\| \leq 2\delta / \Delta$ would imply $a^i \leq \delta$; in other words, taking $\varepsilon = 2\delta / \Delta$ in the stopping criterion would give the desired final gap. However, in general such a bound is not available. Alternatively, one can use the fact that if λ is an eigenvalue of Q then λ^2 is an eigenvalue of $Q^2 = QQ$ and (1.11) to write

$$\|g^i\|^2 = (x^i - x^*)^T Q^T Q(x^i - x^*) \geq \lambda_n^2 \|x^i - x^*\|^2 \implies \|x^i - x^*\| \leq \|g^i\| / \lambda_n ;$$

plugging this in (1.20) gives $a^i \leq \frac{1}{2}\|g^i\|^2 / \lambda_n$, and therefore using $\varepsilon = \sqrt{2\lambda_n\delta}$ in the stopping criterion would imply ensuring that $a^i \leq \delta$ at termination. However, λ_n is typically unknown, and in fact $\lambda_n = 0$ may happen, rendering the rule numerically useless. All this shows that achieving an exact control over final gap by means of choosing the ε in the stopping criterion is not obvious.

A similar, albeit different, note concerns the relationship among the gap at the current iterate and the distance from the optimum. That is, one expects that $a^i \leq \delta$ (assuming it is possible to guarantee it) implies that $\|x^i - x^*\| \leq \gamma$ for some $\gamma > 0$, but the relationships between the two thresholds is again nontrivial. This may be important in view that the overall solution process is ultimately aimed at constructing a “good approximation” of x^* , and this may have to be defined in terms of $\|x^i - x^*\|$ rather than of the gap. Exploiting the same relationships already used above readily shows that $a^i \leq \delta$ implies $\|x^i - x^*\| \leq \sqrt{\delta / \lambda_n}$, again a difficult relationship to use in practice since λ_n is usually unknown and can be 0. This again confirms that controlling the (different aspects of the) accuracy of the obtained final iterate x^i by means of properly choosing the stopping tolerance ε is nontrivial.

Exercise 1.27. Characterise the other direction, i.e., upper bound a^i in terms of $\|x^i - x^*\|$. [solution]

1.4.5 Convergence of the gradient method for $Q \succeq 0$

The previous discussions have repeatedly stressed the importance of the assumption $\lambda_n > 0$, i.e., $Q \succ 0$. This does not, however, mean that the algorithm does not converge in the positive semidefinite case: it does, but the previous analysis no longer applies. In fact, a significantly different analysis is required, which will not be detailed here since it actually is a special case of more general results [3, Theorem 3.3] that will be illustrated in some details later on. It is however worth anticipating the gist of the results, that is the following: for the version of the method with *fixed stepsize* $\alpha^i = 1 / \lambda_1$ for all i , it holds

$$A(x^i) \leq 2\lambda_1 \|x^0 - x^*\|^2 / (i + 2) . \quad (1.21)$$

As previously remarked, $1 / \lambda_1$ is the *lower bound* on the *optimal stepsize* (1.12). Hence, using the optimal stepsize leads to better (not worse) improvements at each iteration, indicating that the standard gradient method should converge faster than the fixed-stepsize version; this in fact typically happens in practice. Thus, the method is still convergent: however, its *efficiency* is (at least in theory) dramatically reduced.

Indeed, the number of iterations required to achieve accuracy δ is now (approximately)

$$i \geq \lambda_1 \|x^0 - x^*\|^2 / \delta ,$$

i.e., grows *linearly* in $1 / \delta$ instead as *logarithmically* as in the previous case. In plain words, if k iterations are required to achieve, say, $\delta = 1\text{e-}4$, it should be expected that $10000k$ iterations will be required to achieve $\delta = 1\text{e-}8$: each additional digit of accuracy requires *one order of magnitude* more iterations. Methods with this kind of efficiency can hardly be considered to obtain accurate solutions with, say, better than $1\text{e-}3 / 1\text{e-}4$ (relative) accuracy, although as always the actual performances are highly dependent on the specific problem.

Although worst-case performance analysis should always be taken with an abundant pinch of salt, in that it does not necessarily represent real-world performances, indications that “some problems may be harder to solve than others” should never be overlooked. In this case, the presence of null eigenvalues can indeed have a detrimental effect on the performances of the gradient method. This is illustrated in Figure 1.10 with reference to two specifically constructed instances with $n = 1000$.

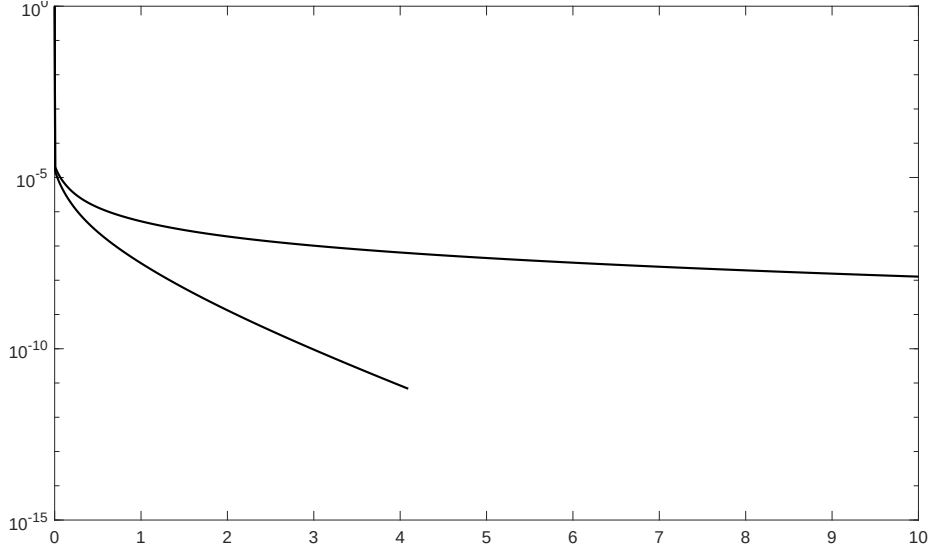


Figure 1.10: Convergence of the gradient method in two larger examples

The matrix Q of the first instance is randomly generated so as to be positive semidefinite, with around 10% of null eigenvalues. Then, q is constructed in such a way as to ensure that the problem is bounded below (by randomly generating a $\bar{x} \in \mathbb{R}^n$ and selecting $q = -Q\bar{x}$). The second instance is obtained by the first by adding to that Q a small multiple of the identity, i.e., $Q' = Q + 10I$ (note that the average of the original diagonal elements Q_{ii} is ≈ 300 , hence 10 is reasonably small in comparison). Figure 1.10 shows the two convergence plots superimposed, with an iteration limit of $1e+5$ (that is, each tick on the axis represents $1e+4$ iterations). Within this time limit, on the original instance (with $Q \succeq 0$) the gradient method reaches a relative gap of $\approx 1e-7$. The convergence curve shows a markedly *sublinear* convergence: the slope of the curve is not constant, but it gradually diminishes. In other words, the convergence speed slows down as the algorithm proceeds. This is what (1.21) suggests: at each iteration the gap is reduced, but the *ratio* $\rho^i = a^{i+1}/a^i$ is *not* constant, as in the linear case: indeed,

$$a^{i+1}/a^i \approx (i+2)/(i+3).$$

While $\rho^i < 1$, indicating convergence, $\lim_{i \rightarrow \infty} \rho^i = 1$: the fraction of the gap that the algorithm manages to reduce diminishes at each iteration. The situation is markedly different for the instance having $Q' \succ 0$: in around $4e+4$ iteration the algorithm achieves a relative gap of $\approx 1e-12$, which would have required an astronomical number of iterations to be achieved with the original semidefinite Q . The corresponding convergence plot does show an initial reduction of the slope, but after $\approx 2e+4$ iterations the convergence speed stabilises and a clear linear convergence rate ensues. This rather dramatic difference in behaviour is due to an apparently minor change in the data of the problem.

We will not delve further in these issues at this point, since this discussion is only meant to foreshadow future arguments that will be made more precise later on. However, it allows to highlight some important facts that will be found true over and over again in these lecture notes:

- achieving accurate solutions of optimization problems may be prohibitively costly;
- the difficulty of solving a problem depends on the characteristics of its data, with apparently minor changes possibly have dramatic effects.

An important remark is appropriate at this point: the above results have been obtained with one specific algorithm, and a not-really-sophisticated one at that. That a given algorithm may struggle at efficiently solving a problem does not necessarily indicate that the problem is inherently “difficult”; rather, the algorithm may not be “smart enough”. The next section will illustrate this point by presenting a “cleverer” version of the gradient method where apparently minor changes (in the algorithm, this time) translate into very substantial efficiency gains. This is *not* meant to imply that the difficulties can always be circumvented: complexity results can be proven that bound the efficiency with which certain classes of problems can be solved in general. However, the

next section does illustrate the point that one should not be discouraged too soon, as there is usually ample scope to improve the performances of approaches for specific problems.

1.5 The Conjugate Gradient Method

As the previous section has shown, ideas are needed to improve the gradient method. The intuition under the proposal of this section is that the gradient method is inefficient because it only “optimise over thin slices of the space each time”. At iteration $i = 0$, the next iterate x^1 is chosen as the best point along the line $\{x = x^0 - \alpha g^0 : \alpha \in \mathbb{R}\}$, i.e., $x^0 + \text{span}(\{g^0\})$, which is clearly a “very low dimensional (affine) subspace of $\mathbb{R}^n$ ” (even when $n = 2$, just a line in the plane). Then, the subsequent iterate x^2 is chosen as the best point in $x^1 + \text{span}(\{g^1\})$. As it was already mentioned, g^0 and g^1 are orthogonal, hence linearly independent: thus, $\text{span}(\{g^0, g^1\})$ would be a 2-dimensional subspace of $\mathbb{R}^n$, as opposed to $\text{span}(\{g^0\}) \cup \text{span}(\{g^1\})$, which remains overall a 1-dimensional space. Thus, if one were able to obtain x^2 which optimises f over all $\text{span}(\{g^0, g^1\})$, then arguably a “much better progress” would have been achieved; in particular, with $n = 2$ this would mean that the optimal solution would have been achieved. Of course this is not true for the gradient method, as the previous numerical examples show. Hence, if one wants to obtain such a property, the *search direction* d^i at each iteration, so that the next point is selected inside $x^i + \text{span}(\{d^i\})$, will have to be different from $(-)g^i$. One could of course question whether such directions actually exist at all, but the next section will show that they indeed do.

1.5.1 Q -conjugate directions

In order to introduce the desired directions we take an apparent detour. This is not strictly speaking required, but it does introduce some powerful ideas that will be repeatedly useful later on. The observation is that directions “with the right property” do surely exist in a very special case: that of a diagonal $Q \succeq 0$, i.e., $Q = \text{diag}(d)$ for some $d \in \mathbb{R}_+^n$. This is a *separable* quadratic function, and the solution procedure sketched in §1.3.2 can be re-interpreted as the following algorithm:

Algorithm 1.2. Reinterpreted Separable Quadratic Optimization

```

procedure  $x = SDQ(d, q, x)$ 
  for ( $i = 1 ; i < n ; ++i$ )
     $x \leftarrow \text{argmin} \{ f(x + \alpha u^i) : \alpha \in \mathbb{R} \};$ 

```

That is, at each iteration i the algorithm focuses on the i -th variable, which means it uses the i -th vector u^i (with all 0s except an 1 in the i -th entry) of the canonical base, and finds the optimal value for x_i . After that iteration, x_i will never be changed again. It is plain to see that the final solution is optimal. Furthermore, the algorithm does have the desired property: after k iterations the variables $x_1, \dots, x_k$ have the optimal value (while $x_{k+1}, \dots, x_n$ have the original one), i.e., *the current iterate x^k minimises the objective over $x^0 + \text{span}(\{u^1, u^2, \dots, u^k\})$.*

The example may look rather artificial and uninteresting, since one wants to work with non-diagonal Q . However, it serves to introduce the really important idea, which in fact had already been used (albeit in a simple way) in §1.2.3 and §1.3.4: *if the problem at hand is “difficult” in its natural space, find a different space in which it is “easy”.* In this particular instance, this could be declined as: *find a space where Q is diagonal.*

This is in fact possible, at least if Q is nonsingular. The relevant object is the *square root of Q* , i.e., the matrix $R = Q^{1/2}$ such that $Q = R^T R$. Such a matrix surely exists if $Q \succeq 0$, and in fact there can be many different ones. A simple way to choose a *symmetric* one is via the spectral decomposition $Q = H \Lambda H^T$, where as usual H is orthonormal: indeed, it is easy to check that

$$R = H \sqrt{\Lambda} H^T$$

has the desired property.

Exercise 1.28. Verify the last statement. **[solution]**

If Q is nonsingular, i.e., $\lambda_n > 0$, then also $\sqrt{\lambda_n} > 0$ and hence R is nonsingular as well: this allows to define the linear transformation

$$z = Rx \quad \equiv \quad x = R^{-1}z$$

which is clearly a bijection from $\mathbb{R}^n$ to $\mathbb{R}^n$. The useful property is that

$$h(z) = f(R^{-1}z) = \frac{1}{2}z^T I z + q R^{-1}z,$$

i.e., in the z -space the objective becomes separable. Thus, one could apply Algorithm 1.2 in the z -space to find the optimal solution z^* , and then map it back as $x^* = R^{-1}z^*$ to solve the original problem. Of course, the line $z + \alpha u^i$ in z -space corresponds to the line $R^{-1}(z + \alpha u^i) = (x + \alpha R^{-1}u^i)$ in x -space: hence, the algorithm can be entirely re-interpreted in x -space as moving along the directions $d^i = R^{-1}u^i$. These directions have the sough-after property, i.e., after k iterations the optimal solution on $x^0 + \text{span}(\{d^1, d^2, \dots, d^k\})$ has been found. The feat is, then, possible.

The outlined approach is not practical, in that computing $Q^{1/2}$ is no less costly than solving $Qx = -q$ in the first place. However, it does suggest what the fundamental property is: the employed directions d^i should be *all orthogonal in z -space*. That is, with v^i the direction in z -space one has $d^i = R^{-1}v^i$, i.e., $Rd^i = v^i$, and wants $\langle v^i, v^j \rangle = 0$ for all $i \neq j$. Intuitively, this leads to an efficient optimization since at every iteration k , the new direction v^k is orthogonal to $\text{span}(\{v^1, \dots, v^{k-1}\})$, and therefore “is exploring a part of the space that has never been searched through before”. Remarkably, this property is not true for the gradient method. In fact, while it is true that $\langle g^i, g^{i+1} \rangle = 0$ for all i , the property does not extend to subsequent iteration: that is $\langle g^i, g^{i+2} \rangle \neq 0$ can happen. Indeed, this surely happens when $n = 2$: since g^i and g^{i+2} are both orthogonal to g^{i+1} , they are necessarily parallel to each other (cf. Figure 1.9), which leads to the detrimental “zig-zagging” phenomenon. Ensuring orthogonality in z -space means $0 = \langle v^i, v^j \rangle = \langle Rd^i, Rd^j \rangle = d^i Q d^j$: directions with this property are called *Q -conjugate*.

It is then possible to prove that using any n Q -conjugate directions in x -space leads to an algorithm that terminates in n iterations. This is still glossing over on how these directions are obtained, which will be discussed later on. Assuming that $p^i \neq 0$, $i = 1, 2, \dots, n$, are Q -conjugate directions, it is easy to see that they are necessarily *linearly independent* from each other, and therefore they form a basis of $\mathbb{R}^n$.

Exercise 1.29. Verify the linear independence. [solution]

One then considers the following abstract algorithm: with arbitrary x^0 , compute $\alpha^i = \text{argmin}\{\varphi_{x^i, p^i}(\alpha) : \alpha \in \mathbb{R}\}$ and set $x^{i+1} = x^i + \alpha^i p^i$. It will now be proven that $x^n = x^*$, i.e., the algorithm terminates in n steps.

Since the p^i are a base of $\mathbb{R}^n$, “the optimal solution is reachable from the initial estimate by moving along the p^i ”; formally speaking, there exist some $\sigma^i \in \mathbb{R}$, $i = 1, \dots, n$, such that $x^* = x^0 + \sum_{i=1}^n \sigma^i p^i$. Let $g^i = Qx^i + q$ be the gradient at iterate x^i of the process: we want to prove that

$$\sigma^i = \alpha^i = \text{argmin}\{\varphi_{x^i, p^i}(\alpha)\} = -\langle g^i, p^i \rangle / ((p^i)^T Q p^i)$$

i.e., that the *optimal stepsizes* produced by minimizing the tomography precisely provide the multipliers σ^i . This immediately proves that the algorithm finds x^* in n steps.

Exercise 1.30. Prove the optimal stepsize formula. [solution]

To prove the claim, fix any $k = 1, \dots, n$ and multiply $x^* - x^0$ by $(p^k)^T Q$: then,

$$(p^k)^T Q(x^* - x^0) = (p^k)^T Q(\sum_{i=1}^n \sigma^i p^i) = \sigma^k (p^k)^T Q p^k$$

(using Q -conjugacy, i.e., $(p^k)^T Q p^i = 0$ if $i \neq k$). Similarly, multiply $x^k = x^0 + \sum_{i=1}^{k-1} \sigma^i p^i$ by $(p^k)^T Q$ to obtain

$$(p^k)^T Q(x^k - x^0) = (p^k)^T Q(\sum_{i=1}^{k-1} \sigma^i p^i) = 0$$

(again, using Q -conjugacy). Putting all together,

$$\begin{aligned} \sigma^k (p^k)^T Q p^k &= (p^k)^T Q(x^* - x^0) = (p^k)^T (-q) - (p^k)^T Q x^0 \\ &= (p^k)^T (-q) - (p^k)^T Q x^k = -(p^k)^T (Q x^k + q) = -\langle g^k, p^k \rangle \end{aligned}$$

which proves the claim. Thus, the algorithm works, provided one is able to generate n Q -conjugate directions. Unsurprisingly, the eigenvectors of Q would do the job, but they are not generally available.

Exercise 1.31. Prove that the eigenvectors are Q -conjugate. [solution]

Fortunately, a very simple formula is available to construct the directions p^i iteratively, just at the moment when they are needed.

1.5.2 The conjugate gradient method for quadratic functions

The *conjugate gradient method* for quadratic functions is based on the following result: the i -th Q -conjugate direction can be obtained by properly *deflecting* the gradient at the i -th iterate g^i and the *previous* direction p^{i-1} (taken as 0 when $i = 0$). More formally, $p^0 = g^0$ and

$$p^i = -g^i + \beta^i p^{i-1} \quad \text{where} \quad \beta^i = \langle g^i, p^{i-1} \rangle / ((p^{i-1})^T Q p^{i-1}) \quad (1.22)$$

for $i > 0$. This is known as the *Fletcher-Reeves formula*. The fundamental property is the following: if the stepsize α^i is chosen as $\operatorname{argmin}\{\varphi_{x^i, p^i}(\alpha) : \alpha \in R\}$, i.e., $\alpha^i = -\langle g^i, p^i \rangle / ((p^i)^T Q p^i)$, then the directions p^i generated in this way are Q -conjugate. The proof, by induction, is somewhat long and tedious and is not reported here; it can be found, e.g., in [10, Theorem 5.2, 5.3]. All in all, this defines the following algorithm:

Algorithm 1.3. Conjugate Gradient for Quadratic Function

```

procedure  $x = CGQ(Q, q, x, \varepsilon)$ 
 $d^- \leftarrow 0$ ;
do forever
   $g \leftarrow Qx + q$ ;
  if ( $\|g\| \leq \varepsilon$ ) then break;
  if ( $d^- = 0$ ) then  $d \leftarrow -g$ ;
  else {  $\beta = (g^T d^-) / ((d^-)^T Q d^-)$ ;  $d \leftarrow -g + \beta d^-$ ; }
   $\alpha \leftarrow -\langle g, d \rangle / (d^T Q d)$ ;  $x \leftarrow x + \alpha d$ ;  $d^- \leftarrow d$ ;

```

The naïve statement of the algorithm contains several matrix-vector products with Q , each of which costs $O(n^2)$. However, the following alternative formulæ

$$\beta^i = \|g^i\|^2 / \|g^{i-1}\|^2, \quad \alpha^i = \|g^i\|^2 / ((d^i)^T Q d^i) \quad (1.23)$$

[10, p. 11] and further appropriate streamlining along the lines of Exercise 1.19 makes it possible to implement the algorithm with approximately the same iteration cost as the gradient method: only one $O(n^2)$ matrix-vector product plus a smattering of $O(n)$ vector operations. The algorithm is therefore going to be competitive if it takes (many) fewer iterations than the gradient method.

Differently from the gradient method, the conjugate gradient algorithm enjoys *finite convergence* in n steps, since it is precisely constructed to achieve this result. However, this is only true in infinite-precision arithmetic, and therefore not in practical implementations. The numerical errors inherent in the use of floating-point arithmetic make it so that the algorithm may require more than n iterations to achieve an approximately optimal solution; on the other hand, depending on the ε of the stopping criterion it may also terminate in less than n iterations. This is fortunate, since for large-scale problems n is not a “small” number of iterations: indeed, such a convergence result is clearly *not dimension independent*, and therefore paradoxically the gradient method may turn out to be preferable (at least for well-conditioned Q) as n grows large. This is generally not the case, as several powerful efficiency results can be proven for the conjugate gradient that show that the method should in general be expected to be significantly more efficient than the gradient method. We will briefly outline the main ones of them, without any proof: these can be found, e.g., in [10, p. 112+] and the references therein, but they are too complex to be reproduced here.

- The convergence rate of the algorithm is at least linear [10, (5.36)], i.e.,

$$a^i \leq [(\sqrt{\kappa} - 1) / (\sqrt{\kappa} + 1)]^i (2a^0).$$

Although the convergence rate still deteriorates as the conditioning increases, it is already significantly better than that of the gradient method. Disregarding the extra factor, the base $r = (\sqrt{\kappa} - 1) / (\sqrt{\kappa} + 1)$ of the exponential is significantly smaller than that of (1.19) due to the fact that $\sqrt{\kappa} < \kappa$. For $\kappa = 1000$, for instance, (1.19) yields $r \approx 0.996$, while the current formula yields $r \approx 0.88$, and the difference is very significant. Indeed, while $0.96^{100} \approx 0.017$, $0.88^{100} \approx 3\text{e-}6$: for the same Q , 100 iterations would give around two significant digits with the gradient method and around six with the conjugate gradient one.

- In fact, the convergence depends on “more than the largest and the smallest eigenvalue”, but on the *whole distribution of the eigenvalues*. The fundamental result [10, Theorem 5.5] is

$$a^i \leq [(\lambda_{i+1} - \lambda_n) / (\lambda_{i+1} + \lambda_n)] a^0,$$

which basically states that “ a^i converges to 0 as fast as the (finite, decreasing) sequence of eigenvalues $\{\lambda_i\}$ converges to its smallest element λ_n ”. This implies, for instance, that if Q has *only k distinct eigenvalues*, then the algorithm terminates in k iterations [10, Theorem 5.4]. Eigenvalues need not to be strictly identical for the result to apply: for instance, if Q has k eigenvalues *clustered around 1*, then the algorithm can be expected to terminate—or at least provide a highly accurate solution—in $n - k$ iterations [10, p. 116].

Thus, the conjugate gradient algorithm can be expected to be more efficient than the gradient method, especially if Q has *some structure*, meaning for instance that it has several identical, or at least closely clustered, eigenvalues.

An illustration of the superior practical efficiency of the conjugate gradient approach is given in Figure 1.11 on

the same two instances with $n = 1000$ of Figure 1.10. The difference is striking. Even in the case where $Q \succeq 0$ (around 10% of null eigenvalues), the conjugate gradient reaches $\approx 1\text{e-}15$ in less than 200 iterations, where the gradient method was still at $1\text{e-}8$ after 100000 iterations. In the positive definite case, a similarly highly accurate solution is reached in about 40 iterations, i.e., *three orders of magnitude less* than those the gradient needed to reach $1\text{e-}12$. In both cases the convergence plot shows signs of *superlinear* convergence, whereby the slope of the curve becomes *steeper* as the iterations progress, or at least of very good linear one (with $r \ll 1$), as the theory suggests.

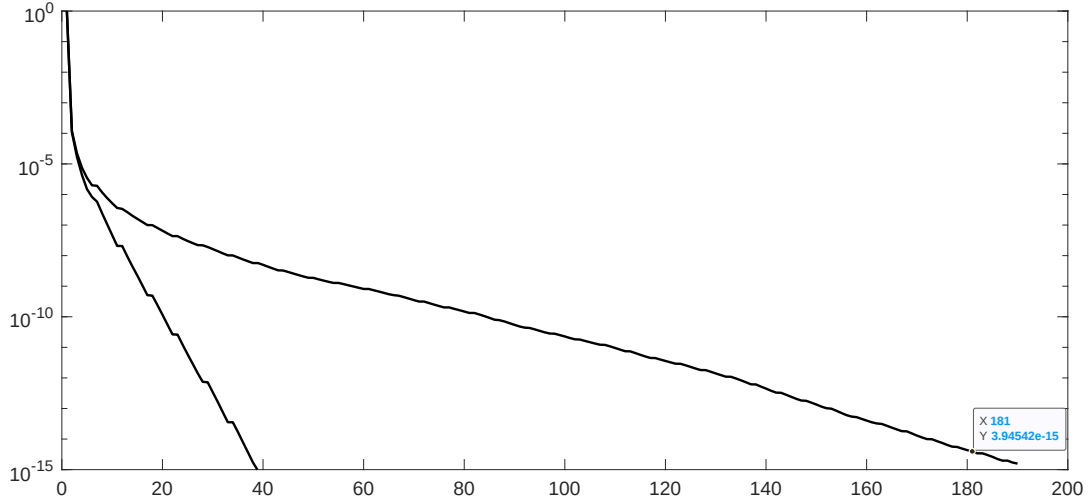


Figure 1.11: Convergence of the conjugate gradient method in two larger examples

1.5.3 Preconditioning the Conjugate Gradient method

Although the conjugate gradient method is already quite (more) efficient (than the gradient method), there can be ways to make it even more efficient. We will rapidly sketch one of these ways, primarily to stress that 1) some powerful ideas will keep showing their worth over and over again, and 2) that there is often ample scope to improve the performances of optimization algorithms exploiting specific forms of *structure* on the instances at hand—or, perhaps more to the point, “forcefully imposing” the structure on the instance.

In this case, the previous analysis has shown that the conjugate gradient method would behave much better if all eigenvalues were clustered around 1. In fact, we have already seen in §1.5.1 that this can be done with an appropriate change in the variables space. We now generalise the approach by selecting any *nonsingular* matrix P and defining the linear bijection in the usual way, i.e., $z = Px \equiv x = P^{-1}z$. The objective function is transformed into

$$h(z) = f(P^{-1}z) = \frac{1}{2}z^T[P^{-T}QP^{-1}]z + qP^{-1}z.$$

The choice $P = R = \sqrt{Q}$ would yield $P^{-T}QP^{-1} = I$, i.e., all eigenvalues identical to 1: the best possible scenario for the conjugate gradient, that would solve the problem—in z -space—in one iteration. In fact, even the humble gradient method would: the first iteration of the two coincide. Yet, as already mentioned, computing $P = R$ and inverting it is in general as costly as solving $Qx + q = 0$ outright, hence again the naïve approach would not be convenient.

Yet, this suggests a clever idea: choose “ $P \approx Q$ ” such that computing P^{-1} is “cheap”. It is outside of the scope of these lecture notes to go in details of what “ $P \approx Q$ ” should exactly mean, but the previous section suggests that what one wants is that $P^{-T}QP^{-1}$ has more identical/clustered eigenvalues. Such a P is called *preconditioner*, on the grounds that, by hopefully “compressing the range of eigenvalues”, it could reduce the conditioning number κ . Applying the conjugate gradient method to the *preconditioned problem*—modified using P —is called the *preconditioned conjugate gradient method*. This can be done without really changing Q and q , but rather streamlining the algorithm so that only extra products between vectors and P^{-1} are required [10, Algorithm 5.3]. It is thus highly convenient if P^{-1} , besides being “cheap to compute”, is also *sparse*.

There is a huge literature about constructing preconditioners for problems with very specific structure that is entirely out of the scope for these lecture notes. A very common and simple “general-purpose” preconditioner is obtained by the *Cholesky factorization* procedure, briefly described in §1.6, that could be used to solve the system outright—basically, computing a “perfect preconditioner”. The *incomplete Cholesky* preconditioner [10, p. 120] is obtained by forcibly dropping (many of the) elements of the factorisation so that it is (much) sparser

and cheaper to use. Rather than going into the (somewhat intricate) details of the method, we briefly describe its “extreme version”: the *diagonal preconditioner* $P = \sqrt{\text{diag}(Q)}$. Basically, one pretends that Q is diagonal and constructs the preconditioner only looking at the diagonal elements. P^{-1} is therefore extremely cheap to compute and extremely sparse. An issue is that P may be singular ($Q_{ii} = 0$ for some i , which implies $Q \not\succeq 0$), but easy heuristics can be used to skirt around this case.

Exercise 1.32. Propose a heuristic to modify the diagonal preconditioner if $Q \not\succeq 0$. [solution]

The effectiveness of such a “poorman’s preconditioner” cannot be expected to be extremely high; it is mostly dependent on how much *diagonally dominant* Q is, a notion that will not be discussed in details but that, roughly speaking, measures “how much the diagonal weighs in the context of the whole matrix”. Obviously, a diagonal matrix is completely diagonally dominant, and a matrix where the diagonal elements are much larger than the non-diagonal ones will tend to be highly diagonally dominant. The effect of the diagonal preconditioner can be expected to be significant on highly diagonally dominant matrices and to decrease, up to basically vanishing, on less and less diagonally dominant Q . An illustration of the effect of preconditioning is given in Figure 1.12 for a problem with $n = 1000$ where Q is “mildly diagonally dominant” (diagonal elements are, on average, between two and three times the non-diagonal ones). Both the gradient method and the conjugate gradient method are tested, with and without preconditioning. All instances were solved to a gap of $\approx 1\text{e-}15$.

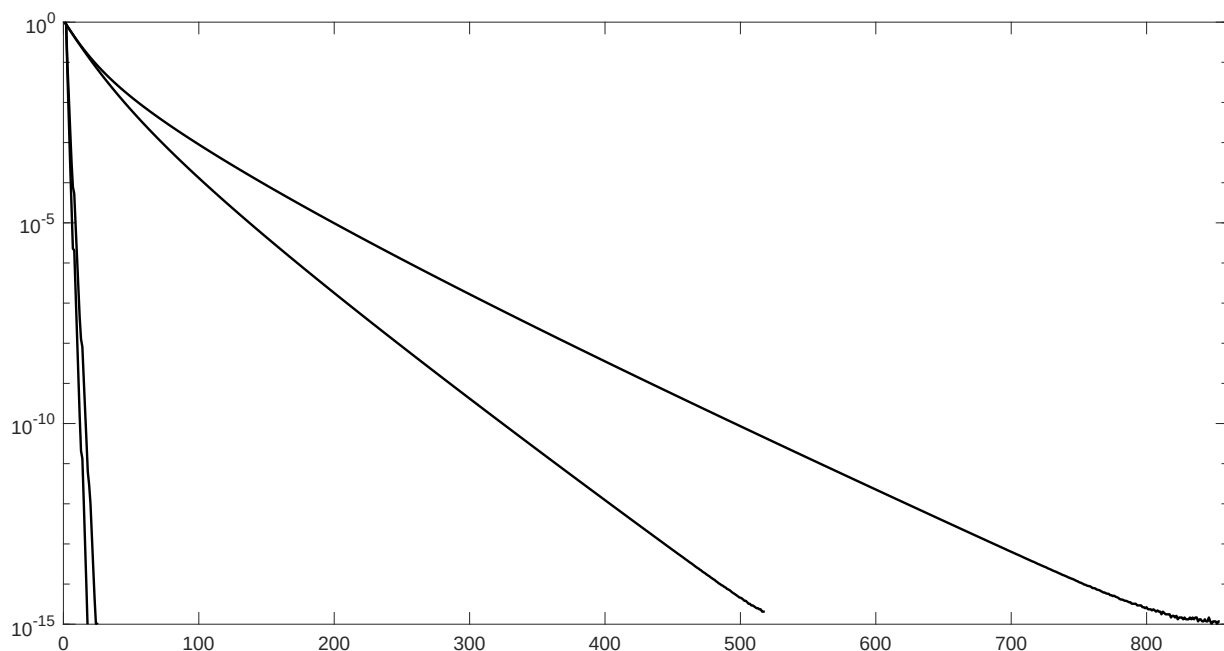


Figure 1.12: Convergence of preconditioned/or not conjugate/or not gradient methods

The example shows that preconditioning can be effective in the gradient method, too. In fact, while the original condition number of Q is $\kappa = 2\text{e}+3$, the condition number of $P^{-1}QP^{-1}$ is $\kappa = 1.4\text{e}+3$. This difference is enough to decrease the required number of iterations from 854 to 518: decreasing the condition number by 30% reduces the number of iterations by 40%, roughly in accordance with the theory. The non-preconditioned conjugate gradient is way faster, solving the problem in only 25 iterations. Yet, the preconditioned version is even faster, requiring only 17 ones; yet another reduction of 30%. This unique example does not pretend to be illustrative of the trade-offs (cost for computing/using the preconditioner vs. decrease in iterations) of the many proposed preconditioned techniques, but just to hint at the fact that appropriate exploitation of the structure of the problem—in this case, of Q —may lead to improved performances. Much more sophisticated ways of exploiting the structure exist, even for problems as simple as the one at hand; a few will be hinted at later on in these lecture notes.

1.6 Multivariate Quadratic optimization: a Direct Method

Since the very simple optimization problem at hand reduced to the solution of the linear system $Qx = -q$, it is worth exploring *direct methods* that aim at producing the solution in one blow. These will no longer be an option on more complex problems, but the techniques discussed here will find several uses down the line. There is a huge amount of sophisticated theory about direct methods for linear systems, e.g., [10, Algorithm A.1, A.2],

and many efficient readily available implementations (`lapack`, ...). In these lecture notes only one technique will be discussed in some detail.

Assuming $Q \succ 0$, the solution to the system can just be written as $x = -Q^{-1}q$. However, in practice computing inverses is not the preferred way for solving systems; rather, *factorizations* are considered to be the better options. In the case of symmetric positive semidefinite matrices, the *Cholesky*—a.k.a. *LLT*—factorization is usually the go-to technique, although not the only possible one. It is based on rewriting

$$Q = LL^T$$

(whence the name) where L is a *lower triangular* matrix. This is clearly related to the preconditioning idea of the previous section: the “perfect preconditioner” would be such that $P^{-T}QP^{-1} = I$, i.e., $Q = P^T P$. However, the L sought after here is clearly not the “abstract” $P = H\sqrt{\Lambda}H^T$; not only because it is nontrivial to compute, but especially because rationale of the Cholesky factorization is that solving linear systems with triangular matrices, be them lower or upper, is “very easy”, as recalled below. This implies that, once the factorization is computed (which is the costly part), the system can be efficiently solved with the two-step process

$$\text{solve } Lv = -q \quad \text{and then solve } L^T x = v.$$

It is worth mentioning that, once L is known, one can basically assume to have L^{-1} available as well, in the sense that “having available” usually means being able to (efficiently) compute matrix-vector products with it. Here, $z = L^{-1}v$ and $z = L^{-T}w$ are clearly available by solving the systems $Lz = v$ and $L^T z = w$, which, as shown next, is “cheap”.

1.6.1 Backsolve

The backsolve procedure for solving a linear system with a (lower or upper) triangular matrix is really trivial, and it is better illustrated by a simple example.

Example 1.4. Backsolve

We have

$$Q = \begin{bmatrix} 4 & 8 & 4 \\ 8 & 25 & 2 \\ 4 & 2 & 12 \end{bmatrix} = LL^T \quad \text{where} \quad L = \begin{bmatrix} 2 & 0 & 0 \\ 4 & 3 & 0 \\ 2 & -2 & 2 \end{bmatrix}$$

(how to compute L out of Q will be seen later on). We want to solve

$$Lx = \begin{bmatrix} 2 & 0 & 0 \\ 4 & 3 & 0 \\ 2 & 2 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 9 \\ 4 \end{bmatrix} = b \quad \text{i.e.} \quad \begin{cases} 2x_1 & & & = & 6 \\ 4x_1 & +3x_2 & & = & 9 \\ 2x_1 & -2x_2 & +2x_3 & = & 4 \end{cases}.$$

Obviously, the first equation immediately gives $x_1 = 6/2 = 3$. Then, the second reduces to

$$4(3) + 3x_2 = 9 \quad \equiv \quad 3x_2 = -3 \quad \equiv \quad x_2 = -1.$$

Similarly, the third gives

$$2(3) - 2(-1) + 2x_3 = 4 \quad \equiv \quad 2x_3 = -4 \quad \equiv \quad x_3 = -2.$$

At each step, all variables save the rightmost one (on the diagonal), if any, have already been fixed; then, it's a trivial matter to compute the remaining one.

Only slightly more formally, at the first step $L_{11}x_1 = b_1$ gives $x_1 = b_1 / L_{11}$. Then, at the k -th step, the variables $x_1, \dots, x_{k-1}$ are known; hence

$$\sum_{i=1}^k L_{ki}x_i = b_k \quad \equiv \quad L_{kk}x_k + \sum_{i=1}^{k-1} L_{ki}x_i = b_k \quad \equiv \quad x_k = (b_k - \sum_{i=1}^{k-1} L_{ki}x_i) / L_{kk}.$$

This is just repeated for $k = 2, \dots, n$; each iteration has a $O(n)$ cost, which means that the whole backsolve is $O(n^2)$. It is completely obvious how the approach is applied to L^T , i.e., in the upper triangular case: just proceed backwards for $k = n, n-1, \dots, 1$.

It is apparent how the whole procedure hinges on the fact that $L_{ii} \neq 0$ (in fact, $L_{ii} > 0$) for all $i = 1, \dots, n$; numerically, then, there could be issues if $L_{ii} \approx 0$ for some i , which means (as it will be clear in the following) that Q is “almost singular”, i.e., some of its eigenvalues are “close to 0”. Apart from this, the procedure is as efficient as it can get. In fact, if L has $m \leq n(n+1)/2$ nonzero entries, then the total cost is $O(m)$ (sums and products, plus $O(n)$ divisions); thus, the procedure readily exploits *sparsity* ($m \ll n(n+1)/2$).

1.6.2 Cholesky factorization

Once backsolve is established, the Cholesky factorization process becomes easy. Again, an example is the efficient way to present it.

Example 1.5. Backsolve

We want to determine

$$L = \begin{bmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{bmatrix} \quad \text{such that} \quad Q = \begin{bmatrix} 4 & & \\ 8 & 25 & \\ 4 & 2 & 12 \end{bmatrix} = LL^T$$

which means

$$\begin{bmatrix} 4 & & \\ 8 & 25 & \\ 4 & 2 & 12 \end{bmatrix} = \begin{bmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{bmatrix} \begin{bmatrix} L_{11} & L_{21} & L_{31} \\ 0 & L_{22} & L_{32} \\ 0 & 0 & L_{33} \end{bmatrix} = \begin{bmatrix} L_{11}^2 & & \\ L_{11}L_{21} & L_{21}^2 + L_{22}^2 & \\ L_{11}L_{31} & L_{21}L_{31} + L_{22}L_{32} & L_{31}^2 + L_{32}^2 + L_{33}^2 \end{bmatrix}$$

(for symmetry only the lower triangular parts need be considered). This is a system of $n(n+1)/2 = 6$ *nonlinear* (containing products of variables, and therefore “quadratic”) equations in the L_{ij} variables. As such it is nontrivial to solve; however, the observation is that *if the equations are approached in just the right order they can be solved as linear ones*:

1. This of course starts with the top-left equation: $L_{11}^2 = 4$ immediately gives $L_{11} = \sqrt{4} = 2$.
2. Since now L_{11} is known, the system corresponding to Q_{21} is $L_{11}L_{21} = 2L_{21} = 8$, yielding $L_{21} = 4$.
3. This paves the way for solving the system corresponding to Q_{22} , i.e., $L_{21}^2 + L_{22}^2 = 16 + L_{22}^2 = 25$, yielding $L_{22} = \sqrt{25 - 16} = 3$.

So far things have been simple, with only one variable at the time involved; but in general the process is more complex and it involves linear systems. In fact, we have now to consider together *both* positions $(3, 1)$ and $(3, 2)$ because they both depend on the variable L_{31} . That is, one has to consider the system

$$\begin{cases} L_{11}L_{31} & = & 4 \\ L_{21}L_{31} + L_{22}L_{32} & = & 2 \end{cases} \quad \equiv \quad \begin{cases} 2L_{31} & = & 4 \\ 4L_{31} + 3L_{32} & = & 2 \end{cases} \quad \equiv \quad \begin{bmatrix} 2 & 0 \\ 4 & 3 \end{bmatrix} \begin{bmatrix} L_{31} \\ L_{32} \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}.$$

This is, however, a lower triangular system and therefore it can be efficiently (back)solved, yielding $[L_{31}, L_{32}] = [2, -2]$. This finally put us in the position of considering the $(3, 3)$ position:

$$12 = L_{31}^2 + L_{32}^2 + L_{33}^2 = 4 + 4 + L_{33}^2 \quad \equiv \quad L_{33} = \sqrt{12 - 8} = 2.$$

Thus, in general the process requires backsolves to compute the off-diagonal elements of L , one row at a time, and then a square root extraction to compute the diagonal ones.

In a more formal way, after having done the trivial “base case” of $L_{11} = \sqrt{Q_{11}}$, the general process looks as follows. At the generic k -th iteration ($k > 1$), one has

$$\begin{bmatrix} L_{-k} & 0 & \dots \\ L_k & L_{kk} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} L_{-k}^T & L_k^T & \dots \\ 0 & L_{kk} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} = \begin{bmatrix} L_{-k}L_{-k}^T & L_{-k}L_k^T & \dots \\ L_kL_{-k}^T & L_{kk}^2 + L_k^TL_k & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} = \begin{bmatrix} Q_{-k} & Q_k^T & \dots \\ Q_k & Q_{kk} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

Here, L_{-k} is the $(k-1) \times (k-1)$ principal submatrix of L , that has been already correctly computed in the previous $k-1$ iterations: that is, $L_{-k}L_{-k}^T = Q_{-k}$, where Q_{-k} is the $(k-1) \times (k-1)$ principal submatrix of Q . One then wants to compute the k -th row of L , distinguishing the $k-1$ off-diagonal elements, L_k , from the diagonal one L_{kk} .

For the former, it is immediate to see that the nonlinear system requires $L_{-k}L_k^T = Q_k^T$, where Q_k are the $k-1$ off-diagonal elements of the k -th row of Q : since L_{-k} is known, this can be (back)solved in $O(k^2)$. Once this is done, for the diagonal element one has

$$L_k^TL_k + L_{kk}^2 = Q_{kk} \quad \equiv \quad L_{kk} = \sqrt{Q_{kk} - \|L_k\|^2}$$

and the process has been iterated. Since it takes n iterations, each of which $O(n^2)$, the total cost is $O(n^3)$.

A variant of the procedure produces the so-called LDLT factorization, where $Q = LDL^T$, L is a lower triangular matrix with all 1s on the diagonal, and D is a diagonal matrix (with all non-negative diagonal entries). This basically requires the same memory (the diagonal elements of L , being all 1s, do not need to be stored and therefore the corresponding n memory locations can be used for the diagonal elements of D) and avoids taking square roots. This is reminiscent of the spectral decomposition $Q = H\Lambda H^T$, but obviously different; yet, sometimes the diagonal elements of D are used as approximation of the eigenvalues. In terms of solving systems, the LDLT factorization is of course also as efficient as the LLT one; both are numerically stable, i.e., they provide numerically accurate solutions (insomuch as this is possible) even if Q is ill-conditioned. Many other interesting results on the Cholesky factorization can be found in the literature, starting from [60].

It is easy to see that the numerically delicate part of the procedure is the square root extraction when computing the diagonal element. This first of all requires $Q_{kk} \geq \|L_k\|^2$; this property is guaranteed to hold for $Q \succeq 0$, but it breaks if a negative eigenvalue exists. In fact, if $Q_{kk} < \|L_k\|^2$ happens at some k the procedure is stopped declaring that the decomposition does not exist. Of course, the square root of a negative number does exist if complex numbers are allowed, but this is of no interest for these lecture notes. Rather, the case where some eigenvalue is 0 merits some discussion. This is revealed precisely by the occurrence that $Q_{kk} = \|L_k\|^2$, yielding a null diagonal element $L_{kk} = 0$ in the factor. Of course numerical safeguards have to be put in place in this

context, as there is no such thing as a check “ $Q_{kk} = \|L_k\|^2$ ” in floating-point arithmetic, but we assume this done without further details. The issue is rather that such an occurrence actually “breaks the process”, in that subsequent iterations rely on backsolves with L_{-k} , but these are ill-defined if some diagonal elements of the factor are 0.

There clearly is, however, a case where this is not an issue: that of $k = n$. In this case, the result is “just” that the last diagonal element of L is null. For instance, it is easy to see that

$$Q = \begin{bmatrix} 4 & 8 & 4 \\ 8 & 25 & 2 \\ 4 & 2 & 8 \end{bmatrix} = LL^T \quad \text{where} \quad L = \begin{bmatrix} 2 & 0 & 0 \\ 4 & 3 & 0 \\ 2 & -2 & 0 \end{bmatrix}.$$

The issue of course is that there is no guarantee that this will only happen at the last iteration. However, a simple and elegant trick can be used to solve the conundrum: *if $Q_{kk} = \|L_k\|^2$ happens, swap the k -th row (and column) of Q with the n -th*. Indeed, for the purpose of solving the linear system $Qx = -q$, the order of the rows of Q is irrelevant and can be permuted arbitrarily (provided that, of course, the entries of q are permuted in the same way). Since Q is symmetric, swapping two rows requires swapping the two corresponding columns as well, but this only entails at permuting the corresponding entries of the solution vector x , so that the original solution can still be efficiently recovered (provided that the final permutation is stored, which can be done efficiently). Thus, if $Q_{kk} = \|L_k\|^2$ happens *just once*, even if for $k < n$, the situation can be handled.

This suggests a way in which the Cholesky factorisation could be extended to the singular case. The idea will only be roughly sketched since this is *not* the approach that is usually advised (e.g., [10, p. 610]), although the two share the fact that they perform row exchanges “when things go badly”. The idea is that the previous result, working for just one row, can be extended to finding a *lower trapezoidal factorization* of Q : after a permutation of the rows and columns of Q , the matrix can be rewritten in such a way that

$$Q = \begin{bmatrix} Q_{++} & Q_{0+}^T \\ Q_{0+} & Q_{00} \end{bmatrix} = LL^T \quad \text{where} \quad L = \begin{bmatrix} L_{++} & 0 \\ L_{0+} & 0 \end{bmatrix},$$

where $Q_{0+} \in \mathbb{R}^{h \times (n-h)}$ with h being the number of null eigenvalues, and $Q_{++} \in \mathbb{R}^{(n-h) \times (n-h)}$ is nonsingular. Obviously, $n - h$ is the number of strictly positive eigenvalues. This is also called a *rank revealing Cholesky factorisation* in that it explicitly reveals the rank of Q , i.e., the number of nonzero eigenvalues.

Exercise 1.33. Prove the existence of the rank revealing Cholesky factorisation. [solution]

Computing such lower trapezoidal factorization is simply a matter of performing appropriate row and column exchanges. That is, at each step of the process one has found a $k \times k$ nonsingular triangular factor L_{-k} of the first k rows of Q (possibly, after permutations performed in previous iterations), and then h subsequent rows that so far give rise to the rank-deficient part, i.e.,

$$P = \begin{bmatrix} L_{-k} & 0 \\ L_h & 0 \end{bmatrix} \quad \text{with} \quad PP^T = \begin{bmatrix} L_{-k}L_{-k}^T & L_{-k}L_h^T \\ L_hL_{-k}^T & L_hL_h^T \end{bmatrix} = \begin{bmatrix} Q_{-k} & Q_h^T \\ Q_h & Q_{hh} \end{bmatrix}.$$

At the beginning $k = h = 0$ and everything is empty. One then proceeds at examining row $k+1+h$, computing $L_{k+1+h} = Q_{k+1+h}L_{-k}^{-T}$ (by backsolve), with Q_{k+1+h} indicating the first k entries of the $(k+1+h)$ -th row of Q (empty if $k = 0$), and then $L_{k+1+h,k+1+h} = Q_{k+1+h,k+1+h} - \|L_{k+1+h}\|^2$. If $L_{k+1+h,k+1+h} = 0$ (note that this can happen at the first iteration if $Q_{11} = 0$) then h is increased by 1 and the process repeated. Otherwise, row (and column) $k+1+h$ of Q is swapped with row (and column) $k+1$, so that $[L_{k+1+h}, L_{k+1+h,k+1+h}]$ becomes the next row $[L_{k+1}, L_{k+1,k+1}]$ of the “nonsingular part” of the factorization, and k is increased by 1. Notice that in this case all the elements $L_{i,k+1}$ for $i = k+1, \dots, k+h-1$ need be computed to keep the invariant that $L_h \in \mathbb{R}^{h \times k}$ when k is increased. It is however obvious how to do that efficiently. The process terminates in n steps when $k+h = n$ having produced the required rank-revealing factorization with an overall cost of $O(n^3)$. All in all, the Cholesky factorization “just works” for Q having an arbitrary number of null eigenvalues, at the cost of accepting an all-zero bottom-left block in the factor.

1.6.3 Using the Cholesky factorization

If $Q \succ 0$ the Cholesky factorization can be computed (albeit at a rather high cost when n is large) and used to solve the $Qx = -q$ system, and hence the optimization problem, as previously discussed. It is interesting to detail, however, what happens when $Q \succeq 0$. As seen in §1.3.4, this may lead to two different outcomes:

1. $Qx = -q$ has infinitely many solutions, each of which is optimal for the optimization problem;
2. $Qx = -q$ has no solution, which means that the optimization problem can be proven unbounded below.

We will now discuss how the Cholesky factorization, in its lower-trapezoidal (rank-revealing) form

$$Q = \begin{bmatrix} L_{++} & 0 \\ L_{0+} & 0 \end{bmatrix} \begin{bmatrix} L_{++}^T & L_{+0}^T \\ 0 & 0 \end{bmatrix},$$

allows to precisely characterize which of the two cases happens, as well as “all the ways in which the problem can be optimal / unbounded below”. As in the nonsingular version, the process starts with solving the system

$$Lv = -q \quad \equiv \quad \begin{bmatrix} L_{++} & 0 \\ L_{0+} & 0 \end{bmatrix} \begin{bmatrix} v_+ \\ v_0 \end{bmatrix} = - \begin{bmatrix} q_+ \\ q_0 \end{bmatrix}.$$

In such a system, the entries of v_0 are always multiplied by 0 and therefore play no role: in fact only those of v_+ matter. The system is therefore *overdetermined*: it has n constraints in $n - k$ variables. Yet, it may still have a solution: to check if this is the case it is sufficient to compute

$$\bar{v}_+ = -L_{++}^{-1}q_+$$

that “solves the first part of the system”, and check whether

$$L_{0+}\bar{v}_+ = -q_0 \tag{1.24}$$

holds or not. This distinguishes the two cases.

In fact, if (1.24) holds, the original system has infinitely many solutions, and therefore the optimization problem has infinitely many optimal solutions. These can be fully characterised as the points satisfying

$$\begin{bmatrix} L_{++}^T & L_{+0}^T \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x_+ \\ x_0 \end{bmatrix} = \begin{bmatrix} \bar{v}_+ \\ 0 \end{bmatrix}$$

(since the entries of v_0 are irrelevant they can as well taken to be 0). Such a system is *underdetermined*: it has $n - k$ constraints (the last k ones being always verified) in n variables, and therefore it has infinitely many solutions, i.e., all vectors of the form

$$\begin{bmatrix} x_+ \\ x_0 \end{bmatrix} = \begin{bmatrix} -L_{++}^{-T}(q_+ + L_{+0}^T x_0) \\ x_0 \end{bmatrix} \quad \text{however chosen } x_0 \in \mathbb{R}^k,$$

as these are all and only the vectors that satisfy $Qx = -q$.

If, conversely, (1.24) does not hold, then the system has no solution: the problem is unbounded below, i.e., there exist directions $\xi \in \mathbb{R}^n$ such that $Q\xi = 0$ but $\langle q, \xi \rangle \neq 0$, so that the tomography $\varphi_{0,\xi}(\alpha)$ is a linear function with nonzero slope. Again, these directions can be fully characterised. In fact, any ξ such that

$$\begin{bmatrix} L_{++}^T & L_{+0}^T \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \xi_+ \\ \xi_0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

clearly has the property that $Q\xi = 0$; again, that system is underdetermined and it has the infinitely many solutions

$$\begin{bmatrix} \xi_+ \\ \xi_0 \end{bmatrix} = \begin{bmatrix} -L_{++}^{-T}L_{+0}^T\xi_0 \\ \xi_0 \end{bmatrix} \quad \text{however chosen } \xi_0 \in \mathbb{R}^k.$$

For each of these

$$\langle q, \xi \rangle = \langle q_+, \xi_+ \rangle + \langle q_0, \xi_0 \rangle = -q_+^T L_{++}^{-T} L_{+0}^T \xi_0 + \langle q_0, \xi_0 \rangle = \langle q_0 - L_{0+} L_{++}^{-1} q_+, \xi_0 \rangle$$

where $\gamma_0 = q_0 - L_{0+} L_{++}^{-1} q_+ = q_0 + L_{0+} \bar{v}_+ \neq 0$ by hypothesis. Hence, taking any ξ_0 such that $\langle \gamma_0, \xi_0 \rangle \neq 0$ (surely some exist, say $\xi_0 = \gamma_0$) provides a direction where the tomography is linear and not constant, providing a “certificate of unboundedness” of the optimization problem. All this provides an (efficient, inasmuch as possible) algorithmic way to “substantiate” the results of §1.3.4.

1.6.4 Efficiency of the Cholesky factorization

There is not too much to discuss about the efficiency of the Cholesky factorization: for better or for worse, it quite closely track the $O(n^3)$ complexity predicted by the theory (save for the usual effect of memory hierarchies). This is, however, mostly “for worse” in one sense: *L may be rather dense even if Q were fairly sparse to start with*. Albeit there are heuristic approaches that permute the rows and columns of Q to try to diminish the effect, the *fill-in* can be very significant. Indeed, it is memory consumption, before computational cost, that usually makes Cholesky factorization unfeasible as n grows. To get a sense of its behaviour, a few experiments about the relative performances of **Matlab**’s highly optimised Cholesky factorization routines and a home-made, by-the-book **Matlab** implementation of the Conjugate Gradient algorithm (even without preconditioning) is reported below for two classes of completely random matrices: dense (all elements nonzero) and mildly sparse (10% of the elements nonzero).

The results show that the growth of the running time is approximately cubic. For it being exactly so, a doubling

n	dense		sparse	
	Chol	CG	Chol	CG
1000	0.005	0.002	0.047	0.007
2000	0.026	0.016	0.176	0.002
4000	0.131	0.083	0.773	0.010
8000	0.895	0.421	5.240	0.042
16000	5.477	1.807	25.65	0.166

Table 1.1: Running times of Cholesky factorization and Conjugate Gradient on dense and sparse metrics

of size would lead to the running time increasing by a factor of 8. However, for Cholesky the time includes that of backsolves, that are rather $O(n^2)$, so the total factor should be somewhat less. For dense instances Cholesky shows a factor between 5 and 6, while CG between 4 and 5 (i.e., slightly more than quadratic). The running time favours CG (despite it being a by-the-book implementation vs. **Matlab**'s highly optimised native Cholesky) by a factor between 2 and 3; a significant but not dramatic advantage. For sparse matrices, however, CG grows by a factor very close to 4, i.e., almost perfectly quadratic. Cholesky does grow somewhat slower than in the dense case, but by a factor always visibly larger than 4; coupled with the fact that CG is already much faster even for $n = 1000$, the advantage grows up to over two orders of magnitude for larger values of n . Interestingly, and somewhat counter-intuitively, Cholesky is significantly *slower* in the sparse case than in the dense one. The main culprit is likely the fact that **Matlab** tries to work with sparse factors due to Q being sparse, but ultimately ending up with dense ones due to catastrophic fill-in: indeed, the obtained Cholesky factors are completely dense. Using sparse data structures for dense matrices is way less efficient, possibly explaining some of the above results. By contrast, sparsity is efficiently exploited in the matrix-vector products that constitutes the computational bottleneck of the Conjugate Gradient method: indeed, for large n the method is close to 10 times faster on sparse than on dense instances (for the same size), closely matching the expected performance advantage due to the fact that the scalar product need work only with 1/10 of the entries.

These few experiments do not pretend to portray the full gamut of the possible computational trade-offs; for instance, CG is significantly influenced by the distribution of eigenvalues while Cholesky is not, hence the relative performances may be rather different on different classes of matrices. However, they at least give a first taste as to how different solution methods applied to the same problem can provide very different performance profiles. Although direct methods like Cholesky factorization are usually not preferred, especially for large-scale problems, they still have their uses in the context of complex optimization algorithms. For instance, a use case that may significantly favour Cholesky factorization is multiple systems, with the same Q but different $q^1, q^2, \dots, q^k$, need be solved. Clearly, in this case the $O(n^3)$ cost is paid only once, and then each solution costs $O(n^2)$; if k is large the amortised cost of the solution of each individual system may be close to $O(n^2)$. This is indeed occurring in some of the more complex optimization algorithms to be explored in the these lecture notes (although typically not with a very large k), justifying the interest in having direct methods in the toolbox of available numerical techniques.

1.7 Ex-post motivation: Polynomial Interpolation

We now provide a motivation for wanting to solve quadratic optimization problems that is related to *learning*. The setting is that of simple *interpolation* problem: we are given a real-world process that can be described by a univariate function $h : \mathbb{R} \rightarrow \mathbb{R}$. The physical foundations of the process may be unclear, or too complex to be described by an *analytical model*, i.e., under the form of algebraic or differential equations that can be explicitly or numerically solved. What is available is a finite set (but possibly large) of *observations*, or *samples*, $S = \{(x_i, y_i) : i \in I\}$, with $y_i = h(x_i)$. The task is to construct a *syntetic, or data-driven, model*, i.e., some function $m : \mathbb{R} \rightarrow \mathbb{R}$ that approximates h . This of course implies that $m(x_i) \approx y_i = h(x_i)$ for $i \in I$; more importantly, the model should be able to *predict* the right value of $h(x)$ even for x that does not coincide with any of the x_i . For a number of good reasons, some of which will be commented upon later on, it is also desirable that m is “simple”.

The standard way in which the problem is addressed is to *choose a-priori a parametric family of simple functions*, and then seek the value of the parameters that identify the function in the family that best *fits* the available data. In our case we choose perhaps a family of simple algebraic functions: *polynomials* of an arbitrarily fixed degree k , i.e.,

$$m_c(x) = c_0 + \sum_{i=1}^k c_i x^i.$$

A polynomial of degree k is identified by $k+1$ real parameters, i.e., a vector $c = [c_0, c_+ = [c_1, \dots, c_k]] \in \mathbb{R}^{k+1}$; we use the notation m_c to stress this dependency. The *fitting problem* could then be stated as: find $c \in \mathbb{R}^{k+1}$

such that $y_i = m_c(x_i)$ for all $i \in I$. It is, however, obvious that this may be impossible to achieve, in particular if the degree k is “small” with respect with the cardinality n of I (the number of samples). It is therefore better necessary to seek for the $c \in \mathbb{R}^{k+1}$ that “fits as well as possible”. This is typically implemented by defining a *loss function* $\mathcal{L} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_+$ that measures “how unequal its two arguments are”; that is, $\mathcal{L}(z, y) = 0$ if and only if $z = y$, with the value becoming larger and larger as z and y become less and less similar. This therefore measures the “loss in accuracy if one accepts z as a substitute value for y ”. Perhaps the most natural loss is $\mathcal{L}(z, y) = (z - y)^2$. Then, the problem becomes to minimize the *total loss* $\mathcal{L}(c) = \sum_{i \in I} \mathcal{L}(y_i, m_c(x_i))$, also called the *empirical risk*: how much the model fails to predict the true function on the available observations.

We can now formally state the polynomial fitting problem (for degree k). It is convenient to define the vector of outputs and the matrix of inputs

$$y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}, \quad X = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^k \end{bmatrix}.$$

The crucial point is that, given S , y and X are *fixed*, i.e., they can be easily computed (in fact, if k is “large” the rows of X may contain numbers of wildly varying magnitude and this may lead to numerical problems, but we avoid to discuss these fine details). Clearly, given the row $X_i = [1, x_i, x_i^2, \dots, x_i^k]$, $m_c(x_i) = \langle c, X_i \rangle$; i.e., the function is *linear* in the coefficients c and in the fixed vector X_i . Hence, the fitting problem is

$$\min \left\{ \mathcal{L}(c) = \sum_{i \in I} \mathcal{L}(y_i, \langle c, X_i \rangle) = \|y - Xc\|^2 : c \in \mathbb{R}^{k+1} \right\}.$$

This is called a *Linear Least Squares* problem, and it is clearly the unconstrained minimization of a quadratic function. In fact,

$$f(c) = \|y - Xc\|^2 = (y - Xc)^T (y - Xc) = \|y\|^2 - 2(y^T X)c + c^T (X^T X)c.$$

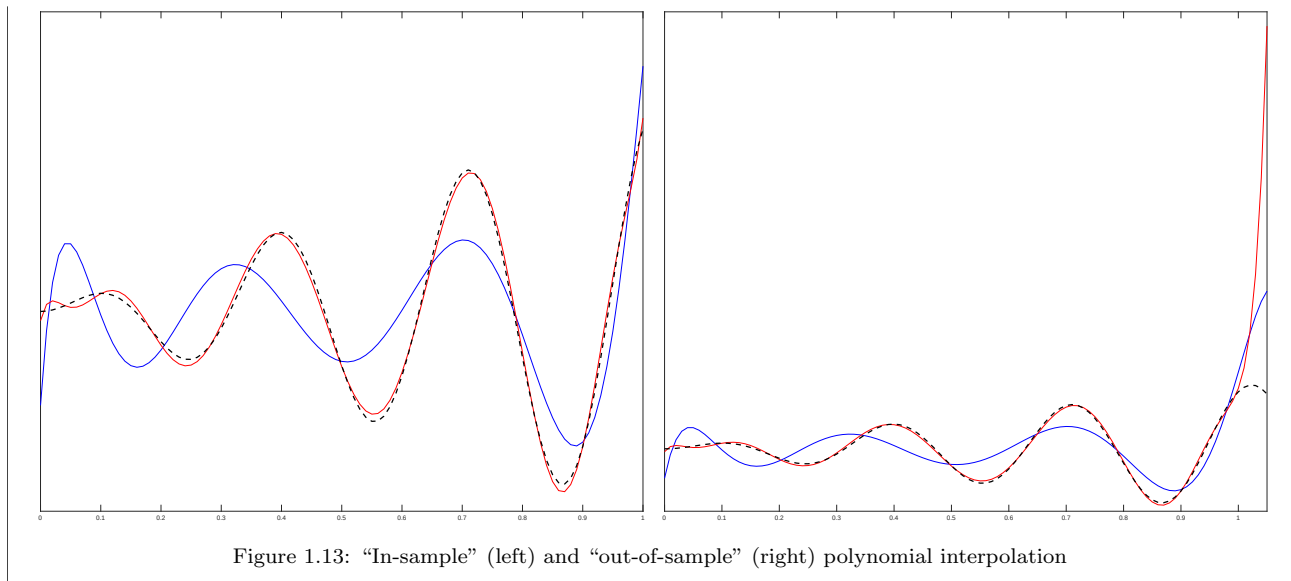
It is immediate to realize that $Q = X^T X \succeq 0$. Furthermore, the problem cannot clearly be unbounded below since the norm is always non-negative: hence, the problem surely has an optimal solution (albeit it may not be unique) that can be found with the techniques seen above. In fact, solving this kind of problems is so common that in **Matlab** this is just obtained with the simple command `c = y / X`.

Example 1.6. Polynomial interpolation

We now illustrate the process, and point to some of its issues, with a small example. We consider the function $h(x) = x \sin(20x)$, and we collect samples in the interval $[0, 1]$ regularly spaced at distance 0.01 from each other. We then perform polynomial interpolation with $k = 8$ and $k = 12$, respectively. In Figure 1.13(left) these are plotted as, respectively, the blue and the red continuous line, while h is plotted as the thicker black dashed line. It is easy to see that the “more complex” model with $k = 12$ tracks h much better than the “less complex” one. Of course, the model is somehow more costly to obtain, but not tremendously so, and the much higher accuracy is arguably worth the increase in computational cost.

However, the issue with “complex” models is that they tend to *overfit*: that is, learn very accurately the phenomenon (function) at the sampled points, but at the cost of accuracy in points different from the sampled ones. This is illustrated (cheating a bit, but this is not a significant issue for these lecture notes) in Figure 1.13(right), where the predicted polynomials are plotted “out of sample”: not only in the interval $[0, 1]$, but also “slightly more on the right” in $[1, 1.05]$. It is easy to see that the quality of the prediction dramatically deteriorates; while this may be expected, the more complex model, that does better “in sample”, does way worse “out of sample”. Again, this is only for illustration purposes and this particular example should not be taken too seriously, but it does point to a real conundrum, perhaps one of the most important ones, in learning known as the *bias/variance dilemma*. In layman terms, models should be “of the right complexity”: if they are not enough complex they are not even capable of faithfully reproducing the phenomenon (function) in the sampled points, but if they are too complex they do not *generalize* well on points that are not very close to the sampled one.

This issue, which would be central in a learning course, is *not* relevant for these lecture notes: here we don’t care if the *fitting problem* we solve leads to good learning outcomes or not, just that we solve it as efficiently as possible. This is reasonable since synthetic models have *hyperparameters* that can be tuned to attain the best learning outcomes; in the case of polynomial interpolation, for instance, this is the degree k . Since it is usually unknown a-priori which (degree) will be best, *hyperparameter tuning* is required that implies solving the fitting problem many times over. Thus, efficient optimization algorithms, that are what these lecture notes are all about, are important for effective learning. This is not to say that learning completely boils down to optimization; on the contrary, a number of issues must be faced that these lecture notes purposely ignore. It remains true that efficient optimization techniques for solving fitting problems, usually much more demanding ones than the very simple one illustrated here, are an important component of effective learning approaches.



1.8 Wrap up

This section has hopefully provided a useful “warm up” for the (much) more complex problems that will be tackled in the next sections of these lecture notes. Although the optimization of quadratic (multivariate) functions is generally “easy”, it is already a complex enough task to clearly delineate the typical workflow for solving some optimization problem:

1. study the properties of the problem and devise *optimality conditions* that characterise the sought-after optimal solutions (comprised understanding when none exists);
2. starting from the optimality conditions devise solution approaches that allow to construct solutions (approximately) achieving them;
3. analyse the efficiency of the devised solution approaches to identify the cases in which the approach may not be performant enough;
4. on the base of this analysis hopefully devise alternative solution methods that (possibly, in some cases) have better performances.

This typically ends up with multiple solution approaches, each of which more appropriate for some cases and less on others. Almost never the “perfect” approach turns up that is always unquestionably better than every other, since there can be many different *structures* in the problem that some approaches may be more capable of exploiting than others.

Many of the tools that have been introduced in this section for “simple” problems will be useful, possibly in more sophisticated versions, for more complex ones. Indeed, one of the most useful ideas is that of *using simple problems as stepping stones for solving complex ones*. This may happen in different forms:

- simple problems are typically easier to analyse and therefore may provide intuition for the solution of more complex, and harder to analyse, ones;
- solving a complex problem may entail solving a sequence of simpler ones;
- a powerful strategy for solving a complex problem is to devise a simpler one that “approximates” it somehow (a *model*) and solve it instead, typically iterating the process (cf. the previous point).

The “simple” linear and quadratic functions are in fact crucial for devising *models* of the more complex ones to be tackled in the following. Indeed, it will soon become apparent that while linear functions are “really too simple”, quadratic ones are “already sophisticated enough to provide excellent models”, justifying the in-depth analysis of this section. As a final remark, the complexity of optimization problems must never be underestimated: even optimising “simple” quadratic functions over “trivial” boxes is an exceedingly difficult ($\mathcal{NP}$ -hard) problem. Keeping on the right side of the complexity landscape will be a constant struggle across all these lecture notes.

1.9 Solutions

- Solution to Exercise 1.1:** The practical solution is to use $\max\{|f_*|, 1\}$ in the denominator instead of $|f_*|$. This can be seen as “use the relative error for large f_* and the absolute one for small f_* ”. That is, one argues that 0 is of the order of $1\text{e}+0$, and therefore the right “scale” for the error is 1. One may contend that if f_* is very close to 0 but not 0, say $1\text{e}-12$, then a relative error of, say $1\text{e}-6$ would correspond to $1\text{e}-18$. However, the issue here becomes that of the numerical accuracy. Roughly speaking, the error one makes for computing some quantity v is related to the machine precision and *the largest* numbers that went in the computation. Since it is very unlikely that none of the numbers involved in the computation of $f(x)$ is at least of the order of $1\text{e}+0$, one should expect the numerical error to be at least of the order of the machine precision. This is a rough argument and debatable, but works most of the time in practice. Another way of seeing it is to assume one is minimizing $g(x) = f(x) + 1$, so that $f_* = 1$; this problem is equivalent to the original one, as discussed next.
- Solution to Exercise 1.2:** If $b > 0$ and $x - z > 0$, then $b(x - z) > 0$, i.e., $bx > bz$, i.e., $f(x) > f(z)$; the other cases are analogous (or simpler).
- Solution to Exercise 1.3:** Let $b > 0$. If $\underline{x} = -\infty$, then obviously $x^* = -\infty = \underline{x}$. If $\underline{x} > -\infty$, since $f(x)$ is increasing, $f(x) > f(\underline{x}) \forall x > \underline{x}$, i.e., $\underline{x}$ is the minimum. The treatment of $\bar{x}$ when maximising is analogous. If $b < 0$, the role of $\underline{x}$ and $\bar{x}$ reverses: $\bar{x}$ is the argmin, and $\underline{x}$ is the argmax. Obviously, if $b = 0$ then every point in X is an optimal solution.
- Solution to Exercise 1.4:** Pick $x > z$, $a > 0$ and $z > 0$: then $f(x) = ax^2 > axz > az^2 = f(z)$. Since $f(x)$ is symmetric ($ax^2 = a(-x)^2$), the fact that f is increasing for $x > 0$ immediately implies that it is decreasing for $x < 0$. When $a < 0$ the sign of the inequalities is inverted: the function is reflected upon the x axis. The case $a = 0$ is trivial.
- Solution to Exercise 1.5:** $f(x)$ has a minimum in 0, is decreasing for $x < 0$ and increasing for $x > 0$. If $\underline{x} > 0$ then $f(x)$ is increasing along all $X = [\underline{x}, \bar{x}]$, hence $\underline{x}$ is the minimum and $\bar{x}$ the maximum. The argument is symmetric if $\bar{x} < 0$. Obviously, if $0 \in X$ then it is the minimum. For the maximization, since the function is increasing when x moves away from 0 in both directions, the maximum has to be one of the two extremes but we don't know which until we test. The rest is trivial.
- Solution to Exercise 1.6:** $f(x) = (ax + b)x$, hence the roots are $x = 0$ and $x = x_p = -b/a$. Clearly, $\bar{x} = -b/2a$ is always in the middle of the interval defined by the roots. If a and b have the same sign then $x_p < \bar{x} < 0$, otherwise $x_p > \bar{x} > 0$.
- Solution to Exercise 1.7:** We assume that i. and ii. hold for f and we want to show that $f(x) = \langle b, x \rangle$ for some $b \in \mathbb{R}^n$. Let $u_i, i \in I$, the i -th vector of the canonical base of $\mathbb{R}^n$ (having 1 in the i -th position and 0 otherwise), and $b_i = f(u_i)$. For any $x \in \mathbb{R}^n$, $x = \sum_{i \in I} x_i u_i$, hence $f(x) = f(\sum_{i \in I} x_i u_i) = \sum_{i \in I} f(x_i u_i)$ (using ii. recursively n times) $= \sum_{i \in I} x_i f(u_i)$ (using i. on each individual term) $= \sum_{i \in I} b_i x_i$ (using the definition of b_i) $= \langle b, x \rangle$ (using the definition of scalar product). The result clearly breaks in the affine case ($c \neq 0$): for $f(x) = x + 1$, $f(2x) = 2x + 1 \neq 2(x + 1) = 2f(x)$.
- Solution to Exercise 1.8:** By contradiction, assume there exists $\gamma \in \mathbb{R}^n \setminus \{0\}$ s.t. $H\gamma = 0$; this implies $0 = \|H\gamma\|^2 = \gamma^T [H^T H] \gamma = \|\gamma\|^2 > 0$ since $H^T H = I$, a contradiction.
- Solution to Exercise 1.9:** This is based on a general result: for $[A^1, A^2, \dots, A^n] = A \in \mathbb{R}^{m \times n}$ (not necessarily square) written by columns, $AA^T = M \in \mathbb{R}^{m \times m}$ (symmetric, prove it using $[AB]^T = B^T A^T$) can be written as the sum of the n rank-one matrices corresponding to the columns, i.e., $M = \sum_{i \in I} [D^i = A^i(A^i)^T]$. In fact, the h -th row of A is $A_h = [A_h^1, A_h^2, \dots, A_h^n]$ and the k -th column of A^T is the k -th row of A , thus $M_{hk} = \langle A_h, A_k \rangle = \sum_{i \in I} A_h^i A_k^i$. But $D_{hk}^i = A_h^i A_k^i$, hence $M_{hk} = \sum_{i=1}^n D_{hk}^i$ for all h and k . To complete the result, for $\Lambda = \text{diag}([\lambda_1, \lambda_2, \dots, \lambda_n]) \in \mathbb{R}^{n \times n}$, $L = A\Lambda = [\lambda_1 A^1, \lambda_2 A^2, \dots, \lambda_n A^n]$. In fact, the h -th row $A_h = [A_h^1, A_h^2, \dots, A_h^n]$ and the k -th column of Λ , i.e., $\lambda_k u_k$ (u_k being the k -th vector of the canonical base) give $L_{hk} = \langle A_h, \lambda_k u_k \rangle = \lambda_k A_h^k$.

- **Solution to Exercise 1.10:** For the first case we have

$$Q = \left(\frac{\sqrt{2}}{2} \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix} \right) \begin{bmatrix} 8 & 0 \\ 0 & 4 \end{bmatrix} \left(\frac{\sqrt{2}}{2} \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix} \right) = \frac{1}{2} \begin{bmatrix} 12 & -4 \\ -4 & 12 \end{bmatrix} = \begin{bmatrix} 6 & -2 \\ -2 & 6 \end{bmatrix}.$$

Analogously, in the other three cases we have respectively

$$Q = \begin{bmatrix} 5 & -3 \\ -3 & 5 \end{bmatrix}, \quad Q = \begin{bmatrix} 4 & -4 \\ -4 & 4 \end{bmatrix}, \quad Q = \begin{bmatrix} 3 & -5 \\ -5 & 3 \end{bmatrix}.$$

- **Solution to Exercise 1.11:** $x = z + \bar{x} \implies \frac{1}{2}x^T Qx + qx = \frac{1}{2}(z + \bar{x})^T Q(z + \bar{x}) + q(z + \bar{x}) = \frac{1}{2}z^T Qz + z^T(Q\bar{x} + q) + [\frac{1}{2}\bar{x}^T Q\bar{x} + q\bar{x}] = \frac{1}{2}z^T Qz + f(\bar{x})$ as $Q\bar{x} + q = Q(-Q^{-1}q) + q = -q + q = 0$.
- **Solution to Exercise 1.12:** $Qv = Q[\sum_{i \in I_0} \beta_i H_i] = \sum_{i \in I_0} \beta_i QH_i = \sum_{i \in I_0} \beta_i \lambda_i H_i = 0$.
- **Solution to Exercise 1.13:** $Q = H\Lambda H^T = \sum_{i=1}^n \lambda_i H_i H_i^T = \sum_{i \in I_0} \lambda_i H_i H_i^T [= 0] + \sum_{i \in I^+} \lambda_i H_i H_i^T$. We want to prove $\exists x$ s.t. $(\sum_{i \in I^+} \lambda_i H_i H_i^T)x = \sum_{i \in I^+} \mu_i H_i = w$. This is true if $\lambda_i H_i^T x = \mu_i$ $i \in I^+ \iff H_i^T x = \gamma_i = \mu_i / \lambda_i$ $i \in I^+$, a linear system of $h \leq n$ equations in n variables (likely underdetermined). Since all H_i are linearly independent, $H_+ = [H_i]_{i \in I^+} \in \mathbb{R}^{n \times h} \implies \text{rank}(H_+) = h$, which implies that $[H_+^T, \gamma] \in \mathbb{R}^{h \times (n+1)}$ has rank h (rank $\leq$ number of rows), i.e., by [84] the system has a solution x (∞ -ly many if $h < n$).
- **Solution to Exercise 1.14:** $\frac{1}{2}x^T Qx + qx = \frac{1}{2}(z + \bar{x})^T Q(z + \bar{x}) + q(z + \bar{x}) = \frac{1}{2}z^T Qz + z^T(Q\bar{x} + q^+ + q^0) + f(\bar{x}) = \frac{1}{2}z^T Qz + q^0 z + f(\bar{x})$ as $Q\bar{x} + q^+ = 0$.
- **Solution to Exercise 1.15:** We know that $g(z) = \frac{1}{2}z^T Qz + f(\bar{x})$, with $z = x - \bar{x}$. For $x = \bar{x} + v$, with $v \in \ker(Q)$, $z = x - \bar{x} = \bar{x} + v - \bar{x} = v$. Hence $g(z) = f(\bar{x})$. On the other hand, $g(z) \geq f(\bar{x})$ for all z since $Q \succeq 0$, thus any such point is a minimum. Any point $x = \bar{x} + v$ with $v \notin \ker(Q)$ has $f(x) = \frac{1}{2}v^T Qv + f(\bar{x}) > f(\bar{x})$ since $v^T Qv > 0$.
- **Solution to Exercise 1.16:** $\varphi(\alpha) = a\alpha^2 + b\alpha$ is quadratic non-homogeneous with $a = \frac{1}{2}(g^i)^T Qg^i \geq 0$ and $b = -\|g^i\|^2 < 0$. If $a > 0$, then $\varphi(\bar{\alpha}) < \varphi(0) = 0 \forall \bar{\alpha} \in (0, -b/a)$; in particular, $\bar{\alpha} = \|g^i\|^2 / ((g^i)^T Qg^i)$ is the minimum of φ . If $a = 0$ then φ is decreasing and $\varphi(\bar{\alpha}) < \varphi(0) = 0 \forall \bar{\alpha} > 0$.
- **Solution to Exercise 1.17:** The variational characterization of the eigenvalues implies that $\lambda_1 \geq d^T Qd / \|d\|^2 \geq \lambda_n$ for all $d \neq 0$; this immediately gives $1/\lambda_1 \leq \|d\|^2 / d^T Qd \leq 1/\lambda_n$ for all d , and therefore in particular $d = g^i$ (knowing that $g^i \neq 0$ otherwise the algorithm would have stopped).
- **Solution to Exercise 1.18:** Because $a < 0$, the step α will be negative, which basically means one is going in direction g rather than $-g$. The algorithm remains the same, except that the extra check has to become $g^T Qg \geq -\delta$.
- **Solution to Exercise 1.19:** Assuming the gradient is computed in the “natural way” as $g = Qx + q$ before the algorithm starts (i.e., with x the initial guess x^0), both quantities depending from matrix-vector products can be recovered by computing the vector $v = Qg$. In fact, $a = g^T Qg = \langle g, v \rangle$. Then, with $x' = x - \alpha g$ one has $g' = Qx' + q = Q(x - \alpha g) + q = (Qx + q) - \alpha Qg = g - \alpha v$. Hence, the gradient at the next iteration can be computed in $O(n)$ out of that of the previous iteration and the vector v . As for the objective function, $x^T Qx/2 + \langle q, x \rangle = (x^T Qx + 2\langle q, x \rangle)/2 = x^T(Qx + q + q)/2 = \langle q + g, x \rangle/2$, i.e., it can be computed in $O(n)$ once g is known.
- **Solution to Exercise 1.20:** All arguments boil down to the crucial $Qx^* + q = 0$. This first of all gives

$$\begin{aligned} f(x^*) &= \frac{1}{2}(x^*)^T Qx^* + \langle x^*, q \rangle = (x^*)^T Qx^* + \langle x^*, q \rangle - \frac{1}{2}(x^*)^T Qx^* \\ &= (x^*)^T(Qx^* + q) - \frac{1}{2}(x^*)^T Qx^* = -\frac{1}{2}(x^*)^T Qx^* \end{aligned}$$

Then, $\frac{1}{2}(x - x^*)^T Q(x - x^*) = \frac{1}{2}x^T Qx + \frac{1}{2}(x^*)^T Qx^* - x^T(Qx^*) = \frac{1}{2}x^T Qx - \langle x, q \rangle + \frac{1}{2}(x^*)^T Qx^* = f(x) - f(x^*)$ (in the penultimate step we have used $Qx^* = -q$).

- **Solution to Exercise 1.21:** $g^i = Q(x^i - x^*) = Qx^i + q$ and $\alpha^i = \|g^i\|^2 / [(g^i)^T Q g^i]$. This gives $g^{i+1} = Qx^{i+1} + q = Q(x^i - \alpha^i g^i) + q = (I - \alpha^i Q)g^i$, and therefore $\langle g^{i+1}, g^i \rangle = \|g^i\|^2 - \alpha^i [(g^i)^T Q g^i] = 0$.

- **Solution to Exercise 1.22:** Since Q is nonsingular $x^i - x^* = Q^{-1}g^i$, whence

$$a^i = \frac{1}{2}(x^i - x^*)^T Q(x^i - x^*) = \frac{1}{2}(g^i)^T Q^{-1}g^i.$$

This gives

$$\begin{aligned} a^{i+1} &= \frac{1}{2}(x^{i+1} - x^*)^T Q(x^{i+1} - x^*) = \frac{1}{2}(x^i - \alpha^i g^i - x^*)^T g^{i+1} = \frac{1}{2}(x^i - x^*)^T g^{i+1} \text{ [using } \langle g^{i+1}, g^i \rangle = 0] \\ &= \frac{1}{2}(x^i - x^*)^T Q(x^i - \alpha^i g^i - x^*) = \frac{1}{2}(x^i - x^*)^T Q(x^i - x^*) - \frac{1}{2}\alpha^i (x^i - x^*)^T Q g^i \\ &= a^i - \frac{1}{2}\alpha^i \|g^i\|^2 \text{ [using } Q(x^i - x^*) = g^i] = a^i - \frac{1}{2}\|g^i\|^4 / (g^i)^T Q g^i \\ &= a^i - \frac{\|g^i\|^4}{((g^i)^T Q g^i)((g^i)^T Q^{-1}g^i)} = a^i \left(1 - \frac{\|g^i\|^4}{((g^i)^T Q g^i)((g^i)^T Q^{-1}g^i)} \right) \end{aligned}$$

- **Solution to Exercise 1.23:** Recall that the eigenvalues of Q^{-1} are $1/\lambda_n \geq \dots \geq 1/\lambda_1 > 0$; from the usual $\lambda_n \|x\|^2 \leq x^T Q x \leq \lambda_1 \|x\|^2$ (applied to Q^{-1} as well) one has $\|g\|^2 / g^T Q g \geq 1/\lambda_1$ and $\|g\|^2 / g^T Q^{-1}g \geq 1/[1/\lambda_n]$.

- **Solution to Exercise 1.24:** Just induction: obvious for $i = 0$, if it holds for $i - 1$ then $A(x^i) \leq rA(x^{i-1}) \leq r(r^{i-1}A(x^0))$.

- **Solution to Exercise 1.25:** $r^i a^0 \leq \delta \equiv r^i \leq \delta / a^0 \equiv \log(r^i) \leq \log(\delta / a^0)$ (log monotone) $\equiv i \log(r) \leq \log(\delta / a^0)$ (property of log); since $r < 1$, $\log(r) < 0$, giving $i \geq \log(\delta / a^0) / \log(r) = [-\log(\delta / a^0)] / [-\log(r)] = \log(a^0 / \delta) / \log(1/r) = \log(a^0 / \delta) [1 / \log(1/r)]$.

- **Solution to Exercise 1.26:** This requires a bit of elementary calculus. The derivative of $\ln(x)$ is $1/x$. The first-order Taylor approximation is $f(x+h) \approx f(x) + f'(x)h$ for $h \approx 0$. Applied to $\ln(\cdot)$ with $x = 1$ gives $\ln(1+h) \approx h$, whence $1/\ln(1/r) = 1/\ln(1+(1-r)/r) = r/(1-r)$. But $\log_a(x) = \log_b(x) / \log_b(a)$, hence $\ln(x) = \log_e(x) = \log_{10}(x) / \log_{10}(e) \approx \log(x) / 0.43 \approx 2.3 \log(x)$, i.e., $\ln(x) \in O(\log(x))$. Then, since $r = (1 - 1/\kappa)$, one has $\kappa = 1/(1-r) \approx r/(1-r)$ since $r \approx 1$.

- **Solution to Exercise 1.27:** $\lambda_1 \|x^i - x^*\|^2 \geq (x^i - x^*)^T Q(x^i - x^*) = 2a^i \equiv \|x^i - x^*\| \geq \sqrt{2a^i / \lambda_1}$, hence $\|x^i - x^*\| \leq \gamma \implies a^i \leq \lambda_1 \gamma^2 / 2$.

- **Solution to Exercise 1.28:** $[H\sqrt{\Lambda}H^T][H\sqrt{\Lambda}H^T] = H\sqrt{\Lambda}[H^TH]\sqrt{\Lambda}H^T = H\Lambda H^T = Q$

- **Solution to Exercise 1.29:** By contradiction, $\exists \gamma \in \mathbb{R}^n \setminus \{0\}$ s.t. $\sum_{i=1}^n \gamma^i p^i = 0 \implies 0 = [\sum_{i=1}^n \gamma^i p^i]^T Q [\sum_{i=1}^n \gamma^i p^i] = \sum_{i=1}^n (\gamma^i)^2 (p^i)^T Q p^i$ [using $(p^i)^T Q p^j = 0$ if $i \neq j$]. But $(p^i)^T Q p^i > 0 \forall i$ since $Q \succ 0$, hence $\gamma \neq 0$ implies $\sum_{i=1}^n (\gamma^i)^2 (p^i)^T Q p^i > 0$: a contradiction.

- **Solution to Exercise 1.30:** $\varphi_{x^i, p^i}(\alpha) = \frac{1}{2}(x^i + \alpha p^i)^T Q(x^i + \alpha p^i) + q(x^i + \alpha p^i) = f(x^i) + \frac{1}{2}\alpha^2 (p^i)^T Q p^i + \alpha \langle Qx^i + q, p^i \rangle = f(x^i) + \frac{1}{2}\alpha^2 (p^i)^T Q p^i + \alpha \langle g^i, p^i \rangle$ then use the results of §1.2.3.

- **Solution to Exercise 1.31:** $H^i Q H^j = H^i (\lambda_j H_j) = \lambda_j \langle H^i, H^j \rangle = 0$ if $i \neq j$.

- **Solution to Exercise 1.32:** $Q \not\succeq 0$ may mean some $Q_{ii} \leq 0$; take $\sqrt{|\text{diag}(Q)|}$ and replace every 0 (small) element with 1

- **Solution to Exercise 1.33:** It all starts with the standard spectral decomposition $Q = H\Lambda H^T$, with H the orthonormal unitary matrix of eigenvectors and Λ the diagonal matrix of eigenvalues. The vector of eigenvalues is partitioned as $\lambda = [\lambda_+, \lambda_0] = [\lambda_+, 0]$, i.e., distinguishing the h null ones from the $n - h$ strictly positive ones; similarly, $H = [H_+, H_0]$, where H_+ are the eigenvectors corresponding to positive eigenvalues and H_0 these corresponding to null ones. Then, obviously $Q = H_+ \Lambda_+ H_+^T$. This is rewritten

in a more explicit form as

$$Q = \begin{bmatrix} Q_{++} & Q_{0+}^T \\ Q_{0+} & Q_{00} \end{bmatrix} = \begin{bmatrix} H_{++} \\ H_{0+} \end{bmatrix} \Lambda_+ [H_{++}^T, H_{0+}^T] = \begin{bmatrix} H_{++}\Lambda_+H_{++}^T & H_{++}\Lambda_+H_{0+}^T \\ H_{0+}\Lambda_+H_{++}^T & H_{0+}\Lambda_+H_{0+}^T \end{bmatrix}.$$

Since $Q_{++} \succ 0$ and $\Lambda_+ \succ 0$, $H_{++} \succ 0$. Thus, $H_{0+}\Lambda_+H_{++}^T = Q_{0+}$ implies $H_{0+}\Lambda_+ = Q_{0+}H_{++}^{-T}$. Hence,

$$Q_{00} = H_{0+}\Lambda_+H_{0+}^T = [Q_{0+}H_{++}^{-T}][\Lambda_+ - 1H_{++}^{-1}Q_{0+}^T] = Q_{0+}[H_{++}^{-T}][\Lambda_+ - 1H_{++}^{-1}]Q_{0+}^T = Q_{0+}Q_{++}^{-1}Q_{0+}^T.$$

Let now $Q_{++} = L_{++}L_{++}^T$ the standard Cholesky factorization of $Q_{++} \succ 0$, and define $L_{0+} = Q_{0+}L_{++}^{-T}$. Then

$$LL^T = \begin{bmatrix} L_{++} & 0 \\ L_{0+} & 0 \end{bmatrix} \begin{bmatrix} L_{++}^T & L_{0+}^T \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} L_{++}L_{++}^T & L_{++}L_{0+}^T \\ L_{0+}L_{++}^T & L_{0+}L_{0+}^T \end{bmatrix} = \begin{bmatrix} Q_{++} & Q_{0+}^T \\ Q_{0+} & L_{0+}L_{0+}^T \end{bmatrix}.$$

But since $L_{0+} = Q_{0+}L_{++}^{-T}$,

$$L_{0+}L_{0+}^T = Q_{0+}L_{++}^{-T}L_{++}^{-1}Q_{0+}^T = Q_{0+}Q_{++}^{-1}Q_{0+}^T = Q_{00},$$

proving that the proposed one is the desired rank-revealing Cholesky factorization.

Part II

Unconstrained Optimization

Chapter 2

Univariate Optimization

2.1 General Univariate Optimization Problems

Proceeding very carefully in our trip among optimization problems, we make a step forward and (apparently) three back: while we now allow the objective f to be any function (for which a pointwise evaluation oracle is available), we restrict at the same time $n = 1$, i.e., $f : \mathbb{R} \rightarrow \mathbb{R}$ to be *univariate*. As for the feasible region $X \subset \mathbb{R}$, we want it to be “easy”; thus, we only consider the (possibly, unbounded on one or both sides) interval $X = [\underline{x}, \bar{x}]$ with $-\infty \leq \underline{x} < \bar{x} \leq +\infty$. In §1.2, such problems were seen to be (outrageously) simple for linear and quadratic functions; however, this is far from being the case for general f . In fact, the problem is basically *impossible to solve in finite time to any fixed accuracy* $\varepsilon > 0$.

This harsh result can be proven formally [4, p. 408], but it basically boils down to the counterexample in Figure 2.1(left). Without any assumption on the oracle, save that it is “cheap”, one could be confronted with an *adversarial* one that basically answers always the same value, say $f(x) = 1$. Any algorithm can only probe the value in a sequence of iterates (x^i) , that can (in theory) be infinite, but *countable*: “after that the sequence is finished”, the oracle could “declare” that the minimum x^* of the function, say with $f_* = f(x^*) = 0$, may live in one of the points of X that have *not* been samples, since these are uncountably many, even if X is a bounded interval $(-\infty < \underline{x} < \bar{x} < +\infty)$. Thus, no algorithm can be guaranteed to find a ε -optimal solution with any given $\varepsilon > 0$ against such an adversarial oracle, albeit (with a bounded interval) one may in fact have produced iterates arbitrarily close to x^* . However, this is hardly useful since, even if this happened, the algorithm (and its user) is not able to understand it, and therefore “save” one of the ‘promising’ iterates (which, however, all formally correspond to arbitrarily bad solutions).

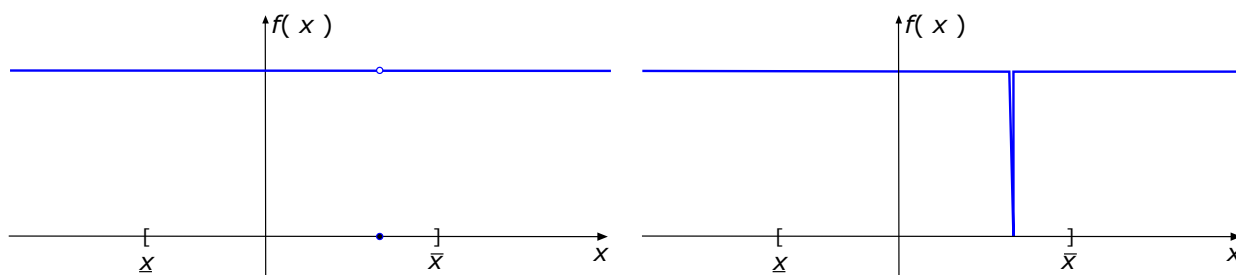


Figure 2.1: Adversarial oracles for general univariate optimization

Of course there are many objections that could be raised against this counterexample, starting from the fact that it clearly assumes x to be a “real real”, while we have repeatedly insisted that in practice x will be, say, a **double** and therefore it only has a finite number of possible values. This may be little solace, as these would be at least of the order of $2^{53} \approx 10^{16}$ (a **double** has 53 bits of mantissa); even with a 10Ghz processor capable of evaluating 10^{10} points per second, checking them all would require $\approx 10^5$ seconds, i.e., more than a day. If this still looks doable, moving to the $2^{113} \approx 10^{34}$ possible values of a 128-bits **quad** would skyrocket to $\approx 10^{24}$ seconds, i.e., $\approx 10^{16}$ years, way more than the current age of the universe. Other nontrivial questions would be how the oracle could store information about all the iterates in order to place x^* somewhere else. Perhaps the most fitting observation is that no-one is really interested in optimizing an adversarial oracle; indeed, the functions involved in the applications of interest for these lecture notes are “decidedly not adversarial”. However, rather than delving in philosophical issues we underline the real gist of the counterexample: *there is no hope of being able to efficiently optimize a function if evaluating it in some x does not somehow convey information about its*

behaviour in points different from x . If the function can change arbitrarily from x to a points x' “extremely close to x ”, every evaluation only conveys “too limited information” to be able to efficiently drive the optimization.

One therefore may surmise that the fundamental issue with the adversarial oracle of the counterexample is that the function “jumps”: it is not *continuous* (cf. A.6). However, Figure 2.1(right) readily dispels the notion. After any finite number of iterates x^i , the oracle can select two ones, x^h and x^k , so that no other one lives in the interval (x^h, x^k) , and then “fit a very narrow downward spike inside the interval”. Such a function is continuous, and still impossible to optimize upon. Of course, the two cases are hardly different when x is actually a floating-point number, as spikes that are narrower than the machine precision actually are basically single points. However, rather than putting too fine a point on the discussion, the gist of the result is the following: *it is impossible to optimise upon a function that is allowed to change arbitrarily rapidly*. If, again, the value $f(x)$ and $f(x')$ can be “arbitrarily far apart” even if x and x' are “near”, then evaluating the function at any point x hardly gives any useful information about its behaviour in any other points. Since *optimizing is learning about the function on the grounds of information extracted by function evaluations*, this means that such ground must be “somehow firm enough” for the process to succeed. Just any function does not provide it, hence we will do for the first time something that will be needed over and over again: *make proper assumptions on the problem* (in this case, its objective function) so that the task at hand becomes possible / efficient enough.

2.2 Lipschitz (Global) Optimization

The analysis in the previous section should have clearly revealed what kind of assumption is needed: something ensuring that “downward spikes can’t be arbitrarily narrow”, i.e., basically, *imposing speed limits on the rate of change of the function*. The useful concept is that of *Lipschitz continuity* (L-c) on X (cf. §A.6), which in this case reads just

$$|f(x) - f(z)| \leq L|x - z| \quad \forall x \in X, z \in X.$$

for some fixed *Lipschitz constant* $L > 0$. This assumption immediately makes it possible to design an algorithm that is capable of finding ε -optimal solutions to the problem for any chosen $\varepsilon > 0$. However, for the algorithm to work it is also necessary that $X = [\underline{x}, \bar{x}]$ is *bounded*, i.e., that its *diameter* $D = \bar{x} - \underline{x}$ is *finite* (trivially, $\underline{x} > -\infty$ and $\bar{x} < \infty$). This allows to proceed as follows [4, p. 411]:

Algorithm 2.1. Global Optimization of a Lipschitz Function

```

procedure  $x = SDQ(f, \underline{x}, \bar{x}, L, \varepsilon)$ 
   $D = \bar{x} - \underline{x}$ ;  $k = \lceil LD / (2\varepsilon) \rceil$ ;  $\Delta = D / k$ ;  $f^{best} = +\infty$ ;
  for ( $i \leftarrow 0$ ,  $z = \underline{x}$ ;  $i \leq k$ ;  $i \leftarrow i + 1$ ,  $z \leftarrow z + \Delta$ )
    if ( $f(z) < f^{best}$ ) then {  $f^{best} \leftarrow f(z)$ ;  $x \leftarrow z$ ; }

```

Note that $\lceil \cdot \rceil$ indicates rounding up to the closest larger integer; similarly, $\lfloor \cdot \rfloor$ will indicate rounding down to the closest smaller integer, and $\lfloor \cdot \rfloor$ just rounding to the closest integer. In plain words, the algorithm samples f uniformly in the interval at the smallest possible number of points such that the distance between two consecutive ones is $\Delta \leq 2\varepsilon / L$, and returns the best among all the function values it finds. This can be easily proven to be a ε -optimal solution. The intuition is that, being z^i and z^{i+1} the end-points of any one of the intervals, then

$$f(x) \geq \min\{f(z^i), f(z^{i+1})\} - \varepsilon \quad \text{for all } x \in [z^i, z^{i+1}].$$

In fact, at any point x inside the interval L-c gives

$$f(x) \geq f(z^i) - L(x - z^i) \quad \text{and} \quad f(x) \geq f(z^{i+1}) - L(z^{i+1} - x);$$

with $\underline{f}^i = \min\{f(z^i), f(z^{i+1})\}$ this gives

$$f(x) \geq \max\{\underline{f}^i - L(x - z^i), \underline{f}^i - L(z^{i+1} - x)\} = \underline{f}^i - L \min\{x - z^i, z^{i+1} - x\}.$$

Of course, the minimum on the rightmost term is maximised when x is exactly in the middle of the interval, i.e., $x - z^i = z^{i+1} - x = \Delta / 2 \leq \varepsilon / L$, which readily yields the desired results. Since this holds for all intervals, wherever the optimal solution x^* lies its optimal value can only have been “missed” by at most ε when iterates z^i and z^{i+1} “close” to it have been evaluated, and this must have happened. This discussion is only meant to highlight the fundamental property: *Lipschitz continuity allows to bound the value of f in points “close” to those where the function is evaluated*, which is what allows to algorithm to work.

Exercise 2.1. Formally prove ε -optimality of the returned x . [solution]

Although this looks finally a positive results, it has a number of significant drawbacks. First of all, it requires

$O(LD/\varepsilon)$ function evaluations, i.e., at least $O(LD/\varepsilon)$ complexity even if the oracle is “very cheap”. As previously discussed, this kind of complexity that is inversely proportional to the accuracy means that finding solutions with “small” ε is often practically impossible: gaining one further digit of accuracy requires one order of magnitude more iterations. The constant of the $1/\varepsilon$ term also worsens as the interval gets larger (D increases) and the function “gets faster”, i.e., L increases, i.e., steeper spikes are possible. A significant issue of the result is also that *the algorithm actually requires and uses the value of L* , which is in general unknown and not easy to estimate.

These are, however, not all the bad news:

- The algorithm above is actually “optimal”, in the sense that it can be proven [21, Theorem 4.4] that for every algorithm capable of finding guaranteed ε -optimal solutions there exists at least one function for which it will require no less than $O(LD/\varepsilon)$ function evaluations. The result again builds an *adversarial function*, precisely constructed to be “nasty” for that very specific algorithm; therefore, it is not necessarily significant for those of interest in these lecture notes. It still is an indication that (global) optimization is “inherently difficult” even for a univariate function; this will actually be confirmed “to the n -th degree” when we will move to multivariate case, i.e., $X \subset \mathbb{R}^n$ with $n > 1$.
- Some other interesting results can be proven, that cannot be discussed any extensively here, that confirm that “optimization is difficult”. Among them, variants of the *no free lunch theorem* [26] fundamentally prove that “all algorithms are equally bad on average”, i.e., “if an algorithm is very good in some cases it has to be very bad in others”. This does not rule out that algorithm exist that can be extremely efficient for specific cases of interest (indeed, this is mostly what these lecture notes are about), but it does reinforce the message that “you have to carefully choose the problem if you want to have a chance at being able to efficiently solve it”.

All in all, these results indicate that efficiently finding (global) ε -optimal solutions for “small” ε may be impossible in general, even in the univariate case. We will therefore be interested in understanding in which specific cases this is indeed possible. We will approach the question in a seemingly roundabout way, that however has important consequences, by posing a different question: is there a *different notion of optimality* that can be considered and that allows efficient solution approaches? Of course the answer is in the positive, as the next sections will detail.

2.3 Local optimization

Finding the (ε -)optimal solution of an (univariate) optimization problem is in general “too difficult”, even when it’s at all possible. However, there is a *weaker* notion of optimality that is conducive to more efficient algorithms: that of *local optimum*. The notion is very intuitive, and just corresponds to the fact that the solution is “better than all those very close by”. Formally speaking, for

$$(P) \quad \min\{f(x) : x \in X\},$$

some $\bar{x} \in X$ is a local optimum if there exists some *neighbourhood* X' of $\bar{x}$ —a set such that $\bar{x} \in I$, in the univariate case typically an interval $[\underline{x}', \bar{x}']$ with $\underline{x}' < \bar{x} < \bar{x}'$ —such that

$$\bar{x} \in \operatorname{argmin}\{f(x) : x \in X \cap X'\}. \quad (2.1)$$

In plain words, $\bar{x}$ is an optimal solution of a *restriction* of (P) where one can only consider “the subset of the feasible region close to $\bar{x}$ ”. It is immediate to see that any optimal solution x^* to (P) is a local minimum; just take $X' = \mathbb{R}$. So, “there are more local optima than bona-fide optimal solutions”. To underline this fact, x^* is typically denoted—if the doubt arises—as the *global optimum* of (P) . It is clearly useful if there are “not too many” local optima: indeed, for the concept of local optimum to make sense it is important to require that “ X' is not too small”: obviously, for any $x \in X$, taking $X' = \{x\}$ would make x a local minima. This is why one asks $\underline{x}' < \bar{x} < \bar{x}'$: it should be possible to “move a little bit both ways” (technically, $\bar{x}$ should be in the interior of X' , but we’ll try to avoid the finer details), and still $\bar{x}$ must remain an optimal solution. While the concept applies to the constrained case, it is perhaps more simply described when $X = \mathbb{R}$. There, the optimal solution of (P) is called the *minimum*, or *global minimum*, of f , while any local optimal solution is a *local minimum*. Of course, one can symmetrically define *global maximum* and *local maximum* of f . It is completely intuitive, by just looking at Figure 2.2(left), that a function may have multiple local minima that are not a global minimum. Of course, it may as well happen that “there is only one local minima, which necessarily has to be the global one”; this important case will be discussed later on. For now we stick to the general case, as depicted in Figure 2.2(left), where local minima may not be the global one.

The interest in the notion of local optimum/minimum is that *they can be found efficiently*, basically due to the

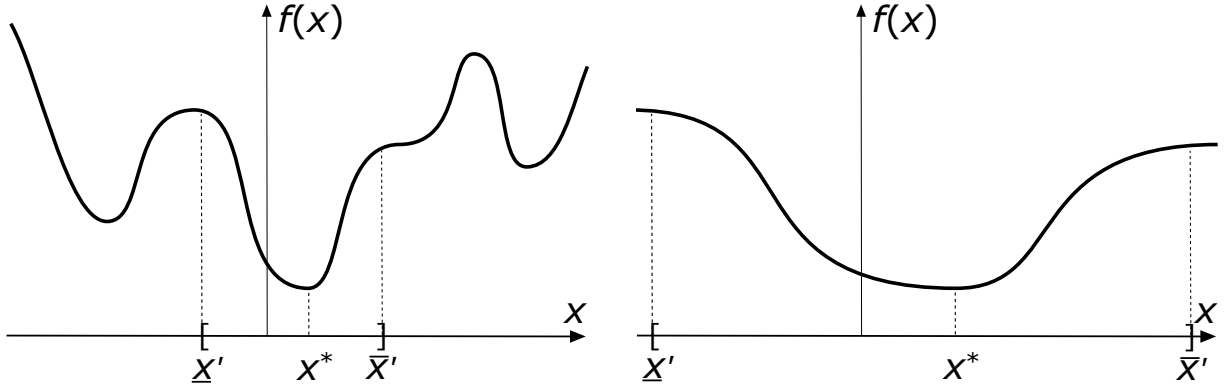


Figure 2.2: A function with local-non-global minima and one of its attraction basins

fact that “near $\bar{x}$, f has a predictable shape”. The precise definition is (almost, cf. [1, Ex. 8.10] for a missing detail): f is (strictly) *unimodal* on $X = [\underline{x}', \bar{x}']$ if

- it has a minimum $\bar{x} \in X$;
- is (strictly) *decreasing* in $[\underline{x}, \bar{x}]$ and (strictly) *increasing* in $[\bar{x}, \bar{x}]$.

Simply put: slightly on the left of the (local) minimum the function need be decreasing, because the minimum still has not reached, but once it is reached it need increase, otherwise it would not be a minimum. The “strict” version of the notion ties in with the notion of *strict* local minimum, which just says that $\bar{x}$ is the unique optimal solution: all other points in $X' \setminus \{\bar{x}\}$ have strictly larger objective value. Of course this is hardly significant for the practical case where all values are actually **double**. Anyway, the useful observation is then: for any (strict) local minimum $\bar{x}$ there exist an interval $X = [\underline{x}', \bar{x}']$ containing it such that f is (strictly) unimodal on X' . The largest such X' can be called the *attraction basin* of $\bar{x}$. Figure 2.2(right) depicts the fact that while f of Figure 2.2(left) is not unimodal on the whole $\mathbb{R}$, it is so when restricted to the attraction basin of one of its local minima (in this case x^* , the global minima at least in the interval that is shown).

2.4 First local optimization algorithms

A fundamental—if obvious—property of an attraction basin is that it contains a local minima. Now, clearly if we knew an attraction basin $[\underline{x}, \bar{x}]$ “of very small diameter $D = \bar{x} - \underline{x} \leq \delta$ ”, for a “small δ ”, then any point x in the basin, e.g., its extremes, could be considered a “good approximation of $\bar{x}$ ”, in that $|x - \bar{x}| \leq \delta$. We will discuss later the nontrivial aspect of how this translates to the standard notion of approximate (local) optimality in terms of function value; cf. §1.4.4. For now, in view of the fact that (approximately) solving an optimization problem in fact amounts at (approximately) finding its optimal solution, we will be more than content with the concept of finding an approximate solution of (2.1), i.e., approximately find $\bar{x}$ to within a given tolerance δ . We assume to know beforehand an initial attraction basin $[\underline{x}, \bar{x}]$ —this assumption will be discussed later on—and we want to iteratively shrink it, “without excluding $\bar{x}$ ”, up until the point where its diameter is smaller than a user-defined threshold δ . The crucial point is that this can be done efficiently by *evaluating f at two arbitrary intermediate points*. That is [1, Th. 8.11 + Ex. 3.60]: if f (strictly) unimodal in $[\underline{x}, \bar{x}]$, with a local minimum $\bar{x}$ lying somewhere inside the interval, then however chosen $\underline{x} < \underline{x}' < \bar{x}' < \bar{x}$:

- if $f(\underline{x}') \geq f(\bar{x}')$ then $\bar{x} \in [\underline{x}', \bar{x}]$;
- if $f(\underline{x}') \leq f(\bar{x}')$ then $\bar{x} \in [\underline{x}, \bar{x}']$.

The principle is simple and it is illustrated in Figure 2.3. If $f(\underline{x}') \geq f(\bar{x}')$ (top), then f is “decreasing in the sub-interval”: $\bar{x}$ may be “further on the right” (top left) or “inside the sub-interval already”, but surely it is *not* in $[\underline{x}, \underline{x}']$. Symmetrically if $f(\underline{x}') \leq f(\bar{x}')$, i.e., f is “increasing in the sub-interval”. Thus, at the cost of *two* evaluation of f we can restrict the attraction basin: clearly, iterating the process we can make it shrink down to any required size.

Once the general concept is established, the next relevant question obviously is: how should one choose $\underline{x}'$ and $\bar{x}'$ so that the interval shrinks as quickly as possible. The issue clearly is that each iteration of the process excludes *either* $[\underline{x}, \underline{x}']$ *or* $[\bar{x}', \bar{x}]$, but one does not know beforehand which of the two. Hence, by having them “of very different size” the behaviour of the algorithm would be “strongly subject to (bad) luck”. That is, if one were to take $\underline{x}'$ “close” to $\underline{x}$ but $\bar{x}'$ “far” from $\bar{x}$, then in the first occurrence the attraction basin would shrink

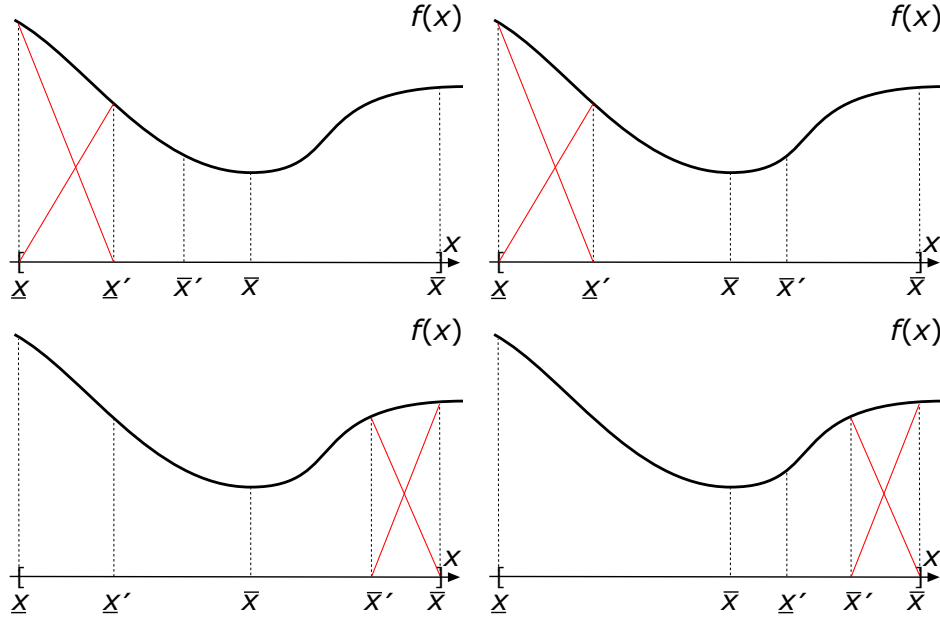


Figure 2.3: Restricting attraction basins by evaluating points inside them

“very little”, while in the second it would shrink “a lot”; and of course symmetrically. Thus, the algorithm may end up being either very efficient or very inefficient (in fact, basically not working at all) depending on how “lucky” one is in choosing the interval. Since this is undesirable, the typical approach is that *optimizing the worst-case behaviour*, i.e., choose the sub-interval in such a way that the attraction basin shrinks as much as possible in the worst case. It is plain to see that this implies to take $[\underline{x}, \underline{x}']$ and $[\bar{x}', \bar{x}]$ of equal size. That is, one selects $r \in (1/2, 1)$ and chooses

$$\underline{x}' = \underline{x} + (1-r)D, \quad \bar{x}' = \underline{x} + rD = \bar{x} - (1-r)D.$$

Thus, both intervals $[\underline{x}, \underline{x}']$ and $[\bar{x}', \bar{x}]$ have size $(1-r)D$, and whichever of the two get excluded the size of the remaining attraction basin is $rD < D$. This immediately implies that the smaller r is, the faster the interval shrinks and therefore the sooner the algorithm ends. On the other hand, r must remain strictly larger than $1/2$. Thus, one would select some small $\sigma > 0$, and have $r = 1/2 + \sigma$. This of course corresponds to

$$\underline{x}' = \underline{x} + D/2 - \sigma, \quad \bar{x}' = \underline{x} + D/2 + \sigma = \bar{x} - D/2 + \sigma;$$

in plain words, “go to the middle of the interval and then step a little bit on the left and a little bit on the right”. For reasons that we have previously discussed, when implemented with floating-point numbers the formula should rather be written with a “relative” σ , i.e., as

$$\underline{x}' = \underline{x} + (1-\sigma)D/2, \quad \bar{x}' = \underline{x} + (1+\sigma)D/2$$

and σ be chosen appropriately w.r.t. the stopping tolerance δ , but we avoid these fine details since, as we will see, this is not the algorithm that one really wants to run. Rather, to simplify the analysis we will ignore σ altogether and pretend that $r = 1/2$: at each time the interval is cut precisely in half. Denoting with $\underline{x}^i$ and $\bar{x}^i$ the values of the extremes at the i -th iteration, and $D^i = \bar{x}^i - \underline{x}^i$ the corresponding diameter, one then has that

$$D^i = \bar{x}^i - \underline{x}^i = D^0 r^i.$$

Thus, with the stopping criterion $D^i \leq \delta$, the approach stops when

$$D^0 r^i \leq \delta \quad \equiv \quad i \geq \log(D/\delta) / \log(1/r).$$

Exercise 2.2. Prove the last bound. [solution]

Thus, the complexity of finding the local minimum only depends on $\log(1/\delta)$, as opposed on $1/\delta$ for finding the global one: the process is “exponentially faster”, and therefore can be ran with “small” δ . While the two complexities are not directly comparable, since one bounds the error ε in the output space and the other the error δ in the input one, it is easy to see that, if f is L -c with known constant $L > 0$, one can easily ensure to achieve a ε -optimal solution in $O(\log(LD/\varepsilon))$, i.e., “exactly exponentially faster” than the global optimization bound.

Exercise 2.3. Prove the last statement. [solution]

Unlike in the case of the gradient method for quadratic functions, r cannot become “too close to 1”. In fact, one can then precisely estimate the constant in the $O(\cdot)$ notation: for $r = 0.5$, $\log(1/r) \approx 0.3$ and $1/\log(1/r) \approx 3.32$. It is important to remark, however, that this is the bound on the number of *iterations*. One should expect that the most costly operation in any such approach is the invocation of the *oracle* computing the f -values, which indeed could have an arbitrary cost. Since all other operations in the algorithm are $O(1)$, the reasonable metric for the cost of the algorithm should therefore be that of the number of *function evaluations*. Since clearly both points $\underline{x}^{i'}$ and $\bar{x}^{i'}$ probed at the i -th iteration are different from any of the ones probed at previous iterations, there are *two* function evaluations at each step; thus, the algorithm requires $\approx 6.64 \log(D/\delta)$ function evaluations to obtain $D^i \leq \delta$. One may therefore wonder whether it is possible to improve such a result, and the answer is positive.

The strategy for improving the efficiency is actually quite simple: accept a slight increase in the number of iterations, but compute the function in only one point at each iteration. This is clearly possible on one condition: one of the two between $\underline{x}^{i'}$ and $\bar{x}^{i'}$ (the one that does *not* become an extreme of the interval) has to “survive” and become one among $\underline{x}^{i+1'}$ and $\bar{x}^{i+1'}$, so that its function value need not be computed again. This just requires some easy algebra. By symmetry, whatever choice is made the interval restricts to being of size rD . Let us consider the case where the interval $[\underline{x}, \underline{x}']$ is eliminated (the other case is symmetric). As shown in Figure 2.4, $\underline{x}$ is at distance rD from $\bar{x}$, while $\bar{x}'$ is at distance $(1-r)D$. The question is whether the “surviving” $\bar{x}'$ is on the left of the middle of the new interval, and therefore becomes $\underline{x}'$ at the next iteration, or on the right and therefore remains $\bar{x}'$. In order for it to be on the left it must hold

$$1 - r > r/2 \quad \equiv \quad r < 2/3;$$

since we prefer r to be as small as possible (whilst $r > 1/2$), to obtain the fastest possible convergence, we assume that this will be the case. Hence, $\bar{x}'$ will become $\underline{x}'$ at the next iteration. This means that (ignoring the constant factor D on all terms) $1 - r$ will have to correspond to the fraction r of the new interval $[\underline{x}', \bar{x}]$ of length r . In formulæ

$$r : 1 = (1 - r) : r \quad \equiv \quad r \cdot r = 1 - r;$$

that is, r has to be a solution of the quadratic equation $r^2 + r - 1 = 0$. These are well-known to be $(-1 \pm \sqrt{5})/2$, one of which is negative: then, the only choice is $r = (\sqrt{5} - 1)/2 \approx 0.618$ ($< 2/3$). This is a somewhat “magic” number: $r = 1/g$, where $g = (\sqrt{5} + 1)/2 \approx 1.618$ is known as the *golden ratio* [71], and it has a number of nice geometric and algebraic properties, such as $g = 1 + r = 1 + 1/g$. However, in our case this just gives us the crucial number that allows us to define the *golden ratio search* algorithm.

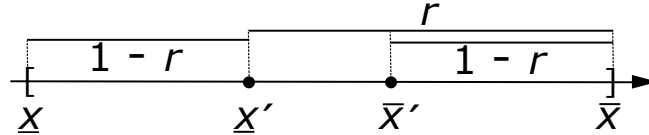


Figure 2.4: Derivation of the Golden Ratio rule

Algorithm 2.2. Golden Ratio Search

```

procedure  $[\underline{x}, \bar{x}] = \text{GRS}(f, \underline{x}, \bar{x}, \delta)$ 
   $\underline{x}' \leftarrow \underline{x} + (1 - r)(\bar{x} - \underline{x})$ ;  $\bar{x}' \leftarrow \bar{x} + r(\bar{x} - \underline{x})$ ; compute  $f(\underline{x}')$ ,  $f(\bar{x}')$ ;
  while  $(\bar{x} - \underline{x} > \delta)$  do
    if  $(f(\underline{x}') > f(\bar{x}'))$ 
      then  $\{ \underline{x} \leftarrow \underline{x}'; \underline{x}' \leftarrow \bar{x}'; \bar{x}' \leftarrow \underline{x} + r(\bar{x} - \underline{x}); \text{compute } f(\bar{x}'); \}$ 
      else  $\{ \bar{x} \leftarrow \bar{x}'; \bar{x}' \leftarrow \underline{x}'; \underline{x}' \leftarrow \underline{x} + (1 - r)(\bar{x} - \underline{x}); \text{compute } f(\underline{x}'); \}$ 

```

By the previous analysis, $\bar{x}^i - \underline{x}^i = D^0 r^i$ with $r \approx 0.618$ gives that the algorithm stops in at most (approximately) $4.78 \log(D/\delta)$ iterations: in fact, $\log(1/0.618) \approx \log(1.618) \approx 0.21$, and $1/0.21 \approx 4.78$. Thus, the algorithm requires about 44% more iterations than the previous version to stop ($4.78/3.32 \approx 1.44$). However, since now each iteration requires only one function evaluation, as opposed to two, the new algorithm requires only the 72% of the function evaluations of the previous one ($4.78/6.64 \approx 0.72$). The difference is not staggering, but significant enough so that the golden ratio search is the preferred algorithm *if no other information is available on the function besides the function value*. Indeed, in this case the algorithm can be proven to be asymptotically optimal [1, p. 355]. It can also be proven that, if the total number n of steps to be performed is fixed in advance (and it is, once D and δ are given), then a slightly better performance can be obtained by using a *variable* r given by the ration of two successive *Fibonacci numbers*: $r^i = F_{n-i}/F_{n-i+1}$, where F_i is the i -th element of the Fibonacci sequence [69]. This is asymptotically the same, as the ratio of two consecutive Fibonacci

numbers tends to the golden ratio as i increases. We leave the details of these entertaining, but ultimately not particularly useful, versions to the interested reader; the point here is to underline the fact that better algorithm can be devised by more carefully studying the problem at hand. In fact, the significance of these results is somewhat limited in view of the fact that *obtaining more information about the function is possible*, which makes it possible in turn to devise even more efficient algorithms.

Before doing so in the next section, let us stress again the main point of this one: *finding local minima is exponentially faster than finding global ones*. This statement should be tempered by the observation that the algorithm is supposed to work on a unimodal function, and many functions are not such. However, *the algorithm can be applied as well to a non-unimodal function*. Indeed, it is clear that nothing in the algorithm “really breaks” if f is not unimodal. What happens if there are multiple local minima, and therefore multiple attraction basins, in the original interval $[\underline{x}, \bar{x}]$ is fairly obvious: at some iteration, the algorithm will *unknowingly eliminate some local minima* and the corresponding attraction basin. Sooner or later, the current interval $[\underline{x}, \bar{x}]$ will be “small enough” so that f will actually be unimodal in there, and the algorithm will eventually (approximately) find the local minimum in there. Thus, in some sense the algorithm operates in *two phases*:

1. in the *global phase*, where $[\underline{x}, \bar{x}]$ contains multiple local minima, the algorithm “randomly” chooses one among the (possibly, many) attraction basins to focus on;
2. in the *local phase*, where $[\underline{x}, \bar{x}]$ finally contains only one attraction basin, the algorithm works to find the corresponding local minimum.

Of course, “randomly” in the previous description has not to be taken literally: the algorithm is rigidly deterministic, and its path is established once the initial interval is given. However, the word underlines the fact that there is in general absolutely no control on which of the (many) local minima the algorithm will ultimately be attracted to: very small changes in the initial interval may dramatically alter the outcome, and it is in general impossible to gauge beforehand if a given choice will yield a “good” local minimum (say, the actual global minimum x^*) or a “bad one”. In other words, there always is the chance that the *local* algorithm will find a global solution, but there is no control on how likely this is to happen. Intuitively, “if there are few local-not-global minima and their attraction basins are small”, there could be a “good chance” that the algorithm indeed finds the global one; but this is only a qualitative statement (unless it can be made precise in the very strong sense of §2.8). To further underline the total lack of control on the quality of the obtained result, let us mention that, at each given iteration, the algorithm has no way of knowing which phase it actually is in.

Hence, local algorithms have rather dramatic trade-offs w.r.t. global ones:

1. the complexity of local optimization is (dramatically) lower;
2. the returned local minimum can be (dramatically) worse than the global one.

While this has been described in details for just this particular case, the principle is general and will hold true throughout all these lecture notes. In plain words: all local minima “look the same”, *comprised the global one*. This makes it finding *some* local optimum “a lot easier”, but on the other hand *once a local minimum has been found, there is no way of telling whether or not it is the global one*. Finding the global minimum, and *proving* it is such, is a completely different problem that these lecture notes will not tackle in any detail, save for a fleeting glimpse—on entirely intuitive terms—in §2.8. The issue can, again, be linked to the *difficulty of estimating the value of f_** ; indeed, if one knew that, it would be easy to ascertain whether or not the found local minima $\bar{x}$ is a global one. While one could then immediately stop in case this were true, it would not be entirely obvious what to do if this turned out not to be the case; again, this cannot be discussed in any detail in these lecture notes, save for the brief hints in §2.8. What matters in our discussion is that, since f_* is generally unknown, local and global optimization are two very different processes.

Due to the lack of any guarantee on the quality of the returned solution, one may therefore deem local algorithms to be scarcely useful. This is not the case, for a number of reasons:

- in some “lucky” cases the two processes actually are the same (cf. again §2.8), and in several applications it is in fact possible to build the optimization problem in order ensure that this actually happens;
- even if the problem cannot be guaranteed to be solved exactly, it may be that in practice the quality of the solutions returned by a local algorithm is most often satisfactory: this is in fact typically the case for the applications of interest in these lecture notes, where the functions to be minimised are *not adversarial*;
- when the size of the problem grows—as it invariably does in the applications of interest in these lecture notes—the complexity of global optimization becomes so dramatically high (cf. §3.1.1) that attempting it is completely out of question: running a local optimization approach, and having a chance (even if possibly slim) at getting a solution of reasonable quality is simply the only option.

For all these reasons, these lecture notes will almost entirely focus on *local algorithms*. The issue of the quality of the obtained solution will simply be “showed under the rug and kept there”: in many applications it will anyway be satisfactory (even optimal), and in case it is not, nothing can be done about it. Of course, many things *can* in fact be done, but these will not be discussed here: the curious readers can refer to the many other available courses in optimization [28].

2.5 Towards faster local optimization algorithms

The discussion of the previous sections can perhaps be summarised by the following (somewhat obvious) tagline: *you can get much more efficiently where you want to if you know which way to go*. In our case, “where we want to go” is a local minimum, and—assuming unimodality—our “directions” are: “go right if the function is decreasing, go left if the function is increasing”. However, to gauge where the function is increasing or decreasing we need its value in no less than four points. It would clearly be interesting to be able to obtain this information more efficiently, and this is of course possible with a very well-known mechanism.

The idea is quite simple: knowing $f(x)$ for some x , one would take some z “very close” to x and compute $f(z) - f(x)$; if this is positive the function “looks increasing” in $[x, z]$, otherwise it “looks decreasing”. However, we know that functions “can change very rapidly”, hence if the interval is “large” this information may not be very significant because the function may “change mode” even many times inside the interval; hence, we want the interval to be “very small”. We could therefore consider $\lim_{h \rightarrow 0} f(x+h) - f(x)$, but this limit should always be 0 if f is continuous, which is a baseline assumption. The solution is to rather consider:

Definition 2.1. The (first) *derivative* of f at $x \in \mathbb{R}$ is the limit of the *incremental ratio* of f between x and $x+h$ when $h \rightarrow 0$, i.e.,

$$f'(x) = \lim_{h \rightarrow 0} (f(x+h) - f(x)) / h, \quad (2.2)$$

provided it *exists and it is finite*.

It is easy to see that the incremental ratio is the slope of the linear function interpolating f at x and $x+h$, i.e., passing by $(x, f(x))$ and $(x+h, f(x+h))$. Thus, $f'(x)$ —if it exists—can be interpreted as the *slope of the line tangent to the graph of f at the point $(x, f(x))$* : this is the *first-order model of f at x*

$$L_x(z) = f'(x)(z-x) + f(x), \quad (2.3)$$

i.e., the “best possible linear approximation of f close to x ”. As such, “the derivative is our GPS”: if it is negative then f is decreasing in x , if it is positive then f is increasing in x .

This can be easily verified for our simple functions of §1.2. Indeed, for $f(x) = bx$,

$$(b(x+h) - bx) / h = b,$$

i.e., the derivative is the constant function $f'(x) = b$: the function has always the same slope (always increasing if $b > 0$, always decreasing if $b < 0$), hence it has no minimum. For the (homogeneous) quadratic function $f(x) = ax^2$,

$$f'(x) = \lim_{h \rightarrow 0} [(a(x+h)^2 - ax^2) / h = a(h^2 + 2hx) / h = a(h + 2x)] = 2ax.$$

Hence, if $a > 0$ the derivative is negative when $x < 0$ and positive if $x > 0$, meaning that the minimum is in $x = 0$ (where $f'(x) = 0$). Thus, at least in this case—not surprising, since the function is clearly unimodal on all $\mathbb{R}$ —the derivative is a reliable indication about “which way the minimum is”. Indeed, in this example it is also a reliable indication about “where exactly the minimum is”: $f'(x) = 0$ in the minimum. However, if $a < 0$ the sign swaps and $x = 0$ is the maximum, as we know well; thus, the derivative being zero is a not necessarily a reliable indicator per se, and it needs to be combined with other information.

Of course, none of this would be any helpful if derivatives were “hard to compute”. However, if f is a “reasonably simple function”, then its derivative also is. That is, if f is an algebraic function, obtained combining the standard “basic” functions (polynomials, exponentials, trigonometric, ...) by the standard algebraic operations (sum, product, composition, ...), then the very same expression tree used to represent the computation of f can be tackled to efficiently compute f' . This is well, known, and briefly recalled in §A.7 and the references therein. In fact, there are efficient ways to compute derivatives even for “rather complicated functions”, as we shall see. In particular, if *actual computer code* is available that implements the computation of $f(x)$, then well-developed, available and efficient *automatic differentiation tools* exist [27] that can automatically—under some assumptions on the original one—*produce code that computes $f'(x)$* . For this reason, in all these lecture notes we will assume that *derivatives are generally available for about the same cost as computing the function, and that doing so is none of our business*. This is a simplification: there are indeed cases, among which some regarding the applications of interest for these lecture notes, where the computation of derivatives is highly

nontrivial and the subject of ongoing research. We will not have the possibility of going in any of these details, though, so the general stance remains that “derivatives are there”.

Of course, computing $f'(x)$ can only be considered if it *exists at all*, which means that the limit in (2.2) must *exist and be finite*. In this case we say that f is *differentiable at x* . If the function $f' : \mathbb{R} \rightarrow \mathbb{R}$ is well-defined and continuous over all points of a set $X \subset \mathbb{R}$ we say that f is *continuously differentiable over X* . Given our reluctance to specify domains and suchlike, we will particularly cherish functions that are continuously differentiable over the whole of $\mathbb{R}$, which we will denote by “ $f \in C^1$ ”. The notation is appropriate, since *if f is differentiable at x then f is continuous at x* ; that is, $f \in C^1 \implies f \in C^0$.

Exercise 2.4. Prove this result. [solution]

Thus, if f is not continuous at x is cannot be differentiable there. However, there are functions that are continuous, but still not differentiable. A convenient concept is that of the *left and right derivatives*

$$f'_-(x) = \lim_{h \rightarrow 0^-} (f(x+h) - f(x)) / h \quad , \quad f'_+(x) = \lim_{h \rightarrow 0^+} (f(x+h) - f(x)) / h ;$$

f is differentiable at x if and only if both the left and right derivative at x exist, are finite, and $f'_-(x) = f'_+(x)$. Our preferred example of nondifferentiable function is $f(x) = |x| = \max\{x, -x\}$, which is continuous everywhere: yet, it is clearly not differentiable at 0 (while it is everywhere else), since it is immediate to see that $f'_-(0) = -1 \neq 1 = f'_+(0)$. In plain words, the function “looks linear both left and right of 0, but with two different slopes”. In fact, it is common to say that “the derivative of the absolute value is the *sign()* function”, which is not continuous; we will see later one some more precise statement that partly justify (but largely confutes) this definition. This is by no means the “worst case”, as functions can be found whose left and right derivative at some x are “as much different as possible”.

Exercise 2.5. Exhibit such a function. [solution]

The example $f(x) = |x|$ clearly illustrates what lack of differentiability means: basically, “there is a kink in the graph of f ”. That is, the graph is not a “smooth line”, with a well-defined tangent at each point; rather, in some point the graph “abruptly changes direction”. This is why *nondifferentiable* functions are also dubbed *nonsmooth*.

Before moving decidedly towards describing how derivatives can be used for optimization we need to introduce the concept of *higher-order derivatives*; in particular, $f''(x)$, i.e., the *second derivative* of f . Fortunately, this is “trivial”: $f'' = (f')'$, i.e., just the derivative of the derivative. In this sense, f' is also called the *first derivative*. Since one is just applying the same concept again, there is not much that needs be said: f'' is typically “easy to compute” by applying to f' the same process used to produce f' out of f , and the sign of $f''(x)$ tells whether f' is increasing or decreasing in x . This information will turn out to be useful for our purposes, first in terms of *optimality conditions*—identifying which x are local minima—and then, as we shall see, to construct actual algorithms. Of course, there is nothing preventing to play the same game an arbitrary number of times: for instance, the *third derivative* $f'''(x)$ is just $(f'')'$, i.e., the first derivative of the second derivative, or alternatively $(f')''$, i.e., the second derivative of the first derivative. In practice, at least for the approaches described in these lecture notes, no derivatives higher than second-order ones will ever be considered. However, occasionally some mention of third derivatives (but not any higher) will be done in some theoretical results. Let us just finish by commenting that, by the result of Exercise 2.4, continuity of higher-order derivatives implies continuity of all lower-order ones and of the function. That is, for C^2 the class of functions with continuous second-order derivatives, one clearly has $f \in C^2 \implies f \in C^1 \implies f \in C^0$. Of course, for C^3 the class of functions with continuous third-order derivatives, $f \in C^3 \implies f \in C^2 \implies f \in C^1 \implies f \in C^0$. The class C^3 is the “strictest” possible one that will ever be mentioned in these lecture notes, and even there only in a few theoretical results. In general one is more than happy with C^2 functions, and C^1 ones (not necessarily C^2) are also common in some applications. Indeed, these lecture notes will even consider non- C^1 , i.e., nonsmooth, functions, since these are required for some important applications.

The last important remark that needs be done concerns the relationship between Lipschitz continuity and the derivative. The result is quite intuitive: L-c implies that the slope of the function cannot be too steep (either positive or negative), and therefore *boundedness of the derivative*. Formally: $f \in C^1$ is *globally L-c* on some (open) set X , with a given constant $L > 0$, implies that $|f'(x)| \leq L$ for all $x \in X$, and the converse also holds.

Exercise 2.6. Prove the last statement. [solution]

It is therefore “reasonably easy” to construct functions that are not globally L-c on the whole of $\mathbb{R}$ by ensuring that the derivative is not bounded: the simple $f(x) = ax^2$, with $f'(x) = 2ax$, immediately provides a clear example. Yet, L-c is easy to achieve on *bounded sets*, in that it just corresponds to continuity of the derivatives. This is due to the *extreme value theorem* [13, Th. 2.2.9], stating that every function that is continuous on a bounded set achieves maximum and minimum there; to keep notation simple, for $f \in C^0$ and $X = [\underline{x}, \bar{x}]$ with

$$-\infty < \underline{x} \leq \bar{x} < +\infty$$

$$\max\{f(x) : x \in X\} < \infty \quad \text{and} \quad \min\{f(x) : x \in X\} > -\infty.$$

As an immediate consequence, if $f \in C^1$ on such a finite X then it is f globally L-c on X , since f' is bounded there. By the same token (and this will be useful), $f \in C^2$ (on a finite X) means that f' is L-c on X , and so on. However, given all the previous description it is not difficult to construct examples of C^0 functions that are *not* L-c on some finite $X = [\underline{x}, \bar{x}]$.

Exercise 2.7. Exhibit such a function. [solution]

We are now in the position of discussing *optimality conditions* for univariate problems based on the derivatives. We introduce the discussion with an example.

Example 2.1. Local minima, maxima and saddle points of a polynomial

We consider the 10-th degree polynomial

$$f(x) = \frac{91}{30}x^2 - \frac{19}{6}x^3 - \frac{54}{25}x^4 + \frac{93}{23}x^5 - \frac{23}{36}x^6 - \frac{121}{93}x^7 + \frac{72}{91}x^8 - \frac{13}{74}x^9 + \frac{9}{640}x^{10}.$$

Using standard results, it is easy to show that

$$\begin{aligned} f'(x) &= \frac{91}{15}x - \frac{19}{2}x^2 - \frac{216}{25}x^3 + \frac{465}{23}x^4 - \frac{23}{6}x^5 - \frac{847}{93}x^6 + \frac{576}{91}x^7 - \frac{117}{74}x^8 + \frac{9}{64}x^9 \\ f''(x) &= \frac{91}{15} - 19x - \frac{648}{25}x^2 + \frac{1860}{23}x^3 - \frac{115}{6}x^4 - \frac{1694}{31}x^5 + \frac{576}{13}x^6 - \frac{468}{37}x^7 - \frac{81}{64}x^8 \end{aligned}$$

The function and its first and second derivative are shown in Figure 2.5. In particular, the six roots of the derivative, x^i , $i = 1, \dots, 6$, are shown (ordered in increasing sense). Since f' is a 9-th degree polynomial, the roots are 9, but the others are complex (in fact I'm slightly cheating, even x^4 is actually a pair of complex roots with real part ≈ 0.8881 , but it is difficult to construct good examples and this one is good enough, so please pretend). The function shows the following:

- x^1 corresponds to a (shallow) local minimum of f . This corresponds to the fact that $f''(x^1) > 0$. That is, f' is increasing in x^1 : it is negative before x^1 and positive after x^1 , meaning that f is decreasing before x^1 and increasing after it.
- Symmetrically, x^2 corresponds to a local maximum of f ; albeit “shallow”, it is in fact the local maximum with highest value (but not a global maximum since the function is unbounded above as $|x| \rightarrow \infty$). This corresponds to the fact that $f''(x^1) < 0$. That is, f' is decreasing in x^1 : it is positive before x^1 and negative after x^1 , meaning that f is increasing before x^1 and decreasing after it.
- x^3 is a local, and global, minimum of f : again this corresponds to the fact that $f''(x^3) > 0$.
- x^4 (disregarding the minor cheating) is a “strange” point, being neither a local minimum nor a local maximum. This corresponds to the fact that $f''(x^4) = 0$. In fact, x^4 is a local minimum of f' : f' is positive *both before and after it*. That is, “ f is increasing, it fractionally stops increasing in x^4 but then immediately resumes increasing”. This is called a *saddle point* of the function.
- x^5 is a local maximum of f ; albeit “not too shallow”, it is not the best local maximum. This corresponds to the fact that $f''(x^5) < 0$.
- x^6 is a local not global minimum of f ; this corresponds to the fact that $f''(x^6) > 0$.

Thus, the example shows that the roots of f' , and the corresponding values of f'' , identify local minima and maxima. However, x^4 also shows that this is only “authoritative” if $f''(x) \neq 0$: for null values of the second derivative the point may be a local minimum, a local maximum, or none of the two (a saddle point). It would be possible to further refine the analysis by looking up the values of even higher-order derivatives which ultimately would give a definite answer, but we will carefully avoid doing it. Anyway, as we will discuss, “even looking at the second derivative is too much in practice”.

A further, rather informal but maybe useful, description is the following: f' gives the slope of the graph of f and f'' gives its curvature. That is, $f' > 0$ says that the graph is “trending upwards” and $f' < 0$ that is “trending downwards”. f' characterise the “rate of change” in the function value. Indeed, the roots of f' are called *stationary point* to indicate that (locally) “the function is not changing”; the graph (if zoomed very close to the point) “looks flat”. Yet, the rate of change in itself would describe a straight line (think a linear function with its constant derivative, or the first-order model). Since f'' characterise the rate of change of f' , it describes how the rate of change of f is (locally) itself changing, i.e., “how the slope of the line is changing”, i.e., “how curved the function is”. A positive f'' implies that the slope is increasing, i.e., a “convex” curvature, while a negative f'' implies a “concave” one. In fact, the roots of f'' —typically, maxima or minima of f' —identify points where “the curvature briefly stops changing” and the graph is “locally linear”; this often happens when “the function changes curvature from convex to concave or vice-versa” (for instance, the root of f'' between x^2 and x^3), although in fact the slope may just briefly stop changing but then immediately resume its previous mode (cf. x^4). Albeit perhaps too pedestrian, this description may turn out to be informative when multivariate functions will be considered.

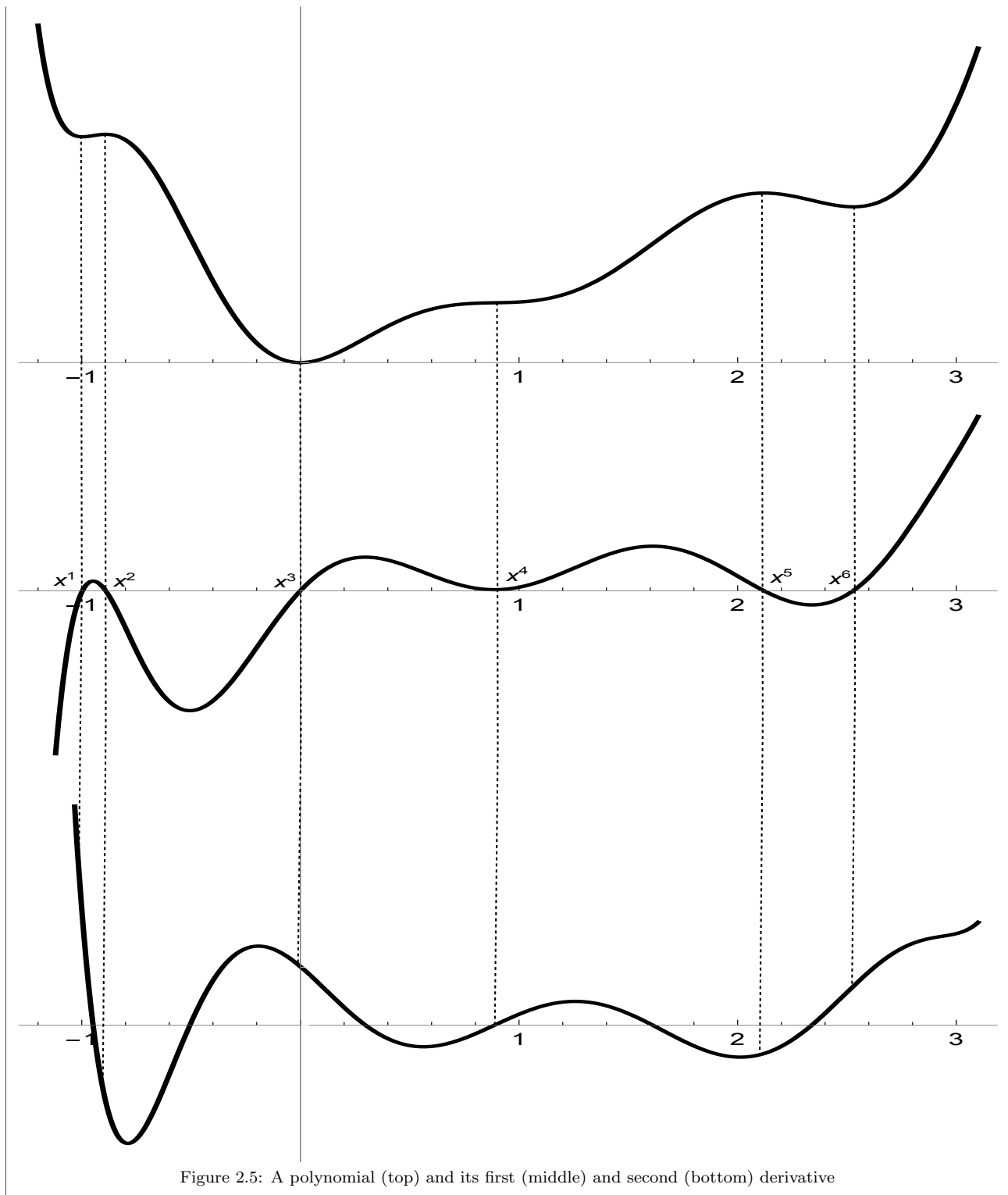


Figure 2.5: A polynomial (top) and its first (middle) and second (bottom) derivative

The example illustrates, among others, the following crucial property (that we won't formally prove as this will be done later on in a more general context): *if $f'(x) \neq 0$, then x cannot be a local minimum (or maximum).* In other words, *if we want to find a local minimum we need to find a root of the derivative.* However, *not all roots of the derivative are local minima: some are local maxima, some other saddle points.*

A final important observation is the following: *even if we find a bona-fide local minimum ($f'(x) = 0$, $f''(x) > 0$), there is no way of saying whether or not it is the global minimum (even assuming a global minimum is known to exist).* In plain words, “all local minima looks equal”. It is possible to efficiently find (approximate) roots of f' , that may (or may not) be local minima, but ascertain whether or not they are *global* minima is *an entirely different process*. This will almost completely ignored in these lecture notes, save for some very brief hints at the end of this Chapter (§2.8). Of course, this is an issue *if there are local-not-global minima* at all. This may not happen, with a (globally) unimodal function providing convenient example. However, for reasons

to be discussed later on, unimodality is not the most convenient concept to use. Rather, the development in this section allows us to define (in a stricter way than necessary, but this will also be discussed later on) “the class of functions we really love to minimise”.

Definition 2.2. $f \in C^2$ is *convex* on $X \subseteq \mathbb{R}$ if $f''(x) \geq 0$ for all $x \in X$.

As we will see this is only a very special case of a much more general definition, but it will do for our purposes here. Needless to say, $f(x) = ax^2$ with $a \geq 0$ is convex. It is immediate to see (again, no formal proof as these will be deferred to the more general treatment later on) that if a convex function has a local minimum—this is not given: a linear function is convex—then it is unimodal on all $\mathbb{R}$, which means that *any local minimum is the global minimum*. Besides, x is a local minimum if and only if $f'(x) = 0$, i.e., *global minima are the same as stationary points*. Thus, convex functions are “perfect to minimise”: not only “you always know the way towards the global minimum”, but also “as soon as you are close to a local minimum you know it because $f' \approx 0$ ”. This is drastically different from the general situation. Fortunately, as we shall see, “there are many convex functions”: a fair number of basic functions are convex, and a number of operations between functions preserve convexity. Thus, *convex minimization* can happen in a reasonable number of cases, which justifies why there is plenty of theory [2], and software [29], for it. Indeed, if one has some choice about the function to be minimised—which happens in some of the applications of interest for these lecture notes, whereby one is constructing *models*—then all other things being equal one should most definitely choose convexity. Even if the convex model is somehow less than ideal for the application, the optimization problem being dramatically easier could very well tilt the balance towards a convex function being chosen anyway.

Motivated by convexity, we now take a fundamental stance that we will keep throughout all these lecture notes (save very briefly in §2.8): *our aim is to find any stationary point, i.e., root of f'* . This is mostly because efficient algorithms exist for doing so. Then, if f is convex this is a guaranteed global minimum. Otherwise, it may hopefully be a local minimum, and therefore it still has a chance of being a global one; this indeed happens frequently in some prominent applications of interest for these lecture notes. If we are “unlucky”, we will just put up with it: at least the “unsatisfactory” solution has been obtained quickly, and if there is still time to spare some other mechanism (which will not be discussed) can be put in place to try to find better ones. That is, *efficient local optimization is the cornerstone of global optimization*, although for us the two “coincide” (in the sense that we only consider global optimization when it is equivalent to local).

2.6 Dichotomic Search

We have then reduced our optimization task—finding the minimum of f —to an essentially algebraic one: *find one of the roots of f'* (hoping there will only be one, and it will correspond to the global minimum). This first and foremost requires $f \in C^1$: the perennial counterexample $f(x) = |x|$ clearly shows that roots may not exist if f' is discontinuous (exactly where the minimum lies). It is obvious that in very simple cases this can actually be done by a closed formula. For instance, for a linear function $f(x) = bx + c$, $f'(x) = b = 0$ is only possible if $b = 0$, reproducing §1.2.1. For the quadratic non-homogeneous $f(x) = ax^2 + bx + c$, $f'(x) = 2ax + b = 0$ yields $x = -b/2a$, clearly the unique minimum if $a > 0$ and maximum if $a < 0$, reproducing §1.2.3. This generalises to cubic and quartic polynomials, whose derivatives have respectively degree 2 and 3 and for which therefore root have a (somewhat involved in the cubic case [61]) closed formula; but almost nowhere else. Algorithms for polynomial factorization exist that can work efficiently up to quite high degrees [68], although they are far from trivial. However, there is little hope for most algebraic functions containing transcendental and/or trigonometric functions. One therefore needs an *algorithm* for solving the *nonlinear* equation $f'(x) = 0$. Fortunately there are efficient algorithms for finding the root of a continuous function over an interval *in which a root is guaranteed to exist*. This is down to the well-known *intermediate value theorem* [13, Th. 2.2.10] (basically, the intuitive definition of continuous function) that guarantees the following: if $f'(\underline{x}) < 0$ and $f'(\bar{x}) > 0$ then there exists $x \in [\underline{x}, \bar{x}]$ such that $f'(x) = 0$. Assuming one knows beforehand an interval with the right property (the assumption will be challenged soon), the *theorem breeds an algorithm*, since it is then trivial to restrict the interval by just computing $f'(x)$ in any point inside the interval. This is the basic *dichotomic search* approach:

Algorithm 2.3. Dichotomic Search

```

procedure  $x = DS(f, \underline{x}, \bar{x}, \varepsilon)$ 
  do forever // invariant:  $f'(\underline{x}) < -\varepsilon, f'(\bar{x}) > \varepsilon$ 
     $x \leftarrow \text{in\_middle\_of}(\underline{x}, \bar{x})$ ; compute  $f'(x)$ ;
    if ( $|f'(x)| \leq \varepsilon$ ) then break;
    if ( $f'(x) < 0$ ) then  $\underline{x} \leftarrow x$ ; else  $\bar{x} \leftarrow x$ ;

```

The trivial (and sensible) choice of choosing the middle point, i.e.,

`in_middle_of( $\underline{x}$ ,  $\bar{x}$ ) { return( $(\bar{x} + \underline{x}) / 2$ ) }`,

clearly gives the standard $\bar{x}^i - \underline{x}^i = Dr^i$ with $r = 0.5 < 0.618$; that is, assuming that the cost of computing $f'(x)$ is close to that of computing $f(x)$ (a nontrivial, but not too wild, assumption) the algorithm should be faster than the Golden Ratio Search. The analysis then follows a well-beaten path: $D^0 r^i \leq \delta$ surely happens when $i \geq \log(D/\delta) / \log(1/r)$, i.e., $i \approx 3.32 \log(D/\delta)$. Now, we know that $f'(x) = 0$ for some $x \in [\underline{x}^i, \bar{x}^i]$; assuming f' is L -c with constant L , then

$$|0 - f'(\underline{x}^i)| \leq L|x - \underline{x}^i| \quad \text{and} \quad |f'(\bar{x}^i) - 0| \leq L|\bar{x}^i - x|.$$

This means that

$$\min\{f'(\underline{x}^i), f'(\bar{x}^i)\} \leq L \min\{|x - \underline{x}^i|, |\bar{x}^i - x|\} \leq L\delta/2$$

(in the worst case, x is equidistant from the extremes); thus choosing $\delta = 2\varepsilon/L$ ensures that the stopping criterion have to be satisfied. All in all, the algorithm will surely terminate after at most $3.32 \log(LD/2\varepsilon)$ iterations. Whether this is better or worse than the Golden Ratio Search also depends on how the Lipschitz constant of f compares with that of f' . In fact, requiring f' to be L -c (i.e., f to be L -smooth) in the first place is nontrivial, but as we shall see is very common when results about the speed of convergence of derivative-based algorithms need be proven. This is intuitive: the “crucial function” is no longer really f , but rather f' . Hence, it is the derivative that need “not to change too fast” (which *can* happen, cf. Exercise 2.7) for being able to exercise some control.

The whole approach hinges on having the invariant ($f'(\underline{x}) < 0$ and $f'(\bar{x}) > 0$) satisfied at the beginning. Of course, even choosing a random x one of the two conditions will surely be satisfied, so the issue is ensuring the other. We will see later on that in many cases $f'(\underline{x}) < 0$ will hold, so we deal with the problem of finding $\bar{x} > \underline{x}$ such that $f'(\bar{x}) > 0$; however, the opposite case would be symmetric. In practice this is hardly a significant problem, since the following very rough solution usually works:

Algorithm 2.4. Right extreme search

```

 $\Delta x \leftarrow 1;$                                 // or whatever value  $> 0$ 
while(  $f'(\bar{x}) \leq -\varepsilon$  ) do
     $\bar{x} \leftarrow \bar{x} + \Delta x; \Delta x \leftarrow 2\Delta x;$     // or whatever factor  $> 1$ 

```

The approach just explores values on the right of $\underline{x}$ “at rapidly increasing steps”. The expectation is that if a minimum is sought for on the right of $\underline{x}$, where f is decreasing, then “sooner or later the function will have to be increasing”, i.e., its derivative will turn positive. A formal way to ensure the property is to require that f is *coercive*, i.e., $\lim_{|x| \rightarrow \infty} f(x) = \infty$; in plain words, “when going away from the origin, sooner or later the function grows unboundedly”. Note that, for instance, both $|x|$ and x^2 are coercive. This is relevant in that in many applications of interest for these lecture notes these functions (or multivariate generalizations thereof) are explicitly added to the “original” objective, which (among other things) may have the nice side-effect to making it coercive. Thus, in the following we will skip all finer analysis and just pretend that Algorithm 2.4, if at all needed, will work. Of course, counterexamples can be carefully constructed so that this fails.

Exercise 2.8. Construct an example where $\bar{x} > \underline{x}$ such that $f'(\bar{x}) > 0$ exists but is not found by Algorithm 2.4 [solution]

Before abandoning this issue for good, it is worth remarking on the obvious case where Algorithm 2.4 cannot but fail: that where f has no local minimum on the right of $\underline{x}$ since it is always decreasing there, i.e., the derivative is always negative. A particularly clear case is that where the function is unbounded below on the right of $\underline{x}$, and $\{f(\bar{x}^i)\} \rightarrow -\infty$. This case is hardly relevant in the applications of interest for these lecture notes (cf. the previous comment about purposely adding a coercive term to the original objective), but it does underline the difficulty of dealing with unbounded problems: how does one *prove unboundedness*? Basically, what one would need is a “finite $-\infty$ ”: a large negative enough value M such that if one finds x with $f(x) \leq M$, then the problem can be safely declared unbounded below. This is actually possible in some cases, but these cannot be discussed here.

2.6.1 Improving the dichotomic search by interpolation

Although the dichotomic is already “fast”, it is possible to make it significantly more efficient in practice. The observation is that choosing the next iterate x “right in the middle” of the current interval $[\underline{x}, \bar{x}]$ is the reasonable choice to optimize the *worst-case* behaviour of the algorithm, but not the *best-case* one. Indeed, if one were able to choose x so that $f'(x) = 0$ (or close enough), then the algorithm would stop in exactly one iteration. This suggests that trying to select x is “close to x_* ” may lead to better performances; of course, the

issue is then to try to predict where x_* actually lies.

A powerful general way to approach the issue is that of exploit available information on f to build a *model of the function*, i.e., a *simple* function g that approximates f and “for which the required information is easy to find”; in our case this means “whose minimum can readily be found”. Then, one can hope that the minimum of g will be an approximation of that of f . This approach is particularly sensible in view of the fact that the algorithm has anyway to compute information about f by evaluating it: at least the values of the derivative $f'(\underline{x})$ and $f'(\bar{x})$ are known, and computing the values of the function $f(\underline{x})$ and $f(\bar{x})$ could obviously also be an option if the cost is limited. It is clear that, in its basic form, the algorithm “only partly uses” knowledge of $f'(\underline{x})$ and $f'(\bar{x})$ in that it only looks at their sign, not at their magnitude. Yet, clearly the magnitude may convey useful information. In fact, the function and derivative values can be used to construct a *model* of f , i.e., a “*simple*” function g that “somehow resembles f ”. This is a very important concept that we will see many other time during the course. Once the model is constructed it can be used to derive information, the typical one being *where the minimum of f would be if it were true that g was identical to f* , i.e., where the minimum of g lies. The obvious use of this information is to drive the selection of the next iterate, although, as we shall see, slightly more involved choices may be better.

Constructing a model of f given function (and derivative) values is *interpolation*, which requires choosing a form of the interpolating function. The simplest choice is that of a *polynomial*, which still requires to select a degree. A degree- k polynomial has $k + 1$ coefficients, and each information we have yields a linear equation in the coefficients to impose that g agrees with f (g' with f') there; thus, the degree cannot be too high. On the other hand, a linear g ($k = 1$) had no minimum. So, a reasonable value is $k = 2$, i.e., $g(x) = ax^2 + bx + c$, for which the interpolation requires

$$\begin{cases} a\bar{x}^2 + b\bar{x} + c = f(\bar{x}) & , & 2a\bar{x} + b = f'(\bar{x}) \\ a\underline{x}^2 + b\underline{x} + c = f(\underline{x}) & , & 2a\underline{x} + b = f'(\underline{x}) \end{cases} .$$

This is an overdetermined system, hence some of the conditions need be dropped. A natural choice is to only keep the two ones on the right, corresponding to imposing that $f'(\bar{x}) = g'(\bar{x})$ and $f'(\underline{x}) = g'(\underline{x})$. Notably, c is not involved in those two equations, but obviously c does not anyway influence where the minimum of g is, hence this does not seem a significant loss. A bit of simple algebra then yields

$$a = \frac{f'(\bar{x}) - f'(\underline{x})}{2(\bar{x} - \underline{x})} \quad , \quad b = \frac{\bar{x}f'(\underline{x}) - \underline{x}f'(\bar{x})}{\bar{x} - \underline{x}} ;$$

these are well-defined values since $\bar{x} > \underline{x}$, and furthermore $a > 0$ since $f'(\bar{x}) > 0$ while $f'(\underline{x}) < 0$. Thus, $g'(x)$ is a (strictly) convex quadratic function, whose (unique) minimum solves $2ax + b = 0$, i.e.,

$$x = \frac{\underline{x}f'(\bar{x}) - \bar{x}f'(\underline{x})}{f'(\bar{x}) - f'(\underline{x})} . \quad (2.4)$$

This is called the *secant formula*, and—as one should geometrically expect—it is easy to prove that $\underline{x} < x < \bar{x}$. Thus it is possible to use x from (2.4) as the next iterate in the dichotomic search, which is known as the “method of false position”.

Exercise 2.9. Prove that x is always in the middle between $\bar{x}$ and $\underline{x}$. [solution]

The process looks therefore pretty reasonable, and in fact it can be proven to (in a certain sense) improve upon the standard dichotomic search. In particular, [11, Theorem 2.4.1] proves the following:

Theorem 2.1. Let $f \in C^3$ be such that $f'(x_*) = 0$ and $f''(x_*) > 0$; then, there $\exists \delta > 0$ such that, however chosen $\underline{x} < \bar{x}$ in the interval $[x_* - \delta, x_* + \delta]$, the dichotomic search with quadratic interpolation started with $\underline{x}$ and $\bar{x}$ produces a sequence of iterates $\{x^i\} \rightarrow x_*$, and the convergence is *superlinear* with $p = (1 + \sqrt{5})/2$ ($1 < p \approx 1.618 < 2$).

In plain words: *if the algorithm is started close enough to a local minimum*, then it eventually converges there superlinearly fast. We don’t delve in the convergence proof since we will shortly see a similar but more significant one. The observation is that the radius δ of the interval around x_* in which the algorithm “works well” can be proven to be nonzero, but it is difficult to characterise in practice; basically, one has to run the algorithm and check how it is actually converging. Hence, in general there is no way of knowing whether or not one is starting inside the interval and will converge fast; but anyway, one does not really even know how many local minima there are in the initial interval and to which one the process will eventually land. Since the interval is (hopefully) shrinking, even if the initial interval is not “appropriate” it will (hopefully) eventually become such. Thus, the algorithm *eventually, in the “tail” of the process*, will be superlinear. Anyway, the definition of convergence always is concerned with the “tail” of the process, hence it is correct to say that the dichotomic search with quadratic interpolation is superlinear.

However, the issue of “when the tail actually starts” is obviously important. In practice, it is not very comforting that the algorithm will eventually be fast when it is close enough to some local minimum, if it will take an inordinate number of iterations to get there in the first place. One may surmise that this is not going to happen since the dichotomic search is at least linearly convergent to start with, but unfortunately this is no longer obviously true with quadratic interpolation. Indeed, linear convergence is predicated on choosing x^{i+1} “in the middle” of the interval; obviously, it keeps holding (with a worse convergence rate) provided that one ensures that x^{i+1} “always remains close enough to the middle”. That is, for $D = \bar{x} - \underline{x}$, if one ensures that $x^{i+1} \in [\underline{x} + \sigma D, \bar{x} - \sigma D]$ for some $\sigma \leq 1/2$ then we know that the interval length will be reduced by at least a fraction of σ , which yields linear convergence with ratio $r = 1 - \sigma$. However, “pure” dichotomic search offers no such guarantee.

The issue is relevant in that it is conceptual: the point is that the efficiency of the approach is predicated on g being a “good model” of f , but there is no guarantee that this will in fact happen. In plain words: *never fully trust a model that you are not certain is “good”*, for otherwise you may be led to making “very bad choices”. In this particular case, the model g can end up being “very skewed”: it is plain to see from (2.4) that if $f'(\bar{x}) \gg -f'(\underline{x})$ then x will end up being “very close” to $\underline{x}$, and, symmetrically, if $f'(\bar{x}) \ll -f'(\underline{x})$ then x will end up being “very close” to $\bar{x}$. This *may* be a bad choice since one of the two intervals is “very small”; if it regularly ends up being the selected one, the algorithm may converge rather slowly. This is not only a theoretical issue, as the behaviour can show up in practice, leading to an algorithm that has a rather inefficient *cruise phase*—the sequence of iterations from the start to when the right interval of the local minimum is finally reached—and that only becomes efficient in the *local phase* at the tail of the convergence process.

This is a common phenomenon in *model-based algorithm*, that can be faced in a general way: *never completely trust the model*. The details of how this is done vary, and we’ll see a number of different occurrences; in this case, however, the solution is pretty easy. One just has to pick a *minimum guaranteed decrease* $\sigma \leq 1/2$ as a *safeguard*, and then choose the next iterate as

$$x \leftarrow \max\{\underline{x} + \sigma(\bar{x} - \underline{x}), \min\{\bar{x} - \sigma(\bar{x} - \underline{x}), x'\}\}$$

with x' the value suggested by (2.4). This guarantees that the next iterate will never be “too close” to the extremes, and therefore linear convergence with ratio $r = 1 - \sigma$ during the cruise phase; eventually, superlinear convergence will hopefully kick-in at the tail.

While simple, the solution has an obvious issue: *how should one choose σ ?* There is no clear answer to this question, as the choice involves navigating an *algorithmic trade-off*. The point is clearly illustrated by considering the extreme choice $\sigma = 1/2$. In this case, quadratic interpolation is basically deactivated: the algorithm will converge linearly with rate $r = 1/2$, even when “very close” to x_* . As σ decreases, the quadratic interpolation has a larger interval in which to choose the iterate, and therefore a better chance to actually drive the algorithm to faster convergence; but, of course, the other extreme $\sigma = 0$ incurs the risk of a rather slow cruise phase. This is a simple but poignant example of an *algorithmic parameter* that need be *tuned* to achieve the desired trade-off; in this case, between the algorithm being “robust” during the cruise phase and “fast” during the local one. We will see that algorithms can have several parameters, whose tuning can be highly nontrivial, especially in that it depends on the need of the specific application that drives the optimization process. For our case: is it more important to guarantee the worst-case behaviour (even if a slow cruise phase may materialise only very infrequently) or to get faster local convergence (even if serious slowdowns may frequently show up in the cruise phase)? Only considering the details of the actual application this can be answered. Fortunately, not all algorithmic parameters are particularly tricky to tune; in some cases, “default” values that tend to work well in almost all the cases can be found experimentally (and in the literature). However, finding the right value of other parameters may be nontrivial, and failing to do so can have a very dramatic impact on the algorithm performances. We will return on the issue as appropriate.

There clearly is another approach to the issue, though: *if the model is bad, make a better model* (vaguely reminiscent of the classical “if life gives you lemons, make a lemonade” of Portal 2 memory). In fact, this could be possible since in (2.4) we only used a part of the available information; arguably, by using more one could obtain a better model, that therefore may lead to a better behaviour (although it must never be forgotten that a model is, by definition, always going to be somehow inaccurate). There could be several ways for doing this even in our simple case. For instance, one could stick with $k = 2$ and choose any three of the four conditions (except as already done) to get different quadratic models; it is not really clear, though, why these should be significantly more accurate than the one we presented. A different approach to quadratic fitting can be found, e.g., in [1, p. 361]. Perhaps more reasonable is to move to $k = 3$, i.e., *cubic* fitting, since this entails finding four coefficients and therefore perfectly fits with the four conditions we can impose. Crucially, the derivative of a cubic polynomial is a quadratic one, hence its zeroes can be easily found; thus, it is possible to characterise where the minimum of the model in the interval lies. While not conceptually difficult, the approach is rather

tedious to write down, analyse and implement; thus we will give it a wide berth, since the details can anyway be found, e.g., in [11, § 2.4.2] and [10, p. 57]. Cubic interpolation is generally considered a good practical approach which can lead to even faster (quadratic) convergence; but rather than insisting on it we move to a more conceptually interesting idea that we will actually prove having (local) quadratic convergence.

2.7 Newton's method

In retrospective, all we have seen so far can be read as different ways to construct models of f that drive the algorithmic choices. Basically, the point is that we need to have information about where a local minima lies, and this requires forming some “quadratic-like model”, i.e., one where there is some information about “where the function decreases and where it starts increasing again”. This is “complex” information, and in fact we have seen it extracted from:

- function values at *four* different points;
- function *and derivative* values at *two* different points.

This could be summarised as “more derivatives, less points”, suggesting that one could try a further step: considering *second* derivatives. This is in fact possible, and a very powerful mechanism.

The crucial idea is the generalization of the first-order model (2.3) to the *second-order model*

$$Q_x(z) = f(x) + f'(x)(z - x) + \frac{1}{2}f''(x)(z - x)^2 = Q_x(z) + \frac{1}{2}f''(x)(z - x)^2, \quad (2.5)$$

i.e., the first-order one plus a second-order term using the second derivative. Taylor's theorem [12, Lecture 4] [13, §2.5] basically state that a higher-order model is “more accurate”—*close to x* —than a lower-order one, in a specific sense that will be discussed in more details when it will be needed. That is, Q_x can be expected to be a better model than L_x , at least close to x . Furthermore, Q_x inherently possesses the *curvature* that had to be somewhat haphazardly extrapolated from the data in the previous cases. This means that choosing the next iterate can be done with the simple formula

$$x^{i+1} = x^i - f'(x^i) / f''(x^i), \quad (2.6)$$

where x^{i+1} is just the minimum of Q_{x^i} computed with the simple tools of §1.2.3 and elementary algebra.

Exercise 2.10. Prove *Newton's step* (2.6). [solution]

This immediately gives rise to the deceptively *Newton's algorithm* as illustrated below; note that, unlike all previous ones, the algorithm does not work with any interval, and therefore it only needs one point to start.

Algorithm 2.5. Newton's method

```

procedure  $x = NM(f, x, \varepsilon)$ 
  while ( $|f'(x)| > \varepsilon$ ) do
     $x \leftarrow x - f'(x) / f''(x);$ 

```

The apparent simplicity of the algorithm hides a number of non-trivial points to discuss. First of all, *it is not at all a minimization algorithm*: (2.6) identifies the point x^{i+1} such that $L'_{x^i}(x^{i+1}) = 0$, but there is no guarantee that this is a minimum. Started close to a local maximum, the algorithm can happily converge there. The issue is that Newton's method is in fact a method for solving *non-linear equations*. The crucial observation is that *the derivative of the second-order model is the first-order model of the derivative*: for $f'(x)$, the first-order model reads

$$L'_x(z) = f'(x^i) + (f')'(x)(x - z) = (Q_x)'. \quad (2.6)$$

Thus, Newton's method can be interpreted as the *tangent method* applied to the solution of the nonlinear equation $f'(x) = 0$, where one just replaces the nonlinear function f' with its first-order model L'_x and then choses the next iterate as the (unique) root of the model; again, $L'_{x^i}(x^{i+1}) = 0$. The principle is illustrated in Figure 2.6. While the approach is sound and, under some conditions, can be extremely efficient, it is clear that it is *not able to distinguish a minimum from a maximum*, as the first derivative itself cannot. The other delicate aspect is that L'_{x^i} may not have any root at all: this happens if $f''(x^i) = 0$. In fact, in this case (2.6) is plainly ill-defined and the method simply breaks down. More in general, Newton's method may significantly suffer if iterates happen that generate “very small” values of $f''(x^i)$, unless the corresponding values of $f'(x)$

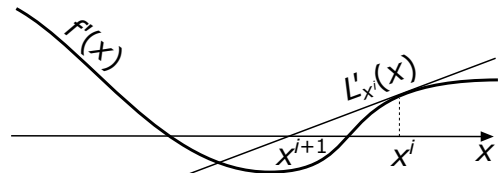


Figure 2.6: Illustration of the tangent method

are not “suitably small” as well.

Yet, the method can be proven to be, under conditions, extremely efficient. While the proof can be found in the literature (e.g., [1, Theorem 8.2.3]) we reproduce it here to give a taste of how these kinds of results can be proven.

Theorem 2.2. Let $f \in C^3$ be such that $f'(x_*) = 0$ and $f''(x_*) \neq 0$; then, there exists $\delta > 0$ such that, however chosen $x^0 \in [x_* - \delta, x_* + \delta]$, Newton's method started at x^0 produces a sequence of iterates $\{x^i\} \rightarrow x_*$, and the convergence is *quadratic*.

The proof hinges on the fundamental results that describes “how accurate” first- and second-(and higher-)order models are, i.e., Taylor's theorem. The particular form required in this case is *second-order Taylor's formula* [13, Theorem 2.5.4] stating that however chosen z there exists $w \in [x, z]$ such that

$$f(z) = L_x(z) + f''(w)(z - x)^2 / 2.$$

This says that “the second-order model would be *exact* at z if the second derivative—but not the first—were computed at some point w between z and x instead of at x ”. Note that the “magical” point w changes with z . However, the more interpretable version is

$$f(z) - L_x(z) = f''(w)(z - x)^2 / 2, \quad (2.7)$$

which says “the error between the true value of f and that of the first-order model is bounded by a term that depends quadratically in $z - x$ and on the value of the second derivative at some point between z and x ”. This is a lesser-known version of the more usual result concerning how the error of the model in z depends on the distance $z - x$ (quadratically, in the case of the second-order one). We will see the use of the more common form later on; the current proof requires (2.7) since it makes it explicit that the coefficient of the $(z - x)^2$ term depends on the value of the second derivative “close” to x .

Proof. Starting from (2.6), we subtract x_* from both sides and use $f'(x_*) = 0$ to get

$$x^{i+1} - x_* = x^i - x_* + (f'(x_*) - f'(x^i)) / f''(x^i).$$

Then, by simple algebra

$$x^{i+1} - x_* = [f'(x_*) - f'(x^i) - f''(x^i)(x_* - x^i)] / f''(x^i).$$

It is plain to see in the rightmost term, at the numerator, the error between *the value of the derivative* in x_* (which is known to be zero) and the value of the *first-order model of the derivative* in x_* . We can therefore use (2.7) *applied to f'* : there exists $w \in [x^i, x_*]$ such that

$$f'(x_*) - [f'(x^i) + f''(x^i)(x_* - x^i)] = f'''(w)(x_* - x^i)^2 / 2.$$

Plugging this in the relation above yields

$$x^{i+1} - x_* = [f'''(w) / 2f''(x^i)](x^i - x_*)^2,$$

where one clearly sees that $x^{i+1} - x_*$ can be bounded in terms of $(x^i - x_*)^2$ as required by quadratic convergence. However, one also needs to bound the term in the middle, that depends on the second and third derivatives of f (for the latter, in some unknown point w between x^i and x_*).

In order to upper bound the ratio, we have to prove that the numerator is bounded above, and the denominator is bounded away from zero. That is, we want to prove that there exists three constants $k_1 < \infty$, $k_2 > 0$, and $\delta > 0$ such that

$$|f'''(w)| \leq k_1 \text{ and } |f''(x)| \geq k_2 \text{ for all } x, w \in [x_* - \delta, x_* + \delta].$$

For k_2 , since $f''(x_*) \neq 0$ one has that $|f''(x_*)| > 0$. Hence, by continuity of f'' ($f \in C^3 \implies f \in C^2$) there exists an interval around x_* where f'' is always strictly positive, and in fact bounded away from 0. Formally speaking, take, e.g., $k_2 = |f''(x_*)| / 2 > 0$: there exists $\delta > 0$ such that

$$|2k_2 - |f''(x)|| \leq k_2 \implies |f''(x)| \geq k_2 \forall x \in [x_* - \delta, x_* + \delta].$$

We can now find k_1 : since f''' is continuous, also $|f'''|$ is. Hence, by the *extreme value theorem* [13, Theorem 2.2.9], the maximum of a continuous function on a compact set is finite:

$$k_1 = \max\{|f'''(w)| : w \in [x_* - \delta, x_* + \delta]\} < \infty.$$

We now rewrite the fundamental relationship as

$$|x^{i+1} - x_*| \leq \frac{k_1}{2k_2} (x^i - x_*)^2 = \left[\frac{k_1}{2k_2} |x^i - x_*| \right] |x^i - x_*| \quad (2.8)$$

(we have used the obvious facts that $a \leq |a|$ and $a^2 = |a|^2$ however chosen $a \in \mathbb{R}$). The crucial point is that for x^i close enough to x_* ,

$$0 < \frac{k_1}{2k_2} |x^i - x_*| < 1.$$

Hence, there is some $\delta > 0$ (possibly smaller than the one in the previous formulæ) such that, if $x^i \in [x_* - \delta, x_* + \delta]$, then $|x^{i+1} - x_*| < |x^i - x_*|$ and therefore $x^{i+1} \in [x_* - \delta, x_* + \delta]$ as well: once the sequence enters the interval it never leaves it, and $\{x^i\} \rightarrow x_*$. But then, (2.8) guarantees that the convergence is quadratic. $\square$

Thus, Newton's method has a very efficient local phase, *provided it is started close enough to some local minima or maxima* x_* . Looking at the proof it is obvious that actually computing the $\delta > 0$ that guarantees convergence, and therefore quadratic convergence, is nontrivial. Furthermore, there is in principle no guarantee that the local phase is ever started, even less how fast this eventually happens. There are ways to *globalize* Newton's method to provide it with a cruise phase that eventually converges to some local minima or maxima, hopefully fast towards the end. We do not delve in these details since we'll see globalization of Newton's method later on in a more general and interesting setting. We just finish the section with a few numerical illustrations.

Example 2.2. Execution of Newton's method

Let us start with the fantastic case of $f(x) = \frac{1}{2}x^2$. Then, $f'(x) = x$ and $f''(x) = 1$. The Newton's step is

$$x - f'(x) / f''(x) = x - x / 1 = 0$$

however chosen x , and 0 is the exact global minimum. In this case, $Q_x = f$: the model is “perfect”, hence minimizing the model immediately provides the true solution of the problem.

Of course, things can go worse. For $f(x) = \frac{1}{6}x^3$, the convergence theory does not work because $f'(x) = \frac{1}{2}x^2$ and $f''(x) = x$; hence, the only place where $f'(x) = 0$ is $x = 0$, where also $f''(x) = 0$. In fact, one has

$$x - f'(x) / f''(x) = x - \frac{1}{2}x^2 / x = \frac{1}{2}x.$$

It is therefore still true that $\{x^i\} \rightarrow x_* = 0$, but the convergence is linear (with rate $r = \frac{1}{2}$) rather than quadratic.

For more practical examples we turn to the polynomial $f(x)$ of Example 2.1. The following is the log of a by-the-book implementation of Newton's method on that f started from $x = -0.1$:

feval	rel gap	x	f(x)	f'(x)	f''(x)
1	3.3243e-02	-1.00000000e-01	3.3243e-02	-6.9098e-01	7.6253e+00
2	2.6968e-04	-9.38338883e-03	2.6968e-04	-5.7755e-02	6.2426e+00
3	5.2563e-08	-1.31629096e-04	5.2563e-08	-7.9871e-04	6.0692e+00
4	2.2300e-15	-2.71140416e-08	2.2300e-15	-1.6449e-07	6.0667e+00

Clearly, $x = -0.1$ is inside the interval predicted by the theorem: the negative exponent of the relative gap *doubles* at each iteration; this basically means that the number of right significant digits in the value of x doubles as well. It is clear that only a fixed number of iterations are required before the accuracy reaches the machine one, making quadratic convergence an exceptionally efficient process.

However, Newton's method is not a minimization process. In fact, started with $x = -0.8$ it runs as

feval	rel gap	x	f(x)	f'(x)	f''(x)
1	-2.4841e-02	-8.00000000e-01	1.6162e+00	-9.7155e-01	-1.2192e+01
2	-3.0160e-04	-8.79687906e-01	1.6568e+00	-9.2538e-02	-8.8770e+00
3	-8.4012e-07	-8.90112321e-01	1.6573e+00	-4.7004e-03	-7.9529e+00
4	-1.0608e-11	-8.90703358e-01	1.6573e+00	-1.6662e-05	-7.8964e+00
5	-1.3398e-16	-8.90705468e-01	1.6573e+00	-2.1350e-10	-7.8962e+00

Here, the gap is computed w.r.t. the f -value at the *local maximum* $x_* \approx -8.907$ where the algorithm is happily converging; note $f'(x_*) \approx 0$, $f''(x_*) < 0$, and the consistently negative relative gap as the function uniformly increase. The algorithm steadfastly maintain its quadratic rate of convergence with which it indeed very efficiently finds a root of f' , except this is not a minimum.

Besides, the unfettered division by $f''(x)$ is clearly risky. Ran with $x = 1$, the algorithm obtains

feval	rel gap	x	f(x)	f'(x)	f''(x)
1	4.3978e-01	1.00000000e+00	4.3978e-01	9.2412e-02	1.1291e+00
2	4.3511e-01	9.18152291e-01	4.3511e-01	3.2736e-02	3.1418e-01
3	4.3144e-01	8.13956159e-01	4.3144e-01	5.6550e-02	-7.6251e-01
4	4.3422e-01	8.88118273e-01	4.3422e-01	2.8010e-02	1.1920e-04
5	7.3305e+21	-2.34102775e+02	7.3305e+21	-3.1148e+20	1.1912e+19

The issue reveals itself at iteration 4, where $x \approx 8.881$ is close to a *saddle point*: it has a reasonably small value of f' , but a much smaller value of f'' . Thus, in one iteration the algorithm “shots away” in a completely unrelated region of space, worsening the gap by 22 orders of magnitude, and it is basically incapable of getting back to a “decent” solution since.

2.8 A Fleeting Glimpse to Global Optimization

We have seen that local optimization can be performed very efficiently, albeit with a fundamental caveat: there is no guarantee on the quality of the local minima one finds (and, possibly, not even if it is a local minima at

all). This turns out to be sufficient in the applications of interests for these lecture notes, and therefore our stance will be to largely ignore the problem. However, it is worth to provide at least a very high-level description of how one could design *global optimization* methods capable of (trying to) find a guaranteed global minimum. We know these methods cannot possibly be “very efficient”, but still there are a number of clever ideas that can be put to use to obtain approaches that are in fact efficient enough in several practical applications.

2.8.1 A very quick glimpse to convexity

The first, obvious (but still poignant) observation is that global optimization is difficult ... until it isn't. The point is that some functions, as we have seen, do not have local minima that are not also global ones. Thus, if one were sure to be minimizing one of these, (s)he could safely employ a local optimization approach with the certainty that a global minimum would be reached. The issue is, therefore, how to be sure that the function has this very desirable property.

The obvious request would be that f be *unimodal*, but in general it is not easy to verify if this is true. A stronger condition turns out to be more convenient, which can be intuitively understood as follows. Looking, e.g., to Figure 2.5, one can make the following intuitive consideration: if f has a local minima that is *not* the global one, then it also has local *maxima* (the function has to grow away from the local minima, then stop growing and start decreasing again to get to the global minima). Thus, *if one were able to avoid local maxima*, the function would have the desired property. Another way of saying this is the following: any stationary point should be a local minima, i.e., whenever $f'(x) = 0$, one also has $f''(x) \geq 0$. There is then an obvious sufficient condition for this to happen: $f''(x) \geq 0$ for all $x \in \mathbb{R}$. In other words, f' is *monotone increasing (non decreasing)*. It is clear that in this case, *all the points such that $f'(x) = 0$ must form a contiguous interval*, before which the derivative is always negative, and after which it is always positive. These are, therefore, all the saddle points of f . Clearly they are all local minima, and clearly they as well are also global minima; let us call such a function *convex*. We avoid proving these statements because we'll see these definitions again in a more formal way later on; for the time being we only stress that we have given a definition that requires $f \in C^1$, which as we will see is not necessary.

The reason why convexity is preferred to unimodality is that convex functions are easier to characterize, and even more to *construct*. In fact, it can be proven (as we will see later on) that a reasonably large number of “basic” functions are convex, and furthermore that *carefully chosen functional operations preserve convexity*. This means that there is a “grammar of convex functions” [2]: by choosing the “atoms” properly, and only applying carefully selected operations, one can be sure that the result is a convex function. Thus, “the convex world is reasonably large”: many different convex functions can be constructed, and there is even software for *disciplined convex programming* that enforces convexity [29]. This means that whenever one is constructing a model of a real-world problem, there is the choice to construct it convex. Of course, this is not always possible, as the model may therefore not properly represent reality. However, in many cases “reality” is not clearly definite; this is, for instance, the case in some *data-driven models* where one is trying to approximate some unknown function by picking the one that fits best in a(n infinite) family of functions with given forms. By properly choosing the target family of functions one can ensure that the resulting optimization problem is convex. We have already seen an example in §1.7, and we will see other cases later on (as well as cases where this does not happen). All in all, the case where a local optimization approach can be guaranteed to actually deliver global minima is by far not uncommon, especially if there is leverage in choosing aspects of the problem to be solved. In optimization, the catchphrase is “if you have the choice, choose convex” (when minimizing, we'll see the other direction); and it does happen that one has the choice. Actually, convexity arguments typically turn out to be crucial even if the problem is not convex.

2.8.2 Exploiting convexity

Thus, *global optimization is easy provided that f is convex*, since it then boils down to local optimization. This can be exploited algorithmically in several ways to help optimize *nonconvex* functions. Here we will provide an *extremely informal* description of the most important class of *exact algorithms* for global optimization, called *Spatial Branch-and-Bound* (SBB) approaches. The description will be provided by means of graphical examples only, leaving out all the (many) intricate algorithmic details that can be found in the literature (e.g., [8]) and that would require a whole course in themselves.

We consider global optimization on a *finite interval* $X = [\underline{x}, \bar{x}]$, i.e., with $-\infty < \underline{x} < \bar{x} < \infty$; finiteness is crucial for the approach. The crucial step on which the SBB is based—and whose absolutely nontrivial implementation we will *not* discuss—is that of finding a *convex valid global underestimator on X* , i.e., a *convex* function $\underline{f}$ such that $\underline{f}(x) \leq f(x)$ for all $x \in X$. This is represented by the blue line in Figure 2.7(left), which always lies below the black line representing (the graph of) f *inside* X ; outside the interval this is not necessary, as the

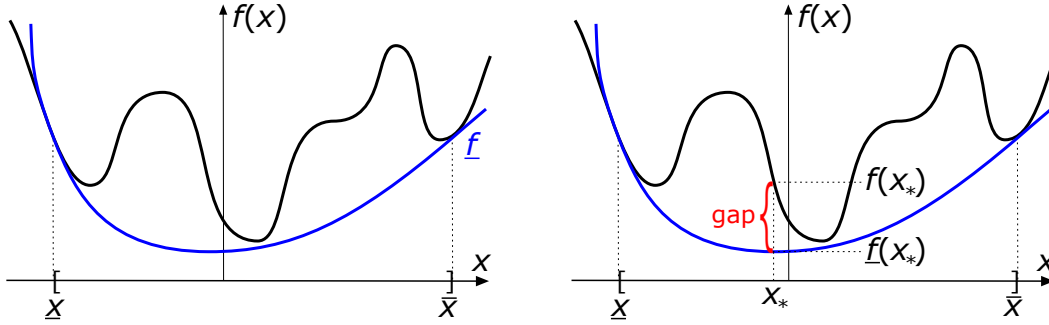


Figure 2.7: Spatial Branch-and-Bound: the root node

figure illustrates. Having obtained $\underline{f}$ (however this is done), one can efficiently compute its *global minimum* x_* by local approaches, especially if $\underline{f}$ is also continuously differentiable (at least once). Then, $\underline{f}_* = \underline{f}(x_*)$ is a *valid global lower bound* on the optimal value f_* of the problem: $\underline{f}_* \leq f_*$. We purposely ignore the fact that computing the exact x_* , and therefore its exact value $\underline{f}_*$, is in general impossible. This is significant, since only the bona-fide optimal value provides a valid lower bound on f_* : the value of any $f(x^i)$ during the optimization process provides an *upper bound* on f_* that is therefore not guaranteed to be a lower bound on f_* . This is not a huge issue in practice since it may lead to an error in the computation of f_* of the same order of magnitude of the error in the computation of $\underline{f}_*$, and in general one has to be content with much worse accuracy in f_* than can be expected in $\underline{f}_*$; but we will eschew all these details and pretend to be able to compute the true value (or a valid lower approximation) of $\underline{f}_*$.

It is then natural (although, strictly speaking, not the only option) to compute the value of f at x_* , since $f(x_*)$ is then a *valid upper bound* on f_* . Hence, from the fact that $f(x_*) \geq f_* \geq \underline{f}_*$ (assuming $\underline{f}_* > 0$ to simplify the notation) we can estimate the (relative) *gap* (cf. Figure 2.7(right)) of x_* :

$$(f(x_*) - \underline{f}_*) / \underline{f}_* > (f(x_*) - f_*) / f_*.$$

Hence, if the gap is smaller than the desired tolerance ε , the algorithm can immediately terminate declaring x_* a ε -optimal solution.

This may, of course, not happen: $\underline{f}_*$ may be “much smaller” than f_* , and/or x_* may be a rather poor approximation of the optimal solution, i.e., $f(x_*)$ may be “much larger” than f_* . Here comes the second crucial element of the algorithm: *divide et impera*, also called *branching*. The idea is simply to divide the original interval $X = [\underline{x}, \bar{x}]$ into two (disjoint) sub-intervals; this is typically done by using x_* as the separation point, although there may be other choices. Then, one has *two* optimization problems defined on $X_0 = [\underline{x}, x_*]$ and $X_1 = [x_*, \bar{x}]$; the algorithm proceeds at solving each of them separately (we incidentally note, but not comment any further, that this can be done in parallel) and then pick the best solution between the two as the optimal solution of the original problem.

Of course, each of the two subproblems is in principle as difficult as the original one, hence “solving them” is easier said than done. However, *each of the two sub-intervals is smaller than the original one*. Since the crucial property of $\underline{f}$ is to be a valid global underestimator *on the relevant interval*, this means that $\underline{f}$ can be recomputed anew for each of the subproblems. Without going in any details, this means that *since the interval is smaller, the global underestimator can be expected to be a better approximation of f* ; this actually happens in practice, for reasons that, again, we cannot even start discussing. The phenomenon is (artistically) depicted in Figure 2.8(left): the underestimator $\underline{f}^1$ now need only to be such over $X_1 = [x_*, \bar{x}]$, which allows a different and “tighter” (the word can be given a formal meaning, but we will keep the informal treatment) one to be generated. Once its minimum x_*^1 is computed, one may discover that $f(x_*^1) - \underline{f}^1(x_*^1)$ is “small enough” so that one can consider the problem *restricted to X^1* to be solved; let us assume that this actually happens. This, of course, does not terminate the process, since there still is X^0 to examine. However, in the picture, something else has happened that is useful: $f(x_*^1) < f(x_*)$. That is, the new point x_*^1 is actually a better solution to the original problem. Clearly, this is the solution that one would report if the algorithm were stopped at this point. In general, an *incumbent solution* x_{best} is kept that is simply the one with lowest f -value found so far, with $f_{best} = f(x_{best})$ being the *incumbent value*, i.e., the best (lowest) upper bound on f_* available to the algorithm. The SBB then proceeds to examine X^0 , which once again requires finding an underestimator $\underline{f}^0$ —again, hopefully “tighter” than $\underline{f}$ —over $X_0 = [\underline{x}, x_*]$ and finding its minimum x_*^0 . Here, Figure 2.8(right) shows the other crucial mechanism, called *bounding*: *since $\underline{f}^0(x_*^0) > f_{best}$, one is sure that there is no better solution than x_{best} in X^0* . In fact, $f(x) \geq \underline{f}^0(x_*^0)$ for all $x \in X^0$, which immediately imply that $f(x) \geq f_{best}$ for all $x \in X^0$. The interval X^0 is “pruned”: there is no reason to further consider *all* the points in there, as surely there is no way

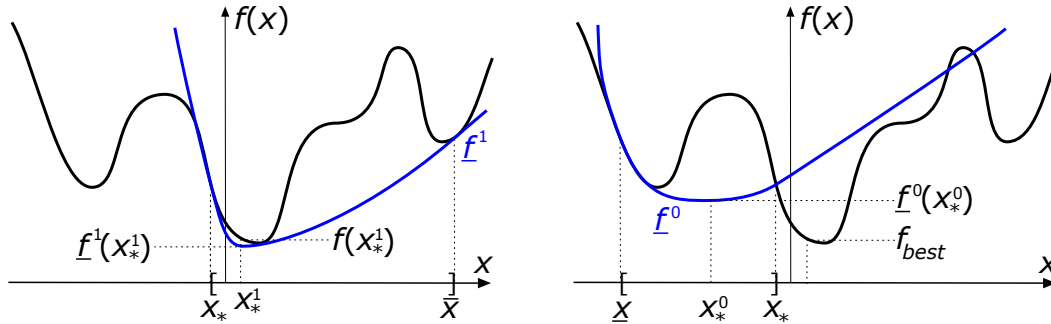


Figure 2.8: Spatial Branch-and-Bound: after “branching”

in which any of those can improve f_{best} . In the example, *the whole problem is solved*: x_{best} is an (approximately) optimal solution. It is clear how the solution of the convex approximations of the original problem (known as *relaxations*), play a crucial role in the approach by allowing to derive *valid lower bounds* on the optimal value (in each sub-interval) and using these to terminate the search without having to “explore all nooks and crannies of the feasible region” as in the approaches of §2.2.

Of course, the example is not complete in that the conditions for terminating the exploration of an interval may not be achieved. However, one can then repeat the approach by *branching again*, i.e., further subdividing the intervals. Under appropriate conditions, the algorithm will keep “slicing and dicing X ” until the sub-intervals are “small enough” so that the problem is solved, one way or another, in each of them and the algorithm finally terminate; it is possible to show that the final incumbent x_{best} at the end of the algorithm is ε -optimal for the whole problem with the same ε that is used to declare optimality in the subproblems. The issue is clearly that the number of intervals to be explored grows exponentially; hence, these algorithms can in principle (and, unfortunately, easily in practice) be very expensive, up to the bounds seen in §2.2. Yet, depending on the f at hand, SBB can even be quite effective. Since we are not really discussing any of the crucial details it is impossible to give a precise account on when this does or does not happen, but intuitively one can think this to depend on: i) “how much nonconvex” f really is, and especially ii) how “tight” the computed $\underline{f}$ are as a lower approximation of f . There are several available well-engineered implementations of the approach, even open-source ones, that tackle the different complex tasks in possibly different ways; they are typically way less inefficient in practice than “blind global search”, but still way more efficient in practice than local optimization. The next example illustrates how this works on a toy problem.

Example 2.3. Global minimization of a polynomial

We want to minimize to global optimality the 10th degree polynomial of Example 2.1 over the interval $[-2, 4]$. To do this, first of all we write the problem in an *algebraic modelling language*, in this case AMPL [36], as follows:

```
var x >= -2, <= 4;

minimize objective:
  91 * x^2 / 30 - 19 * x^3 / 6 - 54 * x^4 / 25
+ 93 * x^5 / 23 - 23 * x^6 / 36 - 121 * x^7 / 93
+ 72 * x^8 / 91 - 13 * x^9 / 74 + 9 * x^10 / 640;
```

This is saved to a simple text file; then (skipping some technical steps for the sake of conciseness), the problem is passed to the open-source global optimization solver Couenne [53], that produces the following output (slightly edited for clarity):

```
Couenne 0.5.8 -- an Open-Source solver for Mixed Integer Nonlinear Optimization
ANALYSIS TEST: Couenne: new cutoff value 4.3977638536e-01 (0.003618 seconds)
NLP0012I
Constraints:          0
Variables:            1 (0 integer)
Auxiliaries:          10 (0 integer)
NLP0014I              1          OPT 2.0488868e-19          6 0.001125
Couenne: new cutoff value 2.0488868292e-19 (0.004999 seconds)
Coin0506I Presolve 52 (-8) rows, 10 (-1) columns and 112 (-16) elements
Clp0006I 0 Obj -70871.048 Primal inf 145848.9 (11)
Clp0006I 13 Obj -57241.771
Clp0032I Optimal objective -57241.77089 - 13 iterations time 0.002, Presolve 0.00
Cbc0012I Integer solution of 2.0488868e-19 found by Couenne Rounding NLP after 0 iterations and 0 nodes
NLP0014I              2          OPT 1.1312381          7 0.001798
Optimality Based BT: 4 improved bounds
Probing: 2 improved bounds
Cbc0031I 1 added rows had average density of 2
```

```
Cbc0013I At root node, 7 cuts changed objective from -57241.771 to 3074.8854 in 1 passes
Cbc0014I Cut generator 0 (Couenne convexifier cuts) - 7 row cuts average 2.0 elements, 2 column cuts (2 active)
Cbc0001I Search completed - best objective 2.048886829156005e-19, took 3 iterations and 0 nodes (0.01 seconds)
```

"Finished"

```
Linearization cuts added in total:      60 (separation time: 0s)
Total solve time:                      0.007846s (0.007845s in branch-and-bound)
Lower bound:                          2.04889e-19
Upper bound:                          2.04889e-19 (gap: 0.00\%)
Branch-and-bound nodes:                0
```

We cannot discuss in details what the output shows, but a few comments are appropriate. First of all, in order to solve the complex problem *Couenne* relies on (open-source) solvers for “simpler” problems, in particular CLP for linear programs [52] and CBC for mixed-integer linear programs [51]. This hinges on the fact that the problem is *reformulated* into a different form that allows to use specific algorithms, in particular by adding *auxiliary variables*

```
Variables:      1 (0 integer)
Auxiliaries:    10 (0 integer)
```

The optimal value is 0, and the corresponding feasible solution (giving an upper bound) is found very early on by local optimization:

```
NLP0014I      1      OPT 2.0488868e-19      6 0.001125
```

However, in order to terminate the algorithm needs to *certify* that this is the optimal solution by computing lower bounds on the optimal value as well. In fact, by not knowing that the obtained solution is already optimal, the solver tries to find a better one

```
NLP0014I      2      OPT 1.1312381      7 0.001798
```

and clearly fails to. At the beginning, computing the lower bounds also fails quite spectacularly:

```
Clp0000I Optimal - objective value -57241.771
```

Intuitively, the convex underestimator is “extremely bad”. If the approach had started branching, as in our simplistic rendition, it would likely have required doing that many times. However, the solver has a fundamental trick up its sleeve: ways to improve the convex underestimator. These are called “cuts”, and they work quite spectacularly:

```
Cbc0013I At root node, 7 cuts changed objective from -57241.771 to 3074.8854 in 1 passes
Cbc0014I Cut generator 0 (Couenne convexifier cuts) - 7 row cuts average 2.0 elements, 2 column cuts (2 active)
Cbc0001I Search completed - best objective 2.048886829156005e-19
```

Improving the convex underestimator requires some nontrivial work, that we cannot even start discussing here, but it gradually lifts the lower bound from $-5.6e+5$ to $2.0e-19$, reducing the gap by some 24 orders of magnitude and solving the problem “at the root node”: no branching at all is needed. This was of course a simple example, but the idea of heavily investing in improving the convex underestimator is one of the fundamental steps that actually dramatically contributes to these approaches being effective—when they are—in practice.

There would be many things to discuss in this line of approach, but these arguments are not appropriate for these lecture notes since exact approaches are typically not feasible for the kind of applications of interest here. Readers interested in global optimization algorithms are referred to the abundant literature on the subject and to the corresponding optimization courses [28].

2.9 Wrap up

Univariate (unconstrained, or box-constrained) optimization is a “good warm-up” to general (unconstrained) optimization: despite being considerably simpler in many respects, as we shall see, it already shows the fundamental traits found in the more complex cases:

- global optimization is in general “difficult”, local optimization is “much easier”;
- the only practical case in which global optimization is “easy” is when it is in fact equivalent to local optimization, i.e., f is *convex* (or, much less frequently, unimodal);
- local, hence global, optimization of convex functions is the computational workhorse of global optimization algorithms (if one is interested in such a thing, which we are not in these lecture notes);
- (local) optimization algorithms can be made faster by cleverly using more information about the function / problem, but this depends on the fact that this information is actually available;
- the fundamental form of information is that provided by *derivatives*: the more *continuous* derivatives one has access to, the faster algorithms one can construct;

- the role of derivatives is to construct *models* (first- and second-order ones) of the objective f , i.e., “simple” functions that “look like f ” (at some points) and that can be used to inform algorithmic decisions;
- but “the map is not the world”: blindly trusting a model may lead to undesirable behaviour, hence one must always be careful not to be overly reliant on the model unless it can be proven / verified that it is indeed accurate enough.

These concepts will show up again and again in the *multivariate* optimization algorithms that we will start examining next.

2.10 Solutions

- **Solution to Exercise 2.1:** Let x_* be any optimal solution in X ; by definition it belongs to (at least) one interval $[z^i, z^{i+1}]$, with $z^{i+1} - z^i = \Delta \leq 2\varepsilon / L$. Assume that $x_* - z^i \leq z^{i+1} - x_*$ (the other case is analogous); then $x_* - z^i \leq \varepsilon / L$. Hence, L-c gives $f(z^i) - f(x_*) \leq L|z^i - x_*| \leq \varepsilon$.
- **Solution to Exercise 2.2:** Basically done this already: $Dr^k < \varepsilon \equiv r^k < \varepsilon / D \equiv \log(r^k) < \log(\varepsilon / D) \equiv k \log(r) < \log(\varepsilon / D) \equiv k > \log(\varepsilon / D) / \log(r)$ as $r < 1 \implies \log(r) < 0$. Hence, $k \geq \log(D / \varepsilon) / \log(1 / r)$.
- **Solution to Exercise 2.3:** Let us assume that $f(\bar{x}^i) \leq f(\underline{x}^i)$, so that the algorithm returns $f^i = f(\bar{x}^i)$ as its best estimate of the optimal value; the other case is analogous. Since f is L-c, $\bar{x}^i - \bar{x} \leq D^i = \bar{x}^i - \underline{x}^i \leq \delta$ implies $r^i = f^i - f(\bar{x}) \leq Ld^i \leq L\delta$. Hence, in order to get $r^i \leq \varepsilon$ it is sufficient to ensure that $d^i \leq \varepsilon / L$, whence the bound.
- **Solution to Exercise 2.4:** Assume that $\lim_{h \rightarrow 0} (f(x+h) - f(x)) / h = L$, with L finite; then, multiplying both sides by h and using the fact that the limit of a product is the product of the limits, one gets $0 = L \lim_{h \rightarrow 0} h = \lim_{h \rightarrow 0} h((f(x+h) - f(x)) / h) = \lim_{h \rightarrow 0} f(x+h) - f(x)$, i.e., f is continuous at x .
- **Solution to Exercise 2.5:** Consider $f(x) = \sqrt[3]{x^2}$, whose derivative is $f'(x) = 2/3x^{1/3}$ (possibly written in the more complex but algebraic-proof form $(2x) / (3(x^2)^{2/3})$). Hence, $\lim_{x \rightarrow 0^-} f'(x) = -\infty$ and $\lim_{x \rightarrow 0^+} f'(x) = \infty$. In plain words, this is because the cubic root is the inverse function of x^3 , which is “flat in 0”; inverse functions “exchange the axes”, which means that if the graph of the function is “horizontal” as some point, the graph of its inverse becomes “vertical” at the same point. A “vertical” graph has infinite derivative.
- **Solution to Exercise 2.6:** The fact that f is L-c implies that $|f(x+h) - f(x)| \leq L|h|$, i.e., $|(f(x+h) - f(x)) / h| \leq L$; now just take the $\lim_{h \rightarrow 0}$. For the other direction one needs use the Mean Value Theorem [13, Theorem 2.3.9]: if $f \in C^1$ in the interval $[x, z]$, then $f(z) - f(x) = f'(w)(z - x)$ for some $x \leq w \leq z$; now take the $|\cdot|$ and use the fact that $|f'(w)| \leq L$.
- **Solution to Exercise 2.7:** As already seen, $f(x) = \sqrt[3]{x^2}$ has $\lim_{x \rightarrow 0^-} f'(x) = -\infty$ and $\lim_{x \rightarrow 0^+} f'(x) = \infty$. Thus $f'(\cdot)$ is not bounded in any interval around 0, and therefore $f(\cdot)$ is not L-c there. Of course, $f'(x)$ is not continuous in 0.
- **Solution to Exercise 2.8:** For $\bar{x} = \Delta x = 1$, the algorithm tries the iterates 1, 2, 4, 8, ..., i.e., 2^i . With $f(x) = \sin(\pi x + 3\pi / 4) \implies f'(x) = \pi \cos(\pi x + 3\pi / 4)$ we have $f'(2^i) = \pi \cos(\pi 2^i + 3\pi / 4) = \pi \cos(3\pi / 4) = -\pi\sqrt{2} / 2 \approx -2.22$ (2^i is always even and $\cos(\cdot)$ has period 2π); that is, the algorithm always finds a “very negative” derivative and never stops, although $f(\cdot)$ has plenty of local minima. Clearly, by only very slightly changing the constants the counterexample would break down
- **Solution to Exercise 2.9:** Rewrite $\underline{x}f'(\bar{x}) - \bar{x}f'(\underline{x}) = \underline{x}f'(\bar{x}) - \bar{x}f'(\underline{x}) + \underline{x}f'(\underline{x}) - \underline{x}f'(\underline{x})$ and regroup to get $\underline{x}(f'(\bar{x}) - f'(\underline{x})) - f'(\underline{x})(\bar{x} - \underline{x})$. Now divide by $f'(\bar{x}) - f'(\underline{x})$ to get $x = \bar{x} + \alpha(\bar{x} - \underline{x})$ with $0 \leq \alpha = -f'(\underline{x}) / (f'(\bar{x}) - f'(\underline{x})) \leq 1$; it is then plain to see that $\underline{x} \leq x \leq \bar{x}$.
- **Solution to Exercise 2.10:** $[Q_i]'(x) = L_i'(x) = f'(x^i) + f''(x^i)(x - x^i) = 0 \equiv x - x^i = -f'(x^i) / f''(x^i)$

Chapter 3

Unconstrained Multivariate Optimality and Convexity

3.1 Unconstrained Multivariate Optimization

We now move to general *unconstrained multivariate problems* where the objective is $f : \mathbb{R}^n \rightarrow \mathbb{R}$. That is,

$$f(x) = f(x_1, \dots, x_n) \quad \text{with} \quad x = [x_i]_{i=1}^n = [x_1, \dots, x_n] \in \mathbb{R}^n ;$$

in layman terms, f has more than one variable. As soon as $n > 1$ a number of issues immediately become more complex, so we will not in general comment on how large n really is. In practice, it can be either quite small (2, 3, ...), small-ish (10s to 100s), quite large (1000s to tens/hundreds of 1000s), and even heniously large (millions, many billions, closing in to trillions). This clearly does have a very significant impact in many senses and we will extensively comment about it in the following.

3.1.1 Multivariate (box-constrained) global optimization

We start with a general (informal) remark: $\mathbb{R}^n$ is the n -times Cartesian product $\mathbb{R} \times \mathbb{R} \times \dots \mathbb{R}$, and therefore *the size of $\mathbb{R}^n$ grows very rapidly with n* . Of course, $\mathbb{R}$ is already uncountably infinite, and in theory $\mathbb{R}^n$ is not larger than $\mathbb{R}$. But in practice, things are quite different. Indeed, consider the case where one has box constraints, so rather than $\mathbb{R}$ the function has to be minimized over, say, $[-1, 1]$. Then, of course, the multivariate case is $[-1, 1]^n$. And then, recall Example 1.3: if f is (quadratic) concave, then the optimal solution is on one of the vertices of the unitary hypercube $\{-1, 1\}^n$. This would seem to be good news, since these are a finite number, and therefore the minimization problem can surely be finitely (exactly) solved. This is indeed good news if n is small, but the number of vertices is 2^n : as $2^{10} \approx 10^3$, 2^{100} is already the astronomical 10^{30} , and no task requiring 2^{1000} operations can ever be considered. Thus, *problems that can be tackled for small n become entirely unfeasible when n grows larger*. Informally speaking, $\mathbb{R}^n$ is “exponentially larger than $\mathbb{R}$ ”: hence, in n dimensions “there is a lot more solutions to look at”, and optimizing is much more difficult.

Of course, this statement holds if it is true that the “size” of the space is directly correlated with the difficulty of finding something in it. In other words, one is assuming that there is no “guiding light” that can be used to efficiently drive an algorithm towards the desired point, and therefore that “looking (almost) everywhere is needed”. Unfortunately, this happens to be the case, unless of course some strong (but not necessarily always unfeasible) assumptions are made. Of course one first requires L-c, for otherwise the impossibility results if §2.1 immediately apply; Appendix A.6 has already provided the definition so that it works for arbitrary n . With L-c, the already mentioned [21, Theorem 4.4] is in fact directly stated for the general multivariate case, and proves that any ϵ -approximate algorithm has an “adversarial nemesis” that will require it to expand—to keep the notation as close as possible to the previous one— $O((LD/\epsilon)^n)$ iteration to get there (with D the size of the underlying hyper-rectangle). This confirms that, at least in the worst-case sense, (approximate) global optimality cannot be achieved “without looking in every nook and cranny of the space”, and therefore that the exponential increase of the “places to look at” can translate in a proportional increase of the computational effort. Similar dependency on $(1/\epsilon)^n$ can be found in results about the expected number of iterations for randomised algorithms [21, Lemma 4.5], proving that randomisation is, in this case, no “silver bullet” for the underlying issue: global optimization is “difficult”. Although practical implementations of global optimization algorithms, e.g., along the lines of §2.8.2, typically do not show all of the dramatic impact that the worst-case result would suggest, it is true that the computational cost can increase very rapidly with n .

This has spurred the development of many *heuristics*, of which efficient implementations are available [30], which can efficiently provide good (but not provably optimal) solutions in many practical cases. These are based on a number of clever ideas, often using randomization, that cannot be discussed in these lecture notes. One of these, however, is that once some iterate x^i is produced by whatever main algorithmic process is employed, it may make sense to *refine it by trying to identify a close-by local minimum*, inasmuch as this can be done efficiently. Indeed, the multivariate extension of (2.1) is trivial with the natural concept of *neighbourhood* provided by §A.8. Hence, efficient *local optimization* is typically the workhorse of global optimization, in particular for *bound computation* purposes as hinted at in §2.8.2. It therefore makes sense to start with—and, in these lectures, to limit ourselves to—local optimization algorithms.

3.1.2 Multivariate (unconstrained) local optimization

We will therefore be mainly interested in *local optimization* approaches aiming at finding a local minima (or maxima). Of course, the fundamental issue here is *how to identify a local minimum* if one is already close by, and *how to reach closer* if one is not. As we have seen, these issues are best solved by the use of *derivatives*. However, derivatives for a multivariate function are “more complex objects” (especially as the order grows) than for univariate functions, and this will require the introduction of an appropriate set of concepts in the next section.

Doing so pays off, in the sense that one can obtain *dimension-independent* efficiency results that are often surprisingly analogous to these seen in Chapter 1 for the (multivariate) quadratic case: the converge speed depends on some properties of the function, but most often *does not explicitly depend on n* . When it does, the dependence is not exponential. A closer analysis will reveal that the analogy between convergence results for “simple” and “not simple” functions is not so surprising after all, since simple functions are used to construct the (simple) *models* that then drive the behaviour of the algorithms. One of the fundamental ideas is that once the algorithm starts converging to some point, successive iterates get closer and closer to each other: as a consequence, the model starts getting a better and better approximation of the function. In other words, *the function almost behaves as the model*, which justifies why convergence results for the model will be relevant, and convergence can be fast. Such dimension-independent results are therefore enticing for very- to extremely-large-scale problems in that they foreshadow algorithms that can converge in “few” iterations even when n is huge.

Of course, this does not mean that these algorithms will necessarily be “fast”; for once, convergence speed may be rather low (“badly linear” or worse). Furthermore, the cost of the algorithm cannot be entirely independent of the dimension of the problem: each iteration will require computing f and its derivatives, whose cost—and, as we will see, memory footprint—necessarily increases with n , sometimes significantly so. Indeed, for very-large-scale problems, even a cost (memory footprint) of the order of $O(n^2)$ may render an algorithm unfeasible, negating any advantage due to superior convergence speed. Besides, the properties of the function on which the convergence depends may not themselves be dimension-independent: in practice they may vary with n , which may introduce “hidden” dependencies from the number of variables. Nonetheless, these algorithms are clearly the ones that make sense to study first and foremost. It therefore makes sense to acquaint oneself with the necessary concepts about derivatives, which these algorithms depend onto. Indeed, as we shall see, *the quality of the available derivative information is the fundamental driver of the efficiency of the algorithms*: “more” and “better” derivatives translate into “faster” algorithms (with caveats), and vice-versa. This is not to say that it doesn’t make sense in practice to consider algorithms that do not make any use of derivative information: this is clearly possible, as illustrated, e.g., in §2.4, and in several cases plainly necessary since, quite simply, derivative information is not there to be had. It is also highly sub-optimal: by dint of the same principle described above, no derivative information typically translates into “slower” algorithms. Since derivatives are naturally—which does not necessarily means easily—available for the applications of interest in these lectures, we will not describe the several available methods that do away with them; the interested reader is referred, e.g., to [10, Chapter 9], [1, §8.4].

3.2 Gradients, Jacobians, and Hessians

The concept of derivative in $\mathbb{R}^n$ is necessarily more complex than that in $\mathbb{R}$. Intuitively, this has to do with the fact that derivatives are limits, but “in $\mathbb{R}^n$ there are many more ways for a sequence $\{x_i\}$ to have limit x ”. This paves the way for cases where functions behave “strangely” when a point is approached in a very specific way, causing unexpected behaviour.

We illustrate this concept with a preliminary example regarding the concept of *continuity* as detailed in §A.6; we have already seen in §2.5 for the univariate case, and we will confirm shortly, that existence of derivatives

imply continuity, hence whether or not a function is continuous in x is clearly relevant for our discussion. We now consider the specific function

$$f(x_1, x_2) = [(x_1^2 x_2) / (x_1^4 + x_2^2)]^2. \quad (3.1)$$

The function is ill-defined for $[x_1, x_2] = [0, 0]$, but one may suspect that it could be *extended by continuity* there. This would be the case if it were true that $\lim_{i \rightarrow \infty} f(x_1^i, x_2^i) = v$ for some value $v \in \mathbb{R}$ *however chosen* a sequence $\{[x_1^i, x_2^i]\} \rightarrow [0, 0]$. As already mentioned, there are “many possible ways” to construct such a sequence. Indeed, if one were to limit to “simple” sequences the result would seem to be true. In particular, we can consider *sequences converging to $[0, 0]$ along a straight line*. A general way to construct one is $[x_1^i, x_2^i] = [d_1/i, d_2/i]$ for some fixed $[d_1, d_2] \in \mathbb{R}^2$; in other words, one is considering points on the half-line $[0, 0] + \alpha[d_1, d_2]$ with the stepsize $\alpha = 1/i$ going to 0. It is easy to prove that $\lim_{i \rightarrow \infty} f(d_1/i, d_2/i) = 0$ however chosen $[d_1, d_2]$, which would seem to support the notion that f is actually continuous in $[0, 0]$. However, this notion is dispelled by considering sequences that “converge to $[0, 0]$ not along a straight line but along a curved one”; take in particular $[x_1^i, x_2^i] = [1/i, 1/i^2]$. While $\{[x_1^i, x_2^i]\} \rightarrow [0, 0]$, it does so describing a parabola in $\mathbb{R}^2$. In this case, it can rather be shown that $\lim_{i \rightarrow \infty} f(1/i, 1/i^2) = 1/4 \neq 0$, proving that f after all is not continuous in $[0, 0]$. This cautionary tale is relevant for this discussion since, as we shall see presently, the concept of derivatives in $\mathbb{R}^n$ is intimately tied to “moving along straight lines”.

Exercise 3.1. Prove the two limits above. [solution]

3.2.1 Directional/partial derivatives, gradient

Indeed, the first concept of derivative in $\mathbb{R}^n$ is precisely based on the concept of *reducing the issue to one for univariate functions and solving it as such*. In particular, given some $x \in \mathbb{R}^n$ at which we want to compute derivative information, we also arbitrarily choose a *direction* $d \in \mathbb{R}^n$ along which the information is computed: that is, we define

$$\frac{\partial f}{\partial d}(x) = \lim_{t \rightarrow 0} \frac{f(x + td) - f(x)}{t} \quad (3.2)$$

as the *directional derivative* at x along the direction d . In other words, one can consider the *tomography* of f at x along d $\varphi_{x,d}(\alpha) = f(x + \alpha d)$, i.e., the univariate function “depicting” the behaviour of f along the line of origin x and direction d ; then, $\frac{\partial f}{\partial d}(x) = \varphi'_{x,d}(0)$. Clearly, the norm of d is largely immaterial in this definition, so that one can typically consider $\|d\| = 1$; in fact, $\frac{\partial f}{\partial(\beta d)}(x) = \beta \frac{\partial f}{\partial d}(x)$.

Exercise 3.2. Prove the property above. [solution]

The definition could therefore easily be extended to that of the *one-sided directional derivative* by taking in (3.2) separately the limit for $t \rightarrow 0_-$ and that for $t \rightarrow 0_+$, as done in §2.5, but this is not tremendously useful, and we will not comment further about it, save for saying that of course one could only consider, say, the limit for $t \rightarrow 0_+$, since the other one is then immediately obtained by taking the opposite direction, i.e., considering $\varphi_{x,-d}(\alpha)$. Since $\varphi_{x,d}$ is a univariate function and the directional derivative is its derivative, once d is fixed it could be computed with the technique alluded to in §A.7 (provided of course f is an algebraic function to start with). However, it would clearly be rather inconvenient if the computation would have to be redone for each d . Fortunately this is not necessary, as one can compute a “basis” of directional derivative information out of which the one for each arbitrary d can be easily obtained. This is done via the definition of *partial derivative* of f with respect to one specific variable x_i at $x \in \mathbb{R}^n$:

$$\frac{\partial f}{\partial x_i}(x) = \lim_{t \rightarrow 0} \frac{f(x_1, \dots, x_{i-1}, x_i + t, x_{i+1}, \dots, x_n) - f(x)}{t} = \frac{\partial f}{\partial u^i}(x). \quad (3.3)$$

In plain words, this is the derivative of f considered as a univariate function of the sole variable x_i , i.e., of the restriction of f to the subspace of x_i ; indeed, it is the directional derivative with respect to u^i , i.e., the i -th vector of the canonical basis of $\mathbb{R}^n$ corresponding to x_i . Algebraically, it is easy to compute: basically, it's just

$$f'(x_1, \dots, x_{i-1}, x, x_{i+1}, \dots, x_n),$$

where *all symbols x_j for $j \neq i$ are treated as constants*, while $x = x_i$ is the only variable. Of course, when n is large the directional derivatives are “many”, but they need to be (symbolically) computed only once (for each variable), which can be done with standard automatic differentiation tools [27]. This provides the all-important *gradient*

$$\nabla f(x) = \left[\frac{\partial f}{\partial x_1}(x), \dots, \frac{\partial f}{\partial x_n}(x) \right]^T \in \mathbb{R}^n \quad (3.4)$$

i.e., the *vector-valued function* $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ that, for each x , returns the (column) vector of all partial

derivatives computed at x . It should already be apparent how the gradient is intimately tied to optimality, and what the condition is: for the quadratic (non-homogeneous) function $f(x) = \frac{1}{2}x^T Qx + qx$, $\nabla f(x) = Qx + q$ is a vector-valued linear function, and Theorem 1.1 can be restated by saying that the optimal solutions to the minimization of f (when $Q \succeq 0$) are all and only the points $x \in \mathbb{R}^n$ such that $\nabla f(x) = 0$. For the linear function $f(x) = \langle b, x \rangle$, $\nabla f(x) = b$ is (a) constant (vector): again, $x \in \mathbb{R}^n$ is a minimum if and only if $\nabla f(x) = b = 0$, i.e., all points are minima if $b = 0$ and the gradient is null everywhere, whereby no point is a minimum if $b \neq 0$.

3.2.2 Differentiability in $\mathbb{R}^n$

As in the univariate case, (directional) derivatives need be “good” for being useful. There, the fundamental requirement is just continuity; in the multivariate case, due to the above-mentioned fact that “there are many more ways for a sequence to converge to a point”, the notion is somehow more involved. A function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is called *differentiable* at x if there exists a *linear (affine) function* $\phi(h) = \langle b, h \rangle + f(x)$ such that

$$\lim_{\|h\| \rightarrow 0} \frac{|f(x+h) - \phi(h)|}{\|h\|} = 0. \quad (3.5)$$

Let us comment extensively on the definition. The numerator in (3.5) plainly evaluates to 0 when $h = 0$, and this is how the constant term in ϕ is univocally chosen. The gist of the definition is that ϕ , which is a *fixed* linear function (once x is fixed), is supposed to *approximate well* f in a small neighbourhood of x . The point is that the denominator in (3.5) converges to 0 by definition: hence, for the limit to be finite (and, in particular, 0), the numerator also have to converge to 0, and do so *faster than the denominator*. Since the latter converges to 0 “linearly” in its argument (it being the argument), the intuition is that the numerator has to “vanish faster than linearly”; but the numerator is the difference between the real function value and that of the approximation provided by ϕ . In other words, ϕ is called the “first order model” of f at x , and it can be pictured as the linear function that best approximates f in the vicinity of x . This is of course the multivariate generalization of the first-order model seen in (2.3), and predictably the gradient then slots in: [13, Theorem 5.3.6] prove that if f is differentiable at x , then $b = \nabla f(x)$. In other words, the general case of the first-order model now reads

$$L_x(z) = \langle \nabla f(x), z - x \rangle + f(x), \quad (3.6)$$

Differentiability of course then requires all partial derivatives to exist in the first place, so that the gradient is well-defined; as we shall see shortly the converse implication is not true, i.e., even if all partial derivatives exist the function may not be differentiable (a stronger condition is needed). However, as in the univariate case differentiability at least implies continuity [13, Theorem 5.3.7].

If f differentiable at x , then the gradient “subsumes all directional derivatives”: that is,

$$\forall d \in \mathbb{R}^n \quad \frac{\partial f}{\partial d}(x) = \langle \nabla f(x), d \rangle \quad (3.7)$$

(cf., e.g., [13, Exercise 5.3.19]). In plain words, the directional derivative is just the scalar product between the gradient and d ; this of course means that differentiability at x implies existence of all directional derivatives, and hence in particular of all partial derivatives. The converse implication requires somewhat stronger conditions: in particular, one needs that all partial derivatives $\frac{\partial f}{\partial x_i}$ exist and are *continuous* at all points z of some ball $\mathcal{B}(x, \delta)$ centred in x for some $\delta > 0$: then, one can be sure that f differentiable at x [13, Theorem 5.3.10, Theorem 5.3.7]. In view of our stance about trying to keep notational burden at a minimum, we will mostly refer to functions where *all* the partial derivatives are continuous *everywhere*: $\frac{\partial f}{\partial x_i} \in C^0$ for all $i = 1, \dots, n$. This means that f is *differentiable everywhere*, which as customary we indicate with $f \in C^1$. We mention in passing that the continuity condition is sufficient but not necessary: there exist (weird) counterexamples of functions f that are indeed differentiable despite having discontinuous partial derivatives [13, Example 5.3.9]. This is basically a curiosity and we will not delve into the concept, since we are rather interested in the opposite case: in applications, continuity of partial derivatives/differentiability is not always given. It is therefore useful to give a quick look at how a function can be *not* differentiable, especially since, given the “flexibility” that $\mathbb{R}^n$ offers to construct weird situations, this can happen in a number of different ways.

A simple example of nondifferentiable function is $f(x_1, x_2) = \| [x_1, x_2] \|_1 = |x_1| + |x_2|$. While the function is obviously continuous, it is as well obviously non differentiable. This is because each of the “components” is an absolute value, which as we discussed in §2.5 is not differentiable in 0 by dint of having different left and right derivatives. Thus, the partial derivatives do not exist in $[0, 0]$, which in view of the previous results means that f cannot be differentiable there. The graph of the function is pictured in Figure 3.1(left), showing how the function is actually “kinky” in $[0, 0]$; nondifferentiability is in fact also referred to as *nonsmoothness*, a concept we will return to.

A subtler case of nondifferentiability is provided by $f(x_1, x_2) = (x_1^2 x_2) / (x_1^2 + x_2^2)$, whose graph is depicted

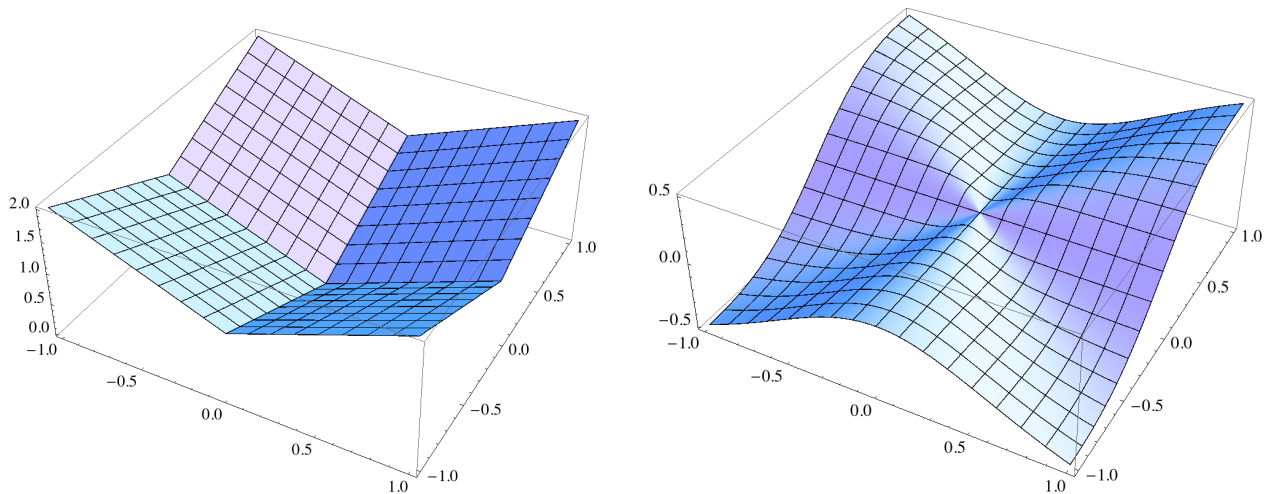


Figure 3.1: Two examples of nondifferentiable functions.

in Figure 3.1(right). While the function is undefined in $[0, 0]$, it can be extended by continuity by posing $f(0, 0) = 0$. It is easy to see that 0 is the only possible value, as it is “the limit along lines”: that is, $\lim_{\alpha \rightarrow 0} f(\alpha d_1, \alpha d_2) = 0$ however chosen $[d_1, d_2] \in \mathbb{R}^2$. This is not sufficient to prove that the limit is always 0, as we have seen for (3.1), but it does prove that if the function is continuous then the value can only be 0. In fact, a bit more work shows that this is indeed the limit however chosen the converging sequence.

Exercise 3.3. Prove the above limits, “along lines” and in general. [solution]

Hence, f is continuous (can be extended by continuity) in $[0, 0]$. Furthermore, however chosen $d = [d_1, d_2] \in \mathbb{R}^2$, the directional derivative $\frac{\partial f}{\partial d}(0, 0)$ is well-defined, and even a continuous function of (d_1, d_2) . However, f is non differentiable in $[0, 0]$, as Figure 3.1(right) clearly illustrates by showing a “kink” in the graph in that point.

Exercise 3.4. Prove all this (hint: compute $[\partial f / \partial d](0, 0)$ for generic $d = [d_1, d_2]$, prove it cannot have the form $\langle v, d \rangle$ for any v). [solution]

Exercise 3.5. Alternatively, compute ∇f and prove it is not continuous in $[0, 0]$ (hint: look at the picture of $\partial f / \partial x_2$ for directions where the limit is different). [solution]

Thus, existence of directional derivatives is not enough to guarantee differentiability: they must also “all agree” with a single vector. Yet, not even this condition is sufficient. This is shown by the “nasty” not continuous function (3.1). Indeed, in this case one can easily see that $\frac{\partial f}{\partial d}(0, 0) = 0$ however chosen $d \in \mathbb{R}^2$. Thus, there does exist a vector v (in particular, $v = [0, 0]$) such that $\frac{\partial f}{\partial d} = \langle v, d \rangle$ for all $d \in \mathbb{R}^2$. Nonetheless, f cannot be differentiable since it is not continuous. This is, again, since “directional derivatives only look at f along lines”, while (3.1) “looks nice along lines but does nasty things along curves”.

Exercise 3.6. Prove that, for (3.1), $\frac{\partial f}{\partial d}(0, 0) = 0$ however chosen $d \in \mathbb{R}^2$. [solution]

3.2.3 Geometric interpretation of the gradient

For our development, it will be useful to get some intuition on how the gradient can be pictured. A hint has already been given in §1.3.1, cf. in particular Figure 1.5: there, the gradient $\nabla f(x) = b$ has been shown to be orthogonal to the level sets. In fact, this property is general, which is not surprising since the gradient identifies a linear function—the first-order model $L_x(x)$ —that “locally behaves like f does”. We illustrate the point in Figure 3.2(left and center), relative to the (non-continuous, and therefore nondifferentiable in $[0, 0]$) function $f: \mathbb{R}^2 \rightarrow \mathbb{R}$ of (3.1). Each picture shows a point $x \in \mathbb{R}^2$, the level set $L(f, f(x))$, and the level set $L(L_x, f(x))$ of the first-order model at x : this is a line passing by x , and $\nabla f(x)$ is orthogonal to that line. This readily generalises to the $\mathbb{R}^n$ case, where $L(L_x, f(x))$ is a(n affine) *hyperplane* passing through x .

In these pictures, x is a point where f is differentiable, so that $\nabla f(x)$ is well-defined. This corresponds to the fact that the level set $L(f, f(x))$ is a smooth line, that has a well-defined tangent. Figure 3.2(right) rather shows the level set $L(f, 0)$, that corresponds to the two axes $x_1 = 0$ and $x_2 = 0$, as it is easy to verify. This illustrates the effect of nondifferentiability in $[0, 0]$. Intuitively, the level sets $L(f, v)$ are smooth for any $v > 0$, but become “progressively less smooth close to $[0, 0]$ ” as $v \rightarrow 0$, up until for $v = 0$ the level set is no longer smooth there; rather, it has a “kink”. Geometrically, $L(f, 0)$ does not have a well-defined tangent in

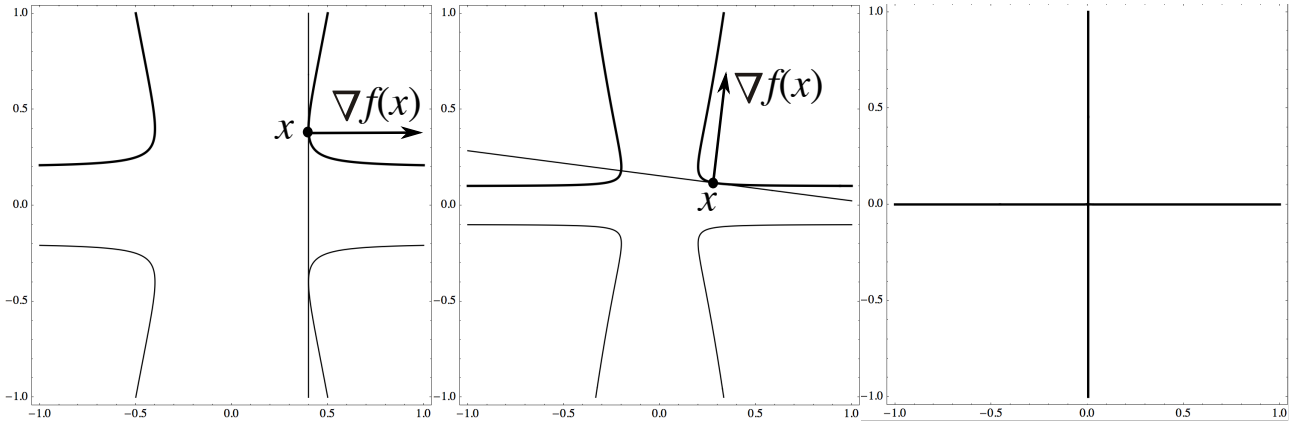


Figure 3.2: Two smooth level sets and a non-smooth one for (3.1)

$[0, 0]$, which corresponds to the fact that ∇f is not well-defined there. This—still keeping the discussion at an intuitive geometric level—generalises to any n : wherever f is differentiable all relevant objects in $\mathbb{R}^{n+1}$ and $\mathbb{R}^n$ are “smooth” and $\nabla f(x)$ is uniquely defined as the gradient of the hyperplane tangent to the graph of f in x . If, rather, f is not differentiable at x , “kinks” appear in the relevant objects (the graph of f and its level sets) and $\nabla f(x)$ is no longer well defined.

A particularly relevant property of the gradient is that of identifying the *steepest ascent direction* of the function at x , i.e., the direction where “the function grows at the fastest clip” when “moving slightly away from x ”. This should already be reasonably obvious by considering what the first-order model is, but it can be seen perhaps more clearly by asking the question: what is the direction along which the directional derivative is larger? Due to (3.7), this is

$$s = \operatorname{argmax}\{\langle \nabla f(x), d \rangle : d \in \mathbb{R}^n, \|d\| = 1\}.$$

By (A.1), it is obvious that the optimal solution (which, as it is easy to see, is unique) corresponds to taking d collinear to $\nabla f(x)$, so that the cosine of the angle between them is 1: in other words, $s = \nabla f(x) / \|\nabla f(x)\|$ (save of course if $\nabla f(x) = 0$, in which case the directional derivative is null along any direction). Thus, $\nabla f(x)$ would intuitively be an “interesting direction” if one was looking to maximise f . By the same token, the opposite of the gradient, i.e., $-\nabla f(x)$ (possibly normalised), is the *steepest descent direction* of the function at x , i.e., the direction where “the function decreases at the fastest clip” when “moving slightly away from x ”. This is of course a relevant direction if one is looking to minimise f , as we will confirm soon. Indeed, $-\nabla f(x)$ is so relevant as to merit its own name, that of *antigradient*.

3.2.4 Derivatives of vector-valued functions

Although vector-valued functions $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ for $m > 1$ are not used in these lecture notes as objective functions (cf. §1.1.7), they still have uses; a very important one will be illustrated right next. The definition of a “derivative” concept along the lines of the gradient is not difficult once one realises that a $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is just “ m -vector of $\mathbb{R}^n \rightarrow \mathbb{R}$ functions”:

$$f(x) = [f_1(x), f_2(x), \dots, f_m(x)]$$

with $f_j : \mathbb{R}^n \rightarrow \mathbb{R}$ for all $j = 1, \dots, m$. As all m -vectors it is usually considered to be a column one, and it is shown as a row one only for compactness of representation. Hence, it is immediate to derive the partial derivative of f_j with respect to x_i for any pair (j, i) with $j \in \{1, \dots, m\}$ and $i \in \{1, \dots, n\}$:

$$\frac{\partial f_j}{\partial x_i}(x) = \lim_{t \rightarrow 0} \frac{f_j(x_1, \dots, x_{i-1}, x_i + t, x_{i+1}, \dots, x_n) - f_j(x)}{t}.$$

Basically, it “just as before save the extra index j ”. This immediately leads to defining the *Jacobian* of f

$$Jf(x) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}(x) & \frac{\partial f_1}{\partial x_2}(x) & \dots & \frac{\partial f_1}{\partial x_n}(x) \\ \frac{\partial f_2}{\partial x_1}(x) & \frac{\partial f_2}{\partial x_2}(x) & \dots & \frac{\partial f_2}{\partial x_n}(x) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(x) & \frac{\partial f_m}{\partial x_2}(x) & \dots & \frac{\partial f_m}{\partial x_n}(x) \end{bmatrix} = \begin{bmatrix} \nabla f_1(x)^T \\ \nabla f_2(x)^T \\ \vdots \\ \nabla f_m(x)^T \end{bmatrix},$$

i.e., the *matrix-valued function* $Jf : \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}$ where the (j, i) -th element of the output is the partial derivative of f_j with respect to x_i . In other words, the j -th row of $Jf(x)$ is (the transpose of) the gradient

of f_j . As already mentioned (cf. §1.1.4), vector-valued functions are typically used to represent the feasible region of optimization problems, so the Jacobian will turn out to be an important concept when dealing with constrained optimization. However, a specific vector-valued function is particularly important already: *the gradient* $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$. As a vector-valued function, ∇f has a Jacobian (which may not be well-defined everywhere or continuous), which is obviously related with the concept of second-order derivatives of f .

3.2.5 Second order (partial) derivatives and the Hessian

Indeed, $\frac{\partial f}{\partial x_i}$ is a multivariate function $\mathbb{R}^n \rightarrow \mathbb{R}$, and as such it has partial derivatives itself. Given a pair (i, j) of variable (indices) with $i = \{1, \dots, n\}$ and $j \in \{1, \dots, m\}$, the *second-order partial derivative of f with respect to x_i and x_j* (in that order) is

$$\frac{\partial^2 f}{\partial x_j \partial x_i}(x) = \frac{\partial f}{\partial x_j} \left[\frac{\partial f}{\partial x_i} \right](x),$$

i.e., the partial derivative with respect to x_j of the partial derivative with respect to x_i ; basically, “just do it twice”. Obviously, the usual algebraic differentiation tools can be used to compute these without any particular extra issue. Obviously, the special case where $i = j$ merits some attention, if only for notational reasons:

$$\frac{\partial^2 f}{\partial x_i \partial x_i}(x) = \frac{\partial^2 f}{\partial x_i^2}(x)$$

is the second-order partial derivative of f with respect to the given x_i , i.e., *the second-order derivative if x_i is considered the only variable* and all the other ones are treated as constants. All this immediately leads to the crucial definition of the *Hessian (matrix-valued function)* of f at x

$$\nabla^2 f(x) = J\nabla f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2}(x) & \frac{\partial^2 f}{\partial x_2 \partial x_1}(x) & \dots & \frac{\partial^2 f}{\partial x_n \partial x_1}(x) \\ \frac{\partial^2 f}{\partial x_1 \partial x_2}(x) & \frac{\partial^2 f}{\partial x_2^2}(x) & \dots & \frac{\partial^2 f}{\partial x_n \partial x_2}(x) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_1 \partial x_n}(x) & \frac{\partial^2 f}{\partial x_2 \partial x_n}(x) & \dots & \frac{\partial^2 f}{\partial x_n^2}(x) \end{bmatrix},$$

i.e., the Jacobian of the vector-valued function $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$, i.e., the matrix-valued function $\nabla^2 : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}$ whose (i, j) -the element is the second-order partial derivative of f with respect to x_i and x_j . Of course, the diagonal elements of $\nabla^2 f$ are the second-order partial derivative of f with respect to the given x_i for $i = 1, \dots, n$. Computing (and storing) $\nabla^2 f$ has in general a (memory) cost of at least $O(n^2)$, and therefore it can be prohibitive when n is large unless $\nabla^2 f$ has a known *sparsity pattern* whereby many of its elements are known beforehand to be zero; this corresponds to the fact that, for some (i, j) , the partial derivative with respect to x_j does not depend on x_i . This is of course the case of linear functions: for $f(x) = \langle b, x \rangle$, $\nabla f(x) = b$ and therefore $\nabla^2 f(x) = 0$, i.e., linear functions have “constantly zero curvature”. For quadratic functions $f(x) = \frac{1}{2}x^T Qx + qx$, $\nabla f(x) = Qx + q$ and $\nabla^2 f(x) = Q$: that is, the matrix Q defining f is precisely the Hessian, which means that quadratic functions have “constant curvature everywhere” and it is known from the beginning whether or not $\nabla^2 f(x)$ is sparse. Of course, the very special case where $n = 1$ leads to $\nabla f(x) = [f'(x)] \in \mathbb{R}$ and $\nabla^2 f(x) = [f''(x)] \in \mathbb{R}$ (any power of 1 is 1), but in general “the cost of the second-order derivatives grows rather quickly with n ”.

The Hessian of f at x therefore defines “the local curvature of the function”, albeit in the nuanced sense typical of the multivariate setting illustrated in §1.3.3. This is best illustrated by introducing one of the fundamental reasons why the Hessian is useful: that of constructing the *second-order model of f at x*

$$Q_x(z) = L_x(z) + \frac{1}{2}(z - x)^T \nabla^2 f(x)(z - x), \quad (3.8)$$

i.e., the extension of the first-order model (3.6) with the *second-order* term describing the local curvature by means of the Hessian. As we shall see—which it can be fully expected in view of the analysis of the univariate case in §2.7 (cf. (2.5) and Theorem 2.2)—the second-order model is “more accurate” than the first-order one in that it captures not only the “linear behaviour” of f around x , but also the “quadratic one”; its curvature. As (non-homogeneous) quadratic functions are “way richer” than linear ones, this more accurate model will be able to provide better information both to characterise optimal solutions of optimization problems and to drive solution algorithms.

Yet, in order to apply all the machinery of §1.3.3 to (3.8) one would need the Hessian to be symmetric, or a symmetrising operation to be applied to it. Fortunately, the first condition is almost invariably true. This basically corresponds to the fact that—under mild conditions—*exchanging the order of (partial) differentiation does not change the results*; i.e.,

$$\frac{\partial^2 f}{\partial x_j \partial x_i}(x) = \frac{\partial^2 f}{\partial x_i \partial x_j}(x) .$$

A simple condition is continuity in a ball around x : that is, there exists some $\delta > 0$ such that

$$\frac{\partial^2 f}{\partial x_j \partial x_i}(z) \quad \text{and} \quad \frac{\partial^2 f}{\partial x_i \partial x_j}(z)$$

exist and are continuous at x however chosen $z \in \mathcal{B}(x, \delta)$ [13, Th. 5.3.3]. This immediately implies that $\nabla^2 f(x)$ is symmetric and therefore all its eigenvalues are real, allowing us to immediately define the notion of *direction of positive / negative / null curvature* for f at x based on that for Q_x . It is, in fact, fairly difficult to construct examples of functions whose Hessian is not symmetric. In our continuous quest to simplify assumptions and notations we will particularly like functions $f \in C^2$, i.e., those where *all second-order partial derivatives $\partial^2 f / \partial x_j \partial x_i$ are continuous everywhere* for all $(i, j) \in \{1, \dots, n\} \times \{1, \dots, n\}$. By the previous result, this means that $\nabla^2 f$ is symmetric everywhere. It also means that $\nabla f \in C^1$, i.e., the gradient is continuously differentiable; since continuity of higher-order derivatives implies that of lower-order ones, it also implies that $\nabla f \in C^0$ (the gradient is continuous everywhere, hence f is differentiable everywhere) and ultimately that $f \in C^0$ (the function itself is continuous everywhere). This is typically the strongest possible condition one could ask for constructing optimization algorithms, and in fact we will most often be more than content with it (although, strictly speaking, for some results one needs $f \in C^3$, i.e., continuity of even third-order partial derivatives). As we shall see, it will sometimes be necessary to make do with (much) less than that, but for the current treatment $f \in C^2$ will be the reference condition. Indeed, we now have all elements needed to elucidate the optimality conditions for (local) optimization of unconstrained functions; for a more detailed mathematical treatment of the concepts of this paragraph the interested reader can consult, e.g., [2, A.4.1][10, p. 625].

3.3 Optimality conditions

We are now ready to generalise the treatment seen in §2.5, starting from the crucial *first-order necessary (local) optimality condition*:

Theorem 3.1. Let f be differentiable at x , and be x a local minimum of f : then $\nabla f(x) = 0$.

That is, local minima (of differentiable functions) can only be found among *stationary points*; of course, not all of the latter are (in general) local minima. Exceptionally we provide a proof of the theorem, since it will immediately provide algorithmically useful insights.

Proof. The proof proceeds by contradiction: let x be local minimum of f and a point where the function is differentiable, but assume that $\nabla f(x) \neq 0$. We want to derive a contradiction, i.e., prove that x cannot be a local minimum as claimed, so let's start with commenting that this is not exactly trivial. Indeed, being a local minimum means that there exists some $\delta > 0$ such that $f(x) \leq f(z)$ however chosen $z \in \mathcal{B}(x, \delta)$. Now, disproving this requires to show that *however chosen $\delta > 0$, one can find $z \in \mathcal{B}(x, \delta)$ such that $f(z) < f(x)$* . In other words, for each $\delta > 0$ ("small enough") one must be able to provide a z_δ ("arbitrarily close to x ") with better function value than x . Luckily, all such z can be taken along a *single* $d \in \mathbb{R}^n$, i.e., they have the form $z = x + \alpha d$ for some $\alpha > 0$. Intuitively, what we need is a d such that f is decreasing along it when moving "slightly away from x "; in other words, the *directional derivative* along d should be negative. It thus makes sense to take "the most negative of them all" which, as we have seen, is $d = -\nabla f(x)$. By the hypothesis, $d \neq 0$.

We are therefore considering the tomography $\varphi_{x, -\nabla f(x)}(\alpha) = \varphi(\alpha) = f(x - \alpha \nabla f(x))$. We want to prove that there exists some $\bar{\alpha} > 0$ such that $\varphi(\alpha) < \varphi(0) = f(x)$ *however chosen $\alpha \in [0, \bar{\alpha}]$* : this yields the desired contradiction, since for any $\delta > 0$ ("small enough") one can then identify $\alpha > 0$ such that $\|x - \alpha \nabla f(x)\| \leq \delta$ and $f(x - \alpha \nabla f(x)) < f(x)$. To prove this we consider the *remainder* of first-order model corresponding to the displacement h from x

$$R(h) = f(x + h) - L_x(h) = f(x + h) - f(x) - \langle \nabla f(x), h \rangle ;$$

since $f \in C^1$, f is differentiable in x and by the very definition (3.5), $\lim_{\|h\| \rightarrow 0} R(h) / \|h\| = 0$. Then, we take $h = -\alpha \nabla f(x)$ to get

$$\varphi(\alpha) = f(x) + \langle -\alpha \nabla f(x), \nabla f(x) \rangle + R(-\alpha \nabla f(x)) .$$

We then have to estimate the sign of the sum $R(-\alpha \nabla f(x)) - \alpha \|\nabla f(x)\|^2$. The second term, which is negative, vanishes linearly in α as $\alpha \rightarrow 0$. But (3.5) implies that the first term, that can possibly be positive, "vanishes faster than linearly" as $\alpha \rightarrow 0$, which implies that for small enough α the second term will dominate and the sum will be negative. Formally, from (3.5)

$$\lim_{\alpha \rightarrow 0} R(-\alpha \nabla f(x)) / \|\alpha \nabla f(x)\| = 0 ,$$

i.e., however chosen $\varepsilon > 0$ there exists $\bar{\alpha} > 0$ such that

$$R(-\alpha \nabla f(x)) / (\alpha \|\nabla f(x)\|) \leq \varepsilon \quad \forall \alpha \in [0, \bar{\alpha}].$$

Hence, it suffices to choose $\varepsilon < \|\nabla f(x)\|$ to get

$$R(-\alpha \nabla f(x)) < \alpha \|\nabla f(x)\|^2 \implies \varphi(\alpha) = f(x) - \alpha \|\nabla f(x)\|^2 + R(\alpha \nabla f(x)) < f(x)$$

however chosen $\alpha \in [0, \bar{\alpha}]$ as desired, finishing the proof. $\square$

Besides providing the formal proof of the expected result—minima can only be found among *stationary points* where $\nabla f(x) = 0$ —the theorem provides valuable algorithmic insight: *a suitably small step along $-\nabla f(x)$ (provided the latter is not 0 already) yields a better point than x .* This is, in fact, all that we would need. However, it is also worth to analyse the conditions under which a stationary point is indeed a minima, since these also give algorithmically useful insights. Intuitively, in a stationary point x the first-order model L_x is “flat” and therefore “gives no information on the behaviour of the function” at $z \neq x$. Getting further information therefore require a “better” model than L_x , which naturally then is Q_x (cf. (3.8)). This is a quadratic function, and §1.3.3 has already illustrated all the relevant concepts about its *curvature*; basically, one just have to consider the eigenvalues of $\nabla^2 f(x)$. Indeed, it is easy to see that

$$\nabla Q_x(x) = \nabla L_x(x) = \nabla f(x).$$

Exercise 3.7. Prove the above relation. [solution]

Hence, $\nabla^2 f(x) \succeq 0 \iff x$ is a (global) minimum of Q_x . Since Q_x is a “good” model of f close to x , we can prove that this *almost* holds for f , too. The “almost” should be expected: being a model means that necessarily “some error has to be expected”. Also, optimality can only be expected to be local, as Q_x can diverge arbitrarily from f “far from x ”.

Theorem 3.2. If $f \in C^2$, and x is a local minimum of f , then $\nabla^2 f(x) \succeq 0$.

Proof. The proof requires second-order Taylor’s theorem [13, Th. 5.4.9] written in “remainder form”: with

$$f(z) = L_x(z) + \frac{1}{2}(z-x)^T \nabla^2 f(x)(z-x) + R(z-x) \quad (3.9)$$

one has $\lim_{\|h\| \rightarrow 0} R(h) / \|h\|^2 = 0$. Intuitively, the denominator “goes to zero quadratically fast” as $\|h\| \rightarrow 0$, and yet the limit is zero, which means that “the remainder goes to zero faster than quadratically”. One can then work by contradiction: assume that x is local minimum of f but $\nabla^2 f(x) \not\succeq 0$, i.e., there exists *direction of negative curvature* $d \in \mathbb{R}^n$ —without loss of generality, with $\|d\| = 1$ —such that $d^T \nabla^2 f(x) d < 0$. In other words, $\varphi(\alpha) = \varphi_{x,d}(\alpha)$ is a quadratic univariate function with negative curvature.

By Theorem 3.1, since x is a local minimum $\nabla f(x) = 0$. Plugging this into (3.9) yields

$$\varphi(\alpha) = f(x) + \frac{1}{2}\alpha^2(d^T \nabla^2 f(x) d) + R(\alpha d).$$

In plain words, the tomography is the sum of a quadratic homogeneous univariate function with negative curvature plus an unknown reminder term that can then be positive, but that “vanishes faster than quadratically” as $\alpha \rightarrow 0$. Formally, $\lim_{\alpha \rightarrow 0} R(\alpha d) / \alpha^2 = 0$ (recall that $\|d\| = 1$). Again, this means that however chosen $\varepsilon > 0$ there exists $\bar{\alpha} > 0$ such that $R(\alpha d) \leq \varepsilon \alpha^2$ for all $\alpha \in [0, \bar{\alpha}]$. Hence, take $\varepsilon < -\frac{1}{2}d^T \nabla^2 f(x) d$ to get $R(\alpha d) < -\frac{1}{2}\alpha^2(d^T \nabla^2 f(x) d)$, which implies $\varphi(\alpha) = f(x) + \frac{1}{2}\alpha^2(d^T \nabla^2 f(x) d) + R(\alpha d) < f(x)$ for all $\alpha \in (0, \bar{\alpha})$, contradicting the local optimality of x and proving the theorem. $\square$

The proof of the Theorem can be illustrated by saying that in a local minimum there cannot be directions of negative curvature, since “when the first derivative is 0, second-order effects prevail” and therefore a direction of negative curvature allows the function to decrease (locally). Conversely, “when the first derivative is not 0 the first-order effects prevail” and direction of negative curvature are irrelevant (locally). However, the Theorem allows for the Hessian to be positive *semidefinite*, i.e., for direction of null curvature to exist. There, both “first and second order effects are null” and therefore not even the second-order model is capable of “properly predicting” how the function behaves (locally). Indeed, the Theorem provides a *necessary* condition which only becomes *sufficient* if directions of null curvature are taken out of the picture.

Theorem 3.3. If $f \in C^2$ and both $\nabla f(x) = 0$, $\nabla^2 f(x) \succ 0$, then x is a local minimum of f .

Proof. The condition $\nabla f(x) = 0$ is obviously necessary in view of Theorem 3.1. Using it we rewrite (3.9) as

$$f(x+d) - f(x) = \frac{1}{2}d^T \nabla^2 f(x) d + R(d)$$

with $\lim_{\|d\| \rightarrow 0} R(d) / \|d\|^2 = 0$. Again, this means that however chosen $\varepsilon > 0$ there exists $\delta > 0$ such that $R(d) / \|d\|^2 \geq -\varepsilon$ for all d such that $\|d\| < \delta$. Rewrite the latter as $R(d) \geq -\varepsilon \|d\|^2$, and consider the minimum eigenvalue $\lambda_n > 0$ of $\nabla^2 f(x)$: then, $d^T \nabla^2 f(x) d \geq \lambda_n \|d\|^2$ by the variational characterization of the

minimum eigenvalues. In plain words, $f(x+d) - f(x)$ is equal to the sum of two terms, one that is positive and grows at least as $\lambda_n \|d\|^2$, and the other that may be negative but “only grows more than quadratically fast close to x ”, and therefore can be bounded below by the negative term $-\varepsilon \|d\|^2$ where ε can be chosen arbitrarily small provided that d is “small enough”. Hence, it is sufficient to take ε “small with respect to λ_n ” say $\varepsilon < \lambda_n/2$, to get

$$f(x+d) - f(x) \geq \frac{1}{2} d^T \nabla^2 f(x) d + R(d) \geq f(x) + \frac{\lambda_n - \varepsilon}{2} \|d\|^2 \geq 0$$

for all d such that $\|d\| < \delta$, i.e., x is a local minimum. $\square$

The proof actually shows that the result is even stronger than what the statement of the Theorem lets transpire: close to x , f grows “at least quadratically fast” where the coefficient of the quadratic term is basically the minimum eigenvalue of the Hessian. However, this requires said eigenvalue to be strictly positive. When a stationary point has directions of zero curvature, one would require even higher-order derivatives to estimate the “higher-order curvature” of the function. Again, the intuition is that lower-order terms prevail on higher-order ones if they are nonzero, but higher-order ones need to be accounted for when all lower-order ones are zero. For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the k -th order derivative is a tensor of order k , i.e., something that requires $O(n^k)$ real numbers to be represented, making more-than-second-order derivatives impractical save for rather small values of n . Indeed, as we shall see, even the $O(n^2)$ real numbers required by the second-order derivative are impractical when n is large, and therefore the use of second-order information will have to be limited. Nonetheless, the theorems give a glimpse of how the first- and second-order model of f at x are crucial to characterise the local behaviour of the function, which will have obvious algorithmic impacts.

3.4 A Quick Look to Convex Functions

Of course, since the results of the previous section only provide local optimality guarantees, it makes sense to discuss under which conditions these also are global optimality conditions. This would call for a multivariate version of unimodality, that would indeed be presented but only briefly at the end of the section. Instead, for reasons that will be discussed, the preferred (sufficient) condition is rather *convexity*. Here we will take a more formal approach w.r.t. the mostly intuitive one of Chapter 2 and provide a general definition [2, §3.1.1]:

Definition 3.1. $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is *convex* if, however chosen $x \in \mathbb{R}^n$, $z \in \mathbb{R}^n$ and $\alpha \in [0, 1]$, it holds

$$\alpha f(x) + (1 - \alpha)f(z) \geq f(\alpha x + (1 - \alpha)z). \quad (3.10)$$

The definition is pictured in Figure 3.3(left): the line drawn between points $[x, f(x)]$ and $[z, f(z)]$ lies always above the graph of f . In some cases, f is convex only if x and z in (3.10) are restricted to some $C \subset \mathbb{R}^n$, in which case one says that f is convex on C . This definition is unrelated from smoothness, and in fact it is easy to see that convex functions need not be differentiable: $f(x) = |x|$ is already trivially so in one dimension, and more in general $f(x) = \|x\|_1$ is, cf. Figure 3.1(left). Convexity of these can be easily verified both geometrically and algebraically, but it will come as easy corollaries of general results discussed next. The fact that convexity implies that desired property, i.e., all local minima are as well global ones, is better understood and demonstrated by using its *first-order characterization*, which implies assuming $f \in C^1$; we will see in Chapter 5 that this can be generalised to the nonsmooth case. We mention in passing that “convex functions are smooth almost everywhere”: technically, the set of points where f is not differentiable has zero measure. This is easily visualised by considering $f(x) = \|x\|_1$: for $n = 1$, the function is differentiable save for the singleton $\{0\}$, a set of 0 measure. For $n = 2$ (cf. again Figure 3.1(left)), the function is differentiable save for the axes $x_1 = 0$ and $x_2 = 0$, two one-dimensional lines in a two-dimensional set and therefore still of 0 measure. Yet, as we shall see, this is enough to make nondifferentiable functions “more difficult” to optimize; one observation (so far, left at the intuitive level) is that, as the examples show, the (local and therefore global) minima of the function can well happen to be precisely in points where the gradient does not exist, i.e., that such points are of 0 measure does not mean that they can (even with high probability) be ignored during the search. But we will delve on these concepts later.

For $f \in C^1$, Definition 3.1 is equivalent to requiring that the gradient ∇f is *monotone*, that is,

$$\langle \nabla f(z) - \nabla f(x), z - x \rangle \geq 0 \quad \forall x \in \mathbb{R}^n, z \in \mathbb{R}^n. \quad (3.11)$$

Exercise 3.8. Justify why (3.11) is called “monotone”. [solution]

While the above characterisation shows that Definition 3.1 conveys in the multi-dimensional case the same concept that we have anticipated in §2.8, it is not the most important one. In fact, the following result [2, §3.1.3] is much more useful: (3.10) and (3.11) are equivalent to

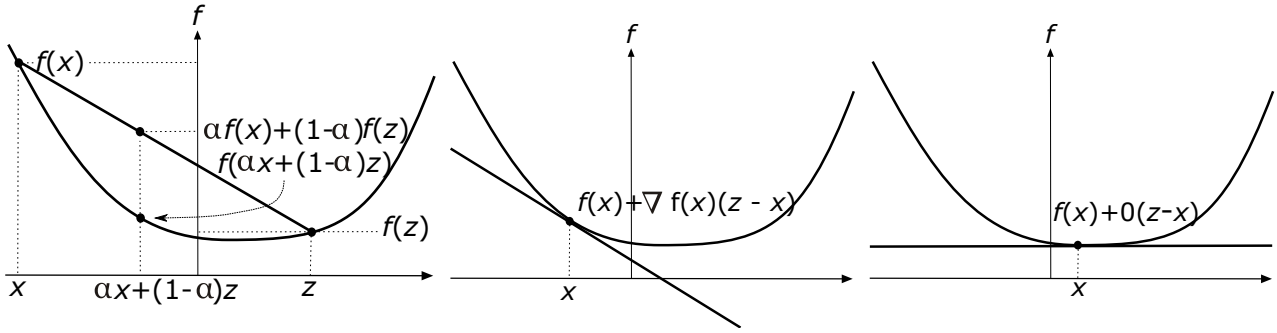


Figure 3.3: Fundamental definitions of convex functions

$$L_x(z) = f(x) + \langle \nabla f(x), z - x \rangle \leq f(z) \quad \forall x \in \mathbb{R}^n, z \in \mathbb{R}^n. \quad (3.12)$$

Exercise 3.9. Prove (3.12) “by prime principles”. [solution]

In plain words, f is convex if and only if its first-order model constructed at *any* point $x \in \mathbb{R}^n$ is always a *valid lower underestimator* of the real function. This is pictured in Figure 3.1(centre). In fact, it can be shown that a function is convex if and only if it is the pointwise maximum of all its first-order models:

$$f(z) = \max\{L_x(z) : x \in \mathbb{R}^n\} \quad \forall z \in \mathbb{R}^n$$

(and this does not, in fact, even require smoothness). That is, the first-order models have a very strong, and unique, property in the case of convex functions: they approximate f —not necessarily “well”, but at least with a sign guarantee—not only locally (close to x), as it is always the case, but also globally (arbitrarily far from x). It therefore stands to reason that the local condition $\nabla f(x) = 0$ can indeed prove that x is a global minimum of f , as immediately shown by (3.12) and pictured in Figure 3.1(right). Thus, (differentiable) convex functions have a very strong *global optimality certificate*: every stationary point, such that $\nabla f(x) = 0$, is a global minimum (and we know that the converse holds in general). The first-order characterization (3.12) also has another immediate but important consequence: no convex function save the constant one(s) can ever have a maxima. Indeed, as soon as $\nabla f(x) \neq 0$, then $f(x) + \alpha \nabla f(x) \geq f(x) + \alpha \|\nabla f(x)\|_2^2$ however chosen $\alpha > 0$. Thus, as a great man once said, “convex analysis is a one-sided world”: minimizing a convex function and maximising it are very different tasks. In fact, a function f is “easy” to globally maximize if it is *concave*, i.e., $-f$ convex. Hence, we will in general refer to convex functions with the implicit assumption that we are minimizing them, while for maximization one is “just changing sign” and assume concavity instead. As already discussed, we will never consider “the wrong problem”, i.e., globally maximizing a convex function / minimizing a concave one (even when this makes sense, i.e., constraints are present that make the problem at least have an optimal solution).

However useful (3.10), (3.11) and (3.12) may be theoretically, a significant issue is how one convinces her/himself that they hold, since the property has to be verified “for too many parameters”. It is, however, encouraging that “more derivatives require less parameters”. In fact, (3.10) requires to check all x, z and α , while (3.11) and (3.12) at least dispense with α . Fittingly, if $f \in C^2$ the characterisation can be had in terms of the Hessian and with “only” one parameter [2, §3.1.4]: f is convex if and only if

$$\nabla^2 f(x) \succeq 0 \quad \forall x \in \mathbb{R}^n. \quad (3.13)$$

The result should have been fully expected in view of the analysis of §1.3.3 for the quadratic (hence, twice differentiable) functions where $\nabla^2 f(x) = Q$: $Q \succeq 0$ is the convex case in which minimizing f may be well-posed, the opposite works for the concave case where $Q \preceq 0$. It is worth noting at this point that convexity does not guarantee existence of the minimum: a linear function $f(x) = \langle b, x \rangle$ is trivially convex and has no minimum (nor maximum). In fact, linear functions play a very special role with regard to convexity: they are the only ones that are *both* convex and concave, and therefore “easy to both minimize and maximize” (provided that some constraints make the feat possible).

Exercise 3.10. Prove the last statement. [solution]

As we have seen, the effect of linear terms in quadratic functions is only relevant—in terms of possessing or not a minimum—only in the semidefinite case: if $Q \succ 0$, the function always has one. In fact, we saw that whether or not the minimum eigenvalue of Q is nonzero does have a significant impact on the properties of solution algorithms. This motivates the definition of *strongly convex* function to generalise the notion. Although this can be given when $f \notin C^2$, the definition is intuitive when the Hessian is available: then, simply f is strongly

convex at x if $\nabla^2 f(x) \succeq \tau I$ for some $\tau > 0$, i.e., minimum eigenvalue of the Hessian is at least τ . In fact, as we will see, the “best” case for (convex) optimization is when the property holds *uniformly*, that is $\nabla^2 f(x) \succeq \tau I$ for *all* $x \in \mathbb{R}^n$ and some fixed $\tau > 0$: the minimum eigenvalue of the Hessian is bounded away from 0. Of course these two conditions are the same for quadratic functions, but not in general. We call such a function τ -convex (since it trivially is convex).

While (3.13) (and its stronger form) are somewhat easier to check than the previous ones, since they only depend on x , it may still be highly nontrivial to prove uncountably infinitely many conditions. However, a highly useful properties of convex functions is that they admit an *algebra*: there are several “easy” functions that are convex, and more importantly there are several *convexity preserving operations* that allow to generate guaranteed convex functions by properly combining functions with the right properties. That is, there is a large *design space* whereby one can construct functions with the assurance that they can be efficiently minimised. Far from being a purely theoretical consideration, this has given rise to available *Disciplined Convex Programming software* [29] that, by only allowing the user to move inside the convex algebra, provides guarantees about all the functions (and, as we shall see, sets) and therefore on the global optimality of the obtained solutions. Due to the matter being well-developed we will only quickly resume the main ideas here in order to provide the results that are most useful for our development. In terms of “basic” functions—besides the already discussed affine ones—one can for instance prove [2, §3.1.5] that:

1. $f(x) = \frac{1}{2}x^T Qx + qx$ is convex if and only if $Q \succeq 0$;
2. $f(x) = e^{ax}$ is convex for any $a \in \mathbb{R}$;
3. $f(x) = -\ln(x)$ is convex if restricted to $x \geq 0$;
4. $f(x) = x^a$ for $a \geq 1$ or $a \leq 0$ is convex if restricted to $x \geq 0$;
5. $f(x) = \|x\|_p$ is convex whenever $p \geq 1$;
6. $f(x) = \max\{x_1, \dots, x_n\}$ is convex;
7. for $Q \in \mathbb{R}^{n \times n}$ symmetric, with (real) eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \lambda_n$, $f_m(Q) = \sum_{i=1}^m \lambda_i$ (sum of the m largest eigenvalues as a function of Q) is convex.

Exercise 3.11. Prove 2., 3. and 4.; for the latter, which a make x^a convex on all $\mathbb{R}$? [solution]

Of course, there are plenty of functions that are trivially *not* convex.

Exercise 3.12. Is $f(x) = \min\{x_1, \dots, x_n\}$ convex? [solution]

Once it is known that some functions are convex, many more can be constructed by (recursively) applying the available *convexity-preserving operations* [2, § 3.2]:

1. if $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $g : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex, $\delta \geq 0$ and $\beta \geq 0$ are real scalars, then $\delta f + \beta g$ is convex [non-negative combination];
2. if $\{f_i\}_{i \in I}$ is a set of (possibly, infinitely many) convex functions $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$, then $f : \mathbb{R}^n \rightarrow \mathbb{R}$ defined as $f(x) = \sup_{i \in I} f_i(x)$ is convex [pointwise supremum];
3. if $f_1 : \mathbb{R}^n \rightarrow \mathbb{R}$ and $f_2 : \mathbb{R}^m \rightarrow \mathbb{R}$ are convex, then $f(x_1, x_2) = f_1(x_1) + f_2(x_2) : \mathbb{R}^{n+m} \rightarrow \mathbb{R}$ is convex [cartesian product];
4. if $f : \mathbb{R}^m \rightarrow \mathbb{R}$ is convex and $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$ define an affine mapping $y = Ax + b : \mathbb{R}^n \rightarrow \mathbb{R}^m$, then $f(Ax + b) : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex [pre-composition with linear mapping];
5. if $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex and $g : \mathbb{R} \rightarrow \mathbb{R}$ is convex and increasing, then $g(f(x)) : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex [post-composition with increasing convex function];
6. if $f_1 : \mathbb{R}^n \rightarrow \mathbb{R}$ and $f_2 : \mathbb{R}^n \rightarrow \mathbb{R}$ are convex, then $f(x) = \inf\{f_1(x_1) + f_2(x_2) : x_1 + x_2 = x\} : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex [infimal convolution];
7. if $g : \mathbb{R}^m \rightarrow \mathbb{R}$ is convex and $A \in \mathbb{R}^{n \times m}$, $b \in \mathbb{R}^n$ define an affine mapping $x = Az + b : \mathbb{R}^m \rightarrow \mathbb{R}^n$, then $f(x) = \inf\{g(z) : Az + b = x\} : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex [value function of a convex constrained problem];
8. if $g(x, z) : \mathbb{R}^{n+m} \rightarrow \mathbb{R}$ is (jointly) convex (in both x and z), then $f(x) = \inf\{g(x, z) : z \in \mathbb{R}^m\} : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex (in x) [partial minimization];
9. if $f(x)$ is convex (in x), then $p(x, u) = uf(x/u)$ is (jointly) convex (on x and u) restricted to $u > 0$ [perspective or dilation function of f].

Exercise 3.13. Prove 1. “from prime principles” (at least 2., 3. analogous). [solution]

Besides constructing new convex functions, the properties above can be used to prove that some function is convex. In fact, convexity of several of the “basic” functions of the first example, which is easy enough to prove “from prime principles”, becomes a trivial application of the convexity-preserving operations once these are established. We leave this to the interested reader, together with the following exercises of which we purposely avoid to provide a solution:

1. Prove in at least two ways that $f(x) = e^{(x-1)^2}$ is convex.
2. Prove that $f(x, y) = xy / (x - y)$ is convex on $\{(x, y) \in \mathbb{R}^2 : x > y\}$.
3. On what subset of $\mathbb{R}^2$ $f(x, y) = 1 / (xy)$ is convex?
4. Prove in at least two ways that $f(x, y) = x^2 / y$ is convex if $y > 0$.
5. Prove that the arithmetic mean of n positive numbers $x_1, \dots, x_n$ is greater or equal to their geometric mean, i.e., $(x_1 + x_2 + \dots + x_n) / n \geq \sqrt[n]{x_1 x_2 \dots x_n}$ (hint: $-\log(x)$ is convex, hence $\log(x)$ is concave).

We finish this section providing a brief discussion of the issue “is convexity the most general condition under which global and local optimality coincide?” To do so, we need to quickly introduce a few concepts that will be crucial later, and in particular in Chapter ???. The main one is that of a *convex set* S , i.e., such that whenever $x \in S$ and $z \in S$, then $\alpha x + (1 - \alpha)z \in S$ however chosen $\alpha \in [0, 1]$. Obviously, convex sets and convex functions are strictly related, and in particular:

- $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a convex function if and only if its *epigraph* $\text{epi}(f) = \{[v, x] : v \geq f(x)\} \subseteq \mathbb{R}^{n+1}$ is a convex set;
- if $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a convex function, then each of its sublevel sets $S(f, l) = \{x : f(x) \leq l\} \subseteq \mathbb{R}^n$ is a convex set.

Crucially, the second property is not an equivalence: there are non-convex functions whose sublevel sets are all convex. These are called *quasiconvex*, and clearly any convex function is quasiconvex but the converse is not true.

Exercise 3.14. Exhibit a quasiconvex function that is not convex. [solution]

In fact, the definition of quasiconvexity can be given in algebraic terms similarly to that of convexity: $\alpha f(x) + (1 - \alpha)f(z) \leq \max\{f(x), f(z)\}$ however chosen $x \in \mathbb{R}^n$, $z \in \mathbb{R}^n$ and $\alpha \in [0, 1]$. This turns out to be the same concept of unimodality in one dimension [1, Ex. 3.57], and equivalent to the definition given in terms of sublevel sets [1, Th. 3.5.2]. Quasiconvexity is a property that makes minimization “easy”. Intuitively, consider having found a minimum x^* , and for simplicity let’s assume it is unique: then, the sublevel set $S(f, f(x^*))$ is the singleton $\{x^*\}$ (not that this is necessarily easy to prove, but let’s keep the discussion at the intuitive level), and this proves that x^* is the global minimum. Thus, quasiconvex functions could be “as useful as convex ones”. However, despite being “more”, it is harder to construct quasiconvex functions that are not convex. This is because the *algebra of quasiconvex functions* is way “weaker” than that of convex ones. In particular, while for any quasiconvex f its positive scaling δf for any $\delta > 0$ remains quasiconvex, it is in general not true that the sum of two quasiconvex functions f and g is quasiconvex

Exercise 3.15. Prove the last two statements. [solution]

Therefore, there is much less leeway to construct a Disciplined QuasiConvex Programming theory and software; besides, applications where f is “naturally” quasiconvex, despite existing, are comparatively rare. On a positive note, the all-important *neural networks* described in the next section, while not strictly quasi convex, are often depicted to be “almost” so, at least in large regions of their parameters space. This may partly explain the—otherwise, somewhat puzzling—fact that, while they are nonconvex, approaches designed to find local minima are usually quite successful in actually finding (pretty good approximations to) global ones.

3.5 Ex-post Motivation: (Artificial, Deep) Neural Networks

We finish this Chapter with a very condensed introduction to a specific class of functions whose importance can hardly be overstated: that of *artificial (deep) neural networks*, NN for short. These are functions that appear in one of the most successful approaches to (multivariate) regression: a set $\mathcal{X} = [x^s \in \mathbb{R}^h, y^s \in \mathbb{R}^k]_{s \in S}$ of *input / output samples* is given that represent some (vector-valued) function $f : \mathbb{R}^h \rightarrow \mathbb{R}^k$ (i.e., $f(x^s) = y^s$ for all $s \in S$) whose behaviour cannot be conveniently described algebraically or algorithmically, either because the underlying

mechanisms are unknown, or because they are known but the computation of f “from prime principles” is too costly. The regression task is that of constructing a function $f_{\mathcal{X}} : \mathbb{R}^h \rightarrow \mathbb{R}^k$ that “best” approximates the “true” f . This of course requires that $f_{\mathcal{X}}(x^s) \approx y^s = f(x^s)$ for all $s \in S$, i.e., $f_{\mathcal{X}}$ “reasonably agrees” with the available input / output data, but the most important part of the requirement is that $f_{\mathcal{X}}$ be able to reasonably approximate the value $f(x)$ even for inputs x not covered by $\mathcal{X}$. Doing so requires creating a *model*, i.e., a computable $f_{\mathcal{X}}$ that properly balances between accurately reproducing the values of f for known inputs and “avoiding unnecessary complications” in fitting these too closely, resulting in a model that is very accurate for the input set but can be wildly inaccurate when interpolating outside of it. In plain words the model has to trade-off accuracy and simplicity, avoiding both underfitting (being so simple as to not representing f well even on the available input / output pairs) and overfitting (being very complex in order to fit the known data, with a significant risk of not really representing the behaviour of f outside it), a conundrum known as the *bias/variance dilemma*. It is completely out of the scope of these lectures to delve into this issue, especially since there are many available courses about machine learning and artificial intelligence that provide a detailed and accurate treatment. Suffices here to say that NN, roughly modeled over the behaviour of biological neurons (whence the name), have been found to be a powerful and general class of functions that allow to efficiently—insomuch as this is possible—navigate this trade-off.

A(n A)NN is typically represented as a L -layered graph $G = (N, A)$, with $L \geq 2$ layers. The set of nodes N is layered, i.e., $N = \cup_{l=1}^L N(l)$, where each layer is a (disjoint sub)set $N(l)$ of the nodes (neurons); we will denote by $n(l) = \#N(l)$ its cardinality. Layers are ordered from $l = 1, \dots, L$, with the first layer ($l = 1$) is the input one, the last layer ($l = L$) is the output one, and the others are known as hidden layers. That is, $n(1) = h$, i.e., the input layer has one neuron for each element of the input vector, $n(L) = k$, i.e., the output layer has one neuron for each element of the output vector, while the cardinality of the hidden layers (as well as their number) can be set arbitrarily. In the standard *feedforward* NN, each arc (i, j) only connects consecutive layers, i.e., $i \in N(l-1)$ and $j \in N(l)$ form some l . Hence, the arc set is also partitioned layer-wise, i.e., $A = \cup_{l=2}^L A(l)$, with $A(l) = \{(i, j) : j \in N(l)\}$ being the arcs entering layer l . A pictorial representation of (the graph G corresponding to) a NN is given in Figure 3.4. In the figure, G is intended as a *complete* graph, i.e., all possible arcs between the nodes of two consecutive layers are present. Although this is not an uncommon arrangements, other ones are preferred in practical applications (i.e., convolutional NN for image analysis), but again this is way above the scope of these lectures where we will be content with the generic graph notation.

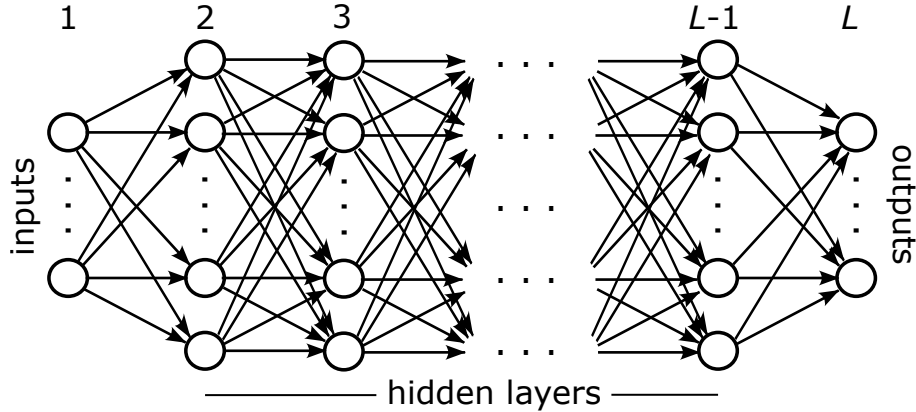


Figure 3.4: A pictorial representation of a(n A)NN

A NN is a function of the *adjustable weights* of its arcs and nodes. That is, each $(i, j) \in A$ has an associated weight w_{ij} , and each node $i \in N \setminus N(1)$ has an associated weight (known as bias) w_i . Then, each node $i \in N$ is endowed with an *output function* o^i depending on the weights of the previous layers; hence, it is convenient to define W^l as the set of all weights (and biases) to all layers $2, \dots, l$. Weights of all arcs entering i and its own bias are summed and then processed through an *activation function* $\sigma^i : \mathbb{R} \rightarrow \mathbb{R}$, of which some typical examples (and their derivatives) are:

- Identity / linear: $\sigma(z) = z$, $\sigma'(z) = 1$;
- Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$;
- Hyperbolic tangent: $\sigma(z) = (e^z - e^{-z}) / (e^z + e^{-z})$, $\sigma'(z) = 1 - \sigma(z)^2$;
- ReLU: $\sigma(z) = \max\{z, 0\}$, $\sigma'(z) = 1$ if $z > 0$, 0 otherwise.

Given all this, it is easy to provide a recursive definition of the output function. Firstly, $o_s^i = x_i^s$ for $i \in N(1)$, i.e., the output of neuron i in the input layer is just the value of the corresponding entry of the input; this

obviously depends on the chosen sample $s \in S$, so we will keep track of it in the subscript. Then,

$$o_s^i(W^l) = \sigma^i \left(\sum_{(j,i) \in A(l)} w_{ji}^l o_m^j(W^{l-1}) + w_s^i \right).$$

Finally, the value of $f_{\mathcal{X}}$ given the will vector of weights (and biases) W^L is just the output function of the output nodes, i.e., $f_{\mathcal{X}}(W^L, x^s) = [o_s^i(W^L)]_{i \in N(L)}$. Computing the function value(s) given its arguments is clearly simple, but $f_{\mathcal{X}}$ is by no means a “simple” function of the weights W^L . Besides, the total number of weights is $n = n(2)(n(1) + 1) + n(3)(n(2) + 1) + \dots + n(L)(n(L-1) + 1)$, i.e., it tends to grow rapidly as the size of each layer grows and the number of layers (the depth of the MM) does. Indeed, current top-of-the-line NN have number of weights in the 100s of billions, and reaching the trillions, so even a single computation of the function—for fixed weights W^l and sample s —requires very significant resources and specialised hardware.

Exercise 3.16. Write the pseudo-code for “Forward propagation”, i.e., computing $f_{\mathcal{X}}(W^L, x^s)$. **[solution]**

Of course, computing the function is by no means the end of the game (although it is very important once the NN is *trained*, i.e., at *inference time* when it is actually used), since first the optimal weights have to be found. These are the solution of the optimization problem

$$\min \{ \mathcal{L}(W^L) + \lambda \mathcal{R}(W^L) : W^L \in \mathbb{R}^n \} \quad (3.14)$$

where $\mathcal{L}$ is the *loss* measuring the (in)accuracy of the prediction over the known data, i.e.,

$$\mathcal{L}(W^L) = \sum_{s \in S} \ell(f_{\mathcal{X}}(W^L, x^s) - y^s)$$

(with $\ell : \mathbb{R} \rightarrow \mathbb{R}$ some simple loss function, such as $\ell(z) = z^2$), $\mathcal{R}$ a *regularisation term* trying keep the model “simple” (e.g., the ridge regularisation $\mathcal{R}(W^L) = \|W^L\|^2$, the Lasso regularisation $\mathcal{R}(W^L) = \|W^L\|_1$, or others [15]), and $\lambda > 0$ is the *hyperparameter* that governs the balance between the two (bias / variance). Clearly, the *fitting problem* (3.14) is a (potentially, extremely-large-scale) highly nonlinear, nonconvex, possibly nonsmooth optimization problem (both the Lasso regularization and the ReLU activation functions are not differentiable). Clearly, solving such a problem—a.k.a., training the NN—would be unfeasibly hard if certified global optima would be required. Fortunately, this is not so, for at least two reasons:

- Especially for “large” NN, local optima typically turns out to be of very good quality; that is, the objective of (3.14) is far from being the kind of “adversarial function” that is designed to fool optimization algorithms. One of the reasons is that typical NN are *overparametrised*, i.e., they have far more weights than these that would be required to find a very good fitting of the available samples. This, intuitively, creates very large regions of the weights space with very low values of the objective, making it easier for solution algorithms to actually find one. Indeed, it is well-known that practical NN can be *pruned*, i.e., a large fraction of their weights can be zeroed (the corresponding arcs in G can be removed outright) without the NN losing any significant fraction of its predictive capabilities [15]. It might seem strange that one should train a huge network—at a huge cost—and then reduce it, but it turns out that this is way more effective than trying to directly train a much smaller one, in part due to the fact that it is easier to find good solutions of a overparametrised NN.
- Furthermore, for the sake of learning, globally optimal solution are not required, and any “good” value of the weights can be used. Indeed, globally optimal solutions may even be less performing than ones with larger objective values in terms of how well the resulting NN interpolates f outside of the sample set. This is due to the fact that the objective in (3.14) is not the “true” objective of the learning process, which is precisely to obtain a model that correctly navigates the bias / variance dilemma. This is in fact measured by a different approach (cross-validation) and requires tuning the hyperparameter λ in ways that we are not interested in describing here at any length, save for commenting that they require the *repeated solution* of (3.14) possibly a large number of times, making efficient approaches even more important.

Example 3.1. A (small-scale, not really deep) NN

Although NNs may seem somewhat “imposing”, in the end they are not particularly complicated functions. We now provide a small example of a NN with 1 input, 1 output, 3 hidden layers of 5 nodes each, and hyperbolic tangent activation function in all layers save the last (where, customarily, the activation function is rather linear). The input for such a function is therefore a vector $w \in \mathbb{R}^{76}$ whose entries are structured as follows:

- $[1 \dots 5]$ are the $[5 \times 1]$ weights of $A(2)$;
- $[6 \dots 10]$ are the $[5 \times 1]$ biases of $N(2)$;
- $[11 \dots 35]$ are the $[5 \times 5]$ weights of $A(3)$;
- $[36 \dots 40]$ are the $[5 \times 1]$ biases of $N(3)$;
- $[41 \dots 65]$ are the $[5 \times 5]$ weights of $A(4)$;
- $[66 \dots 70]$ are the $[5 \times 1]$ biases of $N(4)$;

- [71...75] are the $[5 \times 1]$ weights of $A(5)$.
- [76] is the $[1 \times 1]$ bias of the $N(5)$ (output layer)

Figuring out the exact arrangements of nodes and arcs is left as an easy exercise to the reader. Given such a vector w and a scalar x representing the single input value, **Matlab** code for computing the value $v = f_{\mathcal{X}}(w, x)$ is just the four lines

```
o = tanh( ( ones( 5 , 1 ) * x ) .* w( 1 : 5 ) + w( 6 : 10 ) );
o = tanh( reshape( w( 11 : 35 ) , [ 5 5 ] ) * o + w( 36 : 40 ) );
o = tanh( reshape( w( 41 : 65 ) , [ 5 5 ] ) * o + w( 66 : 70 ) );
v = o' * w( 71 : 75 ) + w( 76 );
```

We also pictorially illustrate the loss function $\mathcal{L}(W^L)$ corresponding to some sample set whose characteristics are not particularly relevant. We do so by plotting three tomographies of $\mathcal{L}$ from the all-zero solution $W^L = 0$ (a not entirely unreasonable starting point for a solution algorithm) along three randomly generated directions d in Figure 3.5, and the same in Figure 3.6 for an “optimal” W^L found with some of the solution algorithms that will be discussed in the following Chapters. The pictures show how $\mathcal{L}$ is by no means convex; however, for $W^L = 0$ it does present directions where it smoothly decreases, which therefore can be exploited to find better values of the weights. In contrast, the “optimal” W^L appears to be in a quite “deep” attraction basin of a local minimum (and seemingly close to it). Although there may be other local minima somewhere else, the found solution (judging by the certainly non-exhaustive means of just a few random direction) would seem to be a promising candidate for the global optimum.

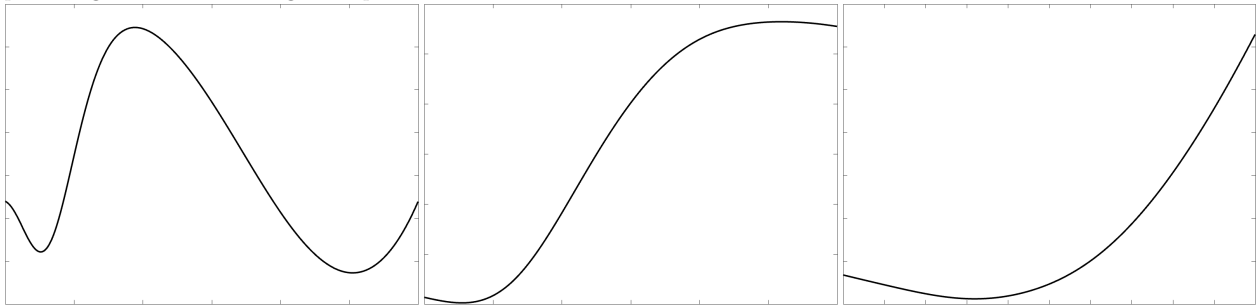


Figure 3.5: Tomographies of a NN from $W^L = 0$.

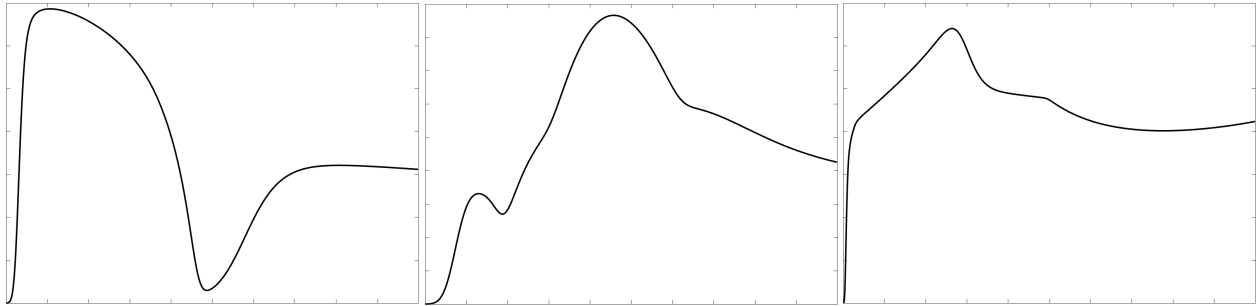


Figure 3.6: Tomographies of a NN from “optimal” W^L .

We finish this section with a brief comment on the gradient of $f_{\mathcal{X}}(W^L, x^s)$ w.r.t. the weights W^L , whose computation is the crucial step for the solution algorithms to the approach. Being a composition of algebraic functions, the gradient could in principle be obtained in closed formulæ. In fact, it is a reasonable exercise to try this for 2 or three layers; however, the exercise would readily show that the formulæ become very quickly very complicated as more layers are added. A *compositional* approach is therefore needed that makes this computation possible and efficient by, basically, treating each layer separately and properly composing the results. This in fact is possible with the celebrated *backpropagation* approach, whose practical importance is clearly indicated by the fact that it has become a field of study in itself [16]. Coherently with our basic stance that “derivatives are easy and are someone else’s problem” we will not delve any further in the subject, save quickly mentioning that the approach exploits layered structure of the computational graph of $f_{\mathcal{X}}(\cdot, x^s)$ and it is obviously closely related to the general field of *automatic differentiation* [27]. Anyway, the corresponding tools are by now well integrated in the many available machine learning tools such as (PyTorch [33], TensorFlow [35], scikit-learn [34], Fido [31], and others, so that the computation of the gradients can be considered a “well-solved problem” that we do not need to discuss here. Yet, computing gradients of “more complicated” functions that may appear as layers of a NN is still an open research area with interesting applications; the interested reader may refer, e.g., to [14] and the references therein.

3.6 Wrap up

As it could have been easily foretold by the previous developments, finding a global minimum of a multivariate function is very hard in general: its cost can be, in theory, exponential in the number of variables (on top of a term that can grow very quickly with the required precision). Although this is a worst-case estimate that is valid for “adversarial” functions, which may not be those of interest here (cf. Example 3.1), it remains generally true the finding a guaranteed local minima of a general function can become extremely costly as the number of variables grow and the required accuracy increases. Intuitively, this is the result of missing an efficient *stopping criterion*: “all local minima look the same”, also the global one, hence once the search leads to one it is hard to tell whether one has reached the true goal or the search should continue. By contrast, finding a local minimum of a multivariate function is “easy” with the right information, i.e., the first-order one (gradient). Many functions are (at least once) differentiable and the gradient is in general “easy” to compute, so this would not seem to be a big deal; however, Section 3.5 has already provided a quick glimpse to the fact that neither of these two things is really given, so making do with less than the expected first-order information is required, and we will discuss it later on.

Even when $f \in C^1$, local optimization is not exactly trivial: indeed, it basically corresponds to the solution of the *nonlinear system* $\nabla f(x) = 0$ (under our assumption that we cannot, and will not try to, distinguish stationary points from bona-fide local minima). As amply discussed in Chapter 1, this is actually simple when f itself is (i.e., quadratic): then, $\nabla f(x) = 0$ is a linear system that can be readily solved with appropriate means. This is relevant in that *quadratic models* of f —the second-order model, or approximations thereof—are going to be very useful algorithmically.

Should one really need a bona-fide global minima, there are chances to efficiently get one if f can be chosen. This can be the case, since f itself is usually a model and there can be leverage to write it in different ways. This is especially true with *synthetic* models built up from samples, since then one really has a choice on the form of the underlying elements. Then, if possible one should choose f to be convex, a then every stationary point is a global minimum: all uncertainties are removed, and solving the nonlinear system $\nabla f(x) = 0$ really is all that is needed (of course, provided that $f \in C^1$, which unfortunately is not a given). Yet, as the Example 3.1 has illustrated, even with synthetic models convexity is not always the right choice: a “better model” may be worth paying the price of non-convexity, provided that an appropriately good solution of the training problem can still be had. Thus, ultimately, it is the efficiency (or lack thereof) of the solution algorithms that dictates the usefulness of a model. All in all, it is finally time to introduce multivariate optimization algorithms.

3.7 Solutions

- **Solution to Exercise 3.1:** For $y = 1/k \rightarrow 0$, $f(d_1y, d_2y) = [d_1^2 d_2 y^3 / ((d_1y)^4 + (d_2y)^2)]^2 \rightarrow 0$ (the degree of the numerator is $>$ of the min degree at the denominator, i.e., the numerator goes to 0 faster than the denominator) however chosen d_1 and d_2 . In the second case $f(y, y^2) = [y^4 / (y^4 + y^4)]^2 = 1/4$.
- **Solution to Exercise 3.2:** $\frac{\partial f}{\partial \beta d}(x) = \lim_{t \rightarrow 0} (f(x + t(\beta d)) - f(x)) / t = \lim_{t \rightarrow 0} \beta (f(x + (t\beta)d) - f(x)) / (\beta t)$. Now, choose $p = \beta t$ and send $t \rightarrow 0$ (hence, $p \rightarrow 0$): $\frac{\partial f}{\partial \beta d}(x) = \lim_{p \rightarrow 0} \beta (f(x + pd) - f(x)) / p = \beta \frac{\partial f}{\partial d}(x)$.
- **Solution to Exercise 3.3:** Fix any $[d_1, d_2]$: since $f(x_1, x_2) = (x_1^2 x_2) / (x_1^2 + x_2^2)$, $\lim_{t \rightarrow 0} f(td_1, td_2) = \lim_{t \rightarrow 0} \frac{t^3 d_1^2 d_2}{t^2(d_1^2 + d_2^2)} = 0$. The general version requires more work: by using the direct definition of limit, given any $\varepsilon > 0$ we want to find $\delta > 0$ such that if $\|[x_1, x_2]\| \leq \delta$, then $|f(x_1, x_2)| \leq \varepsilon$. Now, of course, $\|[x_1, x_2]\| = \sqrt{x_1^2 + x_2^2}$. Thus, $\|[x_1, x_2]\| \leq \delta$ in particular implies $|x_2| \leq \delta$. Hence, $|f(x_1, x_2) - 0| \leq |x_2| \left(\frac{x_1^2}{x_1^2 + x_2^2} \right) \leq |x_2| \leq \delta$ whenever $\|[x_1, x_2]\| \leq \delta$; thus, taking $\delta = \varepsilon$ works, proving that the limit is indeed 0 however chosen the converging sequence.
- **Solution to Exercise 3.4:** However chosen $[d_1, d_2] \in \mathbb{R}^2$, the definition gives $\frac{\partial f}{\partial [d_1, d_2]}(0, 0) = \lim_{t \rightarrow 0} \frac{f(td_1, td_2) - f(0, 0)}{t} = \lim_{t \rightarrow 0} \frac{t^3 d_1^2 d_2}{t^3(d_1^2 + d_2^2)} = \frac{d_1^2 d_2}{d_1^2 + d_2^2} = f(d_1, d_2)$. Thus, this is well-defined and continuous everywhere (f is, even in $[0, 0]$); however it clearly is not a linear function, and therefore it cannot have the form $\langle v, d \rangle$ for any vector $v \in \mathbb{R}^2$.
- **Solution to Exercise 3.5:** $\nabla f(x_1, x_2) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right] = \left[\frac{2x_1 x_2^3}{(x_1^2 + x_2^2)^2}, \frac{x_1^2 (x_1^2 - x_2^2)}{(x_1^2 + x_2^2)^2} \right]$. For $g(x_1, x_2) = \partial f / \partial x_2$, it is easy to check that $g(\alpha, 0) = 1$ while $g(0, \alpha) = 0$. Hence, the limit for $\alpha \rightarrow 0$ along the

directions $[1, 0]$ and $[0, 1]$ is different, showing that g is not continuous in $[0, 0]$ and therefore ∇f is not.

- **Solution to Exercise 3.6:** Strictly speaking, defining $\frac{\partial f}{\partial d}(0, 0)$ requires $f(0, 0)$, which is undefined. However, we can take any generic direction $d = [d_1, d_2] \neq 0$ and prove that $\lim_{\alpha \rightarrow 0} f(\alpha d) = d_1^4 d_2^4 \alpha^4 / (d_2^2 + d_1^4 \alpha^2)^2 = 0$ however chosen d . In fact, if either $d_2 = 0$ or $d_1 = 0$ the numerator is always 0 while the denominator is not (they cannot be both 0). If they are both nonzero, the numerator goes to 0 while the denominator goes to $d_2^2 > 0$. Thus, only looking along lines it would be safe to define $f(0, 0) = 0$ by continuity, and therefore to have $\frac{\partial f}{\partial d}(0, 0) = 0$ for all $d \neq 0$, which gives $\frac{\partial f}{\partial d}(0, 0) = \langle [0, 0], d \rangle$.
- **Solution to Exercise 3.7:** $Q_x(z) = f(x) + \langle \nabla f(x), z - x \rangle + \frac{1}{2}(z - x)^T \nabla^2 f(x)(z - x)$; hence, $\nabla Q_x(z) = \nabla f(x) + \nabla^2 f(x)(z - x)$. Evaluated at $z = x$ this gives $\nabla f(x)$; the result for L_x is trivial since the first-order model is a linear function. The derivation handily reveals that $\nabla Q_x(z)$ is a linear (vector) function of z that coincides with $\nabla f(x)$ at $z = x$, i.e., it is the first-order model of ∇f at x (in fact it uses the “gradient of the gradient”, that is, the Hessian).
- **Solution to Exercise 3.8:** In the univariate case the condition is $(f'(z) - f'(x))(z - x) \geq 0$, i.e., “ $f'(z) - f'(x)$ and $z - x$ have the same sign”. In other words, $z \geq x \implies f'(z) \geq f'(x)$ and $z \leq x \implies f'(z) \leq f'(x)$, i.e., f' is monotone nonincreasing.
- **Solution to Exercise 3.9:** By definition, $\alpha f(z) + (1 - \alpha)f(x) \geq f(\alpha z + (1 - \alpha)x) \forall \alpha \in [0, 1]$. Hence, $\alpha(f(z) - f(x)) + f(x) \geq f(\alpha(z - x) + x)$, i.e., $f(z) - f(x) \geq [f(\alpha(z - x) + x) - f(x)]/\alpha$: send $\alpha \rightarrow 0$ to get $\frac{\partial f}{\partial(z-x)}(x) = \langle \nabla f(x), z - x \rangle$ and the result is proven.
- **Solution to Exercise 3.10:** We want to prove that f is both convex and concave (BCC) if and only if it is linear, i.e., $f(x) = \langle b, x \rangle + c$ for some $b \in \mathbb{R}^n$ and $c \in \mathbb{R}$. This is surprisingly nontrivial. We start with $\Leftarrow$. By definition, BCC means that $f((1 - \alpha)x + \alpha z)$ is equal to $(1 - \alpha)f(x) + \alpha f(z)$, since it has to be both $\geq$ and $\leq$. Now, assume $f(x) = \langle b, x \rangle + c$: then, $f((1 - \alpha)x + \alpha z) = \langle b, (1 - \alpha)x + \alpha z \rangle + c = (1 - \alpha)\langle b, x \rangle + \alpha\langle b, z \rangle + [(1 - \alpha)c + \alpha c] = (1 - \alpha)(\langle b, x \rangle + c) + \alpha(\langle b, z \rangle + c) = (1 - \alpha)f(x) + \alpha f(z)$, proving that f is BCC. Note how this crucially depends on $(1 - \alpha) + \alpha = 1$, it would not be true for generic $\gamma x + \delta z$. For $\implies$, define $g(x) = f(x) - f(0)$, so that $g(0) = 0$. It is trivial to see that, since f is BCC, then also g is. Hence $0 = g(0) = g((1 - (1/2))x + (1/2)(-x)) = (1 - (1/2))g(x) + (1/2)g(-x)$, i.e., g is antisymmetric: $g(-x) = -g(x)$. We now prove two facts: i. $g(\gamma x) = \gamma g(x)$, ii. $g(x + z) = g(x) + g(z)$. For i., We distinguish the case in which $0 \leq \gamma \leq 1$, where $g(\gamma x) = g(\gamma x + (1 - \gamma)0) = \gamma g(x) + (1 - \gamma)g(0) = \gamma g(x)$. If, rather, $\gamma > 1$, then $g(x) = g((1/\gamma)\gamma x) = g((1/\gamma)\gamma x + (1 - 1/\gamma)0) = (1/\gamma)g(\gamma x) + (1 - 1/\gamma)g(0) = (1/\gamma)g(\gamma x)$; multiply both sides by γ to get $\gamma g(x) = g(\gamma x)$. Finally, if $\gamma < 0$ then $g(\gamma x) = g((- \gamma)(-x)) = (-\gamma)g(-x)$ [using the previous results with $-\gamma > 0$] $= (-\gamma)(-g(x))$ [using $g(-x) = -g(x)$] $= \gamma g(x)$. For ii., $g(x + z) = g((1/2)2x + (1/2)2z) = (1/2)g(2x) + (1/2)g(2z) = (1/2)2g(x) + (1/2)2g(z) = g(x) + g(z)$ [using i. with $\gamma = 2$]. Now, i. and ii. are the alternative definition of linear function, hence, there exists $b \in \mathbb{R}^n$ s.t. $g(x) = \langle b, x \rangle$, thus $f(x) = g(x) + f(0)$ is affine with $c = f(0)$, as desired.
- **Solution to Exercise 3.11:** For 2., $[e^{ax}]'(x) = ae^{ax}$, which is positive increasing if $a > 0$, negative increasing if $a < 0$. For 3., $[-\ln(\cdot)]'(x) = -1/x$, which is negative increasing. For 4., $[a^x]'(x) = ax^{a-1}$; for $a < 0$ this is negative increasing, for $a \geq 1$ this is positive increasing. Only positive even integer a make x^a convex on all $\mathbb{R}$, since then ax^{a-1} is positive increasing (as the second derivative, $a(a-1)x^{a-2}$, is always positive).
- **Solution to Exercise 3.12:** No: consider $f(x_1, x_2) = \min\{x_1, x_2\}$ on the line $x_1 + x_2 = 0 \equiv x_2 = -x_1$, i.e., $\min\{x_1, -x_1\} = -|x_1|$ which is concave (and not linear, hence it cannot be convex).
- **Solution to Exercise 3.13:** $\alpha f(x) + (1 - \alpha)f(z) \geq f(\alpha x + (1 - \alpha)z) \implies \delta[\alpha f(x) + (1 - \alpha)f(z)] \geq \delta f(\alpha x + (1 - \alpha)z)$. $\alpha g(x) + (1 - \alpha)g(z) \geq g(\alpha x + (1 - \alpha)z) \implies \beta[\alpha g(x) + (1 - \alpha)g(z)] \geq \beta g(\alpha x + (1 - \alpha)z)$. Hence, $\delta[\alpha f(x) + (1 - \alpha)f(z)] + \beta[\alpha g(x) + (1 - \alpha)g(z)] = \alpha(\delta f(x) + \beta g(x)) + (1 - \alpha)(\delta f(z) + \beta g(z)) \geq \delta f(\alpha x + (1 - \alpha)z) + \beta g(\alpha x + (1 - \alpha)z)$.
- **Solution to Exercise 3.14:** Consider the “downward spike function centred at c ”, i.e., $s_c(x) = \min\{|x - c|, 1\}$. Clearly, s_c is quasiconvex: in fact, $S(f, l) = \emptyset$ if $l < 0$, $S(f, l) = [c - l, c + l]$ if $0 \leq l < 1$, and $S(f, l) = \mathbb{R}$ if $l \geq 1$. However, s_0 is not convex: in fact, $(1/2)s_0(0) + (1/2)s_0(2) = 1/2 < 1 = s_0((1/2)0 + (1/2)2) = s_0(1)$.
- **Solution to Exercise 3.15:** $S(\delta f, l) = \{x : \delta f(x) \leq l\} = \{x : \delta f(x) \leq l/\delta\} = S(f, l/\delta)$: since the latter is a convex set, the former also is. To prove that the sum does not preserve quasiconvexity consider $f(x) = s_{-1}(x) + s_1(x)$ (cf. the previous exercise). Clearly, $f(-1) = f(1) = 0$, but $f(x) > 0$

for all other values of x , i.e., $S(f, 0) = \{-1, 1\}$, which is not convex.

• **Solution to Exercise 3.16:**

```

procedure  $o^L = FP \{w, x\}$ 
   $o^i \leftarrow x_i^s$  for  $i \in N(1)$ ;
  for ( $l = 2$  to  $L$ ) do
    foreach ( $i \in N(l)$ ) do
       $o_i = \sigma_i \left( \sum_{(j,i) \in A(l)} w_{ji} o_j + w_i \right)$ 

```


Chapter 4

Smooth Unconstrained Optimization

4.1 Gradient method for general functions

The developments of the previous Chapters have already clearly delineated an obvious algorithmic approach to minimising any $f \in C^1$: for the current iterate x^i , compute the gradient $\nabla f(x^i)$. If $\nabla f(x^i) = 0$ (in general, it is “small”) then one can stop, otherwise the antigradient $-\nabla f(x^i)$ is a *direction of descent*, which means that for an appropriate (ly small) *stepsize* $\alpha^i > 0$, $f(x^i - \alpha^i \nabla f(x^i)) < f(x^i)$, and therefore another candidate iterate $x^{i+1} = x^i - \alpha^i \nabla f(x^i)$ is available. Before analysing the approach in details, however, we will discuss it as one particular implementation of a more general algorithmic scheme.

The general idea is that, given the current iterate x^i , one builds a *model* f^i that approximates f ; typically, “well” in some neighbourhood of x^i . The, one uses the model to drive the selection of the next iterate, the natural idea being by

$$x^{i+1} \in \operatorname{argmin}\{f^i(x) : x \in \mathbb{R}^n\}. \quad (4.1)$$

The idea is that f^i be a “simple” function, so that (4.1) can be readily solved. The most obvious model would be the first-order one, i.e., $f^i(x) = L^i(x) = L_{x^i}(x) = f(x^i) + \langle \nabla f(x^i), x - x^i \rangle$; however, the scheme is not working with such a choice, since then (4.1) is ill-defined (a linear function has no minimum). This unfortunate fact actually underlines an important general fact: any model will necessarily approximate f , and this approximation will typically only be “good” for points “close” to x^i , while the accuracy will degrade (possibly, dramatically) as x moves farther away from x^i . In other words, *one should never blindly trust a model*: for the first-order one, this would in fact lead to choosing x^{i+1} “infinitely far away” from x^i . On the contrary, derivative information is only completely significant when x is “infinitely near” to x^i , which would suggest to take a vanishing stepsize. In this case, as suggested by Theorem 3.1, the most reasonable choice is that of moving along the antigradient, as—for a vanishing stepsize—this would lead to the largest possible improvement in terms of function value; albeit, clearly, this would also be vanishing, which leads to a tension that we will discuss in some detail. Yet, this discussion foreshadows concepts that will be crucial throughout all this Chapter and beyond: many optimization algorithms can be viewed in terms of navigating the trade-offs between exploiting the information that a model provides without overly relying on it.

4.1.1 Gradient method basics

Since the antigradient $-\nabla f(x^i)$ provides the steepest descent direction, the *Gradient Method* for optimization of general (differentiable) functions, a.k.a. *steepest descent method*, is the immediate generalisation of the one seen in §1.4.2:

Algorithm 4.1. Steepest Descent for Generic Function

```
procedure  $x = SDQ(f, x, \varepsilon)$ 
  while ( $\|\nabla f(x)\| > \varepsilon$ ) do
     $d \leftarrow -\nabla f(x)$ ;  $\alpha \leftarrow \text{stepsize}(f, x, d)$ ;  $x \leftarrow x + \alpha d$ ;
```

Differently from Algorithm 1.1, however, this is not a definite algorithm but rather an *algorithmic scheme*, since a crucial part is left unspecified: how the stepsize is chosen. Obviously, the choice cannot be arbitrary: indeed, $f(x + \alpha d) < f(x)$ is only guaranteed to happen for suitably small α . It is easy to see that taking too large steps not only may not lead to a decrease of the objective value, but to a large increase which may result in $f(x^i) \rightarrow +\infty$: the algorithm *diverges* rather than converging.

Exercise 4.1. Find a too large fixed stepsize for $f(x) = x^2/2$. [solution]

However, “too small” steps are also to be avoided, as they can easily led to the algorithm “stalling far away from x^* ”. This is obvious if $\alpha = 0$, but also a sequence of steps “converging to zero too fast” can have a similar impact. As an example: consider $f(x) = -x$, so that $d^i = -f'(x) = 1$ starting from $x^0 = 0$ with $\alpha^i = 1/i^2$ (which converges to 0 “fast”). Thus, $x^i = -f^i = \sum_{k=1}^i 1/k^2$. Then, $\{f^i\} \rightarrow \pi^2/6 \approx 1.645$ [81], whereas $f_* = -\infty$.

Exercise 4.2. Make the previous example work even if $f_* > -\infty$. [solution]

Exercise 4.3. Is it possible to detect $f_* = -\infty$ as for a quadratic function? Discuss. [solution]

This is not to say that vanishing stepsizes $\{\alpha^i\} \rightarrow 0$ cannot be used; only, the convergence need be “not too fast”. Indeed, the same example as before with $\alpha^i = 1/i$ rather gives $x^i = -f^i = \sum_{k=1}^i 1/k$, so that $\{f^i\} \rightarrow -\infty = f_*$ [81], despite that the step is not bounded away from 0. Intuitively, the “total amount of space travelled”, $\sum_{i=1}^{\infty} \alpha^i \nabla f(x^i)$, must be “large enough to reach from x^0 to x^* ”, since one can’t usually put any finite bound to the distance from the starting point to the optimal solution, the series must diverge so as to be able to cross the required distance, whatever that is.

The previous discussion just wants to underline the concept that the stepsize has to be “just right”. Perhaps the most obvious version of the concept requires finding the optimal stepsize, i.e., the one that minimizes $\varphi_{x,d} = f(x + \alpha d)$:

$$\text{stepsize}(f, x, d) \in \operatorname{argmin}\{\varphi_{x,d}(\alpha)\}, \quad (4.2)$$

known as the *exact Line Search*. Leaving aside the issue of how practical this choice is, the approach completely reproduces Algorithm 1.1 when f is quadratic—and clearly there is no issue in actually computing the optimal stepsize there. Not surprisingly, one then obtains—for good and for bad—basically the same convergence and efficiency results. We skip for the time being the convergence, since it will be discussed in a more general context; then, [10, Th. 3.4] proves that *if* the algorithm converges to a point where “ f looks a strictly convex quadratic”, then the convergence is the one that could be expected for the quadratic function. More formally, if $f \in C^2$ and the sequence $\{x^i\}$ is converging to a local minimum x^* such that $\nabla^2 f(x^*) \succ 0$, then, for all sufficiently large i

$$f(x^{i+1}) - f(x^*) \leq r^2(f(x^i) - f(x^*)) \quad (4.3)$$

for the usual $r = (\lambda_1 - \lambda_n) / (\lambda_1 + \lambda_n)$ with λ_1 and λ_n respectively the maximum and minimum eigenvalue of $\nabla^2 f(x^*)$; that is, the algorithm converges linearly with the coefficient r dictated by the conditioning of $\nabla^2 f(x^*)$. The result is not surprising. In “the tail” of the convergence process, the iterates get closer and closer to x^* ; as a consequence, the original function $f(x)$ and its (fixed) second-order model $Q_{x^*}(x)$ are “very close to each other”. Then, the convergence of Algorithm 4.1 is basically the same as that of Algorithm 1.1 applied to Q_{x^*} . Remarkably—and this is, understandably, a constant all of local convergence / efficiency results—the crucial properties only need to hold in some neighbourhood $\mathcal{B}(x^*, \delta)$ of the point the algorithm is converging to: once the algorithm is “trapped” in the attraction basin of some local minima x^* , only the local behaviour of f around x^* is relevant. However, finding if—and how rapidly—the algorithm converges somewhere may require an entirely different analysis. This is not necessarily only an issue of mathematical proof: algorithms may actually exhibit different *phases*. For instance, a short *startup* phase with a specific behaviour (e.g., a sharp improvement in function values) may be followed by a *cruise* phase (e.g., the convergence speed somehow flattens out and remain stable for a while), finally reaching a *tail* in which the algorithm reaches close to the local minimum it will ultimately converge to and the convergence speed changes (e.g., increases) again; the latter is the one to which the previous result refers, which says little about the other. This behaviour is illustrated in the convergence plot (relative error in logarithmic scale w.r.t. iterations) of Figure 4.1(left) corresponding to one specific function: the three phases alluded to above can be clearly identified. However, such a behaviour is not pervasive. For instance, Figure 4.1(centre) shows a case where the startup phase is missing and the cruise phase is not very stable, while Figure 4.1(right) shows a case in which no clear phases can be identified.

However, the previous result has only scant applications: in general, finding the global optimum of a univariate function is impossible, save for “very simple” functions having “very simple” derivatives with “only a few roots” that can be computed algebraically. One could (rightly) argue that, since only an approximate solution of the whole optimization problem is required, then (4.2) should reasonably be solved approximately as well. This would be problematic as well if one would be looking to a global minimum of (4.2), but it is easy to argue against it: since Algorithm 4.1 cannot guarantee a global minimum anyway, there is no reason why the Local Search should be exact—save of course in the case where f is convex, and therefore φ is, hence exactness comes “for free”. Thus, a local approach to (4.2) makes utter sense: ideally, one would like to converge to the “first attraction basin of φ to the right of 0”, in order to ensure that the function value will improve between x^i and x^{i+1} . Obtaining this algorithmically is nontrivial and it will be discussed later on: for now we concentrate on the most basic issue of *what accuracy should be required to (4.2)*. Clearly, the less accuracy required to the Line

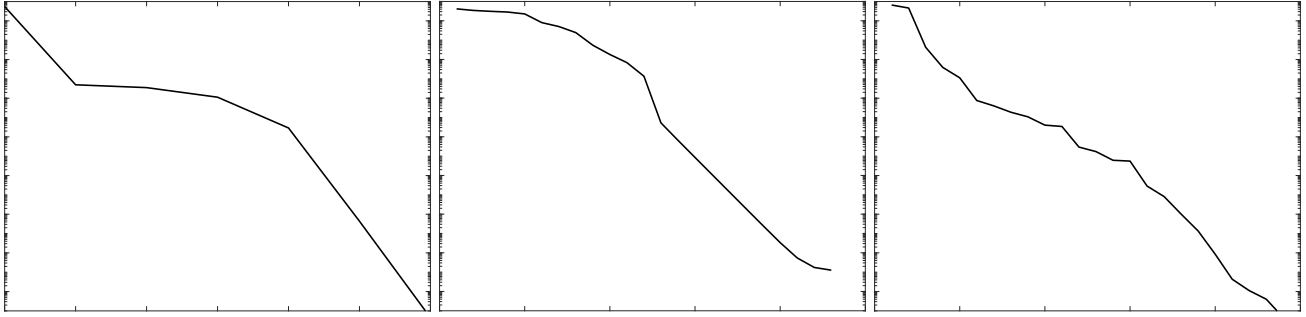


Figure 4.1: Some convergence profiles for Algorithm 4.1

Search the less iterations it can be expected to require, which therefore potentially decreases the total function evaluation cost of the algorithms (although there may be trade-offs). It therefore makes sense to ask what is an appropriate stopping criterion for whatever algorithmic scheme—e.g., among those of Chapter 2—is used to solve (4.2).

Since $f \in C^1$, $\varphi \in C^1$ as well. It is clear that φ' can be computed in terms of ∇f , the exact formula being $\varphi'(\alpha) = \langle \nabla f(x + \alpha d), d \rangle$.

Exercise 4.4. Prove “by prime principles” (definition of φ'). [solution]

Exercise 4.5. Prove using the *chain rule* in $\mathbb{R}^n$: $f : \mathbb{R}^m \rightarrow \mathbb{R}^k$, $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$, $h(x) = f(g(x)) : \mathbb{R}^n \rightarrow \mathbb{R}^k$, $Jh(x) = Jf(g(x)) \cdot Jg(x)$ (note that $Jf \in \mathbb{R}^{k \times m}$, $Jg \in \mathbb{R}^{m \times n}$, in fact $Jh \in \mathbb{R}^{k \times m} \cdot \mathbb{R}^{m \times n} = \mathbb{R}^{k \times n}$). [solution]

We now exploit the fact that $d = -\nabla f(x)$: it is then clear that the standard stopping criterion for Line Search, i.e., $|\varphi'(\alpha)| \leq \varepsilon'$, can be related to that of Algorithm 4.1. The question is, therefore, how one should choose ε' to be able to finally (and finitely) achieve the stopping criterion of Algorithm 4.1. A simple answer is

$$\varepsilon'_i = \varepsilon \|\nabla f(x^i)\|. \quad (4.4)$$

This choice leads to a global convergence result similar in spirit to the previous local one:

Theorem 4.1. For $f \in C^1$ and Algorithm 4.1 with choice (4.4), if $\{x^i\} \rightarrow x$, then $\|\nabla f(x)\| \leq \varepsilon$.

Proof. Let $\{x^i\} \rightarrow x$: from the hypothesis, at each iterate i one has $|\varphi'(\alpha^i)| \leq \varepsilon'_i$. Now, $f \in C^1$ means $\nabla f \in C^0$: thus,

$$\lim_{i \rightarrow \infty} |\langle \nabla f(x^i), \nabla f(x^{i+1}) \rangle| = \langle \nabla f(x), \nabla f(x) \rangle \leq \lim_{i \rightarrow \infty} (\varepsilon'_i = \varepsilon \|\nabla f(x^i)\|) = \varepsilon \|\nabla f(x)\|$$

which yields the desired $\|\nabla f(x)\| \leq \varepsilon$. $\square$

The result is not completely satisfactory, since it depends on the algorithm actually converging somewhere: if this happens, then the point it converges to is an approximate stationary point to the required accuracy. We will not elaborate further the issue of what conditions are required to get the convergence in the first place, since, as we will see, this is not the approach that is best used in practice: “much coarser” line searches can be used, and turn out to be usually better. However, the result establishes important concepts. The first is that it is indeed possible to compute the Line Search approximately, reaching for a(n approximate) stationary point of φ , and still get the best that Algorithm 4.1 will in general provide anyway—an approximate stationary point of f . The second is that the accuracy required to the Line Search does not need to be constant: when “far from the optimum”, i.e., $\|\nabla f(x)\|$ is “large”, then one does not need an accurate solution, i.e., one can stop even if $\varphi'(\alpha)$ is also “large”. However, as the algorithm progresses towards the desired solution, the accuracy has to increase. Indeed, the result suggests that, towards the end of the process, $\varepsilon' \approx \varepsilon^2$ may be required, which could be a rather high accuracy to reach (although algorithms for univariate optimization can be quite fast, making this not entirely unreasonable). However, again, this is not what is typically done in practice, as we shall see soon. Interestingly, (pseudo)convexity of f is nowhere in the picture, although of course “you get what you pay for”: the stronger properties of f give stronger guarantees on the quality of the obtained minima.

Exercise 4.6. Strictly speaking, Theorem 4.1 does not give *finite termination*, i.e., $\exists i$ s.t. $\|\nabla f(x^i)\| \leq \varepsilon$, but the result is clearly “close at hand”: discuss how to get it. [solution]

Exercise 4.7. What if one would want $\varepsilon = 0$, i.e., “asymptotic convergence”? [solution]

Getting somewhat stronger and/or more general convergence result is possible, but they require either conditions

on $f / \nabla f$ other than C^1 , or “exact” Line Search (e.g., [9, p. 206, p. 234]). However, the general gist is that approaches using inexact Line Search should be expected to work. The next section thoroughly analyses how to make this concept practical.

4.1.2 Inexact Line Search: Armijo-Wolfe

A working, complete and practical theory of “how much an inexact a Line Search can be” has been thoroughly developed. It is important to remark that, while discussed for the Gradient method, the theory does not really depend on the specific choice $d = -\nabla f(x)$ and will be easily extended to other approaches later on; this is why in this section we will use “ d ” as much as possible to indicate the direction, actualising it as “ $-\nabla f(x)$ ” only when needed. The analysis starts with defining a condition that just state the obvious: convergence can be obtained providing that, at each iteration, the function value $f(x^i)$ “decreases enough”. The only point is the definition of “enough”, that can be given in terms of the *Armijo condition*

$$\varphi(\alpha) \leq \varphi(0) + m_1 \alpha \varphi'(0), \quad (4.5)$$

where $m_1 \in (0, 1)$ is a constant (typically, $m_1 < \ll 1$, i.e., $1\text{e-}3$). In plain words, the derivative of φ in 0 is negative, since $\varphi'(0) = \langle \nabla f(x^i), d \rangle = -\|\nabla f(x^i)\|^2$, and therefore “promises” that the function value will decrease moving right from 0. The amount of expected decrease is predicted by the first-order model, i.e.,

$$\varphi(\alpha) \approx \varphi(0) + \alpha \varphi'(0)$$

can be expected to hold when α is “small”. As we have discussed previously we don’t want α to be “too small”, hence we can’t expect to closely match the decreased “promised” by the first-order model; then, we “are content with much less than that, provided it is still at least some fixed fraction”. This is pictorially illustrated in Figure 4.2(left): the steeper dashed line is the first-order model, the less steep one the decrease required by (4.5). The figure emphasises the set of points α that satisfy the condition: obviously, it surely contains some interval of the form $[0, \bar{\alpha}]$ for some $\bar{\alpha} > 0$, as (4.5) is basically satisfied with $m_1 = 1$ when “close to 0”, and a fortiori when $m_1 < 0$. This is intuitive, and it will be made formal later on.

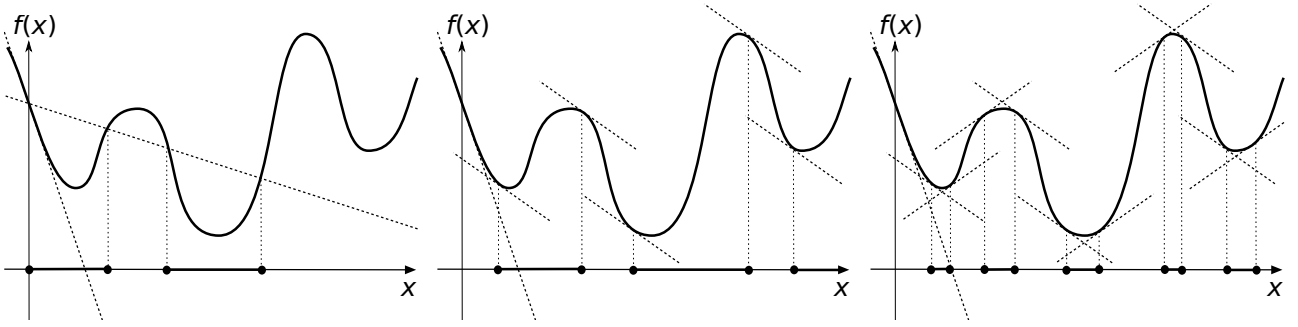


Figure 4.2: Pictorial illustration of the Armijo-Wolfe conditions

Despite its simplicity, it is already possible to show that (4.5) plus some “not disaster” condition on the stepsize—i.e., that it does not converge to 0 “too fast”, which as we have seen can obviously be disastrous—is enough to basically get a convergence-like condition:

Theorem 4.2. Assume that in all iterations of Algorithm 4.1 condition (4.5) holds, and in addition that $\sum_{i=1}^{\infty} \alpha^i = \infty$; then, either $\{f(x^i)\} \rightarrow -\infty$, or $\{\|\nabla f(x^i)\|\} \rightarrow 0$.

Proof. Assume by contradiction that $\{\|\nabla f(x^i)\|\} \geq \varepsilon > 0$ for all i and recall that $-\varphi'_i(0) = \|\nabla f(x^i)\|^2$: thus, $\varphi'_i(0) \leq -\varepsilon^2$ for all i . Summing (4.5) along all iterations, written as

$$f(x^{i+1}) \leq f(x^i) + m_1 \alpha^i \varphi'_i(0) \leq f(x^i) - m_1 \alpha^i \varepsilon^2$$

one readily obtains $f(x^i) \leq f(x^0) - \varepsilon^2 \sum_{h=1}^{i-1} \alpha^h$. Sending $i \rightarrow \infty$, the right-hand side goes to $-\infty$. Therefore, necessarily $\{f(x^i)\} \rightarrow -\infty$, i.e., $\{x^i\}$ is a minimizing sequence (in particular, of the kind showing that $f_* = -\infty$). If this does not happen a contradiction is obtained, which shows that it is not possible that $\|\nabla f(x^i)\|$ remains bounded away from zero. $\square$

A twist of the result is that it proves that every accumulation point of $\{x^i\}$ is stationary, but it does not prove there are any. This is seldom a real issue. For instance, assume one can prove that the level set $S(f, f(x^0))$ is bounded (which often happens in machine learning due to regularization): then, since the algorithm is of descent ($f(x^{i+1}) < f(x^i)$), all iterates belong to a compact set, and Bolzano-Weierstrass Theorem guarantees

that $\{x^i\}$ has at least an accumulation point (cf. Appendix A.10), which is then a stationary one. We will mostly avoid these sophistications in the analysis, focusing rather on the real crux of the matter: on top of condition (4.5), the result requires that α^i does not get “too small” (clearly, $\alpha^i \geq \bar{\alpha} > 0$ satisfies the hypothesis), or at least $\alpha^i \rightarrow 0$ “slowly enough”. As already remarked, there is nothing in condition (4.5) guaranteeing that: thus, a different condition is required. Different ones have been proposed, along which Goldstein’s condition that requires

$$\varphi(\alpha) \geq \varphi(0) + m_2 \alpha \varphi'(0)$$

for some constant $m_2 \in (m_1, 1)$. It is easy to see that such a condition rules out values of α arbitrarily close to 0, and therefore can do the required job. However, the condition may as well exclude *all* local minima of the function from being selectable, that is highly unnatural as one would clearly want the exact Line Search to always be a possible (if extreme) implementation. This is why *Wolfe’s condition*

$$\varphi'(\alpha) \geq m_2 \varphi'(0), \quad (4.6)$$

again for any fixed $m_2 \in (m_1, 1)$, is preferred. Intuitively, an exact Line Search would not only decrease the function value, as required by (4.5), but also an α^i such that $\varphi'(\alpha^i) = 0$, or “close to”: hence, it would reduce the (absolute value) of the derivative from its initial value $\varphi'_i(0) = -\|f(x^i)\|^2 (> \varepsilon^2)$. Thus, (4.6) stipulates that the value of the derivative has to be “a bit closer to 0” than its initial value $\varphi'_i(0)$. This makes sense, since ultimately one wants $\|f(x^i)\|^2$ to become “small”. The condition is pictured in Figure 4.2(centre): again, the steeper dashed line is the first-order model, of which the slope $\varphi'_i(0)$ is relevant here. The other dashed lines corresponds to the points where the (less steep) slope start satisfying (4.6), and again figure emphasises the set of points α that satisfy the condition. It is immediate to verify that points arbitrarily close to 0 are now excluded, but all local minima (where $\varphi'_i(\alpha) = 0$, but then the local maxima as well) are included. However, the “one sided” nature of (4.6) is so that the value of the derivative increases: since the initial value is negative, this may result in a less negative value (closer to 0), but also to a positive value. Hence, there is no guarantee that the absolute value of the derivative decreases, as one would expect since it is its norm that should converge to 0. This motivates the alternative *strong Wolfe condition*

$$|\varphi'(\alpha)| \leq m_2 |\varphi'(0)| = -m_2 \varphi'(0), \quad (4.7)$$

which precisely stipulates that the absolute value of the derivative has to decrease w.r.t. the initial value. This is depicted in Figure 4.2(right), with the dashed lines with positive slope plotted at points where $\varphi'(\alpha) = -m_2 \varphi'(0)$. The picture immediately shows that i) the set of the points satisfying the strong Wolfe (4.7) condition is smaller than that of the points satisfying the “weak” one (4.6), which implies that again points arbitrarily close to 0 are excluded, and ii) that all local minima (and local maxima as well) satisfy it. While of course it is in principle harder to satisfy (4.7) than (4.6), the stronger condition is often considered to be preferable in practice.

The combination of (4.5) and (4.6) (or (4.7)), known as the *Armijo-Wolfe conditions*, makes therefore sense. In particular, one would expect that requiring (4.5) together with (4.6) keeps all the “interesting” local minima, especially the “first one on the right of 0”, while hopefully doing away with some of the maxima due to the fact that the function value is required to decrease w.r.t. the initial one. This is in fact almost always true, but not necessarily so in particular if m_1 is chosen “too large”. This is one of the reasons why m_1 is usually chosen “small”, e.g., $1e-3$ or even $1e-5$ (besides the fact that, obviously, weaker requests are typically cheaper to satisfy). No fixed value can be guaranteed to work for all functions, which means that in order to have absolute guarantees some algorithmic safeguard will be required, as discussed later on; however, a “small” fixed value very often works well in practice.

Exercise 4.8. Find a simple function and a “too large” value of m_1 such that requiring (4.5) excludes all local minima of the function. [solution]

As we shall see soon, this property is very useful to devise algorithmic schemes to efficiently find stepsizes that satisfy both (4.5) and (4.6) (or (4.7)), but before setting to find something one should always be sure that the sought-for is there to be found. It is therefore useful to prove that this is indeed the case, especially since the proof introduces some interesting concepts.

Theorem 4.3. Assume that $\varphi \in C^1$ and $\varphi(\alpha)$ is bounded below for $\alpha \geq 0$: then, there exists some α where both (4.5) and (4.6) are satisfied.

Proof. We will need Rolle’s theorem [13, Th. 2.3.8]: for any univariate differentiable function $f \in C^1$ and any interval $[a, b]$ such that $f(a) = f(b)$, there exists some $c \in (a, b)$ such that $f'(c) = 0$. Intuitively, for having the same value in a and b the function first has to increase and then to decrease, or vice-versa (save in the case of the constant function, which however has constantly null derivative), which means that at least a local maximum or minimum must be found inside the interval.

We then define the *twisted* first-order model of φ in 0, i.e., $l(\alpha) = \varphi(0) + m_1\alpha\varphi'(0)$: (4.5) is satisfied if and only if $\varphi(\alpha) \leq l(\alpha)$. Thus, the distance function between l and φ , $d(\alpha) = l(\alpha) - \varphi(\alpha)$, is such that $d(0) = 0$, and (4.5) is satisfied whenever $d(\alpha) \geq 0$. For future reference, $d'(\alpha) = m_1\varphi'(0) - \varphi'(\alpha)$, and hence in particular $d'(0) = (m_1 - 1)\varphi'(0) (> 0)$.

We first claim that if $d(\alpha) < 0$ for all $\alpha > 0$, then φ unbounded below. Thus, if φ is bounded below, then there exists some $\alpha > 0$ such that $d(\alpha) = 0$, and we denote by $\bar{\alpha} > 0$ the smallest of these values. Clearly, (4.5) is satisfied both in 0 and in $\bar{\alpha}$: we claim that then it is actually satisfied for all $\alpha \in (0, \bar{\alpha})$ as well.

Then, apply Rolle's theorem to d : there exists some $\alpha' \in (0, \bar{\alpha})$ such that $d'(\alpha') = 0$. By the formula above, this is equivalent to $m_1\varphi'(0) = \varphi'(\alpha')$. Since $m_2 > m_1$ and $\varphi'(0) < 0$, one then has that $m_2\varphi'(0) < m_1\varphi'(0) = \varphi'(\alpha')$: hence, (4.6) also holds in α' . $\square$

Exercise 4.9. Prove the first claim in the Theorem. [solution]

Exercise 4.10. Prove the second claim in the Theorem. [solution]

Thus, the Theorem proves that some α' exists that satisfies the Armijo-Wolfe conditions. In fact, since $\varphi'(\alpha') = m_1\varphi'(0) < 0$, α' satisfies (4.7) as well. It is also clear from the proof that α' belongs to the attraction basin of the first local minima of φ on the right of 0. Now that we know that such a point exists, we can discuss how one can set about to find it. We will significantly simplify the discussion by assuming that is m_1 “small enough” such that not all local minima, and in particular the one to whose attraction basin α' belongs, is not “cut” by (4.5). The algorithmic scheme is therefore simple: *go for that local minima, but stop as soon as (4.5) and (4.6) are satisfied*.

In plain words, one could take any of the univariate optimization schemes seen in Chapter 2 and add it with the early stopping criterion “(4.5) and (4.6) satisfied”. However, depending on the fine details of the Line Search algorithm this may not actually work, and some more analysis is required. We briefly discuss the case of Dichotomic Search (cf. §2.6). The standard implementation of the approach may fail to find a point satisfying the Armijo-Wolfe conditions even if one exists in the following scenario: assume there is more than one local minima on the right of 0, but, say, only the first one satisfies (4.5). Also, assume that the maximum stepsize $\bar{\alpha}$ is chosen on the right of (say) the second local minima, and the intermediate point α is chosen on right of the first local minima in the attraction basin of the second, i.e., $\varphi'(\alpha) < 0$: then, the interval is restricted as $[\alpha, \bar{\alpha}]$. Coherently with Dichotomic Search, some local minima α^* , i.e., such that $\varphi'(\alpha^*) = 0$, still exists in the interval, and therefore there are points that satisfy (4.6); but in the attraction basin of α^* there are no points satisfying (4.5), and therefore there is no longer any chance to satisfy both conditions.

Exercise 4.11. Provide a full example proving that dichotomic search may fail to find a point satisfying the Armijo-Wolfe conditions even if one exists. [solution]

Some ad-hoc changes in the logic of the Dichotomic Search is required. A simple one is the following: *refuse to move the left endpoint of the interval unless to points that satisfy (4.5)*. This is easy to implement, and guarantees that the set of points of the interval that satisfy (4.5) is never empty. Thus, hopefully the right endpoint will be decreased until the interval frames the right local minima, and the process will work as intended. This change is nontrivial if, for instance, quadratic interpolation is used to find the middle point: indeed, this requires the sign of the derivatives on the endpoints in the interval to be different, a condition that the modified version may no longer satisfy. Thus, it is not completely guaranteed that the process will work: in particular, the right endpoint of the interval may end up on the left of the local minimum, so that while there are points in the interval that satisfy (4.5), there may be none that satisfies (4.6). However, this can be solved by simply ensuring that the interval does not shrink too small (a condition that is typically guaranteed anyway), i.e., that one finds a point α satisfying (4.5) such that $\alpha \geq \tau$ for some fixed $\tau > 0$. Then, while one can no longer invoke Theorem 4.3, Theorem 4.2 applies (α^i is bounded away from 0) and convergence is still retained. Similar, more refined logic can be employed to build approaches that are resistant to the odd case in which $m_1 = 1e-5$ is not small enough to ensure that the first local minima on the right of 0 satisfies (4.5) [10, Algorithm 3.5]. We will not delve further in the subject, which is not particularly relevant for the applications of interest here, referring the interested reader to the cited bibliography.

Indeed, “very simple” Line Search are traditionally preferred in many applications. Along these lines, the previous discussion shows that one could actually forego Wolfe's condition (4.6) only check the Armijo's one (4.5), provided that one knows that the stepsize does not become “too small”. This leads to the barebones “backtracking” Line Search:

Algorithm 4.2. Backtracking Line Search

<pre> procedure $\alpha = BLS(\varphi, \alpha, m_1, \tau)$ // $\tau < 1$ while $(\varphi(\alpha) > \varphi(0) + m_1\alpha\varphi'(0))$ do $\alpha \leftarrow \tau\alpha$; </pre>
--

From Theorem 4.3, we know that there is a full interval $[0, \bar{\alpha}^i]$ in which (4.5) is satisfied. Provided that the initial stepsize α in Algorithm 4.2 is always the same, the returned stepsize is $\alpha^i = \alpha \tau^{h(i)}$, where $h(i) = \min\{k : \tau^k \leq \bar{\alpha}^i\}$. Thus, an Armijo-only approach (among which, the modification to the Dichotomic Search discussed above) can be guaranteed to work provided that one can show that $\bar{\alpha}^i \geq \bar{\alpha}$ for some fixed $\bar{\alpha} > 0$ and all i : in plain words, the interval on the right of 0 satisfying (4.5) does not vanish. In this case, it is easy to see that $\alpha^i \geq \alpha \tau^h$, with $h = \min\{k : \tau^k \leq \bar{\alpha}\}$: the stepsize is bounded away from 0, and the convergence theory applies. For this to work one needs conditions on f , but not surprising ones. Intuitively, for $\alpha = 0$ the tomography φ is decreasing faster than its twisted first-order model $l(\alpha)$ (cf. Theorem 4.3), since $m_1 < 1$. For the function to catch-up ($d(\alpha) = 0$ again), the derivative of φ must increase, so that is eventually becomes greater than the (fixed) one of $l(\cdot)$. How rapidly this happens depends therefore from how rapidly the derivative of φ (hence, the gradient of f) changes: if this is bounded, then the interval cannot shrink arbitrarily small. This is therefore basically a requirement on how a function changes, i.e., Lipschitz continuity: however, it is not $\varphi(f)$ that matters here, but rather $\varphi'(\nabla f)$. We therefore have to require that f is L -smooth, cf. Appendix A.11. Proving the required result is then a matter of technicalities.

From the already discussed formula $\varphi'_{x,d}(\alpha) = \frac{\partial f}{\partial d}(x + \alpha d) = \langle \nabla f(x + \alpha d), d \rangle$ one immediately gets that $\varphi'_{x, -\nabla f(x)}(0) = \langle \nabla f(x), -\nabla f(x) \rangle = -\|\nabla f(x)\|^2 < 0$: “the farther $\nabla f(x)$ is from 0, the steeper $\varphi'(0)$ is”. The next step is to “translate” L -smoothness from f to φ : if f is L -smooth, then φ is $[L\|d\|^2]$ -smooth.

Exercise 4.12. Prove this “from prime principles”. [solution]

The convergence result is now at hand. Intuitively $\varphi'(0)$ starts “large” and can only “decrease slowly”: thus, the stepsize cannot become too small, which ensures convergence with Armijo-Wolfe or Backtracking Line Search. Formally, recall from the proof of Theorem 4.3 that there exists the smallest value $0 < \alpha' < \bar{\alpha}$ such that (4.5) hold for all $\alpha \in [0, \bar{\alpha}]$, and $\varphi'(\alpha') = m_2 \varphi'(0) > \varphi'(0)$. Now, φ is $[L\|d\|^2]$ -smooth, which implies that α' (and, therefore, $\bar{\alpha}$) is “large”: from

$$L\|d\|^2(\alpha' - 0) \geq \varphi'(\alpha') - \varphi'(0) > (1 - m_2)(-\varphi'(0)) = (1 - m_2)\|d\|^2$$

one immediately gets $\bar{\alpha} > \alpha' > (1 - m_2)/L$. Hence, any stepsize that satisfies Wolfe’s condition is bounded away from 0; as a consequence, the maximum stepsize (in the attraction basin of the first minimum on the left of 0) is also bounded away from 0. Thus, the gradient method converges with practical variants of the Armijo-Wolfe Line Search or even the Backtracking Line Search, provided f is L -smooth.

The next question is whether one can estimate the efficiency of such an approach, i.e., the convergence speed. In the quadratic case, the answer crucially depended on the Hessian having or not directions of null curvature, so one should fully expect the same to happen here (as whatever result one gets can be applied to the quadratic case, and the exact Line Search is a special case of the Armijo-Wolfe one). Of course the appropriate generalisation of the concept is required, which is τ -convexity (cf. Appendix A.11). Without going too much in details, the result is what can be expected: for a L -smooth and τ -convex function, convergence “in the tail” is linear with the usual rate depending on the worst-case condition number $\kappa = L/\tau$ of the Hessian. Again, the crucial properties are only needed to hold in a neighbourhood $\mathcal{B}(x^*, \delta)$ of the point the sequence of iterates is (supposedly) converging to. We refer the interested reader to [9, p. 240], only mentioning here that, as expected, the inexact Line Search worsens convergence somewhat: the rate of convergence has a form $r \approx (1 - \tau/L)$, with “ $\approx$ ” masking some terms depending on m_1 and m_2 . Notably, this result counts *iterations and not function evaluations*, underlying a trade-off that should be fully expected: a smaller m_1 and/or larger m_2 worsen the ratio r , but, by dint of relaxing the stopping criteria, possibly make the Line Search faster. In plain words, as the Line Search becomes more and more inexact the overall convergence worsens, but on the other hand each individual Line Search requires less function calls. This behaviour actually shows up in practice. Fortunately, default parameter values such as $m_1 = 1\text{e-}3$ and $m_2 = 0.9$ work well in very many practical cases: the number of iterations in the Line Search is limited to a handful at most, while the overall convergence rate remains close enough to the maximum that can possibly be expected given the condition number of the Hessian in the convergence point.

The alert reader would have surely have noticed that we are not mentioning the non-strongly-convex case, i.e., we require $\tau > 0$: this is because that case will be discussed next in the context of the “most inexact Line Search”.

4.1.3 Gradient method with fixed stepsize

The previous discussion has clearly revealed that, at least under L -smoothness, an “extremely inexact Line Search” can be used: a *fixed stepsize* $\alpha^i = \bar{\alpha} > 0$. This has clear advantages: it is simple, and it does not require more than one function evaluation per each iteration. On the other hand it is clearly “rigid”: since it has to work at every iteration, the stepsize will necessarily have to be “small enough” that the function value is invariably guaranteed to decrease. This potentially does away with opportunities to decrease the function value

more, thereby revealing again the trade-off between the number of function evaluations in each “Line Search” (one) and the number of iterations. On the other hand, such a setting entirely does away with any issue about the stepsize vanishing: Theorem 4.2 surely works—provided that condition 4.5 is guaranteed to hold.

Before entering the technicalities let’s start with a qualitative note that is not (as, in fact, most of the discussion in the previous section) really specific of the choice $d = -\nabla f(x)$. We aim at constructing a sequence $\{x^i\}$ converging to some x^* , hopefully a stationary point. This, by the very definition, requires that $\{\|x^{i+1} - x^i\| = \alpha^i \|d^i\|\} \rightarrow 0$. As already mentioned, the condition is necessary but not sufficient: not only the norm has to converge, but has to do this “fast enough”.

Exercise 4.13. Provide an example where $\alpha^i \|d^i\| \rightarrow 0$ but the algorithm is not converging to a finite x^* . **[solution]**

Since $\alpha^i = \bar{\alpha}$ is bounded away from 0, this immediately requires that $\|d^i\| \rightarrow 0$. Fortunately, this is “very natural” for $d^i = -\nabla f(x^i)$, as one is precisely aiming at obtaining $\|\nabla f(x^i)\| \rightarrow 0$ (and this will naturally extend to other choices of the direction that will be discussed in the following). This, for once, clarifies a doubt that one may have harboured: since d^i is used as a direction, wouldn’t it be reasonable to take it as normalised, i.e., $d^i = -\nabla f(x^i) / \|\nabla f(x^i)\|$? Of course, for Line Search based approaches there would not be a significant difference, save for the fact that some parameters—say, the initial trial stepsize—may have to be set differently (and may be less convenient to set, as we will discuss in the following). However, for fixed stepsizes this would be a dealbreaker: *if $\|d^i\|$ does not vanish, then α^i has to.* In other words, *α^i can remain bounded away from 0 only if $\|d^i\|$ vanishes.* This apparently obvious comment will turn out to be important later.

Coming back to fixed stepsize, one clearly has to show that there exists a fixed $\bar{\alpha} > 0$ $f(x^{i+1}) = f(x^i - \bar{\alpha}d^i) < f(x^i)$ for all i . Intuitively, “if f varies infinitely rapidly, then only infinitely short steps α^i may be possible”: thus, it is crucial to bound how rapidly f changes, which boils down to bounding how ∇f does, i.e., to L -smoothness. We can then readily re-use the results of the previous section, and in particular:

- if f is L -smooth, then φ is $[L\|d\|^2]$ -smooth;
- there exists the smallest value $\alpha' \in (0, \bar{\alpha})$ such that (4.5) holds however chosen $\alpha \in (0, \bar{\alpha}]$, and $\varphi'(\alpha') = m_2\varphi'(0) > \varphi'(0)$;
- $-\varphi'(0) = \|d\|^2 = \|\nabla f(x)\|^2$.

The intuition is the same as before: L -smoothness implies that φ' cannot change too rapidly, hence there exists a fixed minimal step $\bar{\alpha}$ such that $\varphi'(\bar{\alpha}) = 0$ can ever happen. In other words, $\varphi'(\alpha) < 0$ for α the whole interval $[0, \bar{\alpha})$, i.e., φ is always decreasing up until $\bar{\alpha}$. It therefore makes plenty of sense to choose $\bar{\alpha}$ as the fixed stepsize. Turning the intuition into a proof, however, requires some work.

Using $[L\|d\|^2]$ -smoothness and $\varphi'(0) = -\|d\|^2$ gives

$$\varphi'(\alpha) \leq \varphi'(0) + L\|d\|^2\alpha = \|\nabla f(x)\|^2(L\alpha - 1);$$

hence, $\varphi'(\alpha) \leq 0$ for all $\alpha \in [0, 1/L]$: $\bar{\alpha} = 1/L$ is the proposed fixed stepsize, the maximum one for which the function is guaranteed to be always decreasing. This would already be enough if one was only interested in establishing convergence, as then Theorem 4.2 surely applies for a proper m_1 . However, the analysis can now be made more informative by also discussing efficiency: this requires upper bounding the negative term $\varphi(0) - \varphi(1/L)$, i.e., the improvement of objective function value obtained at each iteration. The intuition to do that is that the derivative φ' is negative ($-\|d\|^2$) in $\alpha = 0$ and null in $\alpha = 1/L$. Furthermore, by $[L\|d\|^2]$ -smoothness, the derivative has to decrease *at least linearly* in the interval, since $\bar{\alpha} = 1/L$ is precisely obtained as the shortest step that can bring the derivative to 0 under the Lipschitz bound. Since the more the derivative is negative the more the function decreases, the worst case is clearly the one in which φ' is actually linear: $\varphi'(\alpha) = (L\alpha - 1)\|d\|^2$. Thus, the worst case for φ is the function exhibiting that as the derivative, i.e., its *antiderivative*: the quadratic function $\varphi(\alpha) = (L\alpha^2/2 - \alpha)\|d\|^2$.

Exercise 4.14. Prove the last statement using the fundamental theorem of calculus. **[solution]**

The above analysis yields the bound on the minimum possible improvement:

$$\varphi(\alpha) - \varphi(0) \leq \|d\|^2 [L\bar{\alpha}^2/2 - \bar{\alpha}] = -\|\nabla f(x^i)\|^2 / 2L.$$

This bound is clearly tight for an appropriate quadratic function, so it cannot be improved, but it is not for general functions in which the quadratic approximation implicit in the bound is not tight. Yet, it immediately provides the estimate of the error decrease:

$$a^{i+1} = f(x^{i+1}) - f_* \leq (a^i = f(x^i) - f_*) - \|\nabla f(x^i)\|^2 / 2L.$$

Unfortunately, this kind of estimate is *not* one producing a linear convergence rate, of the form $a^{i+1} \leq ra^i$

for some $r < 1$, as some quantity Δ^i is *subtracted* to the error rather than multiplied to it. This is typical of *sublinear* convergence results. Indeed, [3, Th. 3.3] proves the sublinear convergence bound

$$a^i \leq 2L \|x^1 - x^*\|^2 / (i - 1),$$

i.e., the number of iterations required to get accuracy ε has the general form $i \in O(LD^2/\varepsilon)$. This is the same form—in fact, precisely the same result—mentioned in §1.4.5 for a quadratic non strictly convex function ($\lambda_n = 0$). This result is not *tight* in the sense that the gradient method is not the one having the best (worst-case) convergence: we will see other approaches showcasing—vastly better, although still sublinear—complexity of the form $O(1/\sqrt{\varepsilon})$ to minimize a L -smooth function. It is useful to remark, however, that the latter result *is tight*: [3, Th. 3.14] shows a specific L -smooth function for which it can be proven that for *any possible algorithm* (only relying on function and gradient computations) it must necessarily hold that $a^i \geq O(LD^2/i^2)$. There is a relevant caveat, i.e., this result only holds “for large n ”, i.e., when $i \leq n$. Very informally speaking, the trick is that any variable of the function identifies a “troublesome direction” that “slows down” the algorithm (whatever it is) for one iteration. Ultimately, every algorithm (with the above caveat) cannot have a better complexity than $O(1/\sqrt{\varepsilon})$; this is indicated by saying that *the problem* of minimising an L -smooth function is $\Omega(1/\sqrt{\varepsilon})$. This is a very important kind of result, proving the inherent limitations of algorithmic capabilities. In layman terms, *algorithms can only go so far with “nasty” problems*.

The alert reader may immediately object that we have seen plenty of algorithms already exhibiting better (in particular, linear) convergence, in fact even the “plain” gradient method with a very implementable inexact line search (cf. §4.1.2, and disregarding the difference between iterations and function evaluations). However, all these results hinged on (local) τ -convexity, i.e., $\lambda_n > 0$. Indeed, the “nasty” function alluded to previously is *not* τ -convex. And if the property is postulated, a different analysis is possible that shows the expected linear convergence. We will now detail on some of the relevant aspects of the analysis, because doing so showcases one of the main reasons to do the analysis in the first place: that of *optimally setting the algorithmic parameters*, in this case the stepsize. Indeed, the analysis will show that $\bar{\alpha} = 1/L$ is not the optimal stepsize in the case of a τ -convex function.

The idea is to try to measure the impact of the choice of α on the distance to the current iterate and x^* . If x denotes the current iterate, and $z = x - \alpha \nabla f(x)$ the next one, then

$$z - x^* = x - \alpha \nabla f(x) - x^*.$$

This would appear to be hardly useful, since x^* is not known, but here comes a weird trick (typically, the hallmark of the elegant proofs): using $\nabla f(x^*) = 0$, one continues the equality as

$$z - x^* = x - \alpha \nabla f(x) - x^* + \alpha \nabla f(x^*) = (x - x^*) - \alpha(\nabla f(x) - \nabla f(x^*)).$$

The reason is that this allows to use the Mean Value Theorem [13, Th. 5.4.5], which informally says: for a differentiable function and any interval, there is at least a point inside the interval in which the derivative has the same slope as line drawn between the two endpoints of the interval (in the graph space). Applied to ∇f (which is continuous since f is L -smooth), this gives that there exists $w \in [x, x^*]$ such that $\nabla f(x) - \nabla f(x^*) = \nabla^2 f(w)(x - x^*)$. As a consequence,

$$z - x^* = (x - x^*) - \alpha \nabla^2 f(w)(x - x^*) = (I - \alpha \nabla^2 f(w))(x - x^*).$$

Passing to the norm, one then has $\|z - x^*\| \leq \|I - \alpha \nabla^2 f(w)\| \|x - x^*\|$ (recall that the norm of the matrix dictates how much at most a vector in the input space gets elongated in the output space, cf. §A.9). The consequence is immediate: *the value of α that minimizes $\|I - \alpha \nabla^2 f(w)\|$ is the best possible fixed stepsize*, i.e., the one leading to the fastest possible reduction of the distance between the current iterate and the optimal solution.

Working out the minimization requires some work, that we reproduce here in detail since it shows the usefulness of familiarising with the appropriate mathematical tools. From §A.9, one has

$$\|I - \alpha \nabla^2 f(w)\| = \max\{|1 - \alpha \lambda_1(\nabla^2 f(w))|, |1 - \alpha \lambda_n(\nabla^2 f(w))|\}.$$

Exercise 4.15. Prove the last statement. [solution]

An interesting observation is that *if $\lambda_n > 0$ then $\alpha = 1/\lambda_1$ “works”*, in the sense that $1 - \alpha \lambda_n = 0$ while $1 - \lambda_n/\lambda_1 < 1$: thus, the algorithm is guaranteed to have a linear convergence rate of at least $(\lambda_1 - \lambda_n)/\lambda_1$: however, this choice is not optimal. Finding the optimal value can be done in several ways: we just observe that if $1 - \alpha \lambda_n \geq 1 - \alpha \lambda_1 \geq 0$ then increasing α decreases the maximum, while if $0 \leq \alpha \lambda_n - 1 \leq \alpha \lambda_1 - 1$ then decreasing α decreases the maximum. As a consequence, the optimal α must be such that $1 - \alpha \lambda_n > 0$ and $1 - \alpha \lambda_1 < 0$; therefore,

$$r = \max\{-1 + \alpha \lambda_1, 1 - \alpha \lambda_n\}.$$

We have taken the shorthand of eliminating “ $\nabla f^2(w)$ ” from the notation, but this hides the fact that these values depend on w (and f) and are generally unknown. Yet, L -smoothness and τ -convexity imply $L \geq \lambda_1$ and $\tau \leq \lambda_n$, whence

$$r = \max\{-1 + \alpha L, 1 - \alpha \tau\}.$$

Exercise 4.16. Prove the last statement. [solution]

Now, it is plain to see that whenever one of the two terms of the maximum increases the other decreases, and vice-versa: this indicates (and it can be easily proven) that the minimum is attained when the two terms are equal, leading to the choice $\alpha = 2 / (L + \tau)$. This ultimately leads to the optimal convergence rate

$$r = (L - \tau) / (L + \tau) = (\kappa - 1) / (\kappa + 1) [< 1]$$

depending in the expected way from the worst-case conditioning of the Hessian $\kappa = L / \tau$.

Exercise 4.17. Check the last details. [solution]

As for the quadratic case, a “small” difference in the properties of f makes a large difference in the convergence properties of the approach. Tellingly, the convergence rate is worse than that of the exact line search (cf. 4.3), and understandably so: at each iteration only the maximum “safe” step is executed, as opposed to the best possible one. Thus, it can be expected that an exact (and, in fact, even a rather coarse like the Armijo-Wolfe one) Line Search will bring in more decrease at each iteration, and this actually shows in practice. Yet, of course, the result as to be tempered by the consideration of the difference between counting iterations and function evaluations, that are the same for the fixed stepsize but not so for any form of “serious” Line Search. However, even the best linear convergence rate achievable with the gradient method can be hopelessly slow when the Hessian is very badly conditioned, and even more so when directions of null curvature can be present. All in all, better algorithms are needed; as we have already seen in the quadratic case, they can indeed be built.

Before doing so we comment again (cf. §1.4.4) on the stopping criterion issue: while one would want to stop as soon as $A(x^i) \leq \delta$ (or, even better, $R(x^i) \leq \delta$) this is not achievable in practice because most often f_* is unknown. One would need a guaranteed lower bound $f \leq f_*$ that is “sufficiently tight” at least towards termination, but getting good guaranteed lower estimates of f_* is “hard” in general. What can be measured, and all algorithms (in this Chapter, but we will see that this is not possible in other setting) use, is $\|\nabla f(x^i)\|$, which is a convenient “proxy” of $A(x^i)$ in the sense that both are null at optimality (with all caveats that we have extensively discussed), so that one can assume that if one of the two is “small” then the other one also is. However, choosing ε in the stopping condition so as to obtain a prescribed δ is not at all obvious, and it is often left to hand-tuning. Some properties of f , and in particular convexity, would be useful to derive values with formal guarantees, but typically one should also have access to information that is not available in practice.

Exercise 4.18. Assume one knows $x^* \in \mathcal{B}(x^i, \delta)$ and f is convex, find the ε ensuring $A(x^i) \leq \delta$. [solution]

Exercise 4.19. Prove: if f is τ -convex, then $\|\nabla f(x^i)\| \leq \sqrt{2\tau\varepsilon}$ implies $A(x^i) \leq \varepsilon$. [solution]

A number of practical tricks can help. One is to use a “relative” stopping condition of the form $\|\nabla f(x^i)\| \leq \varepsilon \|\nabla f(x^0)\|$, whereby, say “ $\varepsilon = 1\text{e-}4$ ” means “reduce the norm of the gradient by four orders of magnitude from what it was at the start”. Alternatively, when f is a model of some “physical” system, $\|\nabla f\|$ may have a “physical” meaning—say, related to cost or robustness of elements in the system—that can be used to choose a proper ε . In this case, the use of other norms than the Euclidean one can be advised: for instance, the ∞ norm guarantees that each entry of ∇f is “small enough”. All this makes sense for applications where “optimality really matters”; say, f measures money (and lots of it), so that any measurable decrease of it really signifies a better solution. For reasons already repeatedly alluded to, this is however not really the case for the applications of primary interest for these lectures: although “good” solutions are useful, very tight accuracies (say, relative errors of $1\text{e-}6$, or even $1\text{e-}4$) are typically not needed. Which is why we will not comment further on the issue, save for saying that—as several of the developments in this Section—it is not specific of the gradient method but common to all (gradient-based) approaches in this Chapter.

4.2 Newton-type methods

Since the previous Section has thoroughly explored all reasonable alternatives for the choice of the stepsize, if one wants to improve the performances of the solution methods it is necessary to consider a different choice of the direction. The first thing to realise is that the previous development has actually provided us with ample results that can be readily generalised to different choices than $d^i = -\nabla f(x^i)$.

4.2.1 “Twisting” the gradient

Indeed, let us recoup the crucial arguments beyond the convergence analyses in the previous section:

1. $\varphi'_i(0) = \langle \nabla f(x^i), d^i \rangle = -\|\nabla f(x^i)\|^2$, i.e., “far from x^* the directional derivative along the chosen direction is very negative”;
2. by a proper choice of the stepsize α^i , “a non-vanishing fraction of the descent promised by $\varphi'_i(0)$ is obtained”.

These two simple arguments already allow to prove at least convergence along the lines of Theorem 4.2: if “disaster can be avoided” for the stepsize, e.g., via an “exact” Line Search or an inexact one (Armijo-Wolfe or Backtracking) under L -smoothness, then a non-vanishing fraction of $\varphi'_i(0)$ is obtained as the decrease at each iteration, implying that $\|\nabla f(x^i)\| \rightarrow 0$ (or $\{f(x^i)\} \rightarrow -\infty$, which is just as well). Of course, point 2. does not really depend on the chosen direction, and there clearly exist many other directions that ensure 1. holds, within some factor, besides the antigradient. To see this, consider the *parodied twisted gradient algorithm*: take d^i as $-\nabla f(x^i)$ arbitrarily rotated by $\pi/4$. Mathematically speaking, this can be obtained as $d^i = R^i(-\nabla f(x^i))$ for an appropriate rotation matrix R^i [80]. Without going into any detail (since the intent is explicitly parodistic), however chosen R^i (in infinitely many ways, if $n > 2$), then $\varphi'_i(0) = \langle \nabla f(x^i), d^i \rangle = -\|\nabla f(x^i)\|^2 \cos(\pi/4) = -\|\nabla f(x^i)\|^2 \sqrt{2}/2 \approx -\|\nabla f(x^i)\|^2 0.7071$: although the directional derivative is “less negative than before”, it still is negative whenever $\nabla f(x^i) \neq 0$. Re-thinking to the developments of the previous section one promptly realises that the convergence proofs (if not necessarily the efficiency ones) largely carry forward unchanged. Also, there is no specific benefit in the choice of an angle of $\pi/4$: any fixed angle would do, of course except $\pi/2$ since then $\cos(\pi/2) = 0$. In other words, there are a very vast choice of possible directions: the only issue is that they remain “significantly of descent”, i.e., that the angle between d^i and $\nabla f(x^i)$ does not become “too close to $\pi/2$ ” so that its cosine does not become “too small”.

One can then consider a general *descent algorithm*, i.e., Algorithm 4.1 in which d^i is an arbitrary *descent direction*, i.e., $\frac{\partial f}{\partial d^i}(x^i) = \langle d^i, \nabla f(x^i) \rangle < 0$, i.e., $\cos(\theta^i) < 0$ with θ^i being the angle between d^i and $\nabla f(x^i)$. The above intuition is brilliantly summarised by Zoutendijk’s Theorem [10, Th. 3.2]:

Theorem 4.4. Let f be an L -smooth function bounded below: applying the descent algorithm to f where the stepsize at each iteration satisfies (4.5) and (4.6), one has

$$\sum_{i=1}^{\infty} \cos^2(\theta^i) \|\nabla f(x^i)\|^2 < \infty \quad (4.8)$$

Theorem 4.4 immediately provides the convergence of the original gradient method: there, $\cos^2(\theta^i) = 1$, and therefore by (4.8) $\{\|\nabla f(x^i)\|\} \rightarrow 0$ (“fast”) otherwise the series must necessarily diverge. But this also works for any descent algorithm that chooses directions “bounded away from being orthogonal to $\nabla f(x^i)$ ”, i.e., so that $\cos(\theta^i)$ is bounded away from 0. And even this is not strictly necessary: $\{\cos(\theta^i)\} \rightarrow 0$ is allowed, providing that it happens “slowly enough” to force $\{\|\nabla f(x^i)\|\} \rightarrow 0$. This clearly opens a vast design space for algorithms to choose their descent direction while attaining guaranteed convergence, but of course this is not the fundamental point: it is not just convergence that one is seeking for, but rather efficiency. So, the issue becomes how to actually choose d^i in this vast design space. As mentioned at the beginning of the Chapter, $d^i = -\nabla f(x^i)$ is arguably the best choice if one is basing it on the indications of the first-order model: so, the natural step is to go beyond this, i.e., use the second-order one.

4.2.2 Newton’s method basics

Indeed, the idea behind (the multivariate version of) *Newton’s method* is very simple: construct the second-order model at the current iterate

$$Q^i(x) = Q_{x^i}(x) = \frac{1}{2}(x - x^i)^T \nabla^2 f(x^i)(x - x^i) + \langle \nabla f(x^i), x - x^i \rangle + f(x^i)$$

and find its minimum:

$$x^{i+1} \in \operatorname{argmin}\{Q^i(x) : x \in \mathbb{R}^n\}. \quad (4.9)$$

This is of course ill-posed if $\nabla^2 f(x^i) \not\succeq 0$, and potentially so if it is “only” $\succeq 0$, so to be on the safe side we will temporarily assume $\nabla^2 f(x^i) \succ 0$, which immediately implies that the minimum is available by simple algebraic operations.

It is worth remarking immediately that Newton’s method actually is an approach for solving *systems of nonlinear equations*. Setting aside all details, one has a vector-valued function $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$, and wants to solve $F(x) = 0$. Given some estimate x^i of the solution, the method constructs the *first-order model* $F^i(x) = JF(x^i)(x - x^i) + F(x^i)$, and chooses x^{i+1} as the solution of the corresponding linear system. In our setting

this is applied to the system of nonlinear equations $\nabla f(x) = 0$, i.e., $F = \nabla f$ and $JF = \nabla^2 f$. This will largely remain an interesting but side observation in this Chapter, but it will turn out to be useful later on.

To proceed, it is convenient to rewrite the model in terms of the *direction* $d^i = x - x^i$; doing so immediately leads to *Newton's direction*

$$d^i = -[\nabla^2 f(x^i)]^{-1} \nabla f(x^i). \quad (4.10)$$

Exercise 4.20. Formally derive (4.10). [solution]

Unlike the antigradient, Newton's direction in fact immediately identifies a point, $x = x^i + d^i$, that is of course the natural candidate for the next iterate x^{i+1} . As we have seen in §2.7, however, there is no guarantee that x is a “good” new iterate (although, as we shall see soon, it will actually be such “often”). This is why it is convenient to rather see d^i as a direction, so that one can perform an appropriate Line Search along it. The easy, but fundamental, observation is that d^i is of descent: $\langle \nabla f(x^i), d^i \rangle = -\nabla f(x^i)^T [\nabla^2 f(x^i)]^{-1} \nabla f(x^i) < 0$ since $[\nabla^2 f(x^i)]^{-1} \succ 0$. The general descent algorithm implemented with (4.10) and any reasonable Line Search, typically Armijo-Wolfe or Backtracking, is called *Newton's method* and it is clearly the multivariate generalization of the univariate one seen in §2.7. As announced there, the Line Search provides the required *globalisation* that—under appropriate assumptions—makes the method globally convergent; this is a consequence of Theorem 4.4.

Theorem 4.5. Newton's method applied to $f \in C^2$ that is L -smooth and τ -convex is globally convergent in the sense of Theorem 4.4.

Proof. The result is immediate from $\nabla^2 f(x^i)d^i = -\nabla f(x^i)$: this gives both

$$\begin{aligned} \langle \nabla f(x^i), d^i \rangle &= -(d^i)^T \nabla^2 f(x^i) d^i \leq -\tau \|d^i\|^2 \\ \|\nabla f(x^i)\| &= \|\nabla^2 f(x^i)d^i\| \leq \|\nabla^2 f(x^i)\| \|d^i\| \leq L \|d^i\|, \end{aligned}$$

whence $\cos(\theta^i) = \langle \nabla f(x^i), d^i \rangle / (\|\nabla f(x^i)\| \|d^i\|) \leq -\tau / L < 0$ and Theorem 4.4 applies. $\square$

Thus, τ -convexity and L -smoothness are the natural conditions for Newton's method to apply. We will discuss shortly how to remove the first, which is necessary in order to be able to apply the approach in the non-convex setting. Yet, endowed with a Line Search the algorithm is globally converging, albeit in the “weak” form that the theorem ensures. As in Chapter 2, if the algorithm is converging, then the convergence is extremely fast:

Theorem 4.6. Let $f \in C^3$, $\nabla f(x^*) = 0$, $\nabla^2 f(x^*) \succ 0$: then, there exists some $\delta > 0$ such that if $x^0 \in \mathcal{B}(x^*, \delta)$ then the “pure” Newton's that always chooses $\alpha^i = 1$ produces a sequence $(x^i) \rightarrow x^*$ and the convergence is quadratic.

The proof is omitted since it is basically just the multivariate generalization of that of Theorem 2.2. Apparently, however, the two results are in tension between them: the first one requires the Line Search, while the second one requires that it is not performed. The tension is only apparent, since the Line Search can exploit the fact that there is a “natural candidate” for the initial value of the stepsize: $\alpha = 1$, leading precisely to $x = x^i + d^i$. In fact, it can be shown that, “in the tail” of the convergence process, this is a “good choice” since it immediately satisfies (4.5) and (4.6). In other words, at some point Newton's method automatically stops doing any Line Search and requires just one function (and gradient, and Hessian) evaluation per iteration. This completely resolves the tension.

Theorem 4.7. Under the hypotheses of Theorem 4.5, if Newton's method produces a sequence $(x^i) \rightarrow x^*$, then there exists some index h such that $\bar{x}^i = x^i + d^i$ satisfies (4.5) for all $i \geq h$ provided that $m_1 < 1/2$.

Proof. We provide the fundamental ideas of the proof. Since the hypotheses of Theorem 4.5 apply then Theorem 4.4 also does and (4.8) holds. Since $\{x^i\} \rightarrow x^*$, then necessarily $\|\nabla f(x^i)\| \rightarrow 0$, which implies $\|d^i\| \rightarrow 0$ by L -smoothness ($\|\nabla^2 f(x^i)\| \leq L$). Note that in this setting τ -convexity is basically only the requirement that $\nabla^2 f(x^i) \succ 0$. Then, we exploit the fact that the first-order and second-order terms in the second-order model, when evaluated in d^i , have a common form, since

$$\langle \nabla f(x^i), d^i \rangle = -\nabla f(x^i)^T [\nabla^2 f(x^i)]^{-1} \nabla f(x^i).$$

Then,

$$\begin{aligned} f(x^i + d^i) &= \frac{1}{2} (d^i)^T [\nabla^2 f(x^i)] d^i + \langle \nabla f(x^i), d^i \rangle + f(x^i) + R(d^i) \\ &= \frac{1}{2} \nabla f(x^i)^T [\nabla^2 f(x^i)]^{-1} \nabla f(x^i) - \nabla f(x^i)^T [\nabla^2 f(x^i)]^{-1} \nabla f(x^i) + f(x^i) + R(d^i) \\ &= -\frac{1}{2} \nabla f(x^i)^T [\nabla^2 f(x^i)]^{-1} \nabla f(x^i) + f(x^i) + R(d^i) \\ &= \frac{1}{2} \langle \nabla f(x^i), d^i \rangle + f(x^i) + R(d^i). \end{aligned}$$

Clearly, $\{\varphi'_{x^i, d^i}(0) = \langle \nabla f(x^i), d^i \rangle\} \rightarrow 0$ as $\{d^i\} \rightarrow 0$, but we know from Taylor's theorem that $\{R(d^i)\} \rightarrow 0$ “faster” than that; hence, eventually the remainder R will be negligible, which means that eventually (4.5) with $m_1 < 1/2$ holds. $\square$

Exercise 4.21. Complete the sketch of the proof of Theorem 4.7. [solution]

Under τ -convexity and L -smoothness, Newton's method therefore has an extremely solid convergence theory. It starts with a “global phase” in which the Line Search is significant (α^i potentially varies), and ends with a quadratically convergent “pure Newton's phase” in which α^i is fixed to 1. Being quadratically convergent, the pure Newton's phase ends in $O(1)$ iterations (typically, a handful) in any practical setting. Under appropriate conditions, also the “global phase” can be proven to be “very fast”, basically $O(1)$: [2, (9.40)] proves that if $\nabla^2 f$ is M -smooth, then the global phase ends in $O(M^2 L^2 (f(x^0) - f_*) / \tau^5)$ iterations. The bound contains some constants that are quite hard to estimate and a fifth power, but in practice the “global phase” of Newton's method typically proves to be quite efficient.

Before moving on to making Newton's method more practical by removing the τ -convexity assumption we comment on another interesting way to look at the approach, along the lines of what is done in §1.5.1 for the Conjugate Gradient method. As in that case, we analyse a quadratic (strictly convex) function $f(x) = \frac{1}{2}x^T Qx + \langle q, x \rangle$: in this case, Newton's direction $d = -x - Q^{-1}q$ has the property that $\nabla f(x + d) = 0$, i.e., Newton's method ends in one iteration. This is of course unsurprising: f and Q_x coincide however x is chosen, and therefore minimizing the second-order model is the same as minimizing the function. However, we present a different way of explaining this property, again based on the square root of Q : $R = Q^{1/2}$ such that $Q = RR$, a nonsingular matrix since Q is. As in §1.5.1 we use this to define the bijective change of variable $z = Rx$, i.e., $x = R^{-1}z$; in this z -space, the function becomes $h(z) = f(R^{-1}z) = \frac{1}{2}z^T I z + \langle q, R^{-1}z \rangle$. Thus, “in z -space $\nabla^2 f(x^i)$ looks like I ”, which implies that even the “humble” gradient method (with exact Line Search) would be “fast”. Perhaps not surprisingly, in z -space Newton's method *is* the gradient method (with *unitary stepsize*): indeed, for $g = -\nabla h(z) = -z - R^{-1}q$ one gets $\nabla h(z + g) = 0$ however z is chosen.

Exercise 4.22. Complete the sketch of the proof of Theorem 4.7. [solution]

In fact, the antigradient in z -space is mapped into Newton's direction in x -space: $R^{-1}g = R^{-1}(-z - R^{-1}q) = -x - Q^{-1}q = d$. In words, R provides a “good” space dilation that, mapping Q into the identity matrix, renders the gradient method “fast”. Analogously to what was already commented in §1.5.3—albeit for different reasons—this is not so much interesting when using the “real” Q , but rather because it suggests that using “approximations” of Q may lead to methods that, while not terminating in strictly one iteration, can still be “fast”. This have of course to be expected here since in general ∇f is not constant; but “in the tail” of the convergence process, where iterates x^i are “very close together”, ∇f changes “little” so that $\nabla f(x^i)$ provides a good approximation to $\nabla f(x^{i+1})$ and—perhaps more importantly— $\nabla f(x^*)$. Yet, approximations they still are, and therefore it stands to reason that “slight” (or not-so-slight) changes in the matrix can still lead to fast convergence. This will be proven to be true, and it is the fundamental concept behind the extension of Newton's method beyond the (strictly) convex case discussed next.

4.2.3 Newton's method: the nonconvex case

The previous discussion immediately suggests a way to deal with the non-positive-definiteness of the Hessian, which is not to use it but rather an appropriate modification. This leads to defining a *Newton-type method* as one in which the direction is chosen by

$$d^i \leftarrow -[H^i]^{-1} \nabla f(x^i) \quad (4.11)$$

where H^i is a positive definite approximation to $\nabla^2 f(x^i)$. This of course opens a vast design space for choosing H^i that we will discuss thoroughly. The first observation, however, is that as soon as $\delta I \preceq H^i \preceq MI$ for any two constants $0 < \delta < M$, then the above theory immediately applies and, as in Theorem 4.7, the algorithm is readily proven to be globally convergent.

This suggests the following recipe for facing the issue of $\nabla^2 f \not\succeq 0$: choose a “small” $\delta > 0$, and then select H^i such that $H^i \succeq \delta I$ and H^i is “as similar as possible to $\nabla^2 f(x^i)$ ”. An obvious formal definition is

$$H^i = \operatorname{argmin}\{\|H - \nabla^2 f(x^i)\|_2 : H \succeq \delta I\}, \quad (4.12)$$

i.e., projecting $\nabla^2 f(x^i)$ on the set of symmetric positive definite matrices with minimum eigenvalue δ . It turns out that this optimization problem has a very simple solution:

$$H^i = \nabla^2 f(x^i) + \varepsilon^i I \quad \text{for} \quad \varepsilon^i = \max\{0, \delta - \lambda^n(\nabla^2 f(x^i))\}.$$

In plain words, if the minimum eigenvalue is smaller than δ , then all eigenvalues are increased by the minimum

amount required to have $\lambda_n = \delta$. This requires fixing an arbitrary value for δ , which is not completely trivial: in principle one would like δ to be “very small”, but on the other hand “any double $\leq 1\text{e-}16$ is 0”, and $1\text{e-}16$ is a very optimistic setting; in practice one would not go below $1\text{e-}12$ to avoid numerical problems. Furthermore, a “very small” δ is likely to bring algorithmic issues: roughly speaking, the axes of the level sets $S(Q_{x^i}, \cdot)$ of the second-order model are “very elongated” (since the length of the one corresponding to H_n is $1/\delta$), which would lead to a “large” d^i from (4.11) and therefore to “ x^{i+1} very far from x^i ”. This is intuitively “not good” since the second-order model is “local” and therefore its quality deteriorates as the distance increases. All this leads to having to appropriately choose a “smallish but not too small” δ , with values that could be something like $1\text{e-}4$, $1\text{e-}8$, or $1\text{e-}12$ and that might have to be experimentally adjusted.

It may be worth remarking that the previous formula is not the only possibility in view of the fact that there are other matrix norms than the Euclidean one: for instance, using the Frobenius norm (cf. §A.9) in (4.12) would lead to the optimal solution that can be obtained by computing the spectral decomposition $\nabla^2 f(x^i) = H\Lambda H^T$ and then setting $H^i = H\bar{\Lambda}H^T$ with $\bar{\lambda}^i = \max\{\lambda^i, \delta\}$; in plain words, rather than increasing all the eigenvalues of the amount required to make the minimum one $\geq \delta$, each one of them is increased by the amount that makes it individually so. One may argue that such a H^i would seem to be “more similar” to $\nabla^2 f(x^i)$ than the previous one, but this would amount at arguing that the Frobenius norm is “better” than the Euclidean one, and this could be done only on the grounds of a more sophisticated analysis that is out of place here. Furthermore, computing a full spectral decomposition is typically more costly than computing the minimum eigenvalue only. However, just computing (4.11) requires the solution of a symmetric linear system with a potentially dense matrix, which is $O(n^3)$. Some clever algorithmic tricks can be devised to lessen the total cost of the approach, comprised the Hessian modification. Without going in any details, using the Cholesky factorization $H^i = L^i(L^i)^T$ is a fast and stable way to solve (4.11) if $H^i \succ 0$; if not, during the decomposition some non-positive diagonal entry is revealed, that could be increased on-the-fly to ensure that the factor remain nonsingular, which would amount at some variant of the approaches outlined above. We will not delve deeply in any of these details, referring the interested reader, e.g., to [10, p. 52+], but use this short discussion to showcase that a vast design space gives rise to many choices and therefore of trade-offs to navigate.

The fundamental observation, however, is that all these Hessian modification techniques are “automatically self-destructing”: if the algorithm is converging to a bona-fide local minimum x^* such that $\nabla^2 f(x^*) \succeq \delta I$, then eventually no modifications will be performed and $H^i = \nabla^2 f(x^i)$, thus the strong quadratic convergence properties “in the tail” of the process will be retained. Once more, Newton’s method is “self-adjusting”. And once more, using approximations to $\nabla^2 f(x^i)$ can still lead to a very efficient approach provided that the fundamental property remains: when converging to some x^* , the approximations H^i must “nicely converge to $\nabla^2 f(x^*)$ ”. As we will see, there are clever ways to obtain that while lessening the $O(n^3)$ cost per iteration of the classical Newton’s method (and the need of computing $\nabla^2 f$ in the first place), which is untenable for very-large-scale optimization.

Before doing so, however, we briefly comment to another possible approach to the issue of non-positive definite Hessian. The issue with $\nabla^2 f(x^i) \not\succeq 0$ is that it breaks the correspondence between (4.9) and (4.10): since there are negative curvature directions for Q^i , (4.9) is unbounded below. However, f decreases along negative curvature directions, and decreasing f is actually what the whole process aims at: therefore, it would seem that pretending they do not exist just because they make the second-order unbounded below is not really a clever reaction. Of course something needs be done about it, but a possibility clearly reveals: while Q^i is unbounded below over $\mathbb{R}^n$, it does have a minimum over any compact set. Thus, one could rather consider

$$x^{i+1} \in \operatorname{argmin}\{Q^i(x) : x \in \mathcal{T}^i\} \quad (4.13)$$

where $\mathcal{T}^i$ is a compact *trust region* such that $x^i \in \mathcal{T}^i$. This makes a lot of sense: being a model, Q^i gets less and less accurate as the distance to x^i grows, so it is sensible to only “trust” it “close” to the current iterate. Of course, (4.13) is a *constrained* problem; even worse, it is $\mathcal{NP}$ -hard even for simple $\mathcal{T}^i$ such as $\mathcal{B}_1(x^i, r)$ or $\mathcal{B}_\infty(x^i, r)$ (cf. Example 1.3). However, it turns out that the problem is actually “easy” for $\mathcal{T}^i = \mathcal{B}_2(x^i, r)$: “round balls are simpler than kinky balls” [18]. Of course, also in Trust Region methods it is completely sensible to use any appropriate H^i approximating $\nabla^2 f(x^i)$, albeit not for the purposes of having it positive definite—since this is no longer required—but for those discussed right next: this leads to problems of the form

$$d^i \in \operatorname{argmin}\left\{\frac{1}{2}d^T H^i d + \langle \nabla f(x^i), d \rangle : \|d\|_2 \leq r^i\right\} \quad (4.14)$$

that can be efficiently solved, as very quickly alluded to next.

In all this, however, we have swept under the rug the real elephant in the room: how “large” should the trust region be? How should we choose the radius $r^i > 0$ in (4.14)? Intuitively, too small a radius will lead to unduly short steps and therefore slow convergence, but a too large one will lead to trusting the model way beyond the region where it actually is accurate at the risk of performing “bad” steps. Thus, some iterative adjustment of the radius could be needed. Yet, this should not be taken as a dramatic disadvantage, since an iterative

adjustment is present also in standard Newton-type methods: that of the stepsize α^i . Indeed, one could argue that there are basically two opposite ways to face the issue that the accuracy of any model (even a “good” one like the second-order one) will necessarily degrade when moving away from x^i :

- *Line Search*: first choose the direction $d^i \in \mathbb{R}^n$, then choose the *stepsize* $\alpha^i \in \mathbb{R}$ giving the next iterate $x^{i+1} = x^i + \alpha^i d^i$;
- *Trust Region*: first choose the *trust radius* $r^i \in \mathbb{R}$ of the trust region, then choose the actual direction $d^i \in \mathcal{B}_2(0, r^i)$ giving the next iterate $x^{i+1} = x^i + d^i$.

It is plain to see how the stepsize and the trust radius play a very similar role, with the main difference being whether they are selected before or after the direction. This does not mean that Line Search and Trust Region methods do not have significant difference in their behaviour and analysis, but is it true that many fundamental ideas and results are shared within the two approaches.

In these lectures we will not discuss (if not tangentially) Trust Region approaches, referring the interested reader, e.g., to [10, Chapter 4]. We will, however, terminate this section by offering a very quick glimpse to the fact that “Line Search and Trust Region are two sides of the same coin”. This is revealed by writing down the optimality conditions of (4.14). While the general theory of these will only be presented in Chapter ??, we beg the forgiveness of the reader for presenting them here without any justification [10, Theorem 4.1]: d^i is optimal for (4.14) if and only if there exists $\lambda^i \geq 0$ such that

$$[H^i + \lambda^i I]d^i = -\nabla f(x^i) \quad (4.15a)$$

$$H^i + \lambda^i I \succeq 0 \quad (4.15b)$$

$$\|d^i\| \leq r^i \quad (4.15c)$$

$$\lambda^i(r^i - \|d^i\|) = 0 \quad (4.15d)$$

That is, there must exist a $\lambda^i \geq 0$ playing the role of ε^i in the Hessian modification approach and making $H^i + \lambda^i I \succeq 0$: then, d^i is a Newton-type direction with the (potentially, doubly-)modified Hessian. Yet, rather than being arbitrarily selected a-priori, ε^i has to be chosen in such a way that the resulting direction remains in the ball. Remarkably, if $H^i \succ 0$ and the “natural” Newton-type direction $d^i = -[H^i]^{-1}\nabla f(x^i)$ of (4.11) satisfies (4.15c), then it clearly satisfies (4.15) with $\lambda^i = 0$. It is only if the “natural” stepsize brings outside of the ball that λ^i enters the play, and in this case (4.15d) guarantees that $r^i = \|d^i\|$, i.e., the direction is on the boundary of the ball. In fact, the typical approach to handle r^i does not change it if $\lambda^i > 0$, which implies $r^i > \|d^i\|$. While converging towards a bona-fide strict local minimum x^* , hopefully this will happen, since towards convergence the steps are naturally shorter and shorter: thus, the standard Newton-type step (with unitary stepsize) is made, and the convergence is fast (if H^i is a good approximation to $\nabla^2 f(x^i)$). Trust Region algorithms are “self-adjusting” similarly to Line Search ones. We will not discuss any more detail, in particular about how (4.14) is solved; the interested reader can consult, e.g., [10, §4.1]. The fundamental points here being that 1) Trust Region is a sensible alternative to Line Search, and that 2) in both cases matrix factorizations may be needed, which means that the cost per iteration is typically $O(n^3)$. This is unbearable for very-large-scale problems, and alternatives are needed.

4.2.4 Quasi-Newton methods

In order to make Newton-type methods practical for very-large-scale problems, the computational cost per iteration has to be reduced to at most $O(n^2)$, or possibly even below. Now, forming the Hessian is in general $O(n^2)$ already, so one fundamental steps of the approach is to *compute second-order information without actually computing the Hessian*. This is not unreasonable, since higher-order information can obviously be approximated starting from first-order one: $f'(x) \approx (f(x + \varepsilon) - f(x))/\varepsilon$ for any conveniently small ε . As we shall see, cleverer forms can be put in place. Before doing so, however, it is worth remarking that the approach makes a lot of sense because *the space of H^i that give “fast convergence” is large*: that is, one needs only a “rather coarse” approximation of the Hessian to actually obtain a more-than-linearly-convergent approach. This is shown, e.g., by [10, Th. 3.6], which proves that one obtains superlinear convergence if H^i “looks enough like $\nabla^2 f(x^i)$ ” just along d^i . A more formal statement is:

Theorem 4.8. For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $f \in C^2$, if the Newton-type algorithm using (4.11) with the stepsize α^i satisfying (4.6) (with $m_1 \leq 1/2$) produces a sequence $\{x^i\} \rightarrow x^*$ such that $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*) \succ 0$, and furthermore

$$\lim_{i \rightarrow \infty} \|(H^i - \nabla^2 f(x^i))d^i\| / \|d^i\| = 0, \quad (4.16)$$

then eventually the unitary stepsize $\alpha^i = 1$ satisfies (4.6), and if eventually the algorithm always accepts the unitary stepsize then the convergence is superlinear.

Condition (4.16) can be phrased by saying that the difference between H^i and $\nabla^2 f(x^i)$, “when measured along d^i ”, goes to 0 faster than d^i does. This means that H^i must “approximate reasonably well” $\nabla^2 f(x^i)$ only on the subspace spanned by d^i (which is “an incredibly small sliver of the whole $\mathbb{R}^n$ ”), while it potentially matters little how good the approximation is on the rest of the space. The condition, as it is stated, does not help in selecting H^i because d^i depends on H^i ; however, it confirms that very coarse approximations to the Hessian are sufficient to significantly increase the convergence speed (“in the tail”) with respect to what is possible for first-order methods. A comprehensive approach for computing “good” H^i has been developed, and it will now be described in some detail.

The general approach is that at each iteration one is considering the approximate second-order model

$$m^i(x) = \langle \nabla f(x^i), x - x^i \rangle + \frac{1}{2}(x - x^i)^T H^i (x - x^i)$$

and performing the standard iteration $x^{i+1} = x^i + \alpha^i d^i$ with d^i from (4.11). As already mentioned this implies that $H^i \succ 0$, for this is necessary to ensure that d^i is of descent, plus of course α^i will have to be properly selected, which typically implies (4.5) and (4.6)—but this will be specifically discussed. At the subsequent iteration one therefore will have the new approximate second-order model

$$m^{i+1}(x) = \langle \nabla f(x^{i+1}), x - x^{i+1} \rangle + \frac{1}{2}(x - x^{i+1})^T H^{i+1} (x - x^{i+1})$$

and the whole point is how to produce the new approximate Hessian H^{i+1} out of the previous one H^i . Fundamentally, this has to be done exploiting the newly produced first-order information $\nabla f(x^{i+1})$ at the next iterate: again, the idea is to approximate the Hessian only using gradient information. This has been approached in an axiomatic way, i.e., by first establishing the properties that one would like H^{i+1} to have, and then working backwards from them to actually compute it. These are three:

1. $H^{i+1} \succ 0$, i.e., the new model is strongly convex;
2. $\nabla m^{i+1}(x^i) = \nabla f(x^i)$, i.e., the new model agrees with the “known” first-order information not only in the current iterate x^{i+1} (which is always given), but also in the previous one x^i ;
3. $\|H^{i+1} - H^i\|$ is “small”, i.e., the new model is “not too different” from the previous one.

The need for i. has already been commented upon. As for iii., the idea is that when $(x^i) \rightarrow x^*$, $(H^i) \rightarrow H^*$ where H^* has to be a “good approximation” to $\nabla^2 f(x^*)$: thus, successive approximations should be “close”, in particular when successive iterates are “close”. Furthermore, conditions i. and ii. “leave a lot of degrees of freedom” in the choice of H^{i+1} , and the closeness requirement allows to select—as we shall see presently—particular specimens among the many possibilities. Before doing so, however, one should always ensure that there actually is something to choose, which is not necessarily given.

Indeed, condition ii. can alternatively be rewritten as

$$H^{i+1}(x^{i+1} - x^i) = \nabla f(x^{i+1}) - \nabla f(x^i),$$

known as “secant equation”.

Exercise 4.23. Verify the statement. [solution]

The interested reader is advised to justify exactly why the name makes sense, but the gist is that “ H^{i+1} explains the difference between $f(x^{i+1})$ and $\nabla f(x^i)$ in terms of the difference between x^{i+1} and x^i ”. Indeed, for the rest of the treatment it is expedient to introduce the shorthand notation $s^i = x^{i+1} - x^i$ and $y^i = \nabla f(x^{i+1}) - \nabla f(x^i)$, which reduces the secant equation to the linear system

$$H^{i+1}s^i = y^i. \quad (4.17)$$

Now, the issue is that condition i. may “clash” with (4.17), because the latter gives $\langle s^i, y^i \rangle = (s^i)^T H^{i+1} s^i$, and by positive definiteness this yields

$$\langle s^i, y^i \rangle > 0 \quad \text{most often written as} \quad \rho^i = 1 / \langle s^i, y^i \rangle > 0, \quad (4.18)$$

known as “curvature condition”: if (4.18) does *not* hold, then there is no way in which all three axioms can be satisfied. Now, at the beginning of the iteration $\nabla f(x^i)$ is fixed, and $s^i = x^{i+1} - x^i = \alpha^i d^i$ it is *almost* so, since the direction d^i is fixed and only the stepsize α^i can be chosen: once this is, and x^{i+1} is revealed, $\nabla f(x^{i+1})$ appears and there is in principle no guarantee that (4.18) is satisfied. In other words, the stepsize has to be properly chosen for the approach to work. Fortunately, it is easy to show that *Wolfe’s condition* (4.6) *implies the curvature condition* (4.18). Indeed, $\varphi'(\alpha^i) = \langle \nabla f(x^{i+1}), d^i \rangle \geq m_2 \varphi'(0) = m_2 \langle \nabla f(x^i), d^i \rangle$ implies $\langle \nabla f(x^{i+1}) - \nabla f(x^i), d^i \rangle = \langle y^i, d^i \rangle \geq (m_2 - 1) \varphi'(0) > 0$ as $m_2 < 1$, but $s^i = \alpha^i d^i$ and $\alpha^i > 0$ then imply $\langle y^i, s^i \rangle = \alpha^i \langle y^i, d^i \rangle > 0$. Thus, if an Armijo-Wolfe Line Search is employed, (4.18) will always be satisfied.

This having been settled, it is natural to choose

$$H^{i+1} = \operatorname{argmin}(\|H - H^i\|_F : (4.17), H \succeq 0).$$

As in the previous case, the judicious choice of the problem allows it to be solved in closed form, which is the *Davidon-Fletcher-Powell formula* [10, p. 138]

$$H^{i+1} = (I - \rho^i y^i (s^i)^T) H^i (I - \rho^i s^i (y^i)^T) + \rho^i y^i (y^i)^T. \quad (4.19)$$

Thus, H^{i+1} can be obtained by the previous H^i by means of *two rank-one corrections*, which means at a cost of $O(n^2)$. This alone would not suffice in making the computation of (4.11) cheap, since it would still be $O(n^3)$, but one can assume that such a computation has already been performed at the previous iteration. That is, one actually has previously computed $B^i = (H^i)^{-1}$, knowing which (4.11) is $O(n^2)$; while in general factorizations are preferred to explicitly computing inverses, this is not the case here. The reason is the *Sherman-Morrison-Woodbury formula* [82]

$$[A + ab^T]^{-1} = A^{-1} - A^{-1}ab^T A^{-1} / (1 - b^T A^{-1}a)$$

that works for any nonsingular matrix $A \in \mathbb{R}^{n \times n}$ and pair of n -vectors a and b : in plain words, the inverse of a rank-one correction is a rank one correction of the inverse. Applying this to (4.19) yields

$$B^{i+1} = B^i + \rho^i s^i (s^i)^T - B^i y^i (y^i)^T B^i / (y^i)^T B^i y^i. \quad (4.20)$$

That is, if B^i is known, then B^{i+1} can be built with two rank-one corrections, which is $O(n^2)$, and therefore (4.11) can be computed with the same complexity: all in all, this is the dominant cost per iteration of the *DFP algorithm*. An open issue is how to select B^0 , but we will discuss this a bit later. All in all, this approach allows to progressively build, and maintain, approximations of the Hessian only using gradients: one could say that $\nabla^2 f$ is “learnt” out of samples of ∇f . These approximations can be shown to be “good” in terms of the fact that the algorithm is—within conditions—superlinearly convergent. Yet, the DFP algorithm based on (4.19)–(4.20) is just one of a larger family of algorithms, that we will briefly describe and for which we will point to the main convergence and efficiency results.

The fundamental observation is that the DFP algorithm takes a somewhat winding road to achieve its results: what is really needed is B^i , approximating $[\nabla^2 f(x^i)]^{-1}$, but what is approximated is rather H^i , approximating $\nabla^2 f(x^i)$. It turns out, however, that one can perform the analysis directly in terms of the inverse: indeed, (4.17) is clearly equivalent to $s^i = B^{i+1} y^i$, and therefore that one could rather consider

$$B^{i+1} = \operatorname{argmin}(\|B - B^i\|_F : s^i = B^{i+1} y^i, B \succeq 0)$$

to define the method. It turns out, this does not yield the same result, although understandably it obtains somewhat symmetric formulæ in which, basically, B is exchanged with H and s is exchanged with y [10, p. 139]:

$$\begin{aligned} B^{i+1} &= (I - \rho^i s^i (y^i)^T) B^i (I - \rho^i y^i (s^i)^T) + \rho^i s^i (s^i)^T \\ &= B^i + \rho^i [(1 + \rho^i (y^i)^T B^i y^i) s^i (s^i)^T - (B^i y^i (s^i)^T + s^i (y^i)^T B^i)] \end{aligned} \quad (4.21)$$

$$H^{i+1} = H^i + \rho^i y^i (y^i)^T - H^i s^i (s^i)^T H^i / (s^i)^T H^i s^i \quad (4.22)$$

The (4.21)–(4.22) is called the *Broyden-Fletcher-Goldfarb-Shanno* update, and the corresponding approach is called the *BFGS algorithm*. It turns out that in practice BFGS is numerically more stable than DFP. But this is still not the most general approach: in fact, it turns out that one can as well use any “convex combination” of the two, i.e.,

$$H^{i+1} = \beta^i H_{DFP}^{i+1} + (1 - \beta^i) H_{BFGS}^{i+1}$$

whose computation is still $O(n^2)$. This is called the *Broyden family* of updates [10, § 6.3], and it can be shown to satisfy both the secant equation (in whatever form) and $H^{i+1} \succeq 0$ however chosen $\beta^i \in [0, 1]$, with DFP and BFGS being of course the extreme cases. Yet, with due care the approach can be made to work even if $\beta^i \notin [0, 1]$ (e.g., [10, § 6.2]), further extending the reach of the approach. This gives a flexible family of updates, with good compromise between iteration cost and convergence speed in practice; general convergence theory is available (in particular, but not exclusively, for strictly convex f), but it is nontrivial and we just refer the interested reader to [10, § 6.4]. We only add a possibly interesting observation [10, Theorem 6.4]: applied to a strictly convex quadratic function, with exact stepsize and $B^0 = I$, any algorithm of the Broyden family exactly reproduces the iterates of the conjugate gradient method (cf. §1.5). In particular, after n iterations one has $B^n = Q^{-1}$. This at least confirms that, at least in this special case, the approach is “good at learning the inverse of the Hessian”.

The only remaining issue to discuss with a modicum of detail is the choice of B^0 . An obvious idea would be to construct H^0 by finite differences and then invert it, but this would risk making the very first iteration so excessively costly as to outweigh all the subsequent ones. The preferred choice is therefore to choose it as a “simple” invertible matrix; I would immediately come to mind, leading to $d^1 = -\nabla f(x^1)$ (the first step is a gradient step), with $B^0 = \delta I$ for some appropriately chosen δ being the obvious next step. Formulæ for

automatically choosing δ have been devised that seem to work well in many practical settings [10, (6.20)].

Exercise 4.24. Discuss how to compute B^0 by finite differences and how much does it cost. [solution]

4.2.5 Limited-memory Quasi-Newton methods

While the $O(n^2)$ iteration complexity of the quasi-Newton methods of the previous section is a vast improvement over the $O(n^3)$ of by-the-book Newton's method, it is still way too high for very-large-scale problems where n is in excess of, say, $1e+4$; and current applications can scale way higher than that. It is therefore necessary to consider versions of the approach that have iteration complexity of the form $O(kn)$, where k is a user-selectable fixed parameter.

This is, in fact, pretty natural for quasi-Newton methods; we will discuss it for BFGS since it is the “simple” version that is usually considered the most efficient and robust, but the idea clearly generalise. The point is that the approach gathers $O(n)$ (memory-wise) new information per iteration under the form of the new iterate x^{i+1} and its gradient $\nabla f(x^{i+1})$; more to the point, the corresponding $s^i = x^{i+1} - x^i$ and $y^i = \nabla f(x^{i+1}) - \nabla f(x^i)$. Then, the idea is simply to only keep, and therefore use, the last k (provided, of course, that $i \geq k$) of these. The point is that (4.21) can be written as

$$B^{i+1} = (V^i)^T B^i V^i + \rho^i s^i (s^i)^T \quad \text{with} \quad V^i = I - \rho^i y^i (s^i)^T.$$

It is therefore possible to “unfold” the formula substituting B^i in it with the information coming from the previous iteration, i.e., $B^i = (V^{i-1})^T B^{i-1} V^{i-1} + \rho^{i-1} s^{i-1} (s^{i-1})^T$. Doing this k times produces a formula like

$$\begin{aligned} B^{i+1} &= (V^{i-k} V^{i-k+1} \dots V^i)^T B^{i-k} (V^{i-k} V^{i-k+1} \dots V^i) + \\ &= \rho^{i-k} (V^{i-k+1} \dots V^i)^T s^{i-k} (s^{i-k})^T (V^{i-k+1} \dots V^i) + \\ &= \rho^{i-k+1} (V^{i-k+2} \dots V^i)^T s^{i-k+1} (s^{i-k+1})^T (V^{i-k+2} \dots V^i) + \\ &= \dots + \rho^i s^i (s^i)^T. \end{aligned}$$

Despite looking imposing, the formula is quite simple to use due to the fact that the only use of B^{i+1} is to multiply it by $\nabla f(x^{i+1})$ to obtain d^{i+1} : this can be applied to the right of the formula, showing that the result can be obtained just by means of scalar products between $\nabla f(x^{i+1})$ and the $O(kn)$ stored information. With a bit of optimization this can be translated into a simple iterative scheme that has $O(kn)$ cost both in memory and computations [10, Algorithm 7.4]. For the first k iterations new information is simply added, after which the oldest information is discarded to make space for the newly computed one.

A somewhat unexpected benefit of this scheme is that one is then given to chose B^{i-k} arbitrarily anew at each iteration i , since its choice is no longer “frozen” in B^{i-1} as per the regular approach. Of course, given the stress on low complexity one would use the simple formula $B^{i-k} = \gamma^i I$, but nonetheless it is possible to freely choose γ^i ; a formula that has been reported to work well in practice (with some theoretical grounds) is $\gamma^i = \langle s^i, y^{i-1} \rangle / \|y^{i-1}\|^2$.

This approach has been shown to yield remarkable trade-offs between cost per iteration and convergence speed. Of course, it can be expected that convergence will somehow deteriorate as k is reduced, with per-iteration cost correspondingly diminishing (linearly). Considering the inherent cost for the computation of function and gradient values, that is typically at least $O(n)$ but with possibly significant constants hidden in the $O(\cdot)$ notation, values of k in the low 100s or even 10s can lead to iteration complexities that are close in practice to that of a pure gradient method, while exhibiting significantly better convergence properties. Even values of k as small as 5, 10 or 20 have been reported to work surprisingly well even for very large n , confirming again the fact that “only a very coarse approximation of the Hessian” is actually needed to achieve significantly better results than what the first-order model alone permits. As usual, the devil is in the details and properly implementing large-scale quasi-Newton variants requires due care; the interested reader is referred, e.g., to [10, Chapter 7].

4.3 Deflected gradient methods

The “twisted gradient” approaches of the previous sections all have the form (4.11): by involving a $n \times n$ matrix-vector product these are almost by necessity bound to have an iteration complexity of at least $O(n^2)$, unless H^i is chosen with a very special structure such as in §4.2.5. In the extremely-large-scale case, even a very few multiple of n could be hardly be tenable, often due to memory problems even before than computing costs (although in practice both, because larger memory pools can always be accessed going to slower tiers of the memory hierarchy, but potentially with orders-of-magnitude effects on latency and efficiency). If one wants to modify the antigradient direction $d^i = -\nabla f(x^i)$ in a “cheap” way, arguably the less costly approach is summing it some vector: $d^i = -\nabla f(x^i) + v^i$ for some $v^i \in \mathbb{R}^n$. We call this *deflecting* the (anti)gradient, and

this clearly has a cost of $O(n)$ —with no hidden large constants—*provided v^i itself can be cheaply computed*. Obviously, the best case would be that where there is actually no need to compute anything, in the sense that v^i is naturally available as a by-product of the previous computations: this immediately leads to the idea of taking $v^i = \beta^i d^{i-1}$, i.e., the direction at previous iteration scaled by some appropriate $\beta^i \in R$.

This makes sense at least in view of the following observation: taking $v^0 = 0$ (i.e., doing the first iteration that the gradient method does), one ends up taking a direction of the form $d^i = -[\sum_{h=1}^i \gamma^h \nabla f(x^h)]$ for some scalars γ^i . That is, d^i is (the opposite of) a weighted sum of all past gradient information, although each computed at a different iterate x^h . In other words, d^{i-1} contains the “memory” of all previous computations, some “history” of the algorithms that is at least conceptually analogous (and, as already remarked, in some cases in fact equivalent) to the approximations of the Hessians / its inverse H^i / B^i in quasi-Newton methods.

There is, however, a significant difference between “twisting” and “deflecting” in terms of keeping the (up to now) crucial property that d^i be a descent direction. Indeed, for twisting this “just” requires $H^i \succeq 0$, which also fits well with the fact that, at least towards termination, one is expecting to converge towards a bona-fide local minimum and therefore H^i has to approximate a positive definite Hessian. Things are much less clear for deflecting, as choosing β^i that ensure that $\varphi'_{x^i, d^i}(0) = \langle \nabla f(x^i), d^i \rangle < 0$ is not obvious. This of course looks doable since the property has to hold for appropriately small β^i , but “if $\beta^i \rightarrow 0$ then $d^i \rightarrow -\nabla f(x^i)$ ”, and therefore there is little hope that the approach can be significantly more efficient than the gradient method. Better ways are needed to properly choose β^i , but these clearly exist since we have seen one already.

Exercise 4.25. Prove that “the limit as $\beta \rightarrow 0$ of deflecting is the gradient method”. [solution]

4.3.1 Nonlinear Conjugate gradient methods

Indeed, the idea of the *Nonlinear Conjugate Gradient* (NCG) algorithm is to copy its counterpart for quadratic functions, Algorithm 1.3, as closely as possible. Of course, the exact Line Search have to be replaced with an approximate one, typically based on Armijo-Wolfe conditions, yielding

Algorithm 4.3. Conjugate Gradient for Nonlinear Function

```

procedure  $x = NCG(f, x, \varepsilon)$ 
   $\nabla f^- = 0$ ;
  while ( $\|\nabla f(x)\| > \varepsilon$ ) do
    if ( $\nabla f^- = 0$ ) then  $d \leftarrow -\nabla f(x)$ ;
    else {  $\beta = \langle \text{right deflection value} \rangle$ ;  $d \leftarrow -\nabla f(x) + \beta d^-$ ; }
     $\alpha \leftarrow \text{LS}(f, x, d)$ ;  $x \leftarrow x + \alpha d$ ;  $d^- \leftarrow d$ ;  $\nabla f^- \leftarrow \nabla f(x)$ ;

```

Then, one has to come up with a replacement of Fletcher-Reeves formula (1.22) that does not use the Hessian Q . However, the way had already been shown by (1.23), where the deflection coefficient β^i was computed only using the current and previous gradient information. Somewhat surprisingly, it turns out that this can be done in no less than four different ways:

1. $\beta_{FR}^i = \|\nabla f(x^i)\|^2 / \|\nabla f(x^{i-1})\|^2$ (Fletcher-Reeves);
2. $\beta_{PR}^i = \langle \nabla f(x^i) - \nabla f(x^{i-1}), \nabla f(x^i) \rangle / \|\nabla f(x^{i-1})\|^2$ (Polak-Ribière);
3. $\beta_{HS}^i = \langle \nabla f(x^i) - \nabla f(x^{i-1}), \nabla f(x^i) \rangle / \langle \nabla f(x^i) - \nabla f(x^{i-1}), d^{i-1} \rangle$ (Hestenes-Stiefel);
4. $\beta_{DY}^i = \|\nabla f(x^i)\|^2 / \langle \nabla f(x^i) - \nabla f(x^{i-1}), d^{i-1} \rangle$ (Dai-Yuan).

When f is convex quadratic and the exact Line Search is performed, all the formulæ above replicate exactly Algorithm 1.3, while they all give in general different results for a general function f . That is, while in the quadratic case the algorithm has the nice convergence properties described (if succinctly) in §1.5.2, the picture for the general case is way more varied. In fact, rather than behaving as minor variants of the same approach, the analysis depicts more a family of four related ones—named F-R, P-R, H-S and D-Y for short—that clearly share several traits in common but that also behave differently. Short of providing any detail, for which the interested reader is referred to [10, §5.2], we quickly point out a few relevant facts:

- F-R requires $m_1 < m_2 < 1/2$ in order to ensure convergence under (4.5) and (4.7);
- conditions (4.5) and (4.7) are not sufficient to ensure that P-R will generate a descent d^i at every iteration, unless the formula is updated as $\beta_{PR+}^i = \max\{\beta_{PR}^i, 0\}$, i.e., negative values are zeroed out;
- the similar modification $\beta_{HS+}^i = \max\{\beta_{HS}^i, 0\}$ turns out to be useful to the H-S variant;

Zeroing-out β^i at some iteration is a general concept, called *restart*. In plain words, “from time to time” one can completely reset the “memory” of the algorithm and restart with a “plain” antigradient step. Restarts clearly make proving convergence easy, but in a “tricky” way [10, p. 127]: basically, one can make the deflection “asymptotically vanish”, so that the approach resembles (for good and bad) a standard gradient approach and inherits all its properties. Despite this, carefully executed restarts have been shown to be useful in practice, especially for some variants among which F-R. This is due to a mechanism whereby the “unrestarted” algorithm may “jam” under some conditions. Roughly speaking, it may happen that the algorithm chancs over a “bad” direction: $\cos(\theta^i) \approx 0$, i.e., $\nabla f(x^i)$ is approximately orthogonal to d^i . If this happens, one can expect a “short” step α^i to be performed, which means $x^{i+1} \approx x^i$, which means $\nabla f(x^{i+1}) \approx \nabla f(x^i)$: thus, $\beta_{FR}^i \approx 1$, and $d^{i+1} \approx d^i$. But since d^i was a “bad” direction previously, d^{i+1} is likely to be such again: “one bad step leads to many bad steps”. Restarting breaks the cycle, allowing the algorithm to “get unstuck”. In theoretical results this calls for rule like performing a restart after n steps, but this is not really a good approach when n is large; in practice, restarts are performed every k —a user-selected parameter—iterations, or automatically by rules like β_{PR+}^i and β_{HS+}^i . It can even be shown that the “unrestarted” P-R, not using β_{PR+}^i , does not converge for some properly selected function [10, Th. 5.8].

In terms of efficiency, results are also varied. An interesting result that is worth mentioning is *n-step quadratic* convergence: under appropriate conditions, $\|x^{i+n} - x^*\| \leq r \|x^i - x^*\|^2$ [10, (5.51)]. A way to state these result is that “ n NCG iterations behave roughly like a Newton iteration”, and after some analysis this makes sense. Indeed, “close to x^* , the function and its second-order model Q_{x^*} behave very similarly”. Since n steps of the conjugate gradient approach (with exact Line Search) suffice to find the exact minimum of a quadratic function, it stands to reason that they can be roughly equivalent to one Newton’s step. Albeit interesting, this result is not particularly significant in practice especially when n is large, and anyway is not really representative of the actual performances of well-designed variants. As already mentioned, the NCG has interesting relationships with quasi-Newton methods [10, §7.2], and “hybrid” versions between the two approaches can be built that can behave quite well in practice, especially the P-R and D-Y forms; however, the results really depend a lot on the specific function to be minimised and on the fine details of the algorithms. All in all, NCG is a powerful and interesting approach, but it is not easy to manage in practice due to the need of choosing between several different variants and tuning their algorithmic parameters. In practice, users may be happy to trade some convergence speed—especially if very high accuracy is not needed, as in the applications of interest for these lectures—for simplicity and robustness. This is why NCG approaches are not widely used in this context, while the simpler ones outlined below are.

4.3.2 Heavy Ball gradient methods

A different version of the deflection approach is the *Heavy Ball* (HB) (gradient) method. The intuition behind the approach is that the fundamental culprit of the inefficiency of the standard gradient method is the “zig-zagging” behaviour whereby at each iteration the next direction is orthogonal to the previous one (cf., e.g., Figure 1.9). In order to avoid this and obtain a “smoother” convergence path the idea is to add some “resistance to change” in the trajectory: that is, the new direction should be “not too different” from the previous one. This is somewhat reminiscent of the fact that an “heavy” object that is progressing along a line will not change abruptly direction if subject to a force, but rather proportionally to the ratio between the force and the mass. In other words, an object has a momentum dependent on its mass that has to be overcome for the direction to change. This inspired the iterative scheme

$$x^{i+1} \leftarrow x^i - \alpha^i \nabla f(x^i) + \beta^i (x^i - x^{i-1}) \quad (4.23)$$

whereby the standard antigradient is modified with the “momentum term” containing the previous displacement $x^i - x^{i-1}$ multiplied by a suitable momentum coefficient $\beta^i > 0$. Obviously, the larger β^i the more similar $x^{i+1} - x^i$ will be to $x^i - x^{i-1}$ —all other things being equal—and therefore the more “stable” the trajectory will be. This has indeed the possibility of improving the performances.

It has to be remarked, however, that this is different from the deflection of the NCG in that the stepsize α^i is applied only to the antigradient part and then the momentum term is applied to the result, as opposed to computing $d^i = -\nabla f(x^i) + \beta^i (x^i - x^{i-1})$ and then applying the stepsize to the whole of d^i . Of course one could argue that the result would be the same for $d^i = -\nabla f(x^i) + (\beta^i / \alpha^i)(x^i - x^{i-1})$, but the point is that α^i is *not* computed as being, say, the “best possible step” along such d^i : in fact, there is no guarantee that such d^i is a direction of descent, and therefore that $f(x^{i+1}) < f(x^i)$ will happen. This is the interesting trait of the HB approach: unlike all those seen so far, it is *not* a descent algorithm, and $f(x^{i+1}) > f(x^i)$ may happen. This means that its analysis has to be based on an entirely different approach (although, as we will see shortly, one whose basic ideas have already been seen). Since the analysis is nontrivial, this section will spend an unusually large effort in presenting it to some detail. Before doing so, it is therefore sensible to show that

the effort is worth: what will be shown is that, for a L -smooth and τ -convex function, with appropriate choices of α^i and β^i the algorithm can achieve a(n almost, with the exact form detailed later) linear convergence rate of the form

$$\|x^{i+1} - x^*\| \leq r \|x^i - x^*\| \quad \text{for } r = (\sqrt{\kappa} - 1) / (\sqrt{\kappa} + 1) \quad (4.24)$$

and $\kappa = L / \tau$. This is significant both theoretically and practically. Indeed, (4.24) can be shown to be the *optimal* rate of convergence for deflection-type methods, i.e., any algorithms where the direction d^i lies in the subspace spanned by the gradients to all previous iterations [3, Th. 3.15]. Furthermore, such a convergence rate is significantly better than that (1.19) of the gradient method, despite HB being hardly more complex: for, say, $\kappa = L / \tau = 1000$, (4.24) yields $r \approx 0.938$ while (1.19) gives $r \approx 0.996$. While the two ratios may seem not too different, one would be severely underestimating the compounded effect of exponentials: while $0.996^{100} = 0.6698$, $0.938^{100} = 0.0016$. In plain words, 100 iterations of the gradient method would reduce the initial error by about one third, while the same number of iterations of the HB would divide it by about 1000. Again, such a striking difference can actually show in practice, confirming the interest for the HB approach (and therefore its analysis).

We will describe the results for constant stepsize and momentum term, i.e., $\alpha^i = \alpha$ and $\beta^i = \beta$ for all iterations i . Extending the ideas seen in §4.1.3, one defines the HB approach by means of a two-terms recurrence:

$$\begin{bmatrix} x^{i+1} - x^* \\ x^i - x^* \end{bmatrix} = \begin{bmatrix} x^i + \beta(x^i - x^{i-1}) - \alpha(\nabla f(x^i) - \nabla f(x^*)) - x^* \\ x^i - x^* \end{bmatrix}$$

exploiting again the fact that $\nabla f(x^*) = 0$. One can then, again, apply the Mean Value Theorem [13, Th. 5.4.5] to ∇f , obtaining that for some $w^i \in [x^*, x^i]$ one has $\nabla f(x^i) - \nabla f(x^*) = \nabla^2 f(w^i)(x^i - x^*)$. Whence

$$\begin{aligned} \begin{bmatrix} x^{i+1} - x^* \\ x^i - x^* \end{bmatrix} &= \begin{bmatrix} (x^i - x^*) - \alpha \nabla^2 f(w^i)(x^i - x^*) + \beta(x^i - x^{i-1}) \\ x^i - x^* \end{bmatrix} \\ &= \begin{bmatrix} [I - \alpha \nabla^2 f(w^i)](x^i - x^*) + \beta(x^i - x^{i-1}) + \beta x^* - \beta x^* \\ x^i - x^* \end{bmatrix} \\ &= \begin{bmatrix} [I - \alpha \nabla^2 f(w^i) + \beta I](x^i - x^*) - \beta(x^{i-1} - x^*) \\ x^i - x^* \end{bmatrix} \\ &= \begin{bmatrix} (1 + \beta)I - \alpha \nabla^2 f(w^i) & -\beta I \\ I & 0 \end{bmatrix} \begin{bmatrix} x^i - x^* \\ x^{i-1} - x^* \end{bmatrix} \end{aligned}$$

(note the weird trick of adding $\beta x^* - \beta x^* = 0$ in the second step). Now, writing $D^i = \nabla^2 f(w^i)$ for notational simplicity, one would like to find α and β such that

$$\|C^i\| = \left\| \begin{bmatrix} (1 + \beta)I - \alpha D^i & -\beta I \\ I & 0 \end{bmatrix} \right\| \leq r < 1,$$

then one would have $\|x^{i+1} - x^*, x^i - x^*\| \leq r^i \|x^1 - x^*, x^0 - x^*\|$, which would prove that $\{x^i\} \rightarrow x^*$ exponentially fast. Unfortunately, what can be proven is different, i.e., that an appropriate choice of α and β makes the *spectral radius* of C^i

$$\rho(C^i) = \max_j (|\lambda_j(C^i)|),$$

i.e., the largest in modulus of its eigenvalues, smaller than 1. The point here is that C^i is *not* symmetric, and therefore the spectral radius and the (Euclidean) norm are not one and the same as we are accustomed to (cf. §A.9); also, note that the eigenvalues can be *complex* numbers, so the modulus here is not the ordinary absolute value. Trying to keep the level of complication at the bare possible minimum, we just mention the important facts that, for any $A \in \mathbb{R}^{n \times n}$, $\rho(A) \leq \|A\|$ (the spectral radius is always smaller than the norm, but strict inequality can hold as it is easy to verify), while *Gelfand's formula* gives [70]

$$\rho(A) = \lim_{k \rightarrow \infty} \|A^k\|^{1/k};$$

in plain words, the spectral radius can be approximated arbitrarily well by raising the matrix to a large power k , taking the norm and then taking the k -th root of the result. These two properties will allow us to obtain bounds on $\rho(C^i)$ and then ultimately use them to prove (almost) linear convergence. This, however, does require a number of nontrivial steps.

The first is to reduce the computation of $\rho(C^i)$ to that of the spectral radius of many smaller matrices; that is,

$$\rho(C^i) = \max\{\rho(C_j) : j = 1, \dots, n\} \quad \text{with} \quad C_j = \begin{bmatrix} 1 + \beta - \alpha \lambda_j(D^i) & -\beta \\ 1 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

That is, one can focus on the n smaller (2×2) matrices built out of the n eigenvalues of D^i , compute their spectral radius, and then take the maximum.

Exercise 4.26. Prove the result above (warning, nontrivial). [solution]

Now, the technical—and really tedious—part of the result consists in proving that if

$$\beta = \max\{|1 - \sqrt{\alpha\tau}|, |1 - \sqrt{\alpha^i L}|\}^2$$

then

$$\rho(C^i) \leq \sqrt{\beta} = \max\{|1 - \sqrt{\alpha\tau}|, |1 - \sqrt{\alpha L}|\}.$$

Exercise 4.27. Prove the result above (warning, extremely nontrivial). [solution]

It is then just the matter of finding the value of α that minimizes the right-hand side. It is not difficult to argue that this is $\alpha = 4/(\sqrt{L} + \sqrt{\tau})^2$. It can be checked that as $\tau \rightarrow 0$ (very elongated level sets) $\alpha \rightarrow 4/L$, i.e., “significantly longer” than the “optimal” stepsize $1/L$ for the gradient method, while if $\tau = L$ (perfectly circular level sets) then $\alpha = 1/L$: the step is “longer the more elongated are the level sets”, i.e., it grows as L/τ does. Anyway, plugging in the value one obtains

$$r = \sqrt{\beta} = (\sqrt{L} - \sqrt{\tau})/(\sqrt{L} + \sqrt{\tau}) = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1) < 1,$$

i.e., (4.24).

Exercise 4.28. Prove the last steps. [solution]

In fact, the last “i.e.” is premature: it would be true if the bound were on $\|C^i\|$ rather than on $\rho(C^i)$, but this is not the case. Yet, the result is almost (but not quite) true. To see this we restrict to the case of a quadratic f , i.e., where $\nabla^2 f$ is constant; thus, $D^i = D$ at all iterations, and $C^i = C$. Then, by recursion

$$\left\| \begin{bmatrix} x^{k+1} - x^* \\ x^k - x^* \end{bmatrix} \right\| \leq \|C^k\| \left\| \begin{bmatrix} x^1 - x^* \\ x^0 - x^* \end{bmatrix} \right\|$$

where, for the avoidance of doubt, C^k now denotes the k -th power of the fixed matrix C . We can now invoke Gelfand’s formula: by the definition of limit, however chosen $\varepsilon > 0$ there exists some h such that

$$\rho(C) - \varepsilon \leq \|C^i\|^{1/i} \leq \rho(C) + \varepsilon$$

for every $i \geq h$. As a consequence

$$\|C^i\| \leq (\rho(C) + \varepsilon)^i$$

for all $i \geq h$: if $r^h = \rho(C) + \varepsilon < 1$, then after iteration h the process starts to converge with rate at least r^h . In fact, the rate gets closer and closer to $\rho(C)$ as the iterates progress. This is called a *quasi-linear* convergence with rate $\rho(C)$: eventually, the algorithm converges with a rate arbitrarily close to $\rho(C)$. Interestingly, the result indicates that in the first iterations the algorithm may *not* converge (not even in distance from x^* , even less so in objective value), and this actually shows in practice: at the beginning, $f(x^i)$ may worsen, but eventually the algorithm “stabilises” and convergence kicks in. At the cost of further complications, clearly unwarranted in our context, the result can be proven for a general L -smooth and τ -convex function.

We illustrate the analysis with some results of the two-dimensional strictly convex quadratic function

$$f(x_1, x_2) = \frac{1}{2}[x_1, x_2]^T \begin{bmatrix} 101 & -99 \\ 99 & 101 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + [10, 5]^T \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad (4.25)$$

whose eigenvalues of the Hessian are $\lambda_1 = 200$ and $\lambda_2 = 2$; thus, the Hessian is moderately ill-conditioned with $\kappa = 100$. The contour lines of the function are plotted in Figure 4.3(left) together with the convergence trajectories of two algorithms: HB, ran with its optimal parameters $\alpha = 4/(\sqrt{200} + \sqrt{2})^2 \approx 0.01652892562$ and $\beta = ((\sqrt{200} - \sqrt{2})/(\sqrt{200} + \sqrt{2}))^2 = 0.6694214876$, and the standard gradient method with optimal fixed stepsize $\alpha = 2/(200 + 2) = 0.00990099$, both started from $x = [0, 2]$. The corresponding convergence plots (relative gap in logarithmic scale vs. iterations) are plotted in Figure 4.3(centre). The algorithms are stopped when the norm of the gradient becomes less than or equal to $1e-5$. HB takes 113 iterations to stop, while the gradient method takes 859. The convergence plot shows that HB is not monotone: for about the first 30 iterations the objective value is worse than $f(x^0)$. However, by dint of making longer steps ($0.0165 > 0.0099$) and of the deflection, HB makes much faster progress towards the minimum, as testified by the much steeper convergence plot that, as the theory predicts, becomes almost linear as the algorithm progresses.

It has to be remarked, however, that the performances of the algorithm are extremely dependent on the right choice of α and β . This is shown in Figure 4.3(left) that plots the convergence graphs for the optimal β and three ever so slightly different values of α : the optimal one 0.01652892562 , 0.0165 and 0.0166 . Just underestimating the optimal value by 0.005% makes the algorithm significantly less stable, with more than one regimes where it is non monotone, although the total number of iterations does not increase (and in fact it slightly decreases).

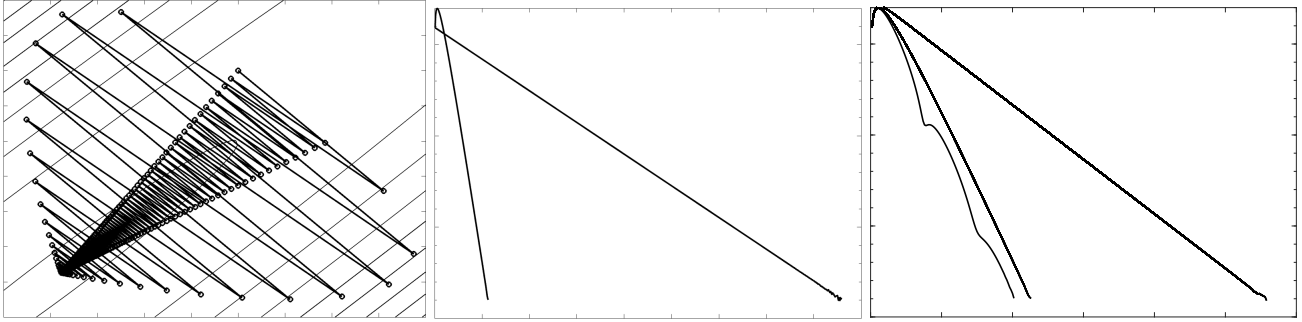


Figure 4.3: Comparing convergence of the Heavy Ball and gradient algorithms.

Overestimating α by a mere 0.04% worsens the convergence significantly, with the algorithm requiring 279 iterations—more than twice as much—to reach the same accuracy. Even more striking, however, are the results with $\alpha = 0.0167$, barely 1% off the optimal value: the algorithm *diverges*. Although not too much must be read from a small example, it is true that a proper setting of α and β is crucial for obtaining good performances, and in fact convergence.

The above analysis clearly hinges on τ -convexity, and therefore convexity. However, many applications of interest for these lectures require minimizing nonconvex functions. As it can be expected, HB generally works well in this case also, provided that “right” values of α and β are experimentally found. Using iteration-dependent values α^i (and, possibly but less frequently, β^i) may make the search easier, typically by gradually diminishing α^i , but this process also has parameters (how gradually ...) that have to be tuned. Convergence theory for the nonconvex case is also available, although even more mathematically involved: for instance, [25, p. 168] proves that—under conditions— β can be arbitrarily fixed $[0, 1)$ provided that $\alpha \in (0, 2(1 - \beta)/L)$. This implies that $\alpha \rightarrow 0$ as $\beta \rightarrow 1$, and possibly indicates that even “small” β give $\alpha \approx 2/L$ which is “better” than the $1/L$ of the gradient method, thus that even a modicum of deflection may improve performances. All in all, the theory basically supports the fact that HB can be expected to work in the nonconvex case, and the practice typically confirms it, provided that α and β are chosen properly.

A final remark concerns the L -smooth, convex but not τ -convex case: there, HB can be proven to have a $O(1/i)$ convergence rate [22], i.e., not better than the gradient method. Although in practice the convergence is better with properly chosen α and β , this raises the question if approaches with better worst-case convergence for this class of functions can be devised. The answer is affirmative for the class of approaches briefly described below.

4.3.3 Accelerated Gradient

The family of *Accelerated Gradient* (AG) algorithms has been devised to achieve the best possible complexity for L -smooth but not necessarily τ -convex functions, that can be proven to be $O(LD^2/\sqrt{\varepsilon})$ [3, Th. 3.14]. To achieve this they are based on a rather sophisticated analysis that will be very lightly alluded to later. The result is a simple approach that resemble, but is somewhat different from, HB. One of the several possible implementations is:

Algorithm 4.4. Accelerated Gradient Algorithm

```

procedure  $y = ACCG(f, x, \varepsilon)$ 
   $x_- \leftarrow x; \gamma \leftarrow 1;$ 
  do { // warning: black magic ahead
     $\gamma_- \leftarrow \gamma; \gamma \leftarrow (\sqrt{4\gamma_-^2 + \gamma_-^4} - \gamma_-^2)/2; \beta \leftarrow \gamma(1/\gamma_- - 1);$ 
     $y \leftarrow x + \beta(x - x_-); g \leftarrow \nabla f(y); x_- \leftarrow x; x \leftarrow y - (1/L)g;$ 
  } while ( $\|g\| > \varepsilon$ );

```

The relevant differences with HB are:

- the stepsize is the fixed $\alpha = 1/L$;
- conversely, the deflection coefficient β changes over time with a very specific formula;
- unlike HB, the algorithm first moves the current iterate to an intermediate point y only using the previous direction information, and then performs a gradient step from y .

The approach can be proven to have optimal $O(LD^2 / \sqrt{\varepsilon})$ complexity [3, Th. 3.19] without τ -convexity, and to match the “optimal” (in the sense already commented upon) linear convergence with $r = (\sqrt{\kappa} - 1) / (\sqrt{\kappa} + 1)$ if f is also τ -convex [3, Th. 3.18]. Like HB the approach is not monotone, but it can be made so at the cost of a minor change that requires *two* function computations per iteration; monotonicity is usually not considered worth the extra cost, which is why we won’t comment further on this option. Interestingly, the algorithm is almost parameter-free, just requiring knowledge of the Lipschitz constant L . Arguably, this is not known (which of course is true for all other approaches that depend on it to set the optimal algorithmic parameter), but there are ways to try to get around the problem both from the theoretical and practical viewpoint. We only comment to one idea, which is that the Backtracking Line Search, if started with a “large enough” initial step, could be used because it is guaranteed to be able to find a stepsize that is at least any fixed fraction of the desired one, e.g., $\alpha^i \geq 1/2L$; cf. §4.1.2. In fact, extending the complexity theory to an iteration-dependent stepsize is possible, provided that the stepsize is monotone nonincreasing, i.e., $\alpha^i \leq \alpha^{i-1}$; yet, this is easy to obtain by just starting the Backtracking Line Search at iteration i from α^{i-1} . Thus, the algorithm is inherently implementable, and it does have less algorithmic parameters to tune than HB.

A detailed analysis of the (family of) AG algorithm(s) is clearly beyond the scope of these lectures, the interested being referred, e.g., to [3, §3.7] and the references therein. However, we provide a very informal, bird’s eye view of some of the more interesting ideas. “Behind the scenes”, the AG approach is based on constructing a there is a *sequence* $\{\psi^i\}$ of *models* of f , starting from $\psi^0(x) = f(x^1) + L\|x - x^1\|_2$; remarkably, this is an *upper model* of f , i.e., $\psi^0(x) \geq f(x)$. Then, at each iteration the model includes new information by making use of the *lower model* corresponding to the newly obtained first-order information:

$$\psi^i(x) = (1 - \gamma_{i-1})\psi^{i-1}(x) + \gamma_{i-1}(f(y^i) + \nabla f(y^i)(x - y^i)).$$

By the careful choice of γ_i , the model incrementally gets a better and better upper estimator, with a property like $\psi^i(x) \leq (1 - \sigma^i)f(x) + \sigma^i\psi^0(x)$ for some $\sigma^i = O(1/i^2)$. In turn, x^i is carefully constructed so that $f(x^i) \leq \min\{\psi^i(x) : x \in \mathbb{R}^n\}$, which ultimately implies that $\|x^i - x^*\| \leq \sigma^i(f(x^i) - f_*)$; the right-hand converges to 0 “fast” ($O(1/i^2)$) as σ^i works. All choices are carefully tailored to make everything work, and the approach is deeply rooted in the fact that f is convex. The main reason for this informal presentation is that, although not apparent in its final incarnation, the algorithm is heavily *model-based* approach, where the model is based on mixing lower and upper ones. These ideas will resurface in the next chapter.

In practice, the AG algorithm has been reported to be generally efficient, but its performances are not always the best ones. Intuitively, the approach is carefully crafted to attain a *given* convergence speed, so it gets what it is constructed for, but its rigidity does not allow to fully exploit opportunities for better performances. It is therefore a good choice for all situations in which consistency of results and robustness are more relevant than raw speed. An example is shown in Figure 4.4(left), where the convergence plots of HB with optimal parameters (cf. Figure 4.3) is compared with that of Algorithm 4.4 with the optimal stepsize $\alpha = 1/L = 1/200 = 0.005$. The AG approach has a much less regular convergence and is slower, requiring 760 iterations to reach the stopping criterion. However, AG is somewhat more stable than the HB: Figure 4.4(right) compares the convergence with $\alpha = 0.005$ and $\alpha = 0.006$: despite being a good 20% larger, the second step is still convergent, and in fact it is even better, taking only 579. This likely has to do with the fact that the β^i parameter is (slowly) decreasing with the iterations in AG, while it is constant in HB. Yet, robustness only goes so far: $\alpha = 0.007$ causes AG to diverge. Again, not too much should be read from a single small-dimensional example, but a point that is generally valid is that better worst-case performances do not always translate into better practical performances.

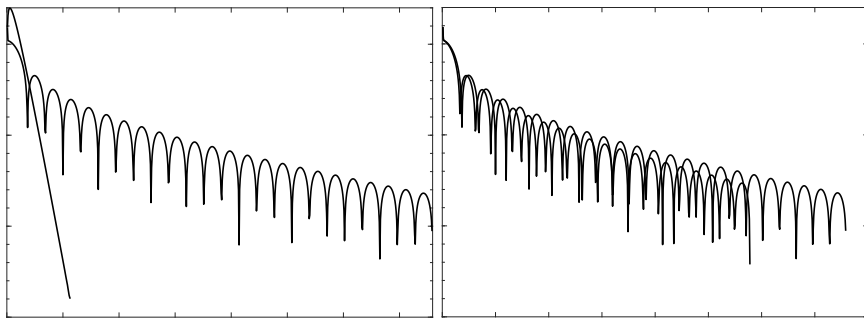


Figure 4.4: Comparing convergence of the Heavy Ball and Accelerated Gradient.

4.4 Wrap up

In this Chapter we have explored a number of variants of algorithms for (locally) minimizing smooth functions. The natural approach is that of descent methods: as soon as $\nabla f(x^*) \neq 0$, a descent direction can be found. There are plenty of ways to “extract enough of the descent predicted by the gradient”, from an exact line search when possible, to more and more inexact one, up to “no search at all”: even a fixed stepsize works (under mild conditions on f). Hence, there are plenty of ways to construct convergent algorithms. However, there is also more than that: convergence speed of “naïve” first-order algorithms is acceptable only if $\nabla^2 f$ is “reasonably well-conditioned”, otherwise it can go from quite bad to downright horrible. As we already mentioned several times, this may not be too much of an issue for the applications of interest for these lecture notes: very accurate solutions, even in terms of local optimality, are hardly a major goal. However, as the size of the problems grow the cost of each single iteration can become very substantial, hence faster convergence to a “good” solution is still beneficial.

For improving convergence rates, one should look no further than the second-order model: by “warping” the space [73] in such a way that $\nabla^2 f$ becomes well-conditioned, extremely fast convergence can be had locally, and a good behaviour can be expected in the “global phase” as well. Importantly, very coarse approximations to $\nabla^2 f$ can be used to excellent effect, keeping the cost per iteration in check for mid-to-large-size problems. Crucially, one can exploit second-order information without ever computing the Hessian once. First-and-a-half-order methods of the quasi-Newton family provide a vast design area whose trade-offs can be fruitfully explored. A lot of details need be considered, numerical aspects are nontrivial, but efficient algorithms can be constructed.

Pushing the boundaries on how little second-order information one can use leads to deflected approaches, starting from the Nonlinear Conjugate Gradient with its many links with quasi-Newton methods. However, at least as interestingly, methods exist that do not fit the general descent approach: it is possible to accept steps that worsen the objective function value, provided that the process can somehow be still driven towards convergence. Heavy Ball and Accelerated Gradient provide compelling examples of this kind of approach. However, the analysis of non-monotone methods is typically more complex than that of monotone ones.

Yet, that one can still reach convergence without ensuring descent at each single iteration is crucial in view of the fact that all approaches in this Chapter still rely on one fundamental assumption: *first-order information, and possibly second-order one, has to exist, be accessible and be continuous*. As the next Chapter will show, this is not given: in many practically relevant cases, the first-order information we have freely relied upon so far is not really accessible, either because it is too costly to obtain, or because it does not exist in the form we have assumed so far. This will require a new set of ideas to be developed, although a number of the principles underlying the algorithms seen upon now will still be important and applicable in the new setting.

4.5 Solutions

- **Solution to Exercise 4.1:** For $f(x) = x^2/2$, $f'(x) = x$. Thus, the algorithm generates $x^{i+1} \leftarrow x^i - \alpha x^i = x^i(1 - \alpha)$. Thus, $f(x^{i+1}) = f(x^i(1 - \alpha)) = (1 - \alpha)^2 f(x^i)$, whence $f(x^i) = r^i f(x^0)$ with $r = (1 - \alpha)^2$. For $\alpha > 2$, $r > 1$ and therefore $\{f^i = f(x^i)\} \rightarrow +\infty$ (exponentially fast), unless $x^i = 0$.
- **Solution to Exercise 4.2:** Take $\alpha^i = 1/2i^2 \leq 1/2$, $f(x) = x^2/2 - x$, $f'(x) = x - 1 \implies x^{i+1} \leftarrow x^i - \alpha^i(x^i - 1) = x^i(1 - \alpha^i) + \alpha^i \leq x^i + \alpha^i$. Take $x^1 = 0$ to get $x^{i+1} \leq \sum_{k=1}^i \alpha^k$; thus, $x^i \leq \sum_{k=1}^\infty \alpha^k = \pi^2/12 \approx 0.8225 < 1 = x^*$ for all i , i.e., the algorithm stalls (long) before reaching the optimal solution
- **Solution to Exercise 4.3:** In a word: no. If $f(x)$ is a “black box”, i.e., one can only evaluate it (and the gradient) but has no clue about how this is done, it’s impossible to declare that $f_* = -\infty$. Indeed, even for $n = 1$, f may decrease “for a very long time” but then abruptly start increasing again, and there is no way of knowing whether or not this will eventually happen. Thus, proving unboundedness is harder than proving (local) optimality, for which $\nabla f(x) = 0$ (approximately) suffices, unless one has more control on the function’s properties such as knowing its exact algebraic form (and even this may not be enough)
- **Solution to Exercise 4.4:** $\varphi'(\alpha) = \lim_{t \rightarrow 0} (\varphi(\alpha + t) - \varphi(\alpha)) / t = \lim_{t \rightarrow 0} (f(x + (\alpha + t)d) - f(x + \alpha d)) / t = \lim_{t \rightarrow 0} (f([x + \alpha d] + td) - f(x + \alpha d)) / t = \frac{\partial f}{\partial d}(x + \alpha d) = \langle \nabla f(x + \alpha d), d \rangle$ (definition of directional derivative).
- **Solution to Exercise 4.5:** $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $Jf(x) = \nabla f(x) : \mathbb{R}^n \rightarrow \mathbb{R}^n$, $g(t) = x + td : \mathbb{R} \rightarrow \mathbb{R}^n$, $Jg(t) = d : \mathbb{R} \rightarrow \mathbb{R}^n$, $h(t) = f(x + td) = f(g(t)) : \mathbb{R} \rightarrow \mathbb{R}$, $Jh(t) = h'(t) = Jf(g(t)) \cdot Jg(t) = \langle \nabla f(x + td), d \rangle$.
- **Solution to Exercise 4.6:** By definition of limit, $\forall \delta > 0 \exists h$ s.t. $\|\nabla f(x^h)\| \leq \varepsilon + \delta$; just use some

$\bar{\varepsilon} < \varepsilon$ as close as you want (anyway, numerical accuracy is limited)

- **Solution to Exercise 4.7:** The question hardly has practical sense, since ε need be (a lot) larger than the machine precision anyway: using `double` (machine precision $\approx 1\text{e-16}$), any $\varepsilon \ll 1\text{e-12}$ is likely to be impractical. Yet, infinite-precision computation is in principle possible (albeit slow), although one cannot expect to get a solution with 0 accuracy in finite time and have to be content to finitely achieving any arbitrary accuracy. For that, it would not be right to just set $\varepsilon = 0$, as then even the first LS may never terminate. The obvious solution is to run the algorithm infinitely many times, at the h -th call using some fixed $\varepsilon^h > 0$, but having $\varepsilon^h \rightarrow 0$ as $h \rightarrow \infty$. Of course, one then have to use the last iteration of the h -th call as the starting point of the $h+1$ -th call, which is all the information one needs to carry forward (unlike other approaches we'll see, the gradient method has "no memory" beyond the current iterate x^i).
- **Solution to Exercise 4.8:** Consider the convex quadratic function $\varphi(\alpha) = a\alpha(\alpha - 2)/2$, with roots $\alpha = 0$ and $\alpha = 2$. It is well-known that the unique global minimum is in the middle of the roots, i.e., for $\alpha = 1$. The derivative is $\varphi'(\alpha) = a(\alpha - 1)$, and therefore $\varphi'(0) = -a$. Armijo condition requires $\varphi(\alpha) \leq \varphi(0) + m_1\varphi'(0)\alpha$, i.e., $a\alpha(\alpha - 2)/2 \leq 0 - m_1a\alpha$. This gives $\alpha \leq 2(1 - m_1)$. Thus, if $m_1 > 1/2$, only points with $\alpha < 1$ satisfy the Armijo condition, and therefore the unique global minimum $\alpha = 1$ is excluded.
- **Solution to Exercise 4.9:** Since $d'(0) > 0$ and $d'(\alpha) = 0$ never happens for $\alpha > 0$, $d'(\alpha) > 0$ for all $\alpha > 0$; in fact, since there exists $\bar{\alpha} > 0$ s.t. $d'(\bar{\alpha}) < 0$, there must also be some $\alpha \in (0, \bar{\alpha})$ such that $d'(\alpha) = 0$ by the Intermediate Value Theorem [13, Th. 2.2.10] (recall that $d'(\cdot) \in C^0$). Hence $d(\cdot)$ is increasing for all $\alpha \geq 0$: since $d(0) = 0$, for all $\alpha \geq 0$ $d(\alpha) \geq 0 \equiv l(\alpha) \geq \varphi(\alpha)$. Since $l(\alpha) \rightarrow -\infty$ as $\alpha \rightarrow \infty$, the same must happen to $\varphi(\alpha)$.
- **Solution to Exercise 4.10:** Again Intermediate Value Theorem, and $d(\cdot) \in C^0$: if there were some $\alpha < \bar{\alpha}$ s.t. $d(\bar{\alpha}) < 0$, then there should be a further $\alpha' \in (0, \alpha)$ s.t. $d(\alpha') = 0$, contradicting the assumption that $\bar{\alpha}$ is the smallest.
- **Solution to Exercise 4.11:** Consider the function $\varphi(\alpha) = \sin(\pi(\alpha + 1))$, so that $\varphi'(\alpha) = \pi \cos(\pi(\alpha + 1))$. Thus, $\varphi'(0) = -1$, and φ has a local minimum in $\alpha = 1/2$, where $\varphi(1/2) = -1$ and $\varphi'(1/2) = 0$. However, there is a second local minimum at $\alpha = 5/2$, again with $\varphi(5/2) = -1$. The attraction basin of the second local minimum starts at $\alpha = 3/2$, where the intervening local maximum is and $\varphi'(3/2) = 0$, and lasts until $\alpha = 7/2$, where the next local maximum is. Thus, if $\bar{\alpha}$ were chosen as $13/4$, one would have $\varphi'(13/4) > 0$. But then, the intermediate point would be $\alpha = 13/8 > 3/2$, i.e., belong to the attraction basin of the second local minima: in fact, $\varphi'(13/8) < 0$, so that at the next iteration the interval would be $[13/8, 13/4]$. However, it is easy to see that, by taking $m_1 = 1/2$, only a (large) part of the attraction basin of the first local minimum satisfies (4.5). Indeed, the only positive root of $d(\alpha) = (\alpha) - \varphi(\alpha) = \sin(\pi(\alpha + 1)) - \alpha/2$ (cf. Theorem 4.3) is $\alpha \approx 1.20603512$, while $13/8 = 1.625$: thus, there is no point in the new interval that satisfies both (4.5) and (4.6).
- **Solution to Exercise 4.12:** $|\varphi'(\alpha) - \varphi'(\beta)| = |\langle \nabla f(x + \alpha d), d \rangle - \langle \nabla f(x + \beta d), d \rangle| = |\langle \nabla f(x + \alpha d) - \nabla f(x + \beta d), d \rangle| \leq \|\nabla f(x + \alpha d) - \nabla f(x + \beta d)\| \|d\| \leq (L\text{-smoothness}) \leq [L\|(x + \alpha d) - (x + \beta d)\|] \|d\| = [L\|d\|^2] |\alpha - \beta|$.
- **Solution to Exercise 4.18:** Convexity implies $f_* = f(x^*) \geq f(x^i) + \langle \nabla f(x^i), x^* - x^i \rangle$, i.e., $\langle \nabla f(x^i), x^* - x^i \rangle \geq f(x^i) - f_* \geq 0$, hence $\|\nabla f(x^i)\| \delta \geq \|\nabla f(x^i)\| \|x^* - x^i\| \geq |\langle \nabla f(x^i), x^* - x^i \rangle| \geq a^i$, which finally gives $\|\nabla f(x^i)\| \leq \varepsilon / \delta \implies a^i \leq \varepsilon$ (looks familiar?)
- **Solution to Exercise 4.19:** $g^i = \nabla f(x^i)$, $f_* = f(x^*) \geq f(x^i) + \langle g^i, x^* - x^i \rangle + \frac{\tau}{2} \|x^* - x^i\|^2 \equiv -h(x^*) = -\langle g^i, x^* - x^i \rangle - \frac{\tau}{2} \|x^* - x^i\|^2 \geq f(x^i) - f_* = a^i$. Since we don't know x^* , we need to overestimate the LHS, i.e., to compute $\max\{-h(x)\} = -\min\{h(x)\}$. As usual, putting $\nabla h(\bar{x}) = 0$ gives $g^i + \tau(\bar{x} - x^i) = 0 \equiv \bar{x} - x^i = -g^i / \tau$, whence $-h(\bar{x}) = \|g^i\|^2 / (2\tau)$. All in all, $\|g^i\| \leq \sqrt{2\tau\varepsilon} \implies \varepsilon \geq \|g^i\|^2 / (2\tau) \geq a^i$
- **Solution to Exercise 4.13:** Seen already: take $f(x) = -x$, so that $f'(x) = -1$, and $\alpha^i = 1/i$: $x^{i+1} - x^i = \alpha^i = 1/i \rightarrow 0$ as $i \rightarrow \infty$, but $x^i \rightarrow \infty$.
- **Solution to Exercise 4.14:** We need to use the fundamental theorem of calculus and monotonicity of integration (cf. §A.12). That is, "the derivative is the inverse of the integral": for $f: \mathbb{R} \rightarrow \mathbb{R}$, $f \in C^0$, F is an antiderivative of f if $F'(x) = f(x)$ for all $x \in \mathbb{R}$. Then, $\int_{\underline{x}}^{\bar{x}} f(x) dx = F(\bar{x}) - F(\underline{x})$. Monotonicity of integration implies that if $f(x) \geq g(x)$ for all x , then $\int_{\underline{x}}^{\bar{x}} f(x) dx \geq \int_{\underline{x}}^{\bar{x}} g(x) dx$. Now we apply this to $\varphi'(\alpha) \leq \|\nabla f(x)\|^2 (L\alpha - 1)$, which gives via monotonicity $\varphi(\alpha) - \varphi(0) = \int_0^\alpha \varphi'(\beta) d\beta \leq \|\nabla f(x)\|^2 \int_0^\alpha (L\beta - 1) d\beta = \|\nabla f(x)\|^2 (A(\alpha) - A(0))$, where $A(\alpha) = L\alpha^2/2 - \alpha$ is the antiderivative of $L\alpha - 1$. All in all, $\varphi(\alpha) - \varphi(0) \leq \|\nabla f(x)\|^2 (L\alpha^2/2 - \alpha)$.

- **Solution to Exercise 4.15:** Recall in general $\|Q\| = \max\{|\lambda_1|, |\lambda_n|\}$ (only simplifies if $\succeq 0 / \preceq 0$), and (λ, v) eep of $Q \implies (1 + c\lambda, v)$ eep of $I + cQ$
- **Solution to Exercise 4.16:** Assuming $\alpha > 0$ is chosen so that $-1 + \alpha L \geq 0$ and $1 - \alpha\tau$ one has $L \geq \lambda_1 > 0 \implies -1 + \alpha\lambda_1 \leq -1 + \alpha L$
 $0 < \tau \leq \lambda_n \implies 1 - \alpha\lambda_n \leq 1 - \alpha\tau$
- **Solution to Exercise 4.17:** $-1 + 2L / (L + \tau) = (-L - \tau + 2L) / (L + \tau) = (L - \tau) / (L + \tau)$
 $1 - 2\tau / (L + \tau) = (L + \tau - 2\tau) / (L + \tau) = (L - \tau) / (L + \tau)$. Then, divide numerator and denominator by τ and use the definition of κ .
- **Solution to Exercise ??:** $g^i = \nabla f(x^i)$, $f_* = \min\{f(x)\} \geq f(x^i) + \min\{q^i(x)\}$, with $q^i(x) = \langle g^i, x - x^i \rangle + \frac{\tau}{2} \|x - x^i\|^2 \implies \nabla q^i(\bar{x}) = 0 \equiv g^i + \tau(\bar{x} - x^i) = 0 \equiv \bar{x} - x^i = -g^i / \tau \implies g^i(\bar{x}) = \langle g^i, -g^i / \tau \rangle + \frac{\tau}{2} \|-g^i / \tau\|^2 = -\|g^i\|^2 / 2\tau = -\|\nabla f(x^i)\|^2 / 2\tau$
- **Solution to Exercise ??:** $\varphi'_{x,d}(\alpha) = \langle d, \nabla f(x + \alpha d) \rangle$, hence $\varphi'_{x,d}(0) = \|d\| \|\nabla f(x)\| \cos(\theta)$. If d where $-\nabla f(x)$ then $\theta = \pi$, since it's rotated by further 45 degrees ($\pi/2$), then either $\theta = 3\pi/4$ or $\theta = 5\pi/4$; in either case, $\cos(\theta) = -\sqrt{2}/2 = -\cos(\pi/4)$, hence $\varphi'_{x,d}(0) = -\|\nabla f(x)\|^2 \cos(\pi/4)$
- **Solution to Exercise 4.20:** $Q^i(d) = \frac{1}{2} d^T \nabla^2 f(x^i) d + \langle \nabla f(x^i), d \rangle + f(x^i)$; asking that $\nabla Q^i(d^i) = 0$ means $\nabla f(x^i) + \nabla^2 f(x^i) d^i = 0$, i.e., $d^i = -[\nabla^2 f(x^i)]^{-1} \nabla f(x^i)$.
- **Solution to Exercise 4.21:** As in Theorem 2.2, $-\langle \nabla f(x^i), d^i \rangle = (d^i)^T \nabla^2 f(x^i) d^i \geq \tau \|d^i\|^2$. By Taylor's theorem, $\lim_{d \rightarrow 0} R(d) / \|d\|^2 = 0 \equiv \forall \varepsilon > 0 \exists h$ s.t. $R(d^i) \leq \varepsilon \|d^i\|^2 \forall i \geq h$. Thus, $R(d^i) \leq \varepsilon \|d^i\|^2 \leq (-\varepsilon / \tau) \langle \nabla f(x^i), d^i \rangle$. Hence, $f(x^i + d^i) - f(x^i) = \frac{1}{2} \langle \nabla f(x^i), d^i \rangle + R(d^i) \leq (\frac{1}{2} - \varepsilon / \tau) \langle \nabla f(x^i), d^i \rangle = (\frac{1}{2} - \varepsilon / \tau) \varphi'_{x^i, d^i}(0)$ eventually holds for all large enough i however chosen ε . Thus, the Armijo condition $f(x^i + d^i) - f(x^i) \leq m_1 \varphi'_{x^i, d^i}(0)$ will eventually hold at every iteration however chosen $m_1 < 1/2$. This uses τ -convexity, that is required for global convergence, but one could rather assume the milder $\nabla^2 f(x^*) \succ 0$ (required anyway for quadratic convergence) and use $\lambda_n(\nabla^2 f(x^*)) > 0$ in place of τ at the cost of complicating the argument somewhat.
- **Solution to Exercise 4.22:** Obvious (we've seen it happening already) by just a few algebraic steps: with $g = -z - R^{-1}q$, $z + g = -R^{-1}q$, hence $\nabla h(z + g) = (-R^{-1}q) + R^{-1}q = 0$.
- **Solution to Exercise 4.23:** Taking the gradient, condition ii. becomes $\nabla m^{i+1}(x) = \nabla f(x^{i+1}) + H^{i+1}(x - x^{i+1})$, and since $\nabla m^{i+1}(x^i) = \nabla f(x^i)$ one obtains $\nabla f(x^{i+1}) + H^{i+1}(x^i - x^{i+1}) = \nabla f(x^i)$.
- **Solution to Exercise 4.24:** For any $f(\cdot)$, a finite difference approximation of the derivative $f'(x)$ can be computed as $(f(x + \varepsilon) - f(x)) / \varepsilon$ for some appropriately chosen "small" ε . Hence, this also holds for $\nabla^2 f(\cdot)$, which is the Jacobian of $\nabla f(\cdot)$. A finite difference approximation of the i -th column of $\nabla^2 f(x)$ can be computed as $(\nabla f(x + \varepsilon u^i) - \nabla f(x)) / \varepsilon$, u^i as usual the i -th vector of the canonical basis; in other words, $[x + \varepsilon u^i]_h = x_h$ for all $h \neq i$, while $[x + \varepsilon u^i]_i = x_i + \varepsilon$. Computing this H^0 costs $n + 1$ gradient computations (i.e., as many gradient computations as iterations, and n can be large), plus it needs be inverted / factorised which is in general $O(n^3)$. Finding the appropriate numerical value for ε is nontrivial either: too large and the approximation will be bad, too small and the numerical errors in the computation of $\nabla f(\cdot)$ will be so large that the noise overwhelms the signal and the approximation will be bad, too.
- **Solution to Exercise 4.25:** Take $\nabla f(\cdot) \in C^0$ and d^{i-1} fixed; then, $\lim_{\beta \rightarrow 0} \varphi'_{x^i, d^i(\beta)}(0) = \lim_{\beta \rightarrow 0} \langle \nabla f(x^i), -\nabla f(x^i) - \|\nabla f(x^i)\|^2 \beta \rangle < 0$ (otherwise the algorithm would have stopped already).
- **Solution to Exercise 4.26:** The first step is diagonalization of the upper-left block: if λ_i and H_i are the eigenvalues and eigenvectors of D^i , then $A = (1 + \beta)I - \alpha D^i$ has eigenvalues $\lambda'_i = (1 + \beta)I - \lambda_i$ and the same eigenvectors H_i . As a result

$$C' = \begin{bmatrix} H & 0 \\ 0 & H \end{bmatrix} \begin{bmatrix} A & -\beta I \\ I & 0 \end{bmatrix} \begin{bmatrix} H^T & 0 \\ 0 & H^T \end{bmatrix} = \begin{bmatrix} (1 + \beta)I - \alpha\Lambda & -\beta I \\ I & 0 \end{bmatrix}$$

Now, since $H^T = H^{-1} C'$ is *similar* to C' and therefore it has the same eigenvalues [83]; hence, $\rho(C') = \rho(C)$. Then, one can block-diagonalise C' with 2×2 block diagonal by exchanges of rows and columns: that is, there exists a *permutation matrix* P [79] such that

$$C'' = PC'P^T = \begin{bmatrix} C_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & C_n \end{bmatrix} \quad \text{with} \quad C_i = \begin{bmatrix} 1 + \beta - \alpha\lambda_i & -\beta \\ 1 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2}$$

Since also for permutation matrices $P^T = P^{-1}$, C'' is similar to C' and therefore to C . The eigenvalues of a block-diagonal matrix are the union of those of the blocks, hence the eigenvalues of C the union of those of C_i .

- **Solution to Exercise 4.27:** The eigenvalues of C_i are the roots of the characteristic polynomial $p(\lambda) = \det(C_i - \lambda I) = \lambda^2 + (1 + \beta - \alpha\lambda_i)\lambda + \beta$. These are extremely tedious (but possible) to compute and write down, the use of a symbolic system is advised; see, e.g., the screenshot below. Once this is done, it is easy (with the symbolic system, see again the screenshot) to check that the largest of the two eigenvalues is always $\leq \sqrt{\beta}$ if $\beta \geq (1 - \sqrt{\alpha\lambda_i})^2$ (and $\leq$ something > 1 , so we don't care).

```

In[62]:= Eigenvalues[{{1+b-a, -b}, {1, 0}}]

Out[62]:= {1/2 (1-a-sqrt((-1+a-b)^2-4 b+b)), 1/2 (1-a+sqrt((-1+a-b)^2-4 b+b))}

In[60]:= Reduce[Abs[1/2 (1-a-sqrt((-1+a-b)^2-4 b+b))] <= sqrt(b) && a >= 0 && b >= 0, {a, b}]

Out[60]:= (a == 0 && b == 1) || (a > 0 && 1-2 sqrt(a)+a <= b <= 1+2 sqrt(a)+a) ||
(0 <= a <= 1 && (0 <= b <= 1-2 sqrt(a)+a || b > 1+2 sqrt(a)+a)) ||
(a > 1 && b > 1+2 sqrt(a)+a) || (a == 1 && b == 0) || (0 <= a <= 1 && b == 0)

In[61]:= Reduce[Abs[1/2 (1-a+sqrt((-1+a-b)^2-4 b+b))] <= sqrt(b) && a >= 0 && b >= 0, {a, b}]

Out[61]:= (a == 0 && b == 1) || (a > 0 && 1-2 sqrt(a)+a <= b <= 1+2 sqrt(a)+a) ||
(a > 1 && (0 <= b <= 1-2 sqrt(a)+a || b == 0)) || (a == 1 && b == 0)

```

Hence, $\beta = \max\{(1 - \sqrt{\alpha\lambda_i})^2 : i = 1, \dots, n\}$ gives $\rho(C) \leq \sqrt{\beta} = \max\{|1 - \sqrt{\alpha\tau}|, |1 - \sqrt{\alpha L}|\}$.

- **Solution to Exercise 4.28:** Since $0 < \tau \leq L$, $\alpha = 4/(\sqrt{L} + \sqrt{\tau})^2 \leq 4/(\sqrt{L})^2 = 4/L$. On the other direction, $\alpha = 4/(\sqrt{L} + \sqrt{\tau})^2 \geq 4/(\sqrt{L} + \sqrt{L})^2 = 4/(2\sqrt{L})^2 = L$. For the rest, $\sqrt{\beta} = \max\{|1 - \sqrt{\alpha\tau}|, |1 - \sqrt{\alpha L}|\} = \max\left\{\left|1 - \sqrt{4\tau/(\sqrt{L} + \sqrt{\tau})^2}\right|, \left|1 - \sqrt{4L/(\sqrt{L} + \sqrt{\tau})^2}\right|\right\} =$
 $= \max\{|(\sqrt{L} + \sqrt{\tau} - 2\sqrt{\tau})/(\sqrt{L} + \sqrt{\tau})|, |(\sqrt{L} + \sqrt{\tau} - 2\sqrt{L})/(\sqrt{L} + \sqrt{\tau})|\} =$
 $= \max\{|(\sqrt{L} - \sqrt{\tau})/(\sqrt{L} + \sqrt{\tau})|, |(\sqrt{\tau} - \sqrt{L})/(\sqrt{L} + \sqrt{\tau})|\} = (\sqrt{L} - \sqrt{\tau})/(\sqrt{L} + \sqrt{\tau}).$

Chapter 5

Nonsmooth Convex Unconstrained Multivariate Optimization

5.1 Motivations

As we have seen throughout the whole of the previous Chapter, first-order information under the form of the gradient is crucial to minimization algorithms; second-order information is “nice to have”, but it can be conveniently approximated with gradients, which is in fact the best choice as the scale of the problem grows. One would therefore like to cling to gradient information. Unfortunately, there are several situations in which this is not possible: either computing it is too costly, or it does not exist. We will now showcase a few settings, purposely chosen among those of interest for these lecture notes, illustrating these issues.

5.1.1 Incremental (a.k.a. Stochastic) Gradient in ML

The first setting is very general in Machine Learning, and in fact was already described in §3.5: one is given a set $\mathcal{X} = [x^s \in \mathbb{R}^h, y^s \in \mathbb{R}^k]_{s \in S}$ of input / output samples that represent some unknown vector-valued function $f : \mathbb{R}^h \rightarrow \mathbb{R}^k$, i.e., $f(x^s) = y^s$ for all $s \in S$, and wants to construct an (arbitrarily complex) *predictor* $\pi(x) : \mathbb{R}^h \rightarrow \mathbb{R}^k$ that conveniently approximates it. A standard and general approach is to build a family of functions $\pi(x, w)$ where $\pi(x, \bar{w}) : \mathbb{R}^h \rightarrow \mathbb{R}^k$ for every fixed $\bar{w} \in \mathbb{R}^n$; that is, they are functions of $x \in \mathbb{R}^h$ once fixed the parameters $w \in \mathbb{R}^n$ (often called weights for reference to Neural Networks). Then, one selects a *loss function* $\ell : \mathbb{R}^k \times \mathbb{R}^k \rightarrow \mathbb{R}$ measuring the difference between the real and the predicted output; think $\ell(y, y') = \|y - y'\|$. This immediately defines the loss for a given choice of the parameters

$$\mathcal{L}_{\mathcal{X}}(w) = \sum_{s \in S} [\mathcal{L}^s(w) = \ell(y^s, \pi(x^s, w))]$$

that basically measures the “regret”, over the known data $\mathcal{X}$, of choosing any specific vector of parameters (weights) $w \in \mathbb{R}^n$. Note that it would be possible as well to have a different loss ℓ^s for each sample, but this is not usually done (and anyway does not impact our discussion). Thus, the “best” vector of parameters (over the known data) is arguably the solution of the training problem

$$\min \{ \mathcal{L}_{\mathcal{X}}(w) : w \in \mathbb{R}^n \}, \quad (5.1)$$

putting it firmly into our favourite setting. One would therefore immediately consider computing the gradient of the objective, which is $\nabla \mathcal{L}_{\mathcal{X}}(w) = \sum_{s \in S} \nabla \mathcal{L}^s(w)$, i.e., the sum of the m gradients of the individual regrets $\mathcal{L}^s$. Each $\nabla \mathcal{L}^s(w)$ can be “cheap” to compute: this is basically true even for “complex” functions like a NN, but for reference consider the simplest possible choices of the *linear least squares*: $\pi(x, w) = \langle x, w \rangle$ a linear function (which immediately implies $h = n$), and $\ell(y, y') = \|y - y'\|_2^2 / 2$. Then,

$$\mathcal{L}^s(w) = (y^s - \langle x^s, w \rangle)^2 / 2 \quad \text{and} \quad \nabla \mathcal{L}^s(w) = -x^s (y^s - \langle x^s, w \rangle). \quad (5.2)$$

Thus, each $\nabla \mathcal{L}^s$ is indeed very cheap to compute; however, $\nabla \mathcal{L}_{\mathcal{X}}(w)$ is the sum of them all. The issue here is that the number m of samples can be exceedingly large; in fact, the power of Machine Learning techniques has only been revealed when sufficiently large data sets (and the computational resources necessary to process them) have become available. Therefore, one has to assume that “ m is very large”; this means from **1e+6** / **1e+7** (millions to tens of millions) of, say, images datasets to **1e+11** (hundredths of billions) of language datasets. Furthermore, in some very relevant cases also the number of *features* (the length of the inputs, h) may grow large; for instances, images are typically at least $256 \times 256 \approx 6.4\text{e}+4$ pixels, and can scale to the ten of millions of pixels (4K and 8K resolutions). Thus, even if the cost of computing each $\nabla \mathcal{L}^s(w)$ is roughly linear in h ,

computing a “full” $\nabla\mathcal{L}_{\mathcal{X}}(w)$ may be prohibitively costly, especially in view of the fact that the data may necessarily have to reside on the lower rungs of the memory hierarchy (RAM or even the filesystem).

The point, however, is that arguably “not all this information is really needed”. Ideally, the input/output pairs $[x^s, y^s]$ should be drawn at random, independently and identically distributed, out of the significant input space. Thus, the complete $\nabla\mathcal{L}_{\mathcal{X}}(w)$ is already, in fact, nothing but a finite approximation gradient of the “real” loss function, $\mathcal{L}(w) = \int_{\mathcal{X}} \ell(f(x), \pi(x, w)) dx$ (which is obviously completely unapproachable, even if well-defined at all). Therefore, there is the obvious question about the smallest size of the sample that is really required to get a “reasonably good approximation”—for our algorithmic purposes—of $\nabla\mathcal{L}_{\mathcal{X}}(w)$, in view of the fact that the latter is anyway really an approximation of $\nabla\mathcal{L}(w)$. In plain words, given any subset $T \subseteq S$ of the samples, one can consider

$$\mathcal{L}_T(w) = \sum_{s \in T} [\mathcal{L}^s(w) = \ell(y^s, \pi(x^s, w))] \quad \text{and} \quad \nabla\mathcal{L}_T(w) = \sum_{s \in T} \nabla\mathcal{L}^s(w) :$$

just compute the gradient on a (small) subset of the samples. Intuitively, “many of the samples will cancel out”, thus almost the same gradient information can be obtained with a much smaller sample than the whole of S , using which is referred to as a *batch* iteration while any $T \subset S$ is a *minibatch* one. At each iteration i , one thus has to decide the size of the subset T^i to be used, with the singleton $T^i = \{s^i\}$ for a single sample $s^i \in S$ being a definite possibility, as well as the exact choice of its elements.

For the second point, intuition and theory indicates that “all the samples should eventually be used, and used frequently enough”. The obvious (although not necessarily the only) way to implement this is to partition S into some disjoint subsets $S^1, S^2, \dots, S^p$ for some $p \leq m$, and use $T^i = S^q$ for some different q at each iteration, so that after p iterations the whole data set have been used. This sequence of iterations is usually referred to as an *epoch*: in the extreme case of batch iterations ($p = 1, S^p = S$) epochs are the same as iterations, in the other extreme case where $p = m$ and each S^q is a singleton an epoch can be a very long sequence of iterations. Thus, the usual approach is to perform a sequence of epochs, in each of which all the inputs in the dataset are used exactly once, up until the epoch is exhausted and a new one begins. Although the choice of the S^q could be the same for two successive epochs, the way in which these are chosen is typically random, and therefore the chosen S^q can—and in fact typically do—change at every epoch. Such an approach is called the *stochastic gradient descent* algorithm, and it is intimately tied with *online* learning approaches in which sample pairs $[x^s, y^s]$ keep appearing over time (typically “fast”), have to be used immediately one by one and immediately discarded because the system is, say, running on very-low-spec hardware “at the edge” that has no memory to retain the past samples. The issue, of course, is that using “small-size” S^q makes each iteration cheaper, but also implies that there is no guarantee that $-\nabla\mathcal{L}_{S^q}(w)$ be a descent direction; thus, there is no hope that such a method can be analysed with the conceptual tools developed in Chapter 4 for descent algorithms. However, at the end of the Chapter we have seen approaches, in particular Heavy Ball and Accelerated Gradient, that are not descent methods; this suggests that it should be possible to get formal convergence results for such methods, and this is indeed true.

However, the analysis of stochastic gradient descent requires conceptual tools that we can’t introduce here, so we’ll restrict to provide the flavour of the results that can be obtained. Since the methods rely on (pseudo-)random choices, the results can only be given in terms of expectation of the relevant quantities, i.e., the gap and/or the distance from an optimal solution. Furthermore, there is inherent “noise” in the iterates themselves: even if some iterate x^i turns out to be “close” to a minimum x^* , it is entirely possible that the next iterate x^{i+1} will not be such, intuitively due to “bad luck” in the choice of the sample subset T^i . In fact, results are typically not given in terms of individual iterates, but rather in terms of their *Cesàro average* $\bar{x}^i = (\sum_{k=0}^i x^k) / (i+1)$. This is the “mean of iterates”, with the obvious property that if $\{x^i\}$ is converging to some x^* , then eventually $\{\bar{x}^i\} \rightarrow x^*$. One could therefore picture the sequence of iterates in this kind of algorithm as subject to some sort of “Brownian motion”: individual iterates may swing around randomly, but as a whole the “cloud” of the iterates will eventually converge (with appropriate choices of the stepsize) towards the area of space where the minimum lies. How fast this happens is clearly dependent on how “informative” the first-order information is: since the actual choices are random, what counts is mostly the number of samples. If, say, $K = |T^i|$ is constant, the results strongly depend on K : as it could be expected, “extreme” mini-batching ($K = 1$) results in rather worse convergence speed than the deterministic case. For instance, [3, Th. 6.3] proves that even for $f \in C^1$ and τ -convex, $\mathbb{E}(f(\bar{x}^i) - f_*) \leq \varepsilon$ can only be guaranteed to happen for $i \geq O(1/\varepsilon^2)$, which is rather dramatically worse than what the deterministic case yields. As it could be expected, things improve somewhat as K grows [3, p. 334], but iteration cost grows (typically, at least linearly in K too). However, these results are still quite interesting for at least two reasons. From the practical side, they imply that these kinds of algorithms can indeed be used to obtain solutions: albeit the slow convergence means these will likely only be rather approximate ones, full batching may simply not be viable for extremely-large-scale problems, and getting anything is always way better than getting nothing. Furthermore, as repeatedly mentioned already, the applications of interest for these lecture notes typically neither require nor really benefit from highly accurate solution—and the above

discussion clearly show why: $\mathcal{L}_{\mathcal{X}}(w)$ is just an approximation of the “real” $\mathcal{L}(w)$ one would like to optimize, whose accuracy really is unknown (although hopefully growing better as $\mathcal{X}$ grows large). On the theoretical side, the results show that first-order methods are *quite robust to errors in ∇f* ; in plain words, one can still get convergence—albeit much slower one—if a rough approximation to the real first-order information is used. This is not surprising, in that it somewhat mirrors what we have already seen with second-order one. Thus, there reasonably is hope to achieve some sort of success even when first-order information is “of bad quality to start with”.

5.1.2 Nondifferentiable Regularisation

Any Machine Learning expert would immediately lecture about the fact that (5.1) is not the training problem that one should be solving. In very simplified terms, this is due to the fact that, again, $\mathcal{L}_{\mathcal{X}}(w)$ is just an approximation of the “real” $\mathcal{L}(w)$ that only “sees” the data in $\mathcal{X}$. Finding (an approximation to) its optimum w^* may therefore lead to “learning f very well on $\mathcal{X}$ and very badly elsewhere”, a phenomenon known as *overfitting*. Since the real goal of the exercise is to be able to predict f on points not seen so far, it turns out that it may be better to learn f less accurately on $\mathcal{X}$ in order to have better *generalization capabilities* outside of it. This is called the *bias / variance dilemma*, and—at a very general and purposely vague level—it can be helped by “keeping the model simpler”; Occam’s razor dictates that a simpler explanation is more likely to be correct than an unnecessarily complicated one. This—for good theoretical reasons that we don’t even try to touch with a bargepole—can be implemented in the setting of the previous section by adding a *regulariser* $\Omega(w)$ to the training problem, turning it into

$$\min \{ \mathcal{L}_{\mathcal{X}}(w) + \lambda \Omega(w) : w \in \mathbb{R}^n \} . \quad (5.3)$$

The regulariser can be a very simple function: the standard *ridge regularisation* is just $\Omega(w) = \|w\|_2^2 / 2$, a C^∞ τ -convex function with the cheapest possible first-order information $\nabla \Omega(w) = w$. Obviously, adding it to $\mathcal{L}_{\mathcal{X}}$ produces a “smaller” (and therefore “simpler”) solution w^* : during gradient iterations, the antigradient is $-\nabla \mathcal{L}_{\mathcal{X}}(w) - \lambda w$, which means that “large” components of w will have a “large” entry of the direction that points towards reducing them (in absolute value). Of course, the issue here is the value of the *hyperparameter* λ that basically dictates what “large” means. It is in general hard to figure out the appropriate value of λ , which is done via appropriate procedures (splitting the data in training / validation subsets, multiple times in k -fold cross validation, and testing the effect of different values of λ) that are not of interest here. What we rather mention is that this is in fact an example of *multiobjective optimization*, as already mentioned in §1.1.7: one has the two contrasting objectives of “learning well on $\mathcal{X}$ ” ($\mathcal{L}_{\mathcal{X}}$) and “keeping the model simple” (Ω), and approaches the vector-valued objective $[\mathcal{L}(w), \Omega(w)]$ by means of its *scalarization* (5.3).

There is, however, another possible approach to “keeping the model simple”: reduce the size of its inputs (h). It should always be remembered that one is approximating a completely unknown function, the underlying mechanisms of which are poorly understood, and therefore even what data really factors in its computation is not necessarily clear. Again by Occam’s razor, if one can explain well the underlying function f without some of the inputs these are likely only “adding noise” and they are better left out: doing so is called *feature selection*. One could therefore consider a different form of regularisation where $\Omega = \|\cdot\|_0$, i.e., the number of nonzero entries in w : if $w_i^* = 0$, this is an indication that the i -th entry of the input space is not needed. However, the zero norm—besides not really being a norm—is a rather “nasty” function, being nonconvex and even not continuous: furthermore, such a regularisation term is only sensitive to the number of nonzeros and insensitive to their size, meaning that it does not capture the concept of “complexity” underpinning the ridge regularisation. A reasonable compromise between these two (again) contrasting objectives has been found in the *Lasso regularisation*, i.e., $\Omega = \|\cdot\|_1$: the L_1 norm is the best convex approximation of $\|\cdot\|_0$ (cf. Figure A.1 and §A.8), and unlike the latter it is sensitive to the actual size of its inputs. We remark in passing that more complex choices are possible and may be more appropriate in some settings (cf., e.g., [15] and the references therein) but their discussion is not fit for these lecture notes. While being convex, the L_1 norm is clearly not differentiable: cf. Figure 3.1 and §3.2.2. One can easily see that the set of points in which the gradient does not exist is “small” (technically, a 0-measure set as for all convex functions), and therefore assume that this would not significantly impact the solution algorithms. Unfortunately, this is very far from being true, as the following example tries to illustrate.

Example 5.1. A toy example of Lasso-regularised problem

Consider the toy example $\mathcal{X} = \{[x^1 = [3, 2], y^1 = 2]\}$ ($h = 2, k = 1$): the linear least-squares model (5.2) is then $\mathcal{L}_{\mathcal{X}}(w_1, w_2) = (3w_1 + 2w_2 - 2)^2$. The Lasso regularisation with $\lambda = 10$ is then $10(|w_1| + |w_2|)$. Some level sets of the corresponding objective $\mathcal{L}_{\mathcal{X}} + \lambda \Omega$ are depicted in Figure 5.1(left), which immediately illustrates that the function is nondifferentiable on all points where $w_1 = 0$ or $w_2 = 0$ via the “kinks” in the level sets there. This is of course due to the fact

that, as anticipated in §2.5, the derivative of the absolute value is not defined in 0: the left derivative is -1 while the right one is $+1$. In other words, there is “uncertainty as to what the derivative actually is”: we will see shortly that *each number in $[0, 1]$ can be considered a valid approximation to the derivative*. In plain words, when one is computing derivatives in nondifferentiable points, there is an arbitrary choice to be made about the value, which finally yield *not just one, but a set* of (candidates to) the (anti)gradient, and therefore the descent direction.

This is illustrated in Figure 5.1(centre), “zooming” on the point $w = [0, 0]$ where the function is (“maximally”) nondifferentiable; incidentally, this would be a pretty reasonable starting iteration x^0 to a minimization algorithm. Recalling Figure 3.2 and §3.2.3, the gradient (and therefore the antigradient, indicated as $-g$ in the figure) should be orthogonal to the level sets. The figure shows that there are multiple ways for this to happen: each of the two “smooth branches of the level set that join in w ”, the upper and the lower one, has a tangent line of which the corresponding orthogonal direction is shown. The theory will shortly confirm that, basically, all directions in the cone generated by the two shown ones can be generated by arbitrarily picking a value in $[0, 1]$ for the derivatives of $|w_1|$ and $|w_2|$, and therefore are candidates to be the (opposite of) the (replacement of) the gradient in w . However, both the depicted “extremes” directions are *not of descent*, as clearly shown by the fact that they “point outside of the level set”: for reference, the minimum of the function is $w^* = [1/9, 0]$. Thus, for nondifferentiable functions there is “uncertainty” as to what the first-order information is, and choosing “randomly” within the possible choices could lead to selecting a direction that misses the fundamental property of the antigradient, i.e., that of being of descent. Hence, any descent-type method trying to use $-g$ as the direction would inevitably find a null stepsize, thereby repeating the iterate and irredeemably stalling.

However, the situation could also be read in a different way: as illustrated in Figure 5.1(right), there *are* directions among those that can be generated that are of descent. Furthermore, we will shortly see that all these directions have a very useful property: *they point roughly in the direction of the minimum w^* , in the sense that $\langle -g, w^* - w \rangle \geq 0$* . Thus, while there may not exist any step that leads to a decrease of the f -value, *there exist steps that lead to a decrease of the distance from w^** . This property, that is intimately related to convexity, will be shown to be crucial for the development of solution algorithms. This is not surprising: after all, the analysis of the Heavy Ball method (cf. §4.3.2) also focuses on decreasing the distance from the minimum as opposed to the function value, so we knew already that such an approach can work. As in the previous case, even a “low quality” first-order information can be enough to properly inform the decisions of an algorithmic scheme, although one should rightly suspect that this can lead to worse convergence properties.

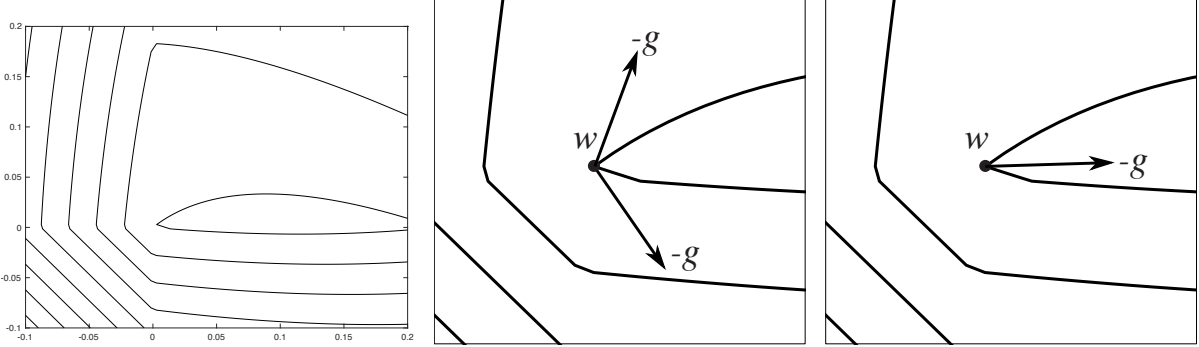


Figure 5.1: A Lasso-regularised linear least squares

5.1.3 Support Vector Machine / Regression

Our third motivation comes from another very well-known Machine Learning model, that is best explained referring to the case of a (supervised) *classification* task. This is basically the same setting as that of the previous paragraphs, except that $y^s \in \{-1, 1\}$. That is, individuals x^s are marked with having or not some specific property, and the task is to find a way to predict whether or not the property holds as a function of the features x describing each individual. It is easy to see that such a problem has countless applications in medicine, spam filtering, fraud detection, and very many others. The task is illustrated in Figure 5.2(left) where individuals are point in the Euclidean plane (i.e., $h = 2$) and the colour of the point dictates to which of the two classes it belongs. In this setting is therefore convenient to use a slightly different notation, whereby $\mathcal{X} = [x^s]_{s \in S} \subset \mathbb{R}^h$ and $\mathcal{X} = (\mathcal{X}^+ = [x^s]_{s \in S^+}) \cup (\mathcal{X}^- = [x^s]_{s \in S^-})$; that is, the data set is partitioned in the (disjoint) subsets of *positive and negative samples*, with the corresponding partition $S = S^+ \cup S^-$ of their indices.

Sticking with the “simplicity is good” ethos already commented about above, one may want to find a simple way to discriminate between the two classes. Arguably, the simplest way is to employ a *separating (affine) hyperplane* $H(w, b) = \{x \in \mathbb{R}^n : \langle w, x \rangle = b\}$; $w \in \mathbb{R}^n$ is the *direction* and $b \in \mathbb{R}$ is the *bias*. The idea is that the hyperplane (line in $\mathbb{R}^2$), i.e., the level set of a linear (affine) function, separates the space in two half-spaces, each of which should contain only points of one class. This is algebraically written

$$s \in S^+ \equiv y^s = 1 \implies \langle w, x^s \rangle > b \quad , \quad s \in S^- \equiv y^s = -1 \implies \langle w, x^s \rangle < b \quad ,$$

or, with a compact notation that will be useful, $y^s(\langle w, x^s \rangle - b) > 1$ for all $s \in S$. One may be slightly stressed by the strict inequality signs in the definition, but we have already extensively commented on the issue and therefore treat this with the usual liberal mindset.

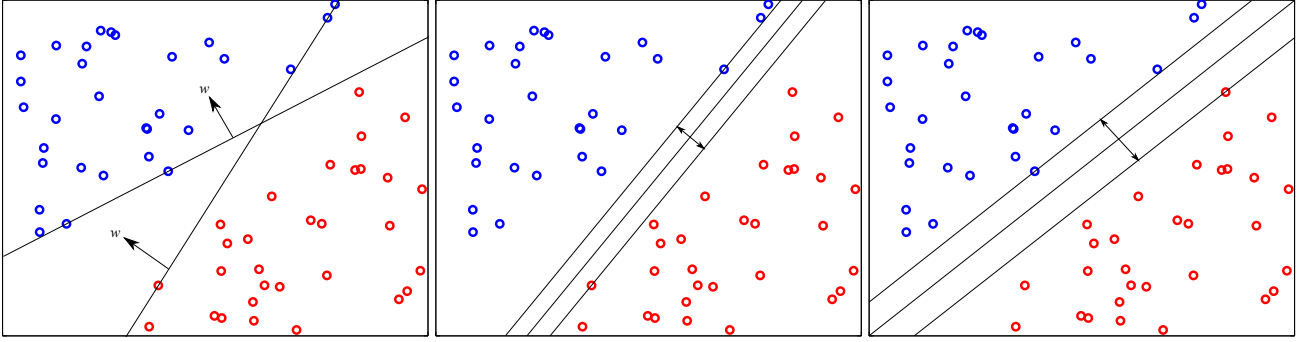


Figure 5.2: Illustration of Support Vector Machine idea

Exercise 5.1. Check the two definitions. [solution]

One issue with the approach is that such H may not exist at all, but this will be dealt with later. Rather, what needs be discussed here is the somewhat opposite problem of having multiple choices: Figure 5.2(left) shows two rather different separating hyperplanes for the same data sets. Clearly, the choice is significant in that it classifies large swaths of the space differently, and therefore may lead to vastly better, or worse, generalization capabilities. A sophisticated theoretical analysis—that we keep at respectful distance from—indicates that the *margin* of the chosen hyperplane is the relevant measure. Geometrically speaking, the margin is “how much one can shift the hyperplane backward and forward without impacting the classification”: Figure 5.2(centre) and (right) show two different choices with significantly different margins, of which the (right) one is then the better choice. Turning the geometric definition in algebraic formulæ requires simple arguments, basically just that the distance between two parallel hyperplanes $H(w, b)$ and $H(w, b')$ —with the same direction but different biases—is $|b - b'| / \|w\|$.

Exercise 5.2. Prove the formula above. [solution]

All in all, the problem of finding the *maximum margin separating hyperplane* can be cast as

$$\min \{ \|w\|^2 : y^s(\langle w, x^s \rangle - b) \geq 1 \quad s \in S \}$$

as maximising the norm minimises its inverse, and it can be easily proven that replacing the norm with its square does not change the optimal solution, while making the objective a (very simple) quadratic (hence smooth) function. However, there is no guarantee that the above problem even has a feasible solution, since $\mathcal{X}^+$ and $\mathcal{X}^-$ may not be linearly separable. It is therefore necessary to allow for some of the constraints to be violated; this leads to the *Support Vector Machine* (SVM) formulation

$$\min \{ \|w\|^2 + C[\mathcal{L}(w, b) = \sum_{s \in S} \max\{1 - y^s(\langle w, x^s \rangle - b), 0\}] \}$$

where a penalty term is added that discourages choosing an hyperplane that does not correctly separate some of the samples. In other words, $\mathcal{L}$ is the *empirical loss* that measures how well the hyperplane classifies the known data, whereas $\|w\|$ represents the *model complexity*, minimizing which (maximizing the margin) hopefully increases the generalisation capabilities of the classifier. The hyperparameter C weighs the two, again implementing a scalarization approach to the two-objective problem of simultaneously minimizing the loss and the complexity. The model is customarily extended to the *regression* task in which y^s are arbitrary real numbers by the Support Vector Regression (SVR) formulation

$$\min \{ \|w\|^2 + C[\mathcal{L}_\varepsilon\{w, b\} = \sum_{s \in S} \max\{|\langle w, x^s \rangle - b - y^s| - \varepsilon, 0\}] \}.$$

The loss function now measures the distance between the model prediction $\langle w, x^s \rangle - b$ and the true value y^s , but another hyperparameter ε is added to define the *insensitive tube*, i.e., the amount of error that is considered negligible and does not incur a penalization. In both cases, the problem is an unconstrained one: however, the loss function $\mathcal{L}$, while being convex, is nondifferentiable. This is illustrated in Figure 5.3 for a Toy SVM with 4 linearly separable points in $\mathbb{R}^2$, there the level sets of $L(w_1, w_2, b)$ are shown for some two-dimensional slices of the space. In particular, the image on left shows the level sets on $\{w_1, w_2\}$ plane for fixed $b = 0$, where the next two show the level sets on $\{w_1, b\}$ plane for two fixed values of w_2 (respectively, 0 and -1). It is plain to see how, due to the max operator, $\mathcal{L}$ is nondifferentiable, and therefore both SVM and SVR require the minimization of nondifferentiable objectives (although later on we will consider *reformulations* that avoid nondifferentiability at the cost of extra variables and constraints). Thus, nondifferentiable objectives do occur “naturally” in Machine Learning tasks.

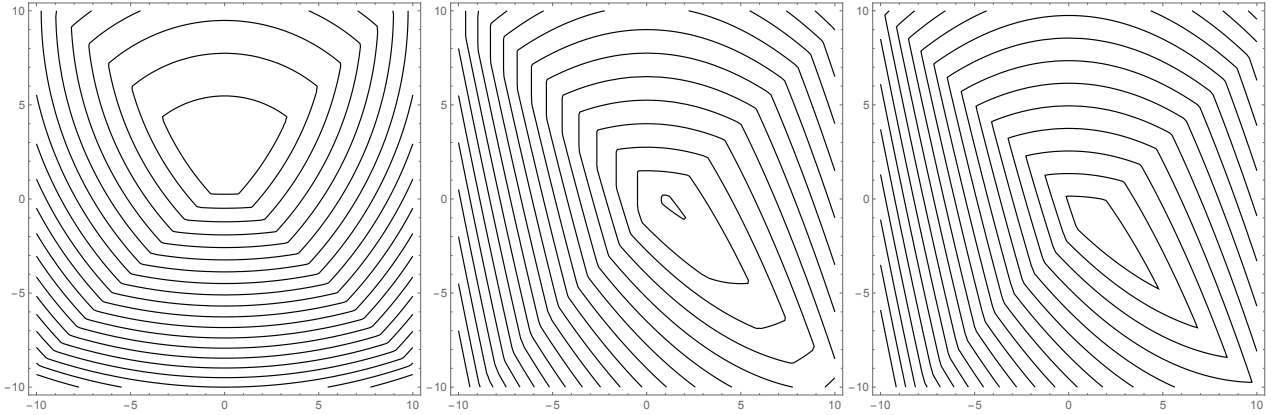


Figure 5.3: Level sets of the loss function of a toy SVM example.

On a similar note, the activation function of a Neural Network can be chosen to be nondifferentiable; cf. the ReLU one in §3.5. There can be compelling practical reasons to do this, despite the fact that this makes $\mathcal{L}_{\mathcal{X}}(w)$ nondifferentiable. All in all, these examples show that differentiability cannot be given for granted, and that the need of minimizing a nondifferentiable function does arise, in particular within important applications of high interest for these lecture notes. This therefore justifies studying the issue in some detail in order to be able to devise bespoke solution methods for this class of problems.

5.2 Convex Nondifferentiable functions

5.3 Nondifferentiable optimization is hard(er)

5.4 Subgradient methods

5.5 Smoothed gradient methods

5.6 Bundle methods

5.7 Wrap up

5.8 Solutions

Solutions

- Solution to Exercise 5.1:** $H(w, b)$ separates $\mathcal{X}^+$ from $\mathcal{X}^-$ if and only if $b^+ = \min\{\langle w, x^s \rangle : s \in S^+\} > b$ and $b^- = \max\{\langle w, x^s \rangle : s \in S^-\} < b$. Geometrically, b^+ is the largest value to which one can increase the bias while keeping all positive samples on one side, and b^- the smallest to which one can decrease the bias while keeping all negative samples on one side. Clearly, then, $b^+ > b > b^-$: hence, the mean $b^* = (b^+ + b^-)/2$ also satisfies $b^+ > b^* > b^-$, i.e., also $H(w, b^*)$ separates $\mathcal{X}^+$ from $\mathcal{X}^-$. Clearly, $H(w, b^*)$ is the hyperplane “in the middle” of the two “extreme” (a.k.a., “supporting”) hyperplanes $H(w, b^+)$ and $H(w, b^-)$ that, respectively, “touch” at least one $x^s \in \mathcal{X}^+$ and one $x^s \in \mathcal{X}^-$. Let $\delta = (b^+ - b^-)/2 > 0$ the “leeway” one has in this movement, i.e., $b^+ = b^* + \delta$ and $b^- = b^* - \delta$. Thus, $S^+ = \{s \in S : \langle w, x^s \rangle \geq b^+ = b^* + \delta\}$: by scaling we can transform (w, b^*) into (ω, β) such that $S^+ = \{s \in S : \langle \omega, x^s \rangle - \beta \geq 1\}$. Indeed, let $\gamma = 1/\delta > 0$, $\omega = \gamma w$ and $\beta = \gamma b^*$: $H(\omega, \beta) = H(w, b^*)$, thus it still separates $\mathcal{X}^+$ from $\mathcal{X}^-$. Since $\langle x^s, w \rangle \geq b^+$ for all $s \in S^+$ we have $\langle x^s, \omega \rangle - \beta \geq b^+ - b^* = \delta$; dividing by δ yields $\langle x^s, \omega \rangle - \beta \geq 1$. Symmetrically, $\langle x^s, \omega \rangle - \beta \leq -1$ for all $s \in S^-$; the two separate conditions can then be concisely written as $y^s(\langle \omega, x^s \rangle - \beta) \geq 1$ for all $s \in S$.
- Solution to Exercise 5.2:** For $H = H(w, b)$ and $H' = H(w, b')$, $d(H, H') = \min_{x, z} \{\|x - z\| : x \in H, z \in H'\}$. H and H' are parallel, and it is geometrically obvious (cf. level sets of an affine function) that we can arbitrarily fix z s.t. $\langle w, z \rangle = b'$ and solve $\min_x \{\|x - z\| : \langle w, x \rangle = b\}$. We will only see later on the optimality conditions for such a constrained problem which would allow to algebraically derive

the solution; here we again rely on the geometric intuition that the solution must clearly be the point $x \in H$ on the half-line emanating from z along the direction $w \perp H'$, i.e., $x = z + \alpha w$ for some $\alpha > 0$. Left-multiplying by w^T gives $\langle w, x \rangle = \langle w, z \rangle + \alpha \langle w, w \rangle \implies b = b' + \alpha \|w\|^2 \equiv \alpha = (b - b') / \|w\|^2$. Thus, $d(H, H') = \|x - z\| = \|\alpha w\| = [|b - b'| / \|w\|^2] \|w\| = |b - b'| / \|w\|$

- Solution to Exercise ??:** $v \in \partial f(x) \equiv f(z) \geq f(x) + \langle v, z - x \rangle$, and $w \in \partial g(x) \equiv g(z) \geq g(x) + \langle w, z - x \rangle$. Hence, $\alpha f(z) + \beta g(z) \geq \alpha[f(x) + \langle v, z - x \rangle] + \beta[g(x) + \langle w, z - x \rangle] = [\alpha f(x) + \beta g(x)] + \langle \alpha v + \beta w, z - x \rangle = [\alpha f + \beta g](x) + \langle \zeta, z - x \rangle$ for $\zeta = \alpha v + \beta w \implies \zeta \in \partial[\alpha f + \beta g](x)$
- Solution to Exercise ??:** $f(x) = |x| = \max\{f_1(x) = x, f_2(x) = -x\}$, hence we can use rule iv. $x > 0 \equiv I(x) = \{1\} \equiv \partial f(x) = \{f'_1(x)\} = \{1\}$. Symmetrically, $x < 0 \equiv I(x) = \{2\} \equiv \partial f(x) = \{f'_2(x)\} = \{-1\}$. Thus, $f(x)$ is differentiable $\forall x \neq 0$. However, $I(x) = \{1, 2\} \implies \partial f(x) = \text{conv}(f'_1(0) \cup f'_2(0)) = \text{conv}(\{1, -1\}) = [-1, 1]$, hence $f(x)$ is not differentiable in 0
- Solution to Exercise ??:** The crucial relationship comes from expanding $\|x^i - x_*\| - \alpha^i g^i\|^2 = \|x^i - x_*\|^2 - 2\alpha^i \langle x^i - x_*, g^i \rangle + (\alpha^i)^2 \|g^i\|^2$, changing sign in the middle term as $+2\alpha^i \langle x_* - x^i, g^i \rangle$, and then using the subgradient inequality $g^i \in \partial f(x^i) \equiv f(z) \geq f(x^i) + \langle g^i, z - x^i \rangle$ at $z = x_*$, yielding $\langle g^i, x_* - x^i \rangle \leq f(x_*) - f(x^i) \leq 0$ as already recalled. Then, the quadratic function $\phi(\alpha) = a\alpha^2 + b\alpha$ with $a = \|g^i\|^2 > 0$ and $b = 2(f_* - f(x^i)) < 0$ notoriously has the two roots $\alpha = 0$ and $\alpha = -b/a > 0$: $\phi(\alpha^i) < 0$ between the two roots, i.e., $\forall \alpha^i \in (0, 2(f(x^i) - f_*) / \|g^i\|^2)$, yielding $\|x^{i+1} - x_*\|^2 < \|x^i - x_*\|^2$, i.e., the algorithm would succeed in decreasing the distance from x_* . In particular, it is well-known that $\phi(\alpha)$ has minimum (most negative) value in the middle of the interval, i.e., $\alpha^i = (f^i - f_*) / \|g^i\|^2$ is the step guaranteeing the largest decrease in the distance from x_* . The issue here is clearly that f_* is unknown, and therefore the “optimal” step cannot be computed
- Solution to Exercise ??:** In the usual $\|x^{i+1} - x_*\|^2 \leq \|x^i - x_*\|^2 + 2\alpha^i(f_* - f^i) + (\alpha^i)^2 \|g^i\|^2$ plug $\alpha^i = \alpha^i_* = (f^i - f_*) / \|g^i\|^2$ to get $\|x^{i+1} - x_*\|^2 \leq \|x^i - x_*\|^2 + 2[(f^i - f_*) / \|g^i\|^2](f_* - f^i) + [(f^i - f_*) / \|g^i\|^2]^2 \|g^i\|^2 = \|x^i - x_*\|^2 - 2(f^i - f_*)^2 / \|g^i\|^2 + (f^i - f_*)^2 / \|g^i\|^2 = \|x^i - x_*\|^2 - (f^i - f_*)^2 / \|g^i\|^2$
- Solution to Exercise ??:** $f(x)$ is the pointwise maximum of (possibly, ∞ -ly many) linear functions $f_z(x)$, one for each $z \in Z$; since each f_z is convex, their maximum also is
- Solution to Exercise ??:** The first step is to write $f(x) = |x| = \max\{x, -x\} = \max\{zx : z \in \{-1, 1\}\}$. One then has to realise that “ $z \in \{-1, 1\}$ ” can equivalently be taken as “ $z \in [-1, 1]$ ”: in fact, for (say) $x > 0$ the maximum is still attained in $z = 1$, as for all $z < 1$ one has $zx < x$ (the case $x < 0$ is symmetric). Hence, $f_\mu(x) = \max\{zx - \mu z^2 / 2 : z \in [-1, 1]\}$. This is the maximum of the (concave) quadratic non-homogeneous univariate function $\phi(z) = zx - \mu z^2 / 2$ on the interval $[-1, 1]$, that we know well how to compute: first we write the unconstrained maximum $z_*(x) = x / \mu$, and then we project it on the interval, i.e., the maximum is $\max\{1, \min\{-1, x / \mu\}\}$. Plugging this formula into the function gives, after a bit of algebra,

$$f_\mu(x) = \begin{cases} x^2 / (2\mu) & \text{if } |x| \leq \mu \\ |x| - \mu / 2 & \text{if } |x| \geq \mu \end{cases}$$
 Hence, $f_\mu(x)$ “has the same shape” as $f(x)$ “far from 0” (i.e., for $|x| \geq \mu$), in that $f_\mu(x) = f(x) - \mu / 2$, whereas $f_\mu(x)$ is a simple quadratic function that “approximates” the absolute value close to 0; in particular, $f_\mu(x) = 0$. It is easy to verify that f_μ is continuous ($f_\mu(\mu) = \mu^2 / (2\mu) = \mu / 2 = \mu - \mu / 2$) and differentiable ($f'_\mu(\mu) = \mu / \mu = 1$), as expected since all convex functions are continuous (on the interior of their domain) and the cardinality of the subdifferential is that of the optimal solutions of the max problem, which always has a unique optimal solution. Thus, $f_\mu(x)$ is indeed a “smoothed version” of $f(x)$
- Solution to Exercise ??:** The crucial formula is $\mu = \varepsilon / (2K)$; in the standard approach, ε is fixed and so μ is $\implies$ if ε is small then so μ is, which makes the algorithm perform very small steps at the beginning (and for a long while) slowing down convergence. A simple idea is to rather take $\mu^i = \max\{f^i - f_*, \varepsilon\}$ and just run the algorithm with this varying μ . This results in much longer steps at the beginning while

the two will tend to behave similarly as $f^i \rightarrow f_*$. Of course, requires either having information about f_* , which is unlikely (but not impossible), or some form of target-level approach

- **Solution to Exercise ??:** $v^* = f_{\mathcal{B}}(x_{\mathcal{B}}^*) \geq f_*$, and $f(x^*) \geq f_*$ by definition, hence $f(x^*) \leq v^* \implies f_* \leq f(x^*) \leq v^* \leq f_* \implies f_* = f(x^*) \implies x^*$ optimal; in fact, it is not possible that $f(x^*) < v^*$, so the check could just be $f(x^*) = v^*$ (save of course for the issue of numerical errors)
- **Solution to Exercise ??:** Since $\mathcal{B}^{i+1} \supset \mathcal{B}^i$, it is immediate to see that $f_{\mathcal{B}}^{i+1}(x) \geq f_{\mathcal{B}}^i(x) \forall x \in \mathbb{R}^n$, whence $\underline{f}^{i+1} = f_{\mathcal{B}}^{i+1}(x^{*,i+1}) = v^{*,i+1} = \min\{f_{\mathcal{B}}^{i+1}(x)\} \geq \min\{f_{\mathcal{B}}^i(x)\} = v^{*,i} = f_{\mathcal{B}}^i(x^{*,i}) = \underline{f}^i$
- **Solution to Exercise ??:** Consider the convex extended-value function $g(x) = f(x) \forall x \in \mathcal{B}(x_*, \varepsilon)$, while $g(x) = \infty$ otherwise. Also, consider the variant to the Cutting Plane algorithm in which the constraint “ $x \in \mathcal{B}(x_*, \varepsilon)$ ” is added to the master problem (which is still a Linear Problem if the ball is, say, in the ∞ -norm, but even with the Euclidean norm it becomes a problem with convex quadratic—in fact, conic as we will see—constraints and therefore still “easy”). The convergence proof of the Cutting Plane algorithm [19, Th. 1] allows for this constraint in the master problem, and in fact it requires it unless $\mathcal{B}^0$ is “large enough” so that the master problem is bounded below; see next slide. So, the algorithm solves $\min\{g(x)\} = \min\{f(x)\}$ (the two problems obviously have the same optimal value and an optimal solution in x_*) when all the iterates are forced to remain in an arbitrarily small ball around x^* . Interestingly, this is not only an abstract proof: [19, Table 1] shows that if one would actually be able to force $x^{*,i} \in \mathcal{B}(x_*, \varepsilon)$ then the practical convergence of the Cutting Plane algorithm would typically be faster (dramatically so when ε is small). This is unfortunately impossible since x_* is usually unknown, but the mechanism does suggest the crucial idea behind the practically useful stabilisation approaches
- **Solution to Exercise ??:** Let $s(x) = \mu\|x - \bar{x}\|^2/2$ be the stabilising term, which clearly is a parabola with curvature μ and centred in $\bar{x}$. The optimality condition of the master problem is $0 \in \partial[f_{\mathcal{B}}(\cdot) + s(\cdot)](x^*) \equiv \exists g \in \partial f_{\mathcal{B}}(x^*) \text{ s.t. } g + \nabla s(x^*) = 0 \equiv g = -\nabla s(x^*)$. That is, the derivative of $f_{\mathcal{B}}$ must be the opposite of that of s in x^* . Of course, $-\nabla s(x^*) = \nabla[-s(\cdot)](x^*)$, and $-s$ is the same parabola “upside-down”. Hence, x^* is the point where the upside-down parabola and $f_{\mathcal{B}}$ have the same derivative. Geometrically, this can be found by imagining the upside-down parabola shifted by a negative constant, i.e., $-s(x) - M$, so that the value in $\bar{x}$ is $-M$. Then, one starts with a “very large” $M > 0$, so that the upside-down parabola is “pushed down a lot”, and gradually decreases M so that it “gradually moves up”. By stopping for the first (smallest) value of M such that the graph of $-s(x) - M$ and that of $f_{\mathcal{B}}$ touch, which must exist (for $M = 0$ the two graphs surely touch), one has that either the two derivatives are equal or $f_{\mathcal{B}}$ is differentiable there, or at least there exists a subgradient g of $f_{\mathcal{B}}$ that does the requisite job (see the pictures). Thus, the x coordinate of the point where the two meet is x_*
- **Solution to Exercise ??:** The first step is due to the fact (already seen in the previous exercise) that $0 \in \partial[f_{\mathcal{B}}(x + \cdot) + \mu\|\cdot\|^2/2](d^*) \equiv \exists g \in \partial f_{\mathcal{B}}(x + d^*) \text{ s.t. } g + \mu d^* = 0 \implies -\mu d^* \in \partial f_{\mathcal{B}}(x + d^*)$ since $\mu d^* = \nabla[\mu\|\cdot\|^2/2](d^*)$. The second step is just the subgradient inequality for $f_{\mathcal{B}}$ evaluated in $x + d^*$: $f(x) = f_{\mathcal{B}}(x) \geq f_{\mathcal{B}}(x + d^*) + \langle -\mu d^*, x - (x + d^*) \rangle$ (properly rearranged)
- **Solution to Exercise ??:** First note that $\{x^i\}$ is the sequence of the stability centres, not of the iterates. However, $\{x^i\} \rightarrow x$ and $\{\|d^{*,i}\|\} \rightarrow 0$ imply that $\{x^i + d^{*,i}\} \rightarrow x$ as well: both the stability centres and the iterates converge to the same point. Note that the stability centres may or may not finitely converge, i.e., after finitely many SS the centre may no longer be changed and only (infinitely many consecutive) NS will be done; yet, this is immaterial for the current result. Now, let $f_{\mathcal{B}}^i$ be the cutting plane model at iteration i , and $f_{\mathcal{B}}^\infty$ be the convex function defined by the set $\mathcal{B}^\infty$ containing all the infinitely many triples (x^i, f^i, g^i) : clearly, $\{f_{\mathcal{B}}^i\} \rightarrow f_{\mathcal{B}}^\infty$ pointwise, i.e., however fixed $z \in \mathbb{R}^n$ one has $\lim_{i \rightarrow \infty} f_{\mathcal{B}}^i(z) = \lim_{i \rightarrow \infty} \max\{l^h(z) : h \leq i\} = \sup\{l^i(z) : i \in \mathbb{N}\} = f_{\mathcal{B}}^\infty(z)$. Thus the theorem applies. Also, since $f_{\mathcal{B}}^i(z) \leq f(z)$ and $f_{\mathcal{B}}^\infty(z) = \lim_{i \rightarrow \infty} f_{\mathcal{B}}^i(z)$, then $f_{\mathcal{B}}^\infty(z) \leq f(z)$, i.e., $f_{\mathcal{B}}^\infty$ is still a correct lower model of f . Now, $-\mu d^{*,i} \in \partial f_{\mathcal{B}}^i(x^i + d^{*,i})$ and (again) $\{\|d^{*,i}\|\} \rightarrow 0$: thus, however chosen $\varepsilon > 0$ and $\delta > 0$ exists $g \in \partial f_{\mathcal{B}}^\infty(x)$, v s.t. $\|v\| \leq \delta$, z s.t. $\|z\| \leq \varepsilon$ and $z = g + v$ (just wait until i is large enough so that both $\|-\mu d^{*,i}\| \leq \delta$ and $\partial f_{\mathcal{B}}^i(x^i + d^{*,i}) \subset \partial f_{\mathcal{B}}^\infty(x) + \mathcal{B}(0, \varepsilon)$ hold). Hence, $\|g\| \leq \|z - v\| \leq \|z\| + \|v\| \leq \varepsilon + \delta$: there are elements in $\partial f_{\mathcal{B}}^\infty(x)$ arbitrarily close to 0. But $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is finite-valued and therefore $f_{\mathcal{B}}^\infty \leq f$ is also finite-valued, hence $\partial f_{\mathcal{B}}^\infty(x)$ is compact and therefore in particular closed: as a consequence, $0 \in \partial f_{\mathcal{B}}^\infty(x)$, i.e., x is a minimum of $f_{\mathcal{B}}^\infty$. Since $f_{\mathcal{B}}^\infty \leq f$,

$f_{\mathcal{B}}^{\infty}(x) \leq f_* \leq f(x)$. Now, $\mathcal{B}^{i+1} = \mathcal{B}^i \cup \{(x^i + d^{*,i}, f(x^i + d^{*,i}), g^i)\} \implies f_{\mathcal{B}}^{i+1}(x^i + d^{*,i}) = f(x^i + d^{*,i})$. Send $i \rightarrow \infty$ to yield $f_{\mathcal{B}}^{\infty}(x) = f(x)$: together with $f_{\mathcal{B}}^{\infty}(x) \leq f_* \leq f(x)$ this gives $f_{\mathcal{B}}^{\infty}(x) = f(x) = f_*$.

In the proof μ is fixed, but it easily extends to μ^i bounded above by some constant, so that $\{\|d^{*,i}\|\} \rightarrow 0 \implies \{\|\mu^i d^{*,i}\|\} \rightarrow 0$

- **Solution to Exercise ??:** Direct from $f_{\mathcal{B}}(x + d^*) - f(x) \leq -\mu\|d^*\|^2$ and the SS condition $f(x + d^*) - f(x) \leq m_1[f_{\mathcal{B}}(x + d^*) - f(x)]: \|d^{i,*}\|^2 \geq \varepsilon \forall i \implies f_{\mathcal{B}}(x^i + d^{i,*}) - f(x^i) \leq -\mu\varepsilon \forall i \implies f(x^i + d^{i,*}) - f(x^i) \leq -\mu m_1 \varepsilon$ at each i where a SS is declared ($x^{i+1} = x^i + d^{i,*}$). Thus, if ∞ -ly many SS are declared, $f(x^i) \rightarrow -\infty$; conversely, if $f_* > -\infty$ (f is bounded below) this cannot happen, which means that $\{\|d^{*,i}\|\} \rightarrow 0$ must happen instead

Note that, again, this proof is using a fixed μ , but is easily extended to μ^i bounded away from 0, or even $\mu^i \rightarrow 0$ provided that $\sum_{i=1}^{\infty} \mu^i = \infty$ (with the series actually only running on the SS iterations i)

Part III

Constrained Optimization

Chapter 6

Constrained Optimality and Duality

Chapter 7

Constrained Optimization

Part IV

Combinatorial Optimization

Chapter 8

A Fleeting Glimpse to Combinatorial Optimization

Part V

Supplementary Material

Bibliography

- [1] M.S. Bazaraa, H.D. Sherali, C.M. Shetty *Nonlinear Programming: Theory and Algorithms*, John Wiley & Sons, 2006.
- [2] S. Boyd, L. Vandenberghe *Convex Optimization*, Cambridge University Press, 2008 [<https://web.stanford.edu/~boyd/cvxbook>]
- [3] S. Bubeck *Convex Optimization: Algorithms and Complexity*, arXiv:1405.4980v2 [<https://arxiv.org/abs/1405.4980>, 2015]
- [4] P. Hansen, B. Jaumard “Lipschitz Optimization” in *Handbook of Global Optimization – Nonconvex optimization and its applications*, R. Horst and P.M. Pardalos (Eds.), Chapter 8, 407–494, Springer, 1995
- [5] J.-B. Hiriart-Urruty, C. Lemaréchal “Convex Analysis and Minimization Algorithms I” Springer-Verlag, 1993
- [6] J.-B. Hiriart-Urruty, C. Lemaréchal “Convex Analysis and Minimization Algorithms II” Springer-Verlag, 1993
- [7] M.R. Garey, D.S. Johnson *Computers and Intractability: A Guide to the Theory of NP-Completeness* Freeman, 1979
- [8] D. Li, X. Sun *Nonlinear Integer Programming* Springer International Series in Operations Research & Management Science, 2006
- [9] D.G. Luenberger, Y. Ye *Linear and Nonlinear Programming*, Springer International Series in Operations Research & Management Science, 2008
- [10] J. Nocedal, S.J. Wright, *Numerical Optimization – second edition*, Springer Series in Operations Research and Financial Engineering, 2006
- [11] W. Sun, Y.-X. Yuan, *Optimization Theory and Methods – Nonlinear Programming*, Springer Optimization and Its Applications, 2006.
- [12] H.C. Pinkham *Analysis, Convexity, and Optimization* Draft of September 4, 2014 [<https://www.math.columbia.edu/departement/pinkham/Optimizationbook.pdf>]
- [13] W.F. Trench, *Introduction to Real Analysis* Free Hyperlinked Edition 2.04, December 2013 [http://ramanujan.math.trinity.edu/wtrench/texts/TRENCH_REAL_ANALYSIS.PDF]
- [14] M. Cacciola, A. Forel, A. Frangioni, A. Lodi “The Differentiable Feasibility Pump” in *Lecture Notes in Computer Science* 15620, 26th International Conference on Integer Programming and Combinatorial Optimization — IPCO 2025, M. Megow and A. Basu (Eds.), 157—171, 2025
- [15] M. Cacciola, A. Frangioni, X. Li, A. Lodi “Deep Neural Networks pruning via the Structured Perspective Regularization” *SIAM Journal on Mathematics of Data Science* 5(4), 1051—1077, 2023
- [16] M. Collins *Computational Graphs, and Backpropagation*
Course notes for NLP, Columbia University <https://www.cs.columbia.edu/~mcollins/ff2.pdf>
- [17] G. d’Antonio, A. Frangioni *Convergence Analysis of Deflected Conditional Approximate Subgradient Methods*
<http://pages.di.unipi.it/frangio/abstracts.html#SIOPT08>
SIAM Journal on Optimization 20(1), 357–386, 2009.

- [18] E. de Klerk “The complexity of optimizing over a simplex, hypercube or sphere: a short survey” *Central European Journal of Operations Research* 16: 111–125, 2008 [<https://link.springer.com/content/pdf/10.1007/s10100-007-0052-9.pdf>]
- [19] A. Frangioni *Standard Bundle Methods: Untrusted Models and Duality* <http://pages.di.unipi.it/frangio/abstracts.html#NDOB> in Numerical Nonsmooth Optimization: State of the Art Algorithms, A.M. Bagirov, M. Gaudioso, N. Karimitsa, M. Mäkelä, S. Taheri (Eds.), 61–116, Springer, 2020
- [20] A. Frangioni, B. Gendron, E. Gorgone *Dynamic Smoothness Parameter for Fast Gradient Methods* <http://pages.di.unipi.it/frangio/abstracts.html#ORL17a> Optimization Letters 12(1), 43–53, 2018.
- [21] M. Gaviano, D. Lera “Complexity of general continuous minimization problems: a survey” *Optimization Methods and Software* 20(4-5), 1–20, 2005
- [22] E. Ghadimi, H.R. Feyzmahdavian, M. Johansson “Global convergence of the Heavy-ball method for convex optimization” *2015 European Control Conference (ECC)*, 310–315, 2015 [<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7330562>]
- [23] K.C. Kiwiel *Convergence of Approximate and Incremental Subgradient Methods for Convex Optimization* SIAM Journal on Optimization 14(3), 807–840, 2003
- [24] Y. Nesterov *Smooth minimization of non-smooth functions* Mathematical Programming 103, 127–152, 2005.
- [25] P. Ochs, *Local Convergence of the Heavy-Ball Method and iPiano for Non-convex Optimization* J. Optim. Theory Appl. 177, 153–180, 2018 <https://doi.org/10.1007/s10957-018-1272-y>
- [26] L. Serafino *Optimizing Without Derivatives: What Does the No Free Lunch Theorem Actually Say?* Notices of the AMS 61(7):750–755, 2014 <https://www.ams.org/notices/201407/rnoti-p750.pdf>
- [27] AutoDiff Org: <https://www.autodiff.org>
- [28] CommaLab: <https://commalab.di.unipi.it/courses>
- [29] CVX: <http://cvxr.com>
- [30] The Derivative-Free Library (DFL) at DIAG: <https://github.com/DerivativeFreeLibrary>
- [31] The Fido Project: <https://fidoproject.github.io>
- [32] OSI (Optimization Solver Interface) <https://github.com/coin-or/OSi>
- [33] PyTorch: <https://pytorch.org>
- [34] scikit-learn: <https://scikit-learn.org>
- [35] TensorFlow: <https://www.tensorflow.org>
- [36] AMPL <https://ampl.com>
- [37] GAMS <https://www.gams.com>
- [38] AIMMS <https://www.aimms.com/platform/aimms-development>
- [39] OPL <https://www.ibm.com/docs/en/icos/12.8.0.0?topic=opl-optimization-programming-language>
- [40] Coliop <http://www.coliop.org>
- [41] ZIMPL <https://zimpl.zib.de>
- [42] FLOPCpp <https://github.com/coin-or/FlopCpp>
- [43] COIN Rehearse <https://github.com/coin-or/Rehearse>
- [44] Gravity <https://github.com/coin-or/Gravity>

- [45] PuLP <https://github.com/coin-or/pulp>
- [46] Pyomo <http://www.pyomo.org>
- [47] JuMP <https://github.com/jump-dev/JuMP.jl>
- [48] YALMIP <https://yalmip.github.io>
- [49] The SMS++ Structured Modelling System <https://gitlab.com/smspp/smspp-project>
- [50] The SCIP (Solver Constraint Integer Programs) solver <https://www.scipopt.org>
- [51] The CBC (COIN Branch & Cut) Solver <https://github.com/coin-or/Cbc>
- [52] CLP: COIN-OR Linear Program Solver <https://www.coin-or.org/Clp>
- [53] Couenne (Convex Over and Under ENvelopes for Nonlinear Estimation) <https://www.coin-or.org/Couenne>
- [54] The HiGHS (high performance software for linear optimization) solver <https://highs.dev>
- [55] The IBM/ILOG CPLEX solver <https://www.ibm.com/products/ilog-cplex-optimization-studio/cplex-optimizer>
- [56] The Gurobi solver <https://www.gurobi.com>
- [57] The Mosek solver <https://www.mosek.com>
- [58] The MCFClass project <https://github.com/frangio68/Min-Cost-Flow-Class>
- [59] Wikipedia – Bolzano-Weierstrass theorem:
https://en.wikipedia.org/wiki/Bolzano%E2%80%93Weierstrass_theorem
- [60] Wikipedia – Cholesky decomposition: https://en.wikipedia.org/wiki/Cholesky_decomposition
- [61] Wikipedia – Cubic equation: https://en.wikipedia.org/wiki/Cubic_equation
- [62] Wikipedia – Determinant: <https://en.wikipedia.org/wiki/Determinant>
- [63] Wikipedia – Derivative: <https://en.wikipedia.org/wiki/Derivative>
- [64] Wikipedia – Double-precision floating-point format:
https://en.wikipedia.org/wiki/Double-precision_floating-point_format
- [65] Wikipedia – Eigenvalues and Eigenvectors https://en.wikipedia.org/wiki/Eigenvalues_and_eigenvectors
- [66] Wikipedia – Eigenvalue Algorithm: https://en.wikipedia.org/wiki/Eigenvalue_algorithm
- [67] Wikipedia – Extreme Value Theorem: https://en.wikipedia.org/wiki/Extreme_value_theorem
- [68] Wikipedia – Factorization of polynomials:
https://en.wikipedia.org/wiki/Factorization_of_polynomials
- [69] Wikipedia – Fibonacci sequence https://en.wikipedia.org/wiki/Fibonacci_sequence
- [70] Wikipedia – Gelfand’s formula [https://en.wikipedia.org/wiki/Spectral_radius#Gelfand’s_formula](https://en.wikipedia.org/wiki/Spectral_radius#Gelfand's_formula)
- [71] Wikipedia – Golden ratio https://en.wikipedia.org/wiki/Golden_ratio
- [72] Wikipedia – Intermediate Value Theorem:
https://en.wikipedia.org/wiki/Intermediate_value_theorem
- [73] Wikipedia – Islands of Space https://en.wikipedia.org/wiki/Islands_of_Space
- [74] Wikipedia – Integral <https://en.wikipedia.org/wiki/Integral>
- [75] Wikipedia – Laplace expansion: https://en.wikipedia.org/wiki/Laplace_expansion
- [76] Wikipedia – Matrix Norm: https://en.wikipedia.org/wiki/Matrix_norm

- [77] Wikipedia – Minifloat: <https://en.wikipedia.org/wiki/Minifloat>
- [78] Wikipedia – Norm: [https://en.wikipedia.org/wiki/Norm_\(mathematics\)](https://en.wikipedia.org/wiki/Norm_(mathematics))
- [79] Wikipedia – Permutation matrix https://en.wikipedia.org/wiki/Permutation_matrix
- [80] Wikipedia – Rotation matrix https://en.wikipedia.org/wiki/Rotation_matrix
- [81] Wikipedia – Series (mathematics) [https://en.wikipedia.org/wiki/Series_\(mathematics\)](https://en.wikipedia.org/wiki/Series_(mathematics))
- [82] Wikipedia – Sherman-Morrison Formula https://en.wikipedia.org/wiki/Sherman-Morrison_formula
- [83] Wikipedia – Matrix Similarity https://en.wikipedia.org/wiki/Matrix_similarity
- [84] Wikipedia – Underdetermined System: https://en.wikipedia.org/wiki/Underdetermined_system

Appendix A

Miscellaneous Mathematical Background

A.1 Infima, suprema and $\overline{\mathbb{R}}$

Some fundamental properties of the set $\mathbb{R}$ of real numbers are briefly recalled. For more detailed/formal treatment consult, e.g., [13, Chapter 1], [12, Lecture 2], [2, A.2.2].

One of the fundamental, although apparently trivial, properties of $\mathbb{R}$ is that of being *totally ordered*: however chosen x and $z \in \mathbb{R}$, at least one among $x \leq z$ and $z \leq x$ holds true. This is crucial for optimization, as discussed in the introduction; formally speaking this means that any set $S \subseteq \mathbb{R}$ has a well identified “best element” when minimizing or maximizing. Formally, the number $\underline{s} \in \mathbb{R}$ is the *infimum* of S , written $\underline{s} = \inf S$, if $\underline{s} \leq s$ for all $s \in S$, but however chosen $t > \underline{s}$ there exists some $s \in S$ such that $s \leq t$. In plain words, $\underline{s}$ is “below” all elements of S , but for any t “above” $\underline{s}$ this is not true, i.e., there is an element of S “below” t . Another way of expressing it is that the infimum is *the largest among all the minorants of S* , i.e., the numbers $t \in \mathbb{R}$ such that $t \leq s$ for all $s \in S$. This however requires defining “the largest”, which is of course the completely symmetric definition of the *supremum* of S : $\overline{s} = \sup S$ if $\overline{s} \geq s$ for all $s \in S$, but however chosen $t < \overline{s}$ there exists $s \in S$ such that $s \geq t$. Again, the supremum is *the smallest of the majorants*.

The issue is that not all set of real numbers have *finite* infimum or supremum: $S = \mathbb{R}$ is the obvious example. It is therefore convenient to introduce the set of *extended reals*: $\overline{\mathbb{R}} = \{-\infty\} \cup \mathbb{R} \cup \{+\infty\}$, where the special elements $-\infty$ and $+\infty$ are, respectively, the infimum of sets with no infimum and the supremum of sets with no supremum. Formally,

$$\inf S = -\infty \iff \forall t \in \mathbb{R} \exists s \in S \text{ such that } s \leq t,$$

i.e., if the set S is *unbounded below* (there are elements “as small (large negative) as one wants them”). Symmetrically,

$$\sup S = +\infty \iff \forall t \in \mathbb{R} \exists s \in S \text{ such that } s \geq t,$$

i.e., if the set S is *unbounded above* (there are elements “as large (positive) as one wants them”). The two special symbols “ $-\infty$ ” and “ $+\infty$ ” can, most of the times, be treated like ordinary real numbers with a bit of tweaking of the standard algebraic rules [13, p. 7], therefore usually we will avoid distinguishing between $\mathbb{R}$ and $\overline{\mathbb{R}}$ unless strictly necessary.

By contrast, *lower/upper bounded* subsets of $\mathbb{R}$ always possess finite inf/sup. Formally, one should distinguish the case where $\underline{s} \in S$ (which implies $\underline{s} \in \mathbb{R}$, i.e., S lower bounded) from the case where $\underline{s} \notin S$ (but still $\underline{s} \in \mathbb{R}$). In the first case $\underline{s}$ is also the *minimum* of S ($\underline{s} = \min S$), whereas in the second case S has no minimum. In our setting where “ $s \in \mathbb{R}$ ” actually means “ s is a fixed-size floating-point number” the distinction is hardly relevant, thus “min” will largely be considered equivalent to “inf”. Symmetrically, $\overline{s} = \max S$ will largely be considered equivalent to “sup”, even if formally $\sup S \notin S$.

As a final note, under the spirit that the infimum is the largest among all the minorants of S one readily obtains the convenient definition that $\inf \emptyset = +\infty$ (any $t \in \mathbb{R}$ is below all elements of $S = \emptyset$, and $+\infty$ is the largest of them all), and symmetrically $\sup \emptyset = -\infty$ since the supremum is the smallest of the majorants.

A.2 Vector space, scalar product

A vector space is any set V that is closed under two operations: addition (sum), and scalar multiplication (by a real value). That is, if $v \in V$, $w \in V$ and $\alpha \in \mathbb{R}$, then both $v + w \in V$ and $\alpha v \in V$.

For each integer $n \geq 1$, the set

$$\mathbb{R}^n = \{x = [x_1, x_2, \dots, x_n] = [x_i]_{i=1}^n : x_1 \in \mathbb{R}, \dots, x_n \in \mathbb{R}\}$$

of all n -dimensional vectors of reals is a vector space once the two operations are defined in the obvious way:

$$\begin{aligned} x + z &= [x_1 + z_1, x_2 + z_2, \dots, x_n + z_n] = [x_i + z_i]_{i=1}^n \\ \alpha x &= [\alpha x_1, \alpha x_2, \dots, \alpha x_n] = [\alpha x_i]_{i=1}^n \end{aligned}$$

$\mathbb{R}^n$ is in particular a *finite-dimensional* vector space in that it has (at least) a finite *basis*: there exists a finite set of vectors, in particular exactly n of them, $V = \langle u^1, u^2, \dots, u^n \rangle = \{v^i\}_{i=1}^n$ such that “all other vectors can be obtained by sum and scalar multiplication out of them”. Formally

$$\forall x \in \mathbb{R}^n \exists \alpha_1, \dots, \alpha_n \text{ such that } x = \alpha_1 u^1 + \dots + \alpha_n u^n.$$

There are of course infinitely many different bases of $\mathbb{R}^n$, but the most obvious is the *canonical base* whereby for each $i = 1, \dots, n$, $u_i^i = 1$ and $u_h^i = 0$ for $h \neq i$; in this case, $\alpha_i = x_i$. Although these lecture notes only deal with $\mathbb{R}^n$ and therefore give finite dimensionality for granted, it is worth remarking that not all vector spaces are finite-dimensional.

After the two defining operations, the most important one is the (*Euclidean*) *scalar product* of $x \in \mathbb{R}^n$ and $z \in \mathbb{R}^n$

$$\langle x, z \rangle = \sum_{i=1}^n x_i z_i = x_1 z_1 + \dots + x_n z_n.$$

The scalar product has the following properties, that are easy to verify:

1. $\langle x, z \rangle = \langle z, x \rangle \forall x, z \in \mathbb{R}^n$ (symmetry);
2. $\langle x, x \rangle \geq 0 \forall x \in \mathbb{R}^n$, and $\langle x, x \rangle = 0 \iff x = 0$;
3. $\langle \alpha x, z \rangle = \alpha \langle x, z \rangle \forall x \in \mathbb{R}^n$ and $\alpha \in \mathbb{R}$;
4. $\langle x + w, z \rangle = \langle x, z \rangle + \langle w, z \rangle \forall x, w, z \in \mathbb{R}^n$.

These could be taken as the definition of the concept of scalar product, of which different versions exist in other vector spaces (matrices, integrable functions, random variables, ...) which are mostly of no concern here, although some “nonstandard” notion of scalar product will turn out to be useful in one particular application. Rather, point 2. immediately introduces the concept of (*Euclidean*) *norm induced by the (Euclidean) scalar product*:

$$\|x\| = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\langle x, x \rangle}.$$

The norm provides a measure of the “size” of a vector: from the properties of the scalar product immediately follows that

1. $\|x\| \geq 0 \forall x \in \mathbb{R}^n$ and $\|x\| = 0 \iff x = 0$
2. $\|\alpha x\| = |\alpha| \|x\| \forall x \in \mathbb{R}^n$ and $\alpha \in \mathbb{R}$;
3. $\|x + z\| \leq \|x\| + \|z\| \forall x, z \in \mathbb{R}^n$ (triangle inequality).

Again, these could be taken as the definition of the concept of norm [78][2, A.1.2][10, p. 600]; in fact, even in these lecture notes different norms than the Euclidean one will be considered, but all satisfying the properties above. This will especially—but not only—be true for *matrices*. However, any norm immediately provides a mean of characterising “how far” (or “close”) two elements of a vector space are by their (*Euclidean*) *distance*

$$d(x, z) = \sqrt{\sum_{i=1}^n (x_i - z_i)^2} = \|x - z\|,$$

i.e., “the norm of x when z is the origin”. Of course, the distance inherits the properties of the norm:

1. $d(x, z) \geq 0 \forall x, z \in \mathbb{R}^n$ and $d(x, z) = 0 \iff x = z$;
2. $d(\alpha x, 0) = |\alpha| d(x, 0) \forall x \in \mathbb{R}^n$ and $\alpha \in \mathbb{R}$;
3. $d(x, w) \leq d(x, z) + d(z, w) \forall x, w, z \in \mathbb{R}^n$ (triangle inequality).

In turn, the distance immediately provides the notion of “all the points close enough to x ”, i.e., the *ball with center $x \in \mathbb{R}^n$ and radius $r > 0$*

$$\mathcal{B}(x, r) = \{z \in \mathbb{R}^n : \|z - x\| \leq r\}.$$

This is a central concept in the *topology* of vector spaces, as briefly discussed later on.

Here we rather focus on the important *geometric characterization of the scalar product*:

$$\langle x, z \rangle = \|x\| \cdot \|z\| \cdot \cos(\theta), \quad (\text{A.1})$$

where θ is the angle formed between x and z (when both are considered applied in the origin). This immediately provides a characterization of “when two vectors point roughly in the same direction”:

- $\langle x, z \rangle > 0$ means that θ is less than a square angle, i.e., “ x and z roughly point in the same direction”;
- $\langle x, z \rangle = 0$ means that the angle θ is square, i.e., x and z are orthogonal ($x \perp z$);
- $\langle x, z \rangle < 0$ means that θ is greater than a square angle, i.e., “ x and z roughly point in opposite directions”.

This interpretation will be repeatedly useful and it is illustrated in Figure A.1.

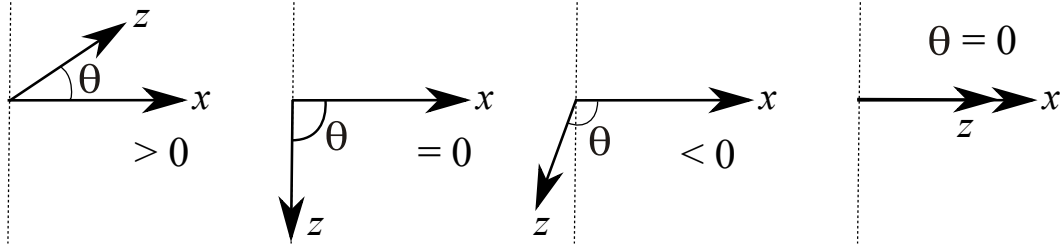


Figure A.1: Geometric characterization of the scalar product

An immediate consequence of the geometric characterization is the *Cauchy-Schwarz inequality*

$$|\langle x, z \rangle| \leq \|x\| \|z\| \quad \forall x, z$$

stating that (absolute value of) the scalar product is always less than or equal to the product of the norms. In particular, the scalar product is *equal* to the product of the norms only when the vectors are *collinear* (cf. Figure A.1(right)), i.e., they lie exactly along same line save possibly for the sign (when $\cos(\theta) = -1$, i.e., x and z point in exactly opposite directions); this will repeatedly come useful. Two other useful properties are

$$\|x + z\|^2 = \|x\|^2 + \|z\|^2 + 2\langle x, z \rangle$$

(this only holds true for the Euclidean norm), and the parallelogram Law

$$2\|x\|^2 + 2\|z\|^2 = \|x + z\|^2 + \|x - z\|^2.$$

A more thorough presentation of these concepts can be found in almost every book, e.g., [2, A.1.1], [12, Lecture 5], [13, §5.1].

A.3 Matrices, transpose, symmetry, products

Matrices [2, A.5] generalise vectors in that, rather than assuming the numbers to be in an “unstructured list”, the elements have a precise geometrical arrangement. That is, a (real) matrix $A \in \mathbb{R}^{m \times n}$ is a rectangular arrangement of $m \times n$ numbers with m rows and n columns. In a formula

$$A = \begin{bmatrix} A_1 \\ A_2 \\ \vdots \\ A_m \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1n} \\ A_{21} & A_{22} & \dots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \dots & A_{mn} \end{bmatrix} = [A^1, A^2, \dots, A^n].$$

That is, each element of A is characterised by two indices: A_{ij} is the element in row i and column j . It is often expedient to see A as formed by m “horizontal” rows $A_i \in \mathbb{R}^n$, $i = 1, \dots, m$, stacked “vertically” on top of one another, or alternatively as formed by n “vertical” columns $A^j \in \mathbb{R}^m$, $j = 1, \dots, n$, stacked “horizontally” next to one another. This may ingenerate confusion as to whether a vector $x \in \mathbb{R}^n$ is “horizontal” or “vertical”; albeit it’s mostly a matter of notation, the usual choice is that $x \in \mathbb{R}^n$ is the same as $x \in \mathbb{R}^{n \times 1}$, i.e., a “column vector”.

A useful operation is exchanging rows with columns, i.e., the transpose. This may start with the humble vector: if $x \in \mathbb{R}^{n \times 1}$ is a column vector, then $x^T \in \mathbb{R}^{1 \times n}$ is a row vector: from “vertical” it becomes “horizontal”. In general, if $A \in \mathbb{R}^{m \times n}$, then $A^T \in \mathbb{R}^{n \times m}$ with

$$A^T = \begin{bmatrix} (A^1)^T \\ (A^2)^T \\ \vdots \\ (A^n)^T \end{bmatrix} = \begin{bmatrix} A_{11} & A_{21} & \dots & A_{m1} \\ A_{12} & A_{22} & \dots & A_{m2} \\ \vdots & \vdots & \ddots & \vdots \\ A_{1n} & A_{2n} & \dots & A_{mn} \end{bmatrix} = [A_1^T, A_2^T, \dots, A_m^T].$$

Of course, transposing the transpose gives back the original matrix: $(A^T)^T = A$. In general, a matrix and its transpose “look very different” although they are formed of the same elements. There is one crucial case, however, when this does not happen: this is the case where $A = A^T$, i.e., A is *symmetric*. This can of course only happen if $n = m$, i.e., A is a *square matrix* and just means that $A_{ij} = A_{ji}$ for all $1 \leq i \leq j \leq n$. Symmetric matrices have the very useful property that they can be represented only with $n(n-1)/2$ numbers instead of n^2 , i.e., they require roughly half the memory of a non-symmetric matrix of the same size. Note that $a \in \mathbb{R}$ can be considered $a \in \mathbb{R}^{1 \times 1}$, and obviously $a = a^T$: a (real) number is a symmetric square (real) matrix.

The fundamental operation with matrices in these lecture notes is the *matrix-vector product*. Since matrices are in general non-symmetric and $m \neq n$ happens, there are two versions of matrix-vector product: *on the left*, which is only possible if the vector is $x \in \mathbb{R}^n$, i.e., it has the same size as rows

$$Ax = \begin{bmatrix} \langle A_1, x \rangle \\ \langle A_2, x \rangle \\ \vdots \\ \langle A_m, x \rangle \end{bmatrix} = \begin{bmatrix} A_{11}x_1 + A_{12}x_2 + \dots + A_{1n}x_n \\ A_{21}x_1 + A_{22}x_2 + \dots + A_{2n}x_n \\ \vdots \\ A_{m1}x_1 + A_{m2}x_2 + \dots + A_{mn}x_n \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_m \end{bmatrix} \in \mathbb{R}^{m \times 1}$$

and produces a (column) vector $v \in \mathbb{R}^m$, i.e., with the same size as columns, and, vice-versa, *on the right*, which is only possible if the vector is $x \in \mathbb{R}^m$, i.e., it has the same size as columns

$$x^T A = [\langle A^1, x \rangle, \langle A^2, x \rangle, \dots, \langle A^n, x \rangle] = [w_1, w_2, \dots, w_n] \in \mathbb{R}^{1 \times n}$$

and produces a (row) vector $w \in \mathbb{R}^n$, i.e., with the same size as rows. The computational cost of the operations is therefore $O(mn)$ in both cases, i.e., as the number of elements in A . This is obvious since each element of A is used only once. In fact, if A is sparse, i.e., its number of nonzero elements is $k \ll nm$, the operation can be implemented with cost $O(k)$ by just ignoring the zero elements, provided of course that the pattern of zero elements is known in advance.

Note that multiplying on the left we insist that x is a column vector, while multiplying on the right we rather transpose it and therefore assume it is a row vector. This is because the matrix-vector product is a special case of the *matrix-matrix product*. This is possible between two matrices $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, i.e., only if the rows of the leftmost matrix are of the same size as the columns of the rightmost one, and it basically consists in computing the scalar product between each row-column pair:

$$AB = \begin{bmatrix} \langle A_1, B^1 \rangle & \langle A_1, B^2 \rangle & \dots & \langle A_1, B^n \rangle \\ \langle A_2, B^1 \rangle & \langle A_2, B^2 \rangle & \dots & \langle A_2, B^n \rangle \\ \vdots & \vdots & \ddots & \vdots \\ \langle A_m, B^1 \rangle & \langle A_m, B^2 \rangle & \dots & \langle A_m, B^n \rangle \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

Of course, then, the matrix-vector product obeys the same rule; note that $Ax = z \in \mathbb{R}^{m \times 1}$, i.e., is a column vector like x , while $x^T A = z \in \mathbb{R}^{1 \times n}$, i.e., it is a row vector like x^T . The cost of computing the matrix-matrix product is $O(nmk)$ in general, unless A and/or B are sparse and their nonzero pattern is known in advance, hence it can be rather steep; indeed, matrix-matrix products will be carefully avoided in these lecture notes (at least in the practical implementation of algorithms). This is of course unless k is “small”, as in the special case

$$\langle x, z \rangle = x^T z = z^T x$$

of the scalar product where $k = 1$, which is of course $O(n)$ (unless at least one among z and x is sparse). By dint of the fact that one usually tries to avoid to distinguish between row vectors and column vectors unless strictly necessary, in these lecture notes the scalar product will also sometimes be written as just “ zx ” or “ xz ” whenever this does not create (too much) confusion.

A.4 Eigenvalues and the determinant, in practice

Given a symmetric $Q \in \mathbb{R}^{n \times n}$, its eigenvalues $\lambda \in \mathbb{R}$ —such that there exist the eigenvector $v \in \mathbb{R}^n$ satisfying $Qv = \lambda v$ —are crucial to determine if Q is singular or not. In particular, the *rank* of Q is the number of eigenvectors having non-zero eigenvalues, and Q is nonsingular if and only if it is full-rank, i.e., the rank is n , i.e., it has no zero eigenvalues. As usual we denote by $\lambda_1, \dots, \lambda_n$ the eigenvalues of Q . Also, recall that if Q is nonsingular and $Qv = \lambda v$ then $(1/\lambda)v = Q^{-1}v$, i.e., the eigenvalues of Q^{-1} are the inverse of those of Q .

One way to study non-singularity of Q is therefore that of considering its *determinant*

$$\det(Q) = \lambda_1 \lambda_2 \dots \lambda_n = \prod_{i=1}^n \lambda_i ,$$

i.e., the product of its eigenvalues. Then, Q is nonsingular if and only if $\lambda_i \neq 0$ for all $i = 1, \dots, n$, i.e., $\det(Q) \neq 0$. Indeed, whenever Q is nonsingular

$$\det(Q^{-1}) = \prod_{i=1}^n (1/\lambda_i) = 1/\det(Q) .$$

This clearly generalises the fact that, if $Q \in \mathbb{R}^{1 \times 1}$ is a single real number, the system $Qx = q$ is guaranteed to always have the solution $x = q/Q$ if and only if $Q \neq 0$. In fact, in this case $Q1 = Q$ means that Q is its own single eigenvalue corresponding to the “eigenvector” $v = 1$, and therefore $Q = \det(Q)$; then, $Q^{-1} = 1/Q$.

One may therefore want to be able to compute the determinant. This is indeed easy for very small values of n : for $n = 2$ (and Q symmetric as always)

$$\det \left(\begin{bmatrix} Q_{11} & Q_{21} \\ Q_{21} & Q_{22} \end{bmatrix} \right) = Q_{11}Q_{22} - Q_{21}^2 .$$

This for instance allows to precisely characterise 2×2 positive (semi) definite matrices by

$$Q_{11} \geq 0 \quad , \quad Q_{22} \geq 0 \quad , \quad Q_{11}Q_{22} \geq Q_{21}^2 .$$

More general formulæ can be defined for any n , but they quickly spiral out of control due to the fact that they require to recursively compute the determinant of (many) smaller matrices. That is, one has to define Q^{ij} as the $(n-1) \times (n-1)$ matrix obtained from Q by removing row i and column j : then

$$\begin{aligned} \det(Q) &= Q_{11}\det(Q^{11}) - Q_{12}\det(Q^{12}) + Q_{13}\det(Q^{13}) \\ &= Q_{11}(Q_{22}Q_{33} - Q_{23}Q_{32}) - Q_{12}(Q_{21}Q_{33} - Q_{31}Q_{23}) + Q_{13}(Q_{21}Q_{32} - Q_{31}Q_{22}) . \end{aligned}$$

This *Laplace expansion* formula [75] can be generalised to any n and recursively applying the definition of the determinant, but this ends up with an exponential number of terms and it is therefore not suitable unless for extremely small values of n .

This is somewhat unfortunate due to the fact that the eigenvalues can be proven to be the roots of *characteristic polynomial*, i.e., the solutions to

$$\det(Q - \lambda I) = 0$$

in the λ variable. This for instance allows to write the explicit formula of the eigenvalues of a 2×2 matrix, since

$$\det \left(\begin{bmatrix} Q_{11} & Q_{21} \\ Q_{21} & Q_{22} \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} \right) = \det \left(\begin{bmatrix} Q_{11} - \lambda & Q_{21} \\ Q_{21} & Q_{22} - \lambda \end{bmatrix} \right) = (Q_{11} - \lambda)(Q_{22} - \lambda) - Q_{21}^2 .$$

Solving the quadratic equation $\det(Q - \lambda I) = 0$ then yields

$$\lambda = \frac{1}{2} (Q_{11} + Q_{22} \pm \sqrt{4Q_{21}^2 + (Q_{11} - Q_{22})^2}) .$$

Unfortunately, this kind of approach does not scale at all; it can be applied to 3×3 matrices, yielding rather ugly formulæ, due to the fact that root of cubic polynomial have a (complicated) closed formula [61], but not further unless Q has some very special structure. The only result that is worth mentioning here is that if Q is (upper or lower) triangular, which of course includes diagonal, then

$$\det(Q) = \prod_{i=1}^n Q_{ii} ,$$

i.e., the determinant is the product of the diagonal elements. For further details check, e.g., [2, A.5.2][10, p. 603][65][62].

Exercise A.1. Prove the last statement. [solution]

Exercise A.2. Find the eigenvalues of $Q = \begin{bmatrix} 6 & -2 \\ -2 & 6 \end{bmatrix}$ and $P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 6 & -2 \\ 0 & -2 & 6 \end{bmatrix}$. [solution]

A.5 Limits and optimization

The basic object here is a(n infinite) sequence $\{v_i\} = \{v_1, v_2, \dots\} = [v_i]_{i=1}^\infty \subset \mathbb{R}$ of real numbers. The sequence has a real limit $v \in \mathbb{R}$, written

$$\{v_i\} \rightarrow v \quad \text{or} \quad \lim_{i \rightarrow \infty} v_i = v$$

if “eventually v_i gets arbitrarily close to v and never leaves”; that is

$$\forall \varepsilon > 0 \exists h \text{ s.t. } |v_i - v| \leq \varepsilon \forall i \geq h.$$

Sequences may not have a limit, the obvious example is $v_i = (-1)^i$ that infinitely alternates between 1 and -1 . Alternatively, a sequence may not have a real but rather an extended real limit $\pm\infty$:

$$\lim_{i \rightarrow \infty} v_i = +\infty \iff \forall M > 0 \exists h \text{ s.t. } v_i \geq M \forall i \geq h$$

$$\lim_{i \rightarrow \infty} v_i = -\infty \iff \forall M > 0 \exists h \text{ s.t. } v_i \leq -M \forall i \geq h.$$

If a sequence is monotone, i.e., either $v_{i+1} \geq v_i$ for all i (increasing) or $v_{i+1} \leq v_i$ for all i (decreasing), then it surely has a limit, although possibly $\pm\infty$ [13, Theorem 4.1.6]. In these lectures, $\{v_i\}$ is often the sequence of function values $\{f^i\}$ of some optimization algorithm, and it is natural to call $\{f^i\} \rightarrow f_*$ a *minimizing sequence* (in fact “minimising sequence” should be referred to as sequence of iterates x^i such that $f^i = f(x^i)$, but in this context is directly used for the sequence of f -values).

In some cases it is not possible, or desired, to ensure that $\{f^i\}$ is monotone. However, a non-monotone sequence can still exhibit useful behaviour when related to some optimization algorithm. Consider for instance $v_i = e^{(-1)^i}$. If this were the f -values sequence of some algorithm for which $f_* = 0$, one could still consider it a minimising sequence in the sense that eventually one gets f -values as close to 0 as desired; only, when some iterate h is reached that $v_i \leq \varepsilon$, it is not guaranteed that *all* subsequent iterates have the same property. Similarly, if one were maximising a function unbounded above, it would be fair to say that one gets f -values “as close as possible to $+\infty$ ”, if disregarding those iterates where f^i is instead “small” is not an issue. In other words, there can be weaker notion of convergence where one does *not* insist that *eventually all iterates become arbitrarily close to an optimal solution*, but rather than *some* do.

A useful mathematical notion that represents this choice is that of *limit inferior* and *limit superior*. This starts by “extract monotone sequences from $\{v_i\}$ the hard way”; that is, define

$$\underline{v}_i = \inf\{v_h : h \geq i\} \quad , \quad \bar{v}_i = \sup\{v_h : h \geq i\}.$$

Since the set of v_h for $h \geq i$ reduces as i increases, it is obvious that $\{\underline{v}_i\}$ is *nondecreasing* and $\{\bar{v}_i\}$ is *nonincreasing*: as such, they both have a limit

$$\liminf_{i \rightarrow \infty} v_i = \lim_{i \rightarrow \infty} \underline{v}_i = \underline{v}_\infty \quad , \quad \limsup_{i \rightarrow \infty} v_i = \lim_{i \rightarrow \infty} \bar{v}_i = \bar{v}_\infty.$$

Since obviously $\underline{v}_i \leq \bar{v}_i$ for all i , it necessarily holds $\underline{v}_\infty \leq \bar{v}_\infty$; in fact, it can be proven that $\lim_{i \rightarrow \infty} v_i = v$ if and only if $\underline{v}_\infty = v = \bar{v}_\infty$, i.e., a sequence has a limit if and only if its limit inferior and limit superior coincide. The operative definition for the limit inferior (the superior one is symmetric) is that *there exists a sub-sequence of $\{v_i\}$ that converges to $\underline{v}_\infty$* . A sub-sequence is just obtained by taking an infinite increasing sequence of natural numbers $\{n_i\}$ and considering $\{v_i\}$ “restricted” to these indices; that is, the sequence $\{r_i\} = \{v_{n_i}\}$. Thus, “not all the numbers in the original sequence $\{v_i\}$ converge to $\underline{v}_\infty$, but an appropriate (infinite) subset of them does”. Thus, a sequence of f -values $\{f^i\}$ such that $\liminf_{i \rightarrow \infty} f^i = \underline{f}_\infty = f_*$ can still be called a minimizing sequence with the intent defined above. These lecture notes will try to steer as clear as possible from this more complicated notion of convergence by focussing on monotone algorithms as much as possible, but the concept may turn up useful in some cases.

A final useful remark is that the definition of limit inferior is somehow “counterintuitive” in that it works “backwards” w.r.t. the one that would be natural in an optimization algorithm. If the sequence $\{f^i\}$ is *not* monotone, it is trivial to make it so by considering that of *record values* $f^{i,rec} = \min\{f^h : h \leq i\}$ (for a minimization problem). In plain words, one is just keeping track of the best f -value f^h found so far (at some iteration $i \leq h$, and typically of the corresponding iterate x^h that generated it). Obviously $\{f^{i,rec}\}$ is monotone *nonincreasing*, while note that $\{\underline{f}_\infty\}$ giving $\underline{f}_\infty$ is *nondecreasing*: in fact, in the former case the minimization is over the (known) *past* values (a set that increases with i), while in the latter case the minimization is over the (unknown) *future* values (a set that decreases with i). One could reasonably call $\{f^i\}$ a minimizing sequence if the corresponding $\{f^{i,rec}\} \rightarrow f_*$, on the ground that some iteration reaches as close as desired to the optimal value. However, this corresponds to a weaker notion of “convergence”, as the following stylised example shows: consider $v_i = f^i = i$, with $f_* = 1$. Clearly, $f^{i,rec} = 1$ for all i , and such a $\{f^i\}$ could then be called a minimising sequence. However, this corresponds to the notion “ f -values close to f_* are generated at some point, but eventually they can become arbitrarily far away and never come back”. By contrast, $\liminf_{i \rightarrow \infty} f^i = f_*$ corresponds to “ f -values close to f_* are generated at some point; much worse ones can be generated later, but eventually the sequence always produce again close ones”, and $\lim_{i \rightarrow \infty} f^i = f_*$ is the strongest one at “ f -values close to f_* are generated at some point, and never become worse since”.

However, all these notions rely on the fact that the sequence is infinite, and hence they are mainly theoretical. In practice the sequence of iterates, and their f -values, must necessarily be finite, and the hope is that algorithms will quickly identify a “good enough” solution and promptly stop. Yet, the issue is precisely that identifying when an iterate x^i is “good enough” is not always possible, and some algorithms need just to be let ran for a

while in the (possibly, theoretically founded) hope that they will eventually provide “good enough” solutions. In this case, the difference between different forms of convergence—i.e., whether all iterated will eventually “cluster around an optimal solution” or only some of them do—becomes relevant.

For further details on sequences and limits check, e.g., [10, p. 623], [13, p. 33], [12, §3.1].

A.6 Continuity

We discuss continuity mainly with the viewpoint of univariate functions $f : \mathbb{R} \rightarrow \mathbb{R}$, but using a general notation so that the concepts can easily be generalised to the multivariate case. This starts by recall that the ball of center x and radius r in the univariate case is just $\mathcal{B}(x, r) = (z \in \mathbb{R} : |z - x| \leq r) = (z \in \mathbb{R} : x - r \leq z \leq x + r)$. Similarly, $\|x\|$ can basically only mean $|x|$ for $x \in \mathbb{R}$. Then, f is continuous at x [12, Definition 3.1.5] [13, §2.2] if, *however chosen* a sequence $\{x^i\} \rightarrow x$ one has $\{f^i = f(x^i)\} \rightarrow f(x)$. In other words,

$$\forall \varepsilon > 0 \exists \delta > 0 \text{ such that } z \in \mathcal{B}(x, \delta) \implies \mathcal{B}(f(x), \varepsilon),$$

or more simply

$$\lim_{i \rightarrow \infty} x^i = x \implies \lim_{i \rightarrow \infty} f(x^i) = f(x).$$

The intuitive concept is that f cannot “jump away from the expected behaviour” at x : if, for some sequence $\{x^i\}$ converging to x the sequence of f -values is converging to some value f^∞ , then the function has no choice but to have $f(x) = f^\infty$. As a consequence, for a continuous function *the limits of all the sequences $\{f(x^i)\}$ for all the possible sequences $\{x^i\} \rightarrow x$ must be equal*. This makes it easy to see how a function can be not continuous: a simple but important one is

$$\text{sign}(x) = \begin{cases} -1 & \text{if } x < 0 \\ 0 & \text{if } x = 0 \\ +1 & \text{if } x > 0 \end{cases}.$$

Clearly, $\text{sign}()$ is continuous for all x save $x = 0$. In fact, the example suggests the introduction of the useful concept of *left limit* and *right limit* (only for univariate functions)

$$\lim_{x^i \rightarrow x_-} f(x^i) = f_- \quad \text{and} \quad \lim_{x^i \rightarrow x_+} f(x^i) = f_+$$

where in the first case the sequence can only be chosen so that $x^i < x$, while in the second $x^i > x$. Obviously, for a continuous function $f_- = f_+$. Here, instead,

$$\lim_{x^i \rightarrow 0_-} \text{sign}(x^i) = -1 < \text{sign}(0) = 0 < \lim_{x^i \rightarrow 0_+} \text{sign}(x^i) = 1.$$

That is, “from whatever direction 0 is approached”, the function suddenly “jumps” to a value that “cannot be predicted by the behaviour in close points”.

If f is continuous at all points of some set X then it is continuous on X . In order to avoid as much as possible to have to deal with “the set of points where the function is continuous”, like for the domain, we will strongly favour functions *continuous everywhere* (on all $\mathbb{R}$, or in general $\mathbb{R}^n$); the set of all such functions will be defined (for reasons to become clear later on), C^0 . Many “simple” functions belong to C^0 , and continuity is easily preserved under many operations between functions: if f and g are C^0 (an useful shortcut for “belong to C^0 ”), then all of $f + g$, $f \cdot g$, $\max\{f, g\}$, $\min\{f, g\}$, and—most importantly— $f(g(\cdot))$ are C^0 . Thus, continuity is not a hard property to come by. Of course there are plenty of simple non-continuous functions: the most obvious example is $1/x$ at $x = 0$ (the left and right limits “cannot be more different”, coming in at $-\infty$ and $+\infty$, respectively), but more contrived examples can be built such as $\sin(1/x)$ (the function keeps oscillating between -1 and 1 ever faster as $x \rightarrow 0$). Thus, continuity cannot always be given for granted.

Although it will not be used much at all in these lecture notes, we still remark that there are weaker forms of continuity that are sometimes useful. In particular, one says that f is *lower semi-continuous* (l.s.c.) at x if

$$\{x^i\} \rightarrow x \implies f(x) \leq \liminf_{i \rightarrow \infty} f(x_i)$$

or, equivalently,

$$\liminf_{z \rightarrow x} f(z) \geq f(x).$$

Symmetrically, an *upper semi-continuous* (u.s.c.) function satisfies

$$\{x^i\} \rightarrow x \implies f(x) \geq \limsup_{i \rightarrow \infty} f(x_i),$$

i.e.,

$$\limsup_{z \rightarrow x} f(z) \leq f(x).$$

In plain words, a l.s.c. function “can jump down wildly, but can never jump up”, and the converse holds for a

u.s.c. one. Being l.s.c. is “convenient for minimization”; if $\{x^i\}$ is a minimising sequence, i.e., $\{f(x^i)\} \rightarrow f_*$, and $\{x^i\} \rightarrow x$, then necessarily $f(x) = f_*$, i.e., x is optimal: in principle the function may jump down, but in a minimizing sequence it has “nowhere further down to go”. The argument readily extends to an ε -minimising sequence ($\{f^i\} \rightarrow f^\infty \leq f_* + \varepsilon$). The symmetric of course holds for an u.s.c. one, which is convenient for maximization. It is easy to see that f is continuous at x if and only if it is *both* l.s.c. and u.s.c. at x , i.e., plain continuity is “good both for minimization and maximization”; we will therefore stick with continuity. The *sign()* example shows a function that is neither l.s.c. nor u.s.c. (at 0).

More interesting for our developments is the *stronger* notion of continuity that “limits how fast f can change close to x ”: f is said to be (locally) *Lipschitz continuous (L-c) at x with Lipschitz constant $L > 0$* [10, p. 624] if there exists some $\varepsilon > 0$ such that

$$\|f(x) - f(z)\| \leq L\|x - z\| \quad \forall z \in \mathcal{B}(x, \varepsilon).$$

The two $\|\cdot\|$ in the definition allow to use it for the general case $f: \mathbb{R}^n \rightarrow \mathbb{R}^k$ even if $k > 1$: although such functions are not suitable to be objectives, they still have important uses throughout these lectures. In plain words, not only $f(z)$ must converge to $f(x)$ as z converges to x , as ordinary continuity requires; this has to happen “uniformly”, i.e., if “how much z is close to x dictates how much $f(z)$ is close to $f(x)$ ”. This basically means that *f cannot change too fast*.

Exercise A.3. Prove that f locally L-c at x imply f continuous at x . [solution]

Note that, in the definition, the constant L depends on x (it should be indicated as L_x to make it apparent, but we avoid it). A function can clearly be L-c with two different constants at x and at $z \neq x$. Of course, if f is L-c with constant $L > 0$, then it is also L-c with any constant $L' \geq L$. Hence, if it is L-c with constant (at most) L on all points of some set X then it is L-c on the whole set. If $X = \mathbb{R}$ ($\mathbb{R}^n$ in general), then f is said to be *globally L-c*. This is a much stronger notion than locally L-c, and a fortiori of continuity. However, this is a crucial notion to make optimization possible, or efficient.

Exercise A.4. Come up with f locally L-c everywhere but not globally L-c. [solution]

Exercise A.5. Come up with f continuous but not L-c on some bounded interval $X = [\underline{x}, \bar{x}]$. [solution]

A.7 (Univariate) Derivatives

Derivatives of univariate algebraic functions are easy to compute, since there is a powerful and complete algebra for this [12, Lecture 3] [13, §2.3]. For all the “basic” functions, the formula for the derivative is computed “from prime principles”, i.e., applying those for limits [13, §2.1] to the very definition of derivative. This yields the derivatives of many simple functions, i.e.,

- $[x^k]' = kx^{k-1}$;
- $[e^x]' = e^x$, $[\ln(x)]' = 1/x$;
- $[\sin(x)]' = \cos(x)$, $[\cos(x)]' = -\sin(x)$

Notably, derivatives of basic continuous functions are typically continuous themselves. Then, for many general functional operations formulæ provide the derivative in terms of these of the arguments, such as

- $[\alpha f(x) + \beta g(x)]' = \alpha f'(x) + \beta g'(x)$;
- $[f(x) \cdot g(x)]' = f'(x) \cdot g(x) + f(x) \cdot g'(x)$;
- $[f(x)/g(x)]' = [f'(x) \cdot g(x) - f(x) \cdot g'(x)]/g(x)^2$;
- $[f(g(x))]' = f'(g(x)) \cdot g'(x)$ (*chain rule*).

Again, most functional operations preserve continuous differentiability, but not all of them. Besides quotient, some important ones that don’t are

$$\max\{f(x), g(x)\} , \min\{f(x), g(x)\}.$$

For more details see, e.g., [13, §2.3, Ex. 2.3.1, Th. 2.3.4 / 2.3.5], [12, §3.1]. The computation of derivatives for “very complex” algebraic functions, such as Deep Neural Networks, may nonetheless pose some interesting problems in terms of being able to *efficiently* compute them. There is, however, the whole field of study of *automatic differentiation* [27] devoted to solving these problems, in terms of both developing the requisite theory and implementing it in efficient, well-engineered. available software packages for many different programming languages / systems that can be effectively integrated with the applications that require derivatives computation.

This is true in particular for Deep Neural Networks, whose layered structure allows layer-wise algorithms [16]. Thus, in these lecture notes we will just assume that “derivatives are efficiently computed somehow” and avoid to delve much further in the process.

A.8 Topology and limit in $\mathbb{R}^n$

When generalising the concept of limit—which itself is crucial for that of derivative—to $\mathbb{R}^n$, some aspects naturally become more involved. The point is that “there are many more ways for two points in $\mathbb{R}^n$ to be close to each other (or not)”. This requires to formally state some *topological concepts* that serve to generalize the one seen in univariate optimization. This starts with the very easy one of *ball* of center $x \in \mathbb{R}^n$ and radius $r > 0$ in some norm

$$\mathcal{B}(x, r) := \{z \in \mathbb{R}^n : \|z - x\| \leq r\},$$

i.e., the points “close” to x (in the chosen norm), that we have extensively used already in §A.6. There $n = 1$ and there is basically only one possible norm, but things are different when $n > 1$. Indeed, while it is very natural to imagine the Euclidean norm to be used in the definition above, any other norm satisfying the conditions set out in §A.2 will do, and there are very many different one.

In fact, the Euclidean norm just one member of the large family of *p-norms*, or *L_p-norms*, defined as

$$\|x\|_p := \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

however chosen the scalar $p > 0$; obviously, $p = 2$ for the Euclidean one, which in fact is also known as the *L₂* norm. The balls of radius 1 are depicted in the figure here on the right for $p = 0, 1/24, 1, 3/24, 2, 3, \infty$. It is easy to see that the ball for smaller p is strictly included in that for larger p . Indeed, the *L_∞* norm (obtained when $p \rightarrow \infty$)

$$\|x\|_\infty = \max\{|x_i| : i = 1, \dots, n\}$$

is the outermost square, as it is plain to see, while the *L₀* norm

$$\|x\|_0 = \#\{i : |x_i| > 0\}$$

that counts the number of nonzero entries in the vector x is the “very thin” cross in the centre. It should be noted that the *L₀* norm does not satisfy all the requirements and it is therefore not a norm, but it is still called such due to being one of the two extremes element of the *L_p* family; it is also relevant in that it is the natural measure of the *sparsity* (or lack thereof) of the vector x . Although the first time this Appendix is referenced we have not yet formally defined the concept for multivariate functions, we also mention that the *L_p* norms are convex functions (and, therefore, the corresponding balls are convex sets, which we will define later) for $p \geq 1$, and nonconvex ones for $p < 1$. Hence, the *L₁*—also known as *Lasso*—norm

$$\|x\|_1 = \sum_{i=1}^n |x_i|$$

it is relevant in that it is the *closest convex approximation of the L₀ norm*, and it is often used as such (i.e., as a convenient proxy for “the sparsity of x ”). While there are other relevant norms than the *L_p* ones, the family already has many interesting properties, starting from the fact that *p-norms* come in pairs: $\|\cdot\|_p$ and $\|\cdot\|_q$ are called *dual* of each other if $1/p + 1/q = 1$ [2, A.1.6], and dual norms satisfy *Hölder’s inequality*

$$|\langle x, z \rangle| \leq \|x\|_p \|z\|_q.$$

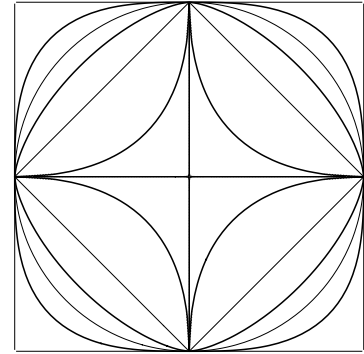
It is easy to see that for dual norms, p is “large” if and only if q is “small”; indeed, *L₁* and *L_∞* are dual. These are both convex and their balls are “particularly simple sets” (*polyhedra*, as we shall see), hence they are used in many applications. The only *self-dual* norm, i.e., for which $p = q$, is clearly the Euclidean one ($p = 2$), confirming its special role among norms. We will not make too much use of these concepts, though, so we don’t delve any further.

In fact, for the matter at hand here, the exact choice of the norm is immaterial. Indeed, all concepts in topology are related to properties like “there exists a ball” or “for all small balls”, as we shall see, but *all norms are topologically equivalent* [78]: however chosen two norms $\|\cdot\|$ and $|||\cdot|||$, there exist two constants $0 < \alpha < \beta$ such that $\alpha\|x\| \leq |||z||| \leq \beta\|x\|$ however chosen x and z in $\mathbb{R}^n$. Thus, for the subsequent treatment we can avoid specifying which norm—and distance—is actually intended.

Then, the definition of $\{x_i\} \rightarrow x$ in $\mathbb{R}^n$ can be stated as

$$\lim_{i \rightarrow \infty} x_i = x \quad \equiv \quad \forall \varepsilon > 0 \exists h \text{ s.t. } x_i \in \mathcal{B}(x, \varepsilon) \quad \forall i \geq h.$$

In layman terms, the points of $\{x_i\}$ *eventually all* come arbitrarily close to x . In fact, the definition can be



given in terms of distance as

$$\forall \varepsilon > 0 \exists h \text{ s.t. } d(x_i, x) \leq \varepsilon \forall i \geq h \equiv \lim_{i \rightarrow \infty} d(x_i, x) = 0.$$

It should be noted, however, that $\mathbb{R}^n$ is “exponentially larger” than $\mathbb{R}$; that is, “there are many more ways for $\{x_i\} \rightarrow x$ in $\mathbb{R}^n$ than in $\mathbb{R}$ ”. This requires care when some arguments are made, as we shall see.

A.9 Eigenvalues & matrix norms

While in some sense “matrices are just large vectors arranged differently”, the useful concepts of norm of a matrix are somewhat more involved than these for vectors and require a separate discussion [10, A1][65, 76]. A matrix $A \in \mathbb{R}^{m \times n}$ is a linear mapping $y = Ax : \mathbb{R}^n \rightarrow \mathbb{R}^m$, thus a general definition of norm can be built in terms of arbitrary norms in the two natural spaces, the input one $\mathbb{R}^n$ and the output one $\mathbb{R}^m$:

$$\|A\|_{\alpha, \beta} = \sup\{\|Ax\|_{\alpha} : \|x\|_{\beta} \leq 1\}.$$

In plain words, this is the largest norm of output vectors obtained by input vectors of unit size, i.e., “how much the matrix stretches the input space into the output space”. It is quite natural to take $\alpha = \beta$, obtaining the $\|\cdot\|_{\alpha}$. It is also natural to consider p -norms (cf. §A.8), and among them of course primarily the Euclidean one $p = 2$. In our cases where A is invariably symmetric (which is convenient since $m = n$, making it even more natural to consider the same norm in the two spaces), this norm is strongly tied to the eigenvalues of A .

Denoting by (λ, v) an eigenvalue / eigenvector pair (eep) of a symmetric matrix $Q \in \mathbb{R}^{n \times n}$, i.e., $Qv = \lambda v$, the following properties are trivial to verify:

1. $(c\lambda, v)$ is an eep for cQ however chosen $c \in \mathbb{R}$;
2. $(1 + \lambda, v)$ is an eep for $I + Q$;
3. (λ^2, v) is an eep for $Q^2 = QQ$ (extends to Q^k).

Then, the Euclidean matrix norm induced by the Euclidean vector norm is

$$\|Q\|_2 = \max\{\|Qd\|_2 : \|d\|_2 \leq 1\} = \max\{\|Qd\| / \|d\| : d \neq 0\} = \sqrt{\lambda_1(Q^T Q)}$$

by the variational characterization of eigenvalues. This proves the fundamental relationship between the (Euclidean – but all norms are topologically equivalent) matrix norm of Q and its eigenvalues. Easy and useful consequences are:

- $\|Qd\|_2 \leq \|Q\|_2 \|d\|_2$ for all $d \in \mathbb{R}^n$;
- $\|Q\|_2 = \max\{|\lambda_1(Q)|, |\lambda_n(Q)|\}$;
- if $Q \succeq 0$, then $\|Q\| = \lambda_1(Q)$, thus $\|Qd\|_2 \leq \lambda_1(Q) \|d\|_2$ for all $d \in \mathbb{R}^n$.

Exercise A.6. Prove 2. [solution]

While computing $\|Q\|_2$ is therefore nontrivial, it is easy to upper bound it in terms of the easier *Frobenius Norm* $\|Q\|_F = \sqrt{\sum_i \sum_j Q_{ij}^2}$ (basically, the norm of Q “flattened” as a $n \times n$ vector): $\|Q\|_2 \leq \|Q\|_F$, with equality holding if and only if Q is rank-one, i.e., $Q = uv^T$ for some $u \in \mathbb{R}^n$ and $v \in \mathbb{R}^n$ (or null).

A.10 Topology and feasibility

We now build upon the concept of §A.8 to define some properties of sets that are useful in the analysis of *constrained* optimization algorithms. In these lecture notes we keep a “light” approach to details and we will try to avoid to be too specific about these issues, but it makes sense nonetheless to at least mention these here. This starts with the definition of *interior* and *boundary* of an arbitrary set $S \subseteq \mathbb{R}^n$:

- $x \in \text{int}(S)$, i.e., is a point of the *interior* of S , if *there exists* a small enough ball centred in x and completely contained in S , i.e., any $r > 0$ such that $\mathcal{B}(x, r) \subseteq S$;
- $x \in \partial(S)$, i.e., is a point of the *boundary* of S , if *any* ball centred in x contains both points of S and points outside of S , that is, for each $r > 0$ there exist z and w , both belonging to $\mathcal{B}(x, r)$, such that $z \in S$ while $w \notin S$.

Note that by definition $x \in \text{int}(S)$ implies $x \in S$, but this is not true for points in the boundary. In fact, S is *open* if $S = \text{int}(S)$, i.e., “no points of S are on the boundary”. One can then define the *closure* of S as

$cl(S) = S \cup \partial S$, i.e., the set containing S and all the points on its boundary; then, S is *closed* if $S = cl(S)$, i.e., the closure operation does nothing since “all the points on the boundary of S already belong to it”. It can be proven that S is closed if and only if its complement, $\bar{S} = \mathbb{R}^n \setminus S$, is open. More relevant for our application is the following characterisation: S is closed if and only if *all limit points of sequences in S also belong to S* . Formally, however chosen $\{x_i\} \subset S$ such that $\{x_i\} \rightarrow x$, one is guaranteed that $x \in S$. It can be seen why this is (theoretically) important when $S = X$: provided that all iterates remain feasible, one is sure that also any limit point is such.

It could be mentioned that many sets in $\mathbb{R}^n$ are “too thin” to have an interior; just picture for instance a segment, or a line, in $\mathbb{R}^2$. Having a point in the interior requires a “full dimensional ball” to be included in S , which means that S itself must be *full dimensional* (while a line in $\mathbb{R}^2$ has dimension 1, i.e., smaller than that of the space). We will not delve too much on these aspects, but let us mention that sometimes the concept of *relative interior* can be useful; picture all the points in the segment except the two extremes.

Thus, being closed is a very useful (in theory, necessary) property to ask to our feasible regions X ; it is therefore reasonable to briefly discuss when this can be obtained, i.e., the *algebra of closed (and open) sets*. As usual, some sets can be easily seen to be closed (or open); this will be discussed in due time. The issue is therefore that of set operations that preserve closeness (or openness). The recap is simple enough:

- if $\{S_i\}$ is a family of (even infinitely many) open sets, then $\bigcup_i S_i$ is open;
- if S_1 and S_2 are open, then $S_1 \cap S_2$ is open;
- if $\{S_i\}$ is a family of (even infinitely many) closed sets, then $\bigcap_i S_i$ is closed;
- if S_1 and S_2 are closed, then $S_1 \cup S_2$ is closed.

We leave the proofs as exercises, together with simple facts such as: i) $\mathbb{R}^n$ and $\emptyset$ are *both* closed and open (hint: where’s the boundary?); ii) the union of infinitely many closed sets need not be closed, and iii) the intersection of infinitely many open sets need not be open.

Closed sets are therefore preferred in optimization, especially if also *bounded*: $\exists r > 0$ such that $S \subseteq \mathcal{B}(0, r)$, i.e., “the set does not go all the way down to infinity”. A closed and bounded set is called *compact*, and compactness equates to “piece of mind” for optimization (of continuous functions), at least theoretically, due to *Bolzano-Weierstrass Theorem* [59]. Let us recall that for a sequence $\{x_i\} \subset \mathbb{R}^n$ and a(n increasing) *index sequence* $\{n_i\}$, with $n_i \in \mathbb{N}$, the corresponding *subsequence* of $\{x_i\}$ is just $\{x_{n_i}\}$; in layman terms, only some of the elements of the original sequence. Then, x is an *accumulation point* of $\{x_i\}$ if *there exists a subsequence converging to it*: $\{x_{n_i}\} \rightarrow x$, i.e., $\liminf_{i \rightarrow \infty} d(x_i, x) = 0$. Clearly, this is a weaker requirement than the whole of $\{x_i\}$ converging to x : some iterates x_i may “stray away” from x occasionally, but “sooner or later a subsequent iterate will be again close to x ”. Now, assume that $\{x_i\}$ is a minimizing sequence, which in particular means that all iterates are feasible. Bolzano-Weierstrass Theorem states that if X is compact, then there exists a subsequence $\{x_{n_i}\}$ that converges to some x_* ; obviously $x_* \in X$ by closeness, and therefore clearly x_* is a candidate at being an optimal solution (it surely is if, e.g., $\{f(x_i)\}$ is monotone) [10, A2]. This is somewhat weird but very useful result: if you “fire enough (∞) shots in a compact, some of them eventually have to bunch very closely together. On first thought this could be considered intuitive, since a sequence (x^i) only have countably infinite elements, while typical $X \subseteq \mathbb{R}^n$ have uncountably many ones, and “ $\aleph_0 \ll \aleph_1$ ”. However, upon further reflection one realises the result is not so strange, since with infinitely many points “it is very hard to keep all them far from each other”. Consider for instance the unitary hypercube $[0, 1]^n$, and divide each side equally in h parts by means $h+1$ equally spaced points: this gives $(h+1)^n$ points and a regular mesh of h^n hypercubes, each of volume $(1/h)^n$. This is clearly a very good way of keeping each point as far away as possible from all the others, but still the volume vanishes as $h \rightarrow \infty$, i.e., eventually the points have to become arbitrarily close. Again, this is thanks to $\mathbb{R}^n$ being a finite-dimensional vector space, as this would not be true for infinite-dimensional ones.

For our purposes, the consequence is that “optimization is easy with a compact feasible region”: *all you need to do is to generate a minimizing sequence*, and—taking a subsequence if necessary—that will “automatically converge to some optimal solution”. Of course, this is only a very abstract statement: constructing a minimizing sequence, in particular for a nonconvex problem, remains an extremely challenging task.

All this discussion is largely theoretical, though, due to the oft-repeated fact that “there is no sharp boundary when using floating-point numbers”. So, for instance, one may have $X = (0, 1)$, a clearly open set, and a sequence of feasible iterates $\{x_i\} \rightarrow 1$ that “points towards” an unfeasible extreme. While this would be a serious issue mathematically, it has been already discussed in §1.1.5 why this is not so in practice. Thus, we close this section by making our stance explicit: *as far as we are concerned, all sets are closed*. And this is it.

A.11 Stronger forms of regularity

The concept of Lipschitz continuity (cf. §A.6) is related to that of (maximum) rate of change in a function, and therefore to that of derivatives (gradients). In fact, for any $f \in C^1$ it is easy to prove that:

1. if $\sup\{\|\nabla f(x)\|\} = L < \infty$ then f Lipschitz continuous (with constant L);
2. vice-versa, if f (is globally) Lipschitz continuous with constant L then $\|\nabla f(x)\| \leq L$.

Exercise A.7. Prove 1. out of Mean Value Theorem [13, Th. 5.4.5]. **[solution]**

Exercise A.8. Prove 2. “from prime principles”. **[solution]**

In turn, the concept of Lipschitz continuity is very important when applied to derivatives (gradients, Hessians, ...) so as to merit a specific name: $f \in C^1$ is *L-smooth* (as usual, one should add “on some subset $X \subseteq \mathbb{R}^n$ ” and then be careful about properties of X , but we’ll cut that short assuming $X = \mathbb{R}^n$ unless very strictly necessary) if ∇f is Lipschitz continuous (on X) with Lipschitz constant L . In formulæ,

$$\|\nabla f(x) - \nabla f(z)\| \leq L\|x - z\| \quad \forall x, z \in X.$$

The formula above is precisely why in §A.6 we insisted in defining Lipschitz continuity for the general case $f: \mathbb{R}^n \rightarrow \mathbb{R}^k$.

For a twice differentiable function $f \in C^2$, the initial result clearly applies to concept of *L-smoothness*: since $\nabla^2 f$ is the “gradient” (Jacobian) of ∇f , f being *L-smooth* means that ∇f is Lipschitz continuous is equivalent to $\nabla^2 f$ being *bounded*. That is, f is *L-smooth* means (cf. §A.9)

$$-LI \preceq \nabla^2 f(x) \preceq LI \equiv \max\{|\lambda^1|, |\lambda^n|\} \leq L \quad \forall x \in X.$$

For a *convex* function $f \in C^2$, since the eigenvalues are bounded below from 0 the previous results simplifies to

$$0 \preceq \nabla^2 f(x) \preceq LI \equiv [0 \leq \lambda^n \leq \lambda^1 \leq L \quad \forall x \in X].$$

Thus, in the all-important convex C^2 case, *L-smoothness* boils down to boundedness of eigenvalues of the Hessian. As we have seen, however, null eigenvalues and the corresponding directions of null curvature also bring their share of problems, hence also eigenvalues being bounded away from zero is potentially important. This is what the next definitions are about.

All definitions can be given in three forms: for any functions (even nondifferentiable), for C^1 functions and for C^2 functions. All definitions stipulate properties that must be true for all $x \in \mathbb{R}^n$ (more in general, $x \in X$ for the subset $X \subset \mathbb{R}^n$ where the property holds), plus possibly other parameters (the less, the more regular the function is). We start recalling the results for convexity, that are respectively:

- $\alpha f(x) + (1 - \alpha)f(z) \geq f(\alpha x + (1 - \alpha)z)$ for all $\alpha \in [0, 1]$ and $z \in \mathbb{R}^n$;
- $f \in C^1$ and $f(z) \geq f(x) + \langle \nabla f(x), z - x \rangle$ for all $z \in \mathbb{R}^n$;
- $f \in C^2$ and $\nabla^2 f(x) \succeq 0$

A(n ever so slightly) stronger notion is that of *strictly convex*:

- $\alpha f(x) + (1 - \alpha)f(z) > f(\alpha x + (1 - \alpha)z)$ for all $\alpha \in [0, 1]$ and $z \in \mathbb{R}^n$;
- $f \in C^1$ and $f(z) > f(x) + \langle \nabla f(x), z - x \rangle$ for all $z \in \mathbb{R}^n$;
- $f \in C^2$ and $\nabla^2 f(x) \succ 0$.

We have multiple times commented on our resistance to taking too seriously strict inequalities like “ $>$ ” and “ $\succ$ ” without any guarantee on how strict the relationship can become, due to the fact that real numbers can become “too close for comfort” in terms of their numerical representation as floating-point numbers. The definition of *strongly convex modulus* $\tau > 0$ (for short, τ -convex) goes in this direction by saying not only that f “strictly grows”, but that it “grows at least as fast as $\lambda_n \|x\|^2$ ”:

- $\alpha f(x) + (1 - \alpha)f(z) \geq f(\alpha x + (1 - \alpha)z) + \frac{\tau}{2}\alpha(1 - \alpha)\|z - x\|^2$ for all $\alpha \in [0, 1]$ and $z \in \mathbb{R}^n$;
- $f \in C^1$ and $f(z) \geq f(x) + \langle \nabla f(x), z - x \rangle + \frac{\tau}{2}\|z - x\|^2$ for all $z \in \mathbb{R}^n$;
- $f \in C^2$ and $\nabla^2 f(x) \succeq \tau I [\succ 0]$.

Exercise A.9. Prove the equivalence above. **[solution]**

The shorthand for the general definition is: f is τ -convex if and only if $f(x) - \frac{\tau}{2}\|x\|^2$ is convex: “removing a convex quadratic still leaves a convex function” (which is absolutely not obvious: while the sum of two convex functions is invariably convex, their difference is in general not). This is the definition that captures the case of a convex quadratic function with $\lambda_n > 0$, i.e., where no directions of null curvature exists *and the second-order information “clearly says it”*. Indeed, in this case (but, in fact, this is true even under strict convexity) the necessary and sufficient optimality conditions are equivalent.

Exercise A.10. Prove that any $f \in C^1$ strictly/strongly convex has a unique minimum, if any. **[solution]**

The “most convenient class” of functions for minimization are those that are C^2 , L -smooth and τ -convex, which means

$$\tau I \preceq \nabla^2 f \preceq LI \equiv [0 <] \tau \leq \lambda^n \leq \lambda^1 \leq L;$$

that is, eigenvalues of $\nabla^2 f$ are both bounded above and away from 0.

A.12 A super-brief recall on integrals

Due to the pervasive presence of derivatives in our treatment, may sometimes useful to deal with the inverse operation to derivation, i.e., integration. Unlike derivation, that is “easy”, integration is “difficult” but this is completely of no concern for us: never we deal with actually computing the integrals (and very little with computing derivatives, indeed). We will just recall the main definitions and the Fundamental theorem of calculus [74] inasmuch as it can be useful for some theoretical argument.

Given some $f : \mathbb{R} \rightarrow \mathbb{R}$, $f \in C^0$, another function $F : \mathbb{R} \rightarrow \mathbb{R}$, $f \in C^1$, is said to be the *antiderivative* of f if $F'(x) = f(x)$ for all $x \in \mathbb{R}$. Obviously the antiderivative is not unique, as “constant disappear when deriving”: if F is an antiderivative of f , then $F + c$ is so for any real constant c . This just needs to be taken into account, but constants mostly disappear automatically since it is the difference of antiderivatives that actually matter most. The antiderivative difficult to compute in general but easy to verify, which is completely fine to us.

A definition that to the uninitiated would appear to be completely unrelated is that of *integral*: given $g : \mathbb{R} \rightarrow \mathbb{R}$ and some nonempty real interval $X = [\underline{x}, \bar{x}]$, one defines the tagged partition of the interval in terms of $2n + 1$ numbers $x_0 \leq t_1 \leq x_1 \leq \dots \leq t_n \leq x_n$ with $x_0 = \underline{x}$ and $x_n = \bar{x}$. Then, the Riemann sum of g with respect to that tagged partition is $\sum_{i=1}^n g(t_i)(x_i - x_{i-1})$. Basically, the integral of g over X is “the limit of Riemann sums as the size of the tagged partition goes to ∞ ”. There is the implicit requirement that the partition is “balanced”, i.e., all the sub-intervals are vanishing: basically, one looks at the mesh $\Delta = \max\{x_i - x_{i-1} : i = 1, \dots, n\}$ of the partition. Then, S is the value of the integral of g over X , written $S = \int_{\underline{x}}^{\bar{x}} g(x)dx$, if for all $\varepsilon > 0$ there exists some $\Delta > 0$ such that $|S - \sum_{i=1}^n g(t_i)(x_i - x_{i-1})| \leq \varepsilon$ for all tagged partitions of X of mesh Δ . It goes without saying that as $\Delta \rightarrow 0$, $n \rightarrow \infty$ since $\Delta \geq (\bar{x} - \underline{x})/n$. The integral famously measures “the area below the graph of g ”.

The fundamental theorem of calculus then relate the integral and the antiderivative:

- if F is the antiderivative of f , then for all $X = [\underline{x}, \bar{x}]$, $\int_{\underline{x}}^{\bar{x}} f(x)dx = F(\bar{x}) - F(\underline{x})$;
- for any $f \in C^0$ and however chosen $\underline{x}$, if $F(x) = \int_{\underline{x}}^x f(t)dt$ for all $x \geq \underline{x}$, then F is an antiderivative of f .

As announced, integration is the inverse operation to derivation. A useful (and quite obvious) property is that integration is monotone: if $f(x) \geq g(x)$ for all $x \in [\underline{x}, \bar{x}]$, then $F(x) = \int_{\underline{x}}^x f(x)dx \geq G(x) = \int_{\underline{x}}^x g(x)dx$.

Solutions

- **Solution to Exercise A.1:** The only nonzero in the first row is Q_{11} , hence all the other terms vanish; repeat the argument in Q^{11} .
- **Solution to Exercise A.2:** We want to solve $\det(Q - \lambda I) = (6 - \lambda)(6 - \lambda) - 4 = \lambda^2 - 12\lambda + 32 = 0$, a quadratic equation whose discriminant is $\Delta = 12^2 - 4 \cdot 1 \cdot 32 = 16$. Since $\sqrt{\Delta} = 4$, $\lambda_{1,2} = (-(-12) \pm 4) / 2 = \{8, 4\}$; this is confirmed by the provided closed formula. Now, since P is block-diagonal, applying the Laplace expansion one has $\det(P) = P_{11}\det(P^{11}) = 1 \cdot \det(Q)$, hence $\det(P - \lambda I) = (1 - \lambda)\det(Q - \lambda I)$. Thus, the eigenvalues of P are the same as these of Q with the addition of $\lambda_3 = 1$.

- **Solution to Exercise A.3:** Take $\delta = \varepsilon / L$; then, for all $z \in \mathcal{B}(x, \delta)$ one has $|f(z) - f(x)| \leq L\|x - z\| \leq L\delta \leq L(\varepsilon / L) = \varepsilon$.
- **Solution to Exercise A.4:** Note: we'll have a much simpler proof later, after we present the relationships between L-c and derivatives. $f(x) = x^2$ is locally but not globally L-c; we prove this for $x \geq 0$, but the same arguments work for $x \leq 0$ (the function is symmetric). If $\delta > 0$ then $0 \leq f(x + \delta) - f(x) = (2x + \delta)\delta \leq 3x\delta$ if $\delta \leq x$; hence, f is L-c at x with Lipschitz constant $3x$ in some (right) interval around x . However, $\delta > 0$ also gives $f(x + \delta) - f(x) = (2x + \delta)\delta \geq 2x\delta$; hence, f cannot be L-c at x with Lipschitz constant less than $2x$, and that value is not bounded as $x \rightarrow \infty$. Symmetric arguments works for left intervals ($f(x) - f(x - \delta)$).
- **Solution to Exercise A.5:** The standard example is $f(x) = \sqrt{|x|}$, which is easily verified to be continuous and to become "infinitely steep" as $x \rightarrow 0$, because it is the inverse function of $y = x^2$ (for $x \geq 0$), and x^2 becomes "infinitely flat" as $x \rightarrow 0$. Again, we'll have a much simpler proof later, after we present the relationships between L-c and derivatives; in fact, the proof is so much simpler that it is not worth proceeding now, we just wait until we have the right tools.
- **Solution to Exercise A.6:** $\|Q\| = \sqrt{\lambda_1(Q^T Q)} = \sqrt{\lambda_1(Q^2)}$. The eigenvalues of Q^2 are the square of those of Q , hence their square root is the absolute value of those of Q . Clearly, the largest of the absolute values is either that of maximum eigenvalue or that of the minimum eigenvalue.
- **Solution to Exercise A.7:** Let $f \in C^1$. By the mean value theorem, however chosen x and z there exists some $w \in \text{conv}\{x, z\}$ (the segment with extremes x and z) such that $f(z) - f(x) = \langle \nabla f(w), z - x \rangle \leq \|\nabla f(w)\| \|z - x\| \leq L\|z - x\|$ (directly by the definition of L).
- **Solution to Exercise A.8:** Let $g = \nabla f(x)$ and $d = g / \|g\|$; $[0 \leq] \|g\| = \langle g, g \rangle / \|g\| = \langle g, d \rangle = \frac{\partial f}{\partial d}(x) = \lim_{t \rightarrow 0} (f(x + td) - f(x)) / t = \lim_{t \rightarrow 0} |f(x + td) - f(x)| / |t| = \lim_{t \rightarrow 0} |f(x + td) - f(x)| / |t|$. Since f is L-c, $|f(x + td) - f(x)| \leq L|t|$; hence, $\|\nabla f(x)\| \leq L$.
- **Solution to Exercise A.9:** For $g(x) = f(x) - \frac{\tau}{2} \|x\|^2$, $\nabla^2 g(x) = \nabla^2 f(x) - \frac{\tau}{2} I$, hence $\lambda_n(\nabla^2 g(x)) = \lambda_n(\nabla^2 f(x)) - \tau$.
- **Solution to Exercise A.10:** Let us assume that some minimum x^* of f exists: then, $\nabla f(x^*) = 0$. Pick any $z \neq x^*$: for strictly convex, $f(z) > f(x^*) + \langle \nabla f(x^*), z - x^* \rangle = f(x^*)$. For strongly convex, $f(z) \geq f(x^*) + \langle \nabla f(x^*), z - x^* \rangle + \frac{\tau}{2} \|z - x^*\|^2 > f(x^*)$ as $\|z - x^*\| > 0$. In fact, " $f \in C^1$ " is not needed: the result is true for nondifferentiable functions, it will be easy to prove when we'll get there.